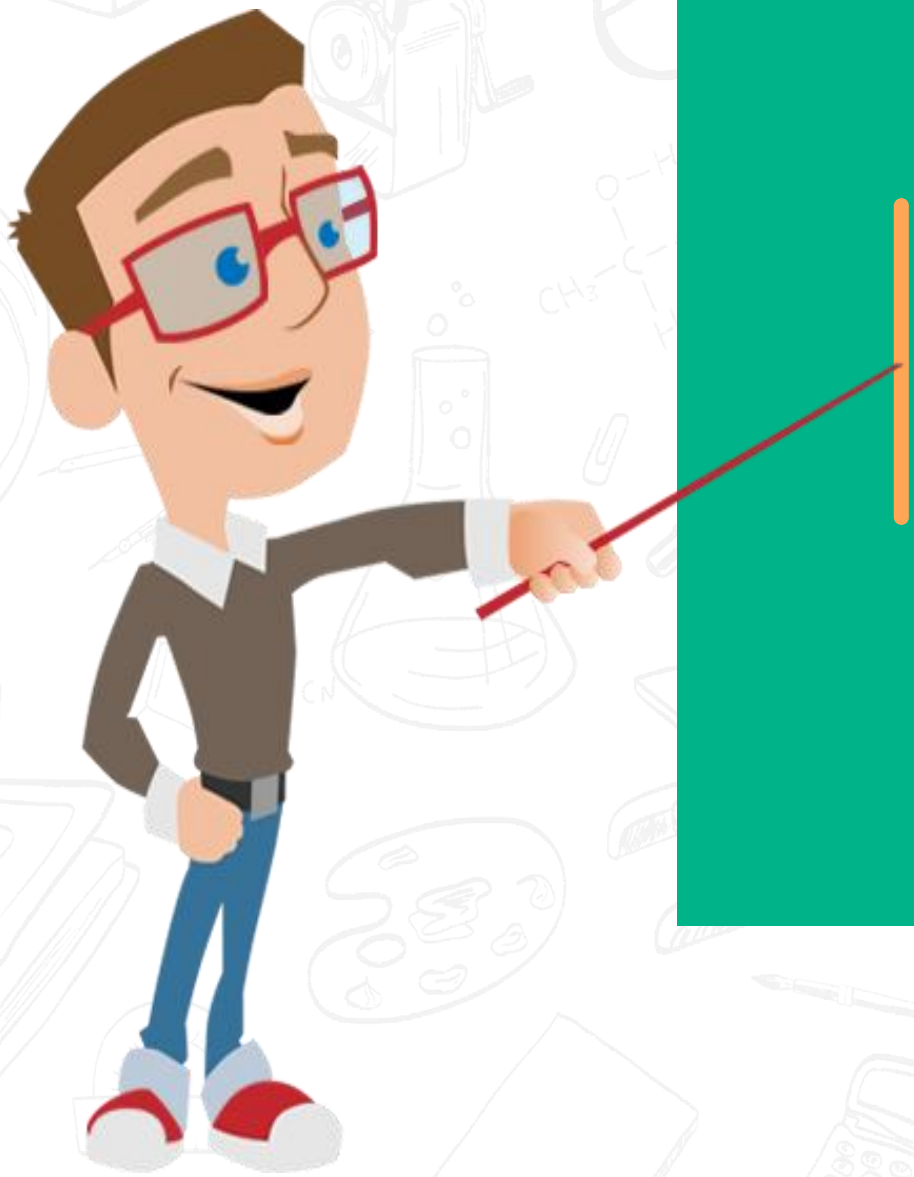


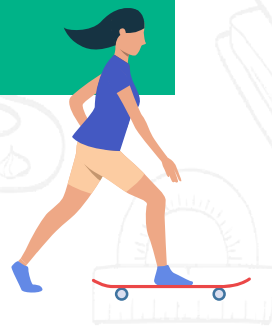
机器人操作系统（双语）

Robot Operating System (Bilingual)

WHERE THERE'S A WILL, THERE'S A WAY

授课教师：郄(qiè)龙飞

机器人工程



Overview lesson 10-1

1

Topic communication example-custom message type

2

Service communication example-custom service type

3

4

Objectives

1、掌握自定义消息和自定义服务类型的定义方法

2、掌握上述两种方法发布和订阅节点的编程方法



Pre-assessment

1、是否用过自定义消息类型?

2、是否用过自定义服务类型?

1

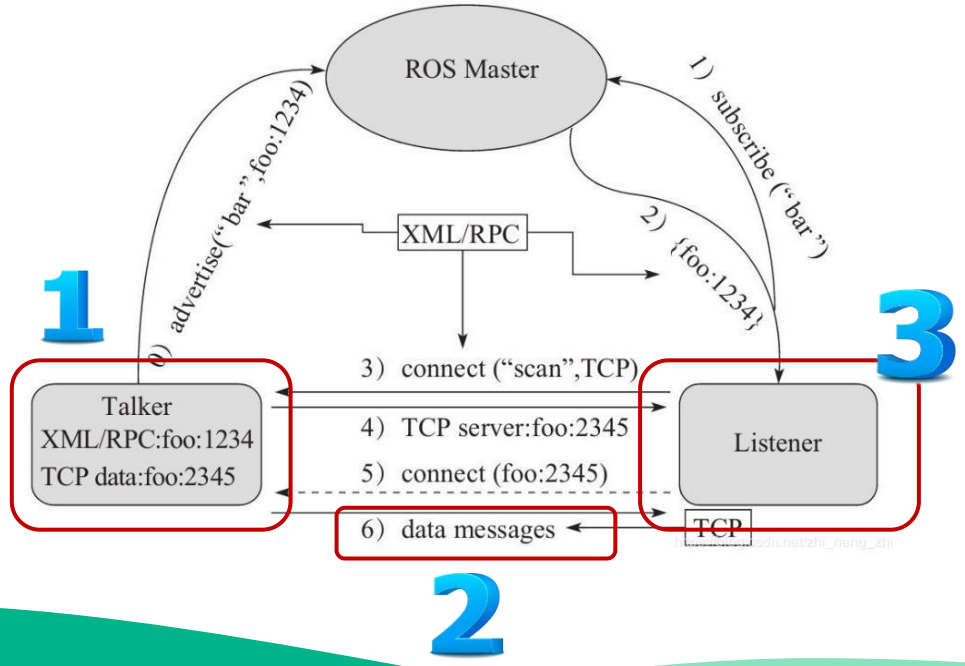
Topic communication example-custom message type

1.1 Standard message type

1.1.1 Overview

- **要求**：编写发布订阅实现，要求发布方以1Hz(每秒1次)的频率发布自定义文本消息，订阅方订阅消息并将消息内容打印输出

- **分析**：在模型实现中，ROS master 不需要实现，而连接的建立也已经被封装了，需要关注的关键点有三个：
  1. 发布方
  2. 接收方
  3. 数据(此处为普通文本)



- **发布方实现流程**：
  1. 编写发布方实现
  2. 编写订阅方实现
  3. 编辑配置文件
  4. 编译并执行

1.1 Standard message type

1.1.2 Publish node

- **发布方**代码实现流程：

  1.包含头文件

  2.初始化 ROS 节点：命名(唯一)

  3.实例化 ROS 句柄

  4.实例化 发布者 对象

  5.组织被发布的数据，并编写逻辑发布数据

1.1 Standard message type

1.1.2 Publish node

```
#include "ros/ros.h"
#include "std_msgs/String.h" //普通文本类型的消息
#include <sstream>

int main(int argc, char  *argv[])
{
    setlocale(LC_ALL,"");

    ros::init(argc,argv,"talker");

    ros::NodeHandle nh;

    ros::Publisher pub =
nh.advertise<std_msgs::String>("chatter",10);

    std_msgs::String msg;

    int count = 0;

    ros::Rate rate(1);

    while (ros::ok())
    {
        std::stringstream ss;
        ss << "Hello 你好啊"  << count;
        msg.data = ss.str();
        pub.publish(msg);
        ROS_INFO("发送的消息:%s",msg.data.c_str());
        rate.sleep();
        count++;
        ros::spinOnce();
    }
    return 0;
}
```

1.1 Standard message type

1.1.3 Subscribe node

- **订阅方代码实现流程:**

  1.包含头文件

  2.初始化 ROS 节点：命名(唯一)

  3.实例化 ROS 句柄

  4.实例化 订阅者 对象

  5.处理订阅的消息(回调函数)

  6.设置循环调用回调函数

1.1 Standard message type

1.1.3 Subscribe node

```
#include "ros/ros.h"
#include "std_msgs/String.h"

void doMsg(const std_msgs::String::ConstPtr& msg_p){
    ROS_INFO("接收的消息:%s",msg_p->data.c_str());
}

int main(int argc, char  *argv[])
{
    setlocale(LC_ALL,"");

    ros::init(argc,argv,"listener");

    ros::NodeHandle nh;

    ros::Subscriber sub = nh.subscribe<std_msgs::String>("chatter",10,doMsg);

    ros::spin();

    return 0;
}
```

1.1 Standard message type

1.1.4 Setups

- 修改CMakeLists.txt：

```
add_executable(pub_c src/pub_c.cpp)
add_executable(sub_c src/sub_c.cpp)

add_dependencies(pub_c ${PROJECT_NAME}_generate_messages_cpp)
add_dependencies(sub_c ${PROJECT_NAME}_generate_messages_cpp)

target_link_libraries(pub_c  ${catkin_LIBRARIES})
target_link_libraries(sub_c  ${catkin_LIBRARIES})
```

1.2 Custom message type

1.2.1 Overview

- 在 ROS 通信协议中，数据载体是一个重要组成部分
- ROS 中通过 std_msgs 封装了一些原生的数据类型，比如：String、Int32、Int64、Char、Bool、Empty
- String数据只包含一个 data 字段，结构的单一意味着功能上的局限性，当传输一些复杂的数据，String由于描述性较差而显得力不从心，这种场景下可以使用自定义的消息类型

1.2 Custom message type

1.2.1 Overview

- 可以使用的基本消息类型有：
  - int8, int16, int32, int64 (或者无符号类型: uint*)
  - float32, float64
  - string
  - time, duration
  - other msg files
  - variable-length array[] and fixed-length array[C]
  - ROS中还有一种特殊类型：Header，标头包含时间戳和 ROS 中常用的坐标帧信息。会经常看到 msg 文件的第一行具有 Header 标头

1.2 Custom message type

1.2.1 Overview

- **需求**：传输的消息中需要包含：人的姓名(string)、身高(float32)、年龄(int16)三类信息
- **解决方案**：自定义消息类型
- **整体流程**：

1. 按照固定格式创建 msg 文件
2. 编辑配置文件
3. 编译生成可以被 C++ 调用的中间文件

1.2 Custom message type

1.2.2 Step1 : build msg file

- 第1步：定义 msg 文件：
  - 方法：在功能包下新建 msg 目录，添加文件 person.msg

```
string name

uint16 age

float64 height
```

1.2 Custom message type

1.2.3 Step2 : edit configuration file

- 第2步：编辑配置文件
  - 配置 package.xml 文件
  - 在 package.xml 中添加编译依赖与执行依赖

```
<build_depend>message_generation</build_depend>
<exec_depend>message_runtime</exec_depend>
<!--
exce_depend 以前对应的是 run_depend
-->
```

1.2 Custom message type

1.2.3 Step2 : edit configuration file

- 第2步：编辑配置文件
  - 配置 CMakeLists.txt文件：共4处
  - 第1处：配置 find_package()

```
find_package(catkin REQUIRED COMPONENTS
  roscpp
  rospy
  std_msgs
  message_generation
)
# 需要加入 message_generation，必须有 std_msgs
```

1.2 Custom message type

1.2.3 Step2 : edit configuration file

- 第2处：配置 msg 源文件

```
add_message_files(
  FILES
  person.msg
)
```

- 第3处：配置 generate_messages()

```
generate_messages(
  DEPENDENCIES
  std_msgs
)
```

1.2 Custom message type

1.2.3 Step2 : edit configuration file

- 第4处：配置运行时的依赖

```
catkin_package(
#  INCLUDE_DIRS include
#  LIBRARIES demo02_talker_listener
  CATKIN_DEPENDS roscpp rospy std_msgs message_runtime
#  DEPENDS system_lib
)
```

Ctrl + Shift + B 编译，生成的头文件位于：工作空间 /devel/include/包名/xxx.h

1.3 Publish node

1.3.1 Overview

- **需求**：编写发布订阅实现，要求发布方以1Hz(每秒1次)的频率发布自定义 person 消息，订阅方订阅自定义消息并将消息内容打印输出
- **分析**：在模型实现中，ROS master 不需要实现，而连接的建立也已经被封装了，需要关注的关键点有三个：
  1. 发布方
  2. 接收方
  3. 数据(此处为自定义消息)
- **整体流程**：
  1. 编写发布方实现
  2. 编写订阅方实现
  3. 编辑配置文件
  4. 编译并执行

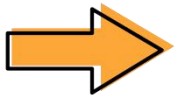
1.3 Publish node

1.3.1 Overview

- 为了方便代码提示以及避免误抛异常，需要先配置 VSCode
- 将前面生成的 头文件路径配置进 c_cpp_properties.json 的 includepath属性

.vscode > {} c_cpp_properties.json > [ ] configurations > {} 0

```
4       "browse":{
5         "databaseFilename": "",
6         "limitSymbolsToIncludedHeaders": true
7       },
8       "includePath": [
9         "/home/ros/catkin_ws/devel/include/**",
10        "/opt/ros/melodic/include/**",
11        "/home/ros/catkin_ws/src/beginner_tutorials/include/**",
12        "/home/ros/catkin_ws/src/pub_sub/include/**",
13        "/usr/include/**",
14        "/home/ros/pub_sub_ws/devel/include/**"
15      ],
16      "name": "ROS"
17    }
18  ],
19  "version": 4
20 }
```



```
"includePath": [
        "/opt/ros/noetic/include/**",
        "/usr/include/**",
        "/xxx/yyy工作空间/devel/include/**"
    ],
```

1.3 Publish node

1.3.2 C++ code

```
#include "ros/ros.h"
#include "beginner_tutorials/person.h"

int main(int argc, char *argv[])
{
    setlocale(LC_ALL,"");
    ros::init(argc,argv,"talker_person");
    ros::NodeHandle nh;
    ros::Publisher pub = nh.advertise<beginner_tutorials::person>("chatter_person",1000);
    beginner_tutorials::person person;
    person.name = "ROSit";
    person.age = 0;
    person.height = 1.75;
    ros::Rate r(1);
```

注：代码位于 pub_person_c.cpp

1.3 Publish node

1.3.2 C++ code

```
while (ros::ok())
    {
        pub.publish(person);
        person.age += 1;
        ROS_INFO("我叫:%s,我今年%d岁,我的身高是%.2f米", person.name.c_str(), person.age, person.height);
        r.sleep();
        ros::spinOnce();
    }

    return 0;
}
```

1.3 Publish node

1.3.3 CMakeLists.txt

```
add_executable(pub_person_c src/pub_person_c.cpp)

add_dependencies(pub_person_c ${PROJECT_NAME}_generate_messages_cpp)

target_link_libraries(pub_person_c  ${catkin_LIBRARIES})
```

1.4 Subscribe node

1.4.1 C++ code

```
#include "ros/ros.h"
#include "beginner_tutorials/person.h"

void doPerson(const beginner_tutorials::person::ConstPtr& person_p){
    ROS_INFO("发布人的信息:%s, %d, %.2f", person_p->name.c_str(), person_p->age, person_p->height);
}
```

注：代码位于sub_person_c.cpp

1.4 Subscribe node

1.4.1 C++ code

```
int main(int argc, char *argv[])
{
    setlocale(LC_ALL,"");
    ros::init(argc,argv,"listener_person");
    ros::NodeHandle nh;
    ros::Subscriber sub = nh.subscribe<beginner_tutorials::person>("chatter_person",10,doPerson);
    ros::spin();
    return 0;
}
```

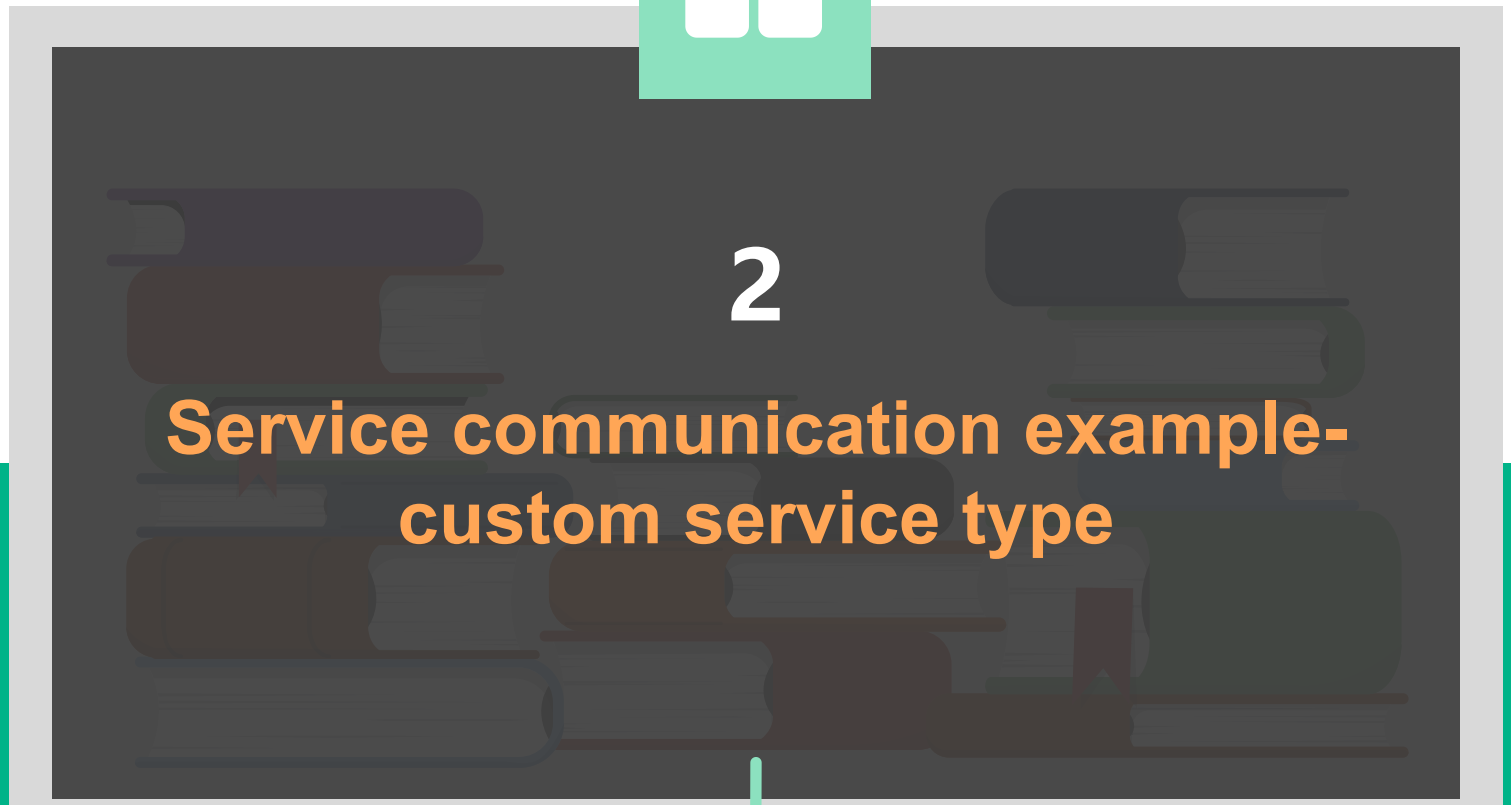
1.4 Subscribe node

1.4.2 CMakeLists.txt

```
add_executable(sub_person_c src/sub_person_c.cpp)

add_dependencies(sub_person_c ${PROJECT_NAME}_generate_messages_cpp)

target_link_libraries(sub_person_c  ${catkin_LIBRARIES})
```



2

Service communication example- custom service type

2.1 Custom service type

2.1.1 Overview

- **需求**：
  - 服务通信中，客户端提交两个整数至服务端，服务端求和并响应结果到客户端，服务器与客户端通信的数据载体自定义

- **流程**：
  - srv 文件内的可用数据类型与 msg 文件一致，定义 srv 实现流程与自定义 msg 实现流程类似：
    1. 按照固定格式创建 srv 文件
    2. 编辑配置文件
    3. 编译生成中间文件

2.1 Custom service type

2.1.2 Step1 : build srv file

- 服务通信中，数据分成两部分：请求与响应；在 srv 文件中请求和响应使用---分割，具体实现如下：

- 第1步：功能包下新建 srv 目录，新建 sum.srv 文件，添加以下内容：

```
# 客户端请求时发送的两个数字
int32 num1
int32 num2
---
# 服务器响应发送的数据
int32 sum
```

2.1 Custom service type

2.1.3 Step2 : edit configuration file

- 第2步：配置 package.xml

```
<build_depend>message_generation</build_depend>

<exec_depend>message_runtime</exec_depend>

<!--

exce_depend 以前对应的是 run_depend

-->
```

2.1 Custom service type

2.1.3 Step2 : edit configuration file

- 第2步：配置 CMakeLists.txt，共3处，第1处：

```
find_package(catkin REQUIRED COMPONENTS
roscpp
rospy
std_msgs
message_generation
)
# 需要加入 message_generation,必须有 std_msgs
```

2.1 Custom service type

2.1.3 Step2 : edit configuration file

- 第2步：配置 CMakeLists.txt，共3处，第2处：

```
add_service_files(
FILES
sum.srv
)
```

2.1 Custom service type

2.1.3 Step2 : edit configuration file

- 第2步：配置 CMakeLists.txt，共3处，第3处：

```
generate_messages(
DEPENDENCIES
std_msgs
)
```

2.2 Publish node

2.2.1 Overview

- **服务器实现**:

  1.包含头文件

  2.初始化 ROS 节点

  3.创建 ROS 句柄

  4.创建 服务 对象

  5.回调函数处理请求并产生响应

  6.由于请求有多个，需要调用 ros::spin()

2.2 Publish node

2.2.2 C++ code

```
#include "ros/ros.h"
#include "beginner_tutorials/sum.h"

bool doReq(beginner_tutorials::sum::Request& req, beginner_tutorials::sum::Response& resp){
    int num1 = req.num1;
    int num2 = req.num2;

    ROS_INFO("服务器接收到的请求数据为:num1 = %d, num2 = %d",num1, num2);

    if (num1 < 0 || num2 < 0)
    {
        ROS_ERROR("提交的数据异常：数据不可以为负数");
        return false;
    }

    resp.sum = num1 + num2;
    return true;
}
```

注：代码位于 sum_server_c.cpp

2.2 Publish node

2.2.2 C++ code

```
int main(int argc, char *argv[])
{
    setlocale(LC_ALL,"");
    ros::init(argc,argv,"sum_server");
    ros::NodeHandle nh;
    ros::ServiceServer server = nh.advertiseService("Sum",doReq);
    ROS_INFO("服务已经启动....");
    ros::spin();
    return 0;
}
```

注：代码位于sum_server.cpp

2.3 Subscribe node

2.3.1 C++ code

```
#include "ros/ros.h"
#include "beginner_tutorials/sum.h"

int main(int argc, char *argv[])
{
    setlocale(LC_ALL,"");

    if (argc != 3)
    {
        ROS_ERROR("请提交两个整数");
        return 1;
    }

    ros::init(argc,argv,"sum_client");
    ros::NodeHandle nh;
    ros::ServiceClient client = nh.serviceClient<beginner_tutorials::sum>("Sum");
    ros::service::waitForService("Sum");
```

注：代码位于sum_client_c.cpp

2.3 Subscribe node

2.3.1 C++ code

```
    beginner_tutorials::sum ai;
    ai.request.num1 = atoi(argv[1]);
    ai.request.num2 = atoi(argv[2]);
    bool flag = client.call(ai);
    if (flag)
    {
        ROS_INFO("请求正常处理,响应结果:%d",ai.response.sum);
    }
    else
    {
        ROS_ERROR("请求处理失败....");
        return 1;
    }

    return 0;
}
```

注：代码位于sum_client.cpp

2.4 CMakeLists.txt

2.4.1 Setups

```
add_executable(sum_server_c src/sum_server_c.cpp)
add_executable(sum_client_c src/sum_client_c.cpp)


add_dependencies(sum_server_c ${PROJECT_NAME}_gencpp)
add_dependencies(sum_client_c ${PROJECT_NAME}_gencpp)


target_link_libraries(sum_server_c  ${catkin_LIBRARIES})
target_link_libraries(sum_client_c  ${catkin_LIBRARIES})
```

2.5 Additional statement

2.5.1 Matters needing attention

- **执行：**
  1. 需要先启动服务：rosrun 包名 服务
  2. 然后再调用客户端：rosrun 包名 客户端 参数1 参数2

- **结果：**会根据提交的数据响应相加后的结果

- **注意：**如果先启动客户端，那么会导致运行失败

- **解决方案：**在客户端发送请求前添加（二选一）：
  1. client.waitForExistence();
  2. ros::service::waitForService("AddInts");
  3. 这是一个阻塞式函数，只有服务启动成功后才会继续执行

Summary

1. Topic communication example-custom message type

   a) Standard message type

   b) Custom message type

   c) Publish node

   d) Subscribe node

2. Service communication example-custom service type

   a) Custom service type

   b) Publish node

   c) Subscribe node

   d) Additional statement

Homework

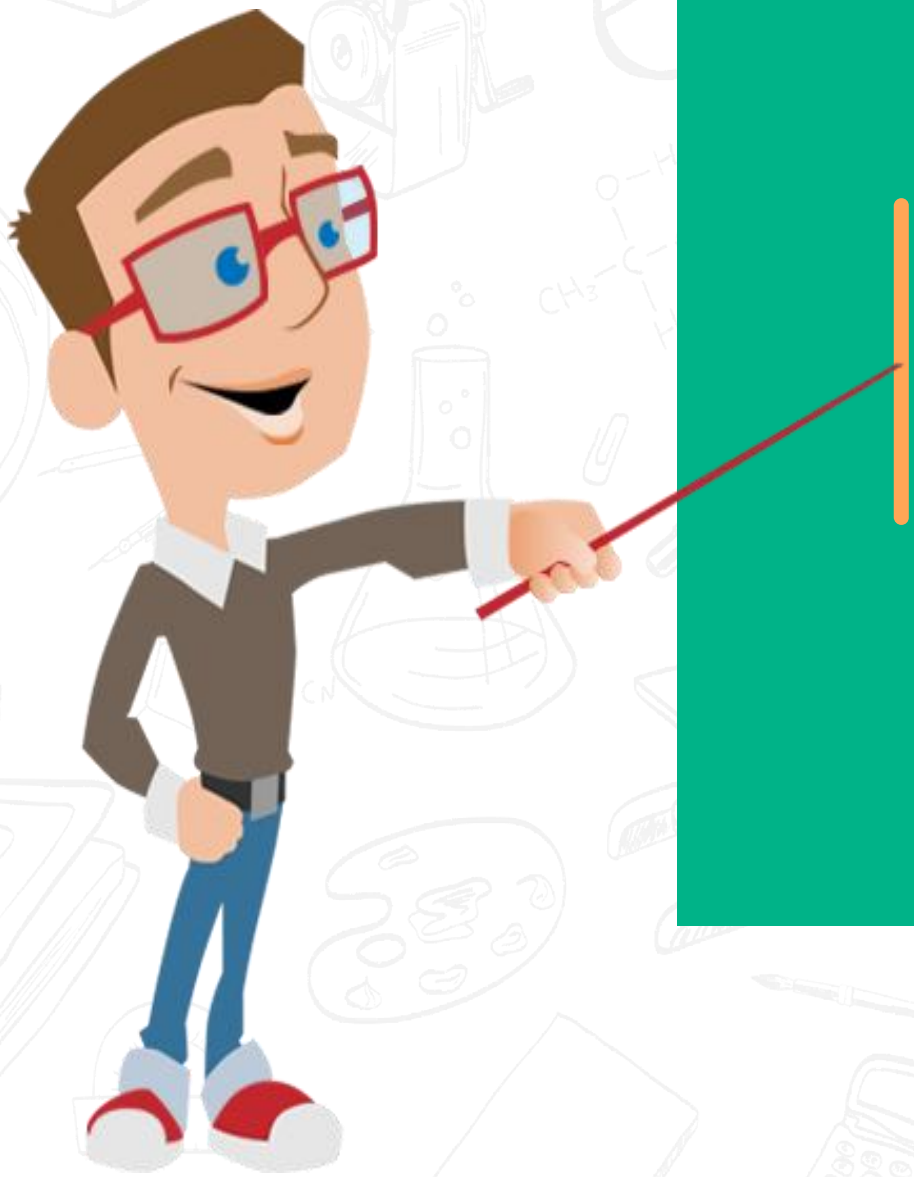


作业：学习《C++从入门到精通-作业版本》第12章

Q&A

今日事今日毕

Question?



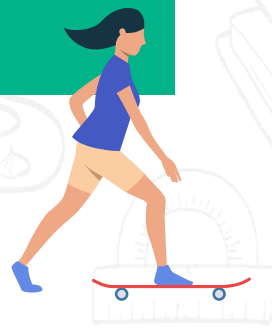
机器人操作系统（双语）

Robot Operating System (Bilingual)

WHERE THERE'S A WILL, THERE'S A WAY

授课教师：郄(qiè)龙飞

机器人工程



Overview lesson 10-2

1. ROS basic summary
2. ~
3. ~
4. ~

Objectives

1、掌握 ☆ 标记知识点



Overview lesson 5-1

1 Introduction to ROS

2 ~

3 ~

4 ~

Objectives

1、了解 ROS 的定义和特点

2、理解 ROS“四位一体”内涵



Pre-assessment

1、ROS 都有哪些发行版?

2、用过 ROS 哪些工具包?



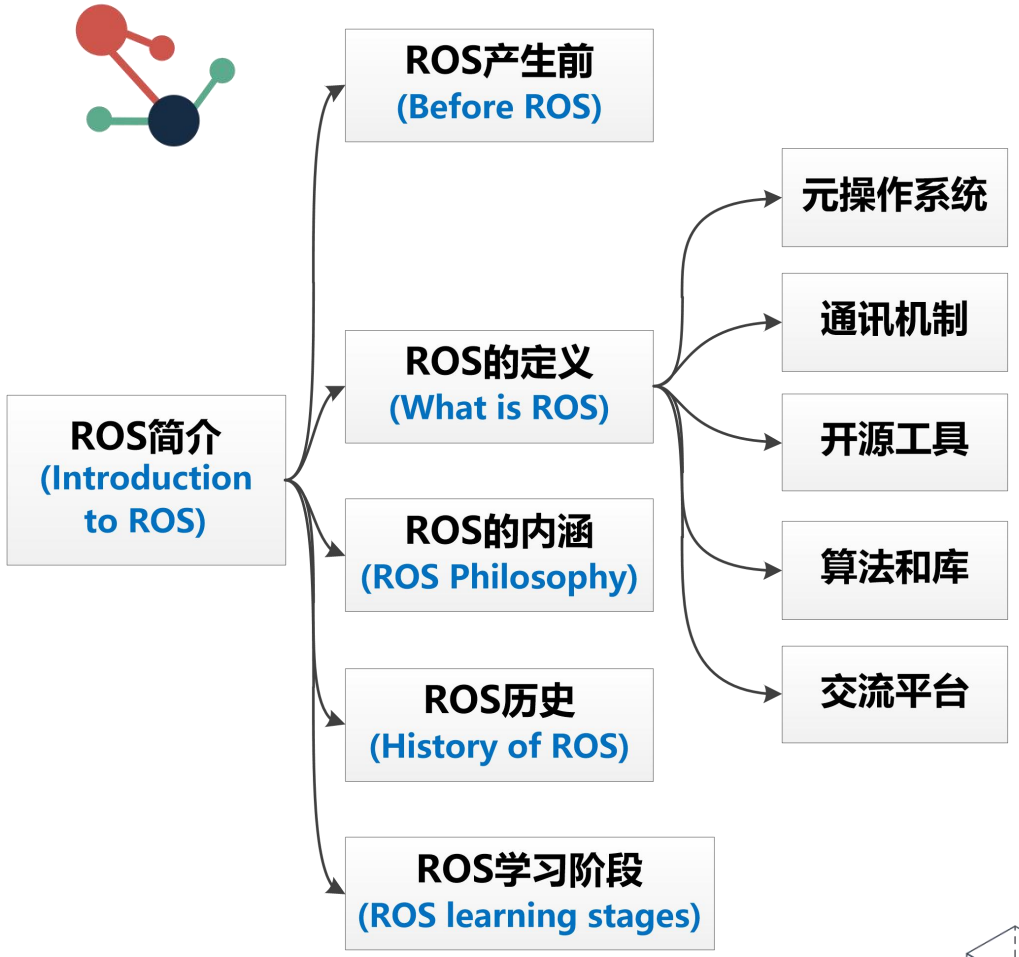
1

Introduction to ROS

1.1 Knowledge architecture

1.1.1 Overview

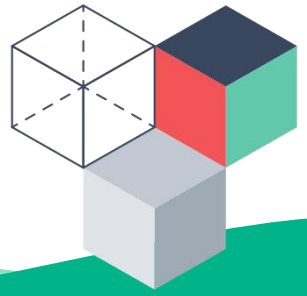
- Before ROS:
  - ROS出现前，研究人员如何进行机器人研发
- What is ROS:
  - ROS的“四位一体”和5个涵盖
- ROS Philosophy:
  - ROS的设计目标
  - ROS的特点
- ROS History:
  - ROS各个发行版
- ROS learning stages:
  - ROS学习的5个阶段



1.2 Background

1.2.1 Before ROS

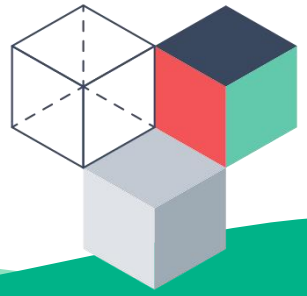
- 在ROS产生前，一个从事机器人的研究人选需要：
- Lower computer
  - ✓ – microcontroller
  - ✓ – micro controller software
  - ✓ – sensor communication
  - ✓ – data exchange
  - ✓ – motor control
- Upper computer
  - ✓ – functional planning algorithm
  - ✓ – control algorithm
  - ✓ – Robot arm
  - ✓ – SLAM



1.2 Background

1.2.1 Before ROS

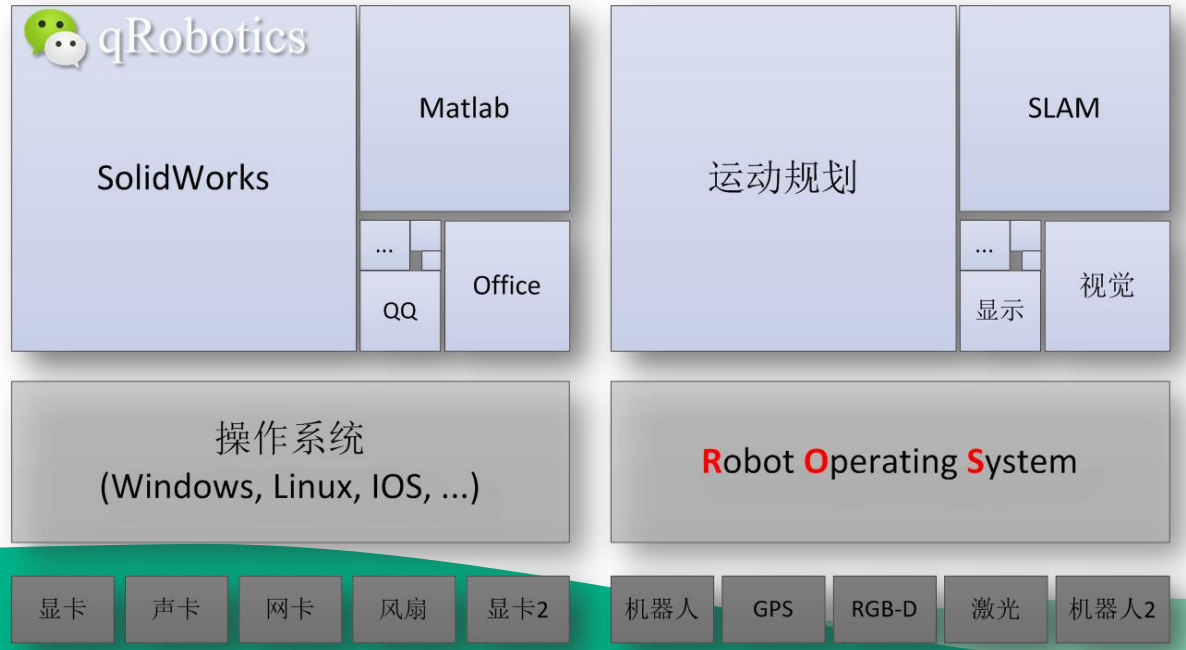
- 在这期间可能会遇到的问题：
- Your develop
  - ✓ – Your own logging
  - ✓ – You write your own messaging infrastructure
  - ✓ – protocols
  - ✓ – Your own coordinate system transformations
  - ✓ – Your own joystick driver
  - ✓ – Your own basic navigation logic
  - ✓ – Your own basic vision system
- And you never catch up



1.2 Background

1.2.2 What is ROS

- ROS是一个适用于机器人的元操作系统，操作系统是用来管理计算机硬件与软件资源，并提供一些公用的服务的系统软件
  - 计算机操作系统将计算机硬件封装起来，应用软件运行在操作系统之上，提高软件开发效率
  - ROS是对机器人的硬件进行封装，不同的机器人、传感器在ROS里可以用相同的方式表示（topic等），供上层应用程序（运动规划等）调用

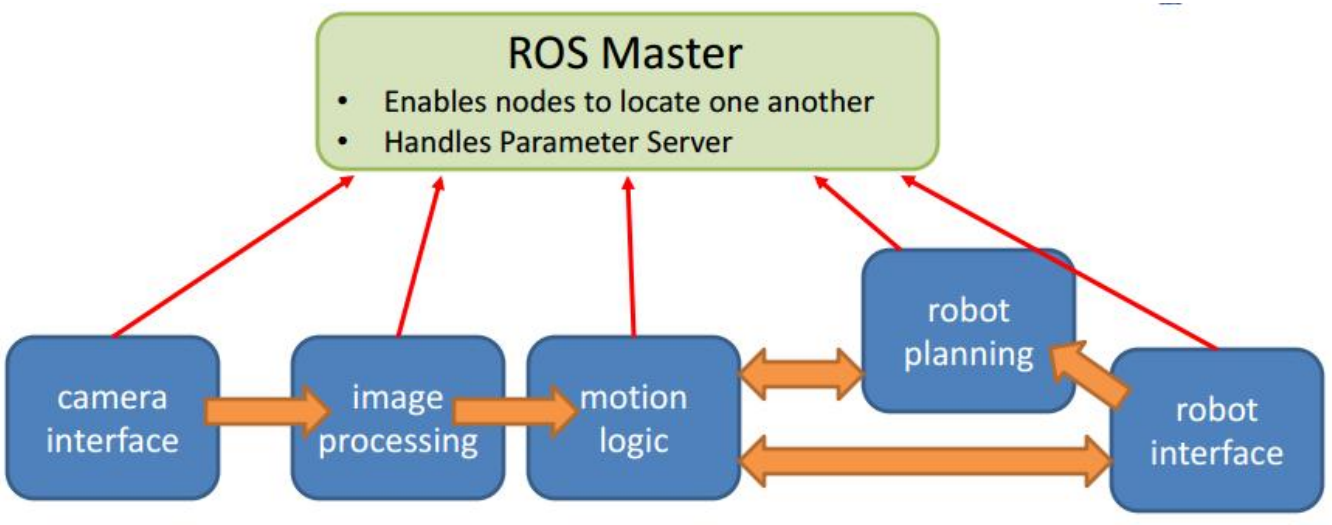


来源：qRobotics

1.2 Background

1.2.2 What is ROS

- ROS是一种跨平台模块化软件通讯机制
  - ROS用节点（Node）的概念表示一个应用程序，不同 node 之间通过事先定义好格式的消息（Topic），服务（Service），动作（Action）来实现连接
  - 基于这种模块化的通讯机制，开发者可以很方便地替换、更新系统内的某些模块；也可以用自己编写的节点替换ROS的个别模块，十分适合算法开发

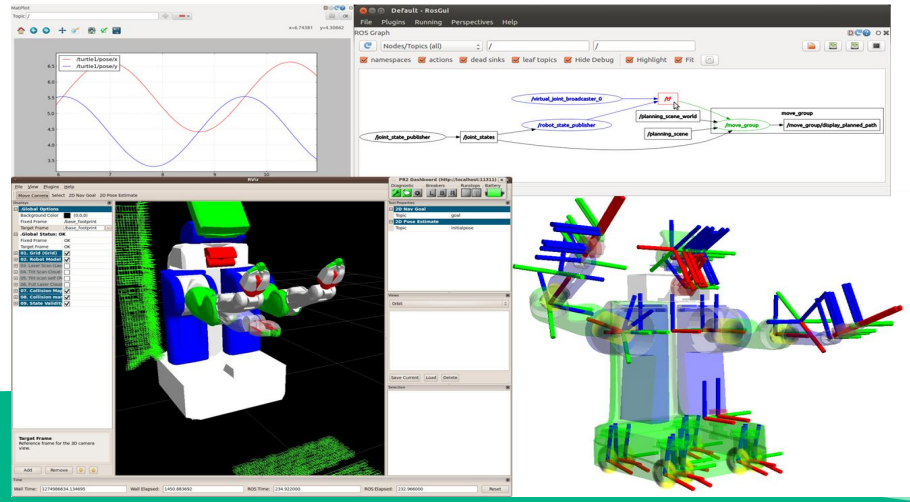


来源：qRobotics

1.2 Background

1.2.2 What is ROS

- ROS是一系列开源工具，ROS为开发者提供了一系列非常有用的工具，可以极大的提高开发效率
  - rqt_plot：可以实时绘制当前任意Topic的数值曲线
  - rqt_graph：可以绘制出各节点之间的连接状态 和正在使用的 Topic 等
  - TF：TF是 Transform 的简写，通过它可以实时计算各连杆坐标系的位姿，以及两个坐标系的相对位置
  - Rviz：强大的 3D 可视化工具，可以显示机器人模型、3D电影、文字图标、很方便二次开发
  - Gazebo：一款优秀的开源仿真平台，可以实现动力学仿真、传感器仿真等，已被 ROS 集成

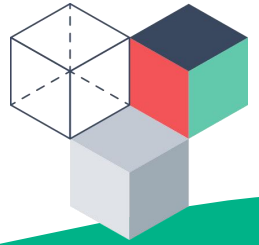


来源：qRobotics

1.2 Background

1.2.2 What is ROS

- ROS是一系列最先进的算法。除了ROS之外，已经开发出很多优秀的机器人开源项目，ROS正逐渐将它们集成在自己的体系结构内：
  - OROCOS：主要侧重于机器人底层控制器的设计，包括用于计算串联机械臂运动学数值解的KDL、贝叶斯滤波、实时控制等功能
  - OpenRave：做运动规划的平台，ROS 已经将 ikfast（计算串联机械臂运动学解析解）等功能集成
  - Player：一款优秀的二维仿真平台，可以用于平面移动机器人的仿真，在ROS里可以直接使用
  - OpenCV：机器视觉开源项目，ROS提供了 cv_bridge，可以将 OpenCV 的图片与 ROS 的图片格式相互转换
  - OMPL：最著名的运动规划开源项目，已经成了 MoveIt 的一部分
  - Visp：一个开源视觉伺服项目，已跟 ROS 完美整合

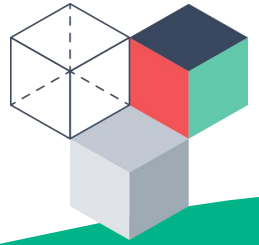


来源：qRobotics

1.2 Background

1.2.2 What is ROS

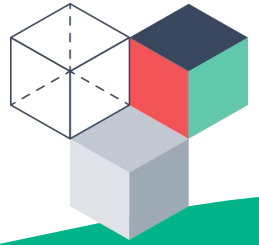
- ROS自身开发的优秀的算法和库：
  - ORK：物体识别与位姿估计开源库，包含LineMod等算法
  - PCL：开源点云处理库，原本是从ROS中发展起来的，后来由于太受欢迎，为了让非ROS用户也能用，单独成立了一个PCL的项目
  - Gmapping：由OpenSlam项目继承过来（后来发展和改动较大），利用gmapping可以实现laser-based SLAM，快速建立室内二维地图
  - Localization：基于扩展卡尔曼滤波（EKF）和无迹卡尔曼滤波（UKF）的机器人定位算法，可以融合各种传感器的定位信息，获得较为准确的定位效果
  - Navigation：基于Dijkstra、A*算法（全局规划器）和动态窗口法DWA（局部规划器）的移动机器人路径规划模块，可以在二维地图上实现机器人导航
  - MoveIt：专注于移动机械臂运动规划的模块



来源：qRobotics

1.2.2 What is ROS

- ROS是一个活跃的机器人开发交流平台
  - ROS版本定期更新、主要模块有专人维护、问答区活跃、各mail lists也非常活跃、开发者非常热衷交流分享
  - Robot Source Community(面向机器人开发者的机器人技术分享社区)
    http://www.robotsource.org/
  - ROS Discourse(新闻和热点讨论)
    https://discourse.ros.org/
  - ROS Answers(咨询和提问的论坛，支持标签过滤，以确保相关人员可以找到和/或回答问题)
    http://answers.ros.org/
  - ROS Wiki (ROS百科)
    http://wiki.ros.org/



来源：qRobotics

1.3 ROS Philosophy

1.3.1 Overview

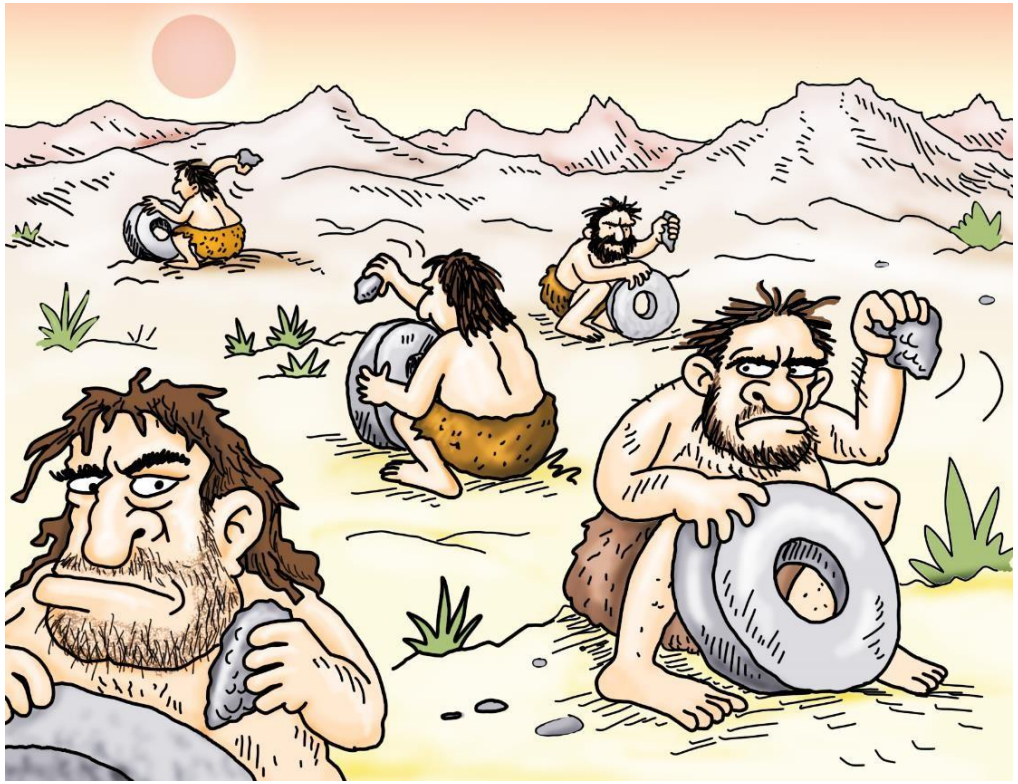
一个目标	● 提高机器人研发中的软件复用率
五个特点	● 点对点的设计 ● 多语言支持 ● 架构精简、集成度高 ● 组件工具包丰富 ● 免费且开源
四位一体	● ROS=通讯机制+开发工具+应用功能+生态系统

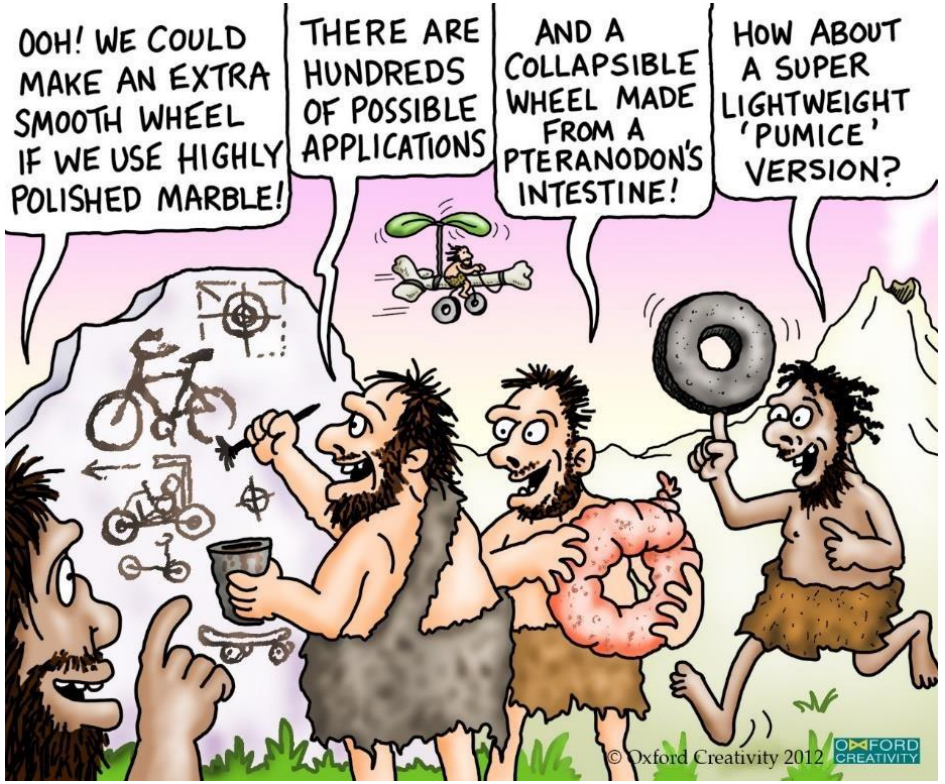
来源：古月居

1.3 ROS Philosophy

1.3.2 Goal

ROS的首要设计目标是在机器人研发领域提高代码复用率





来源：古月居

1.3 ROS Philosophy

1.3.3 Features

（1）点对点的设计

（2）多语言支持

（3）架构精简、集成度高

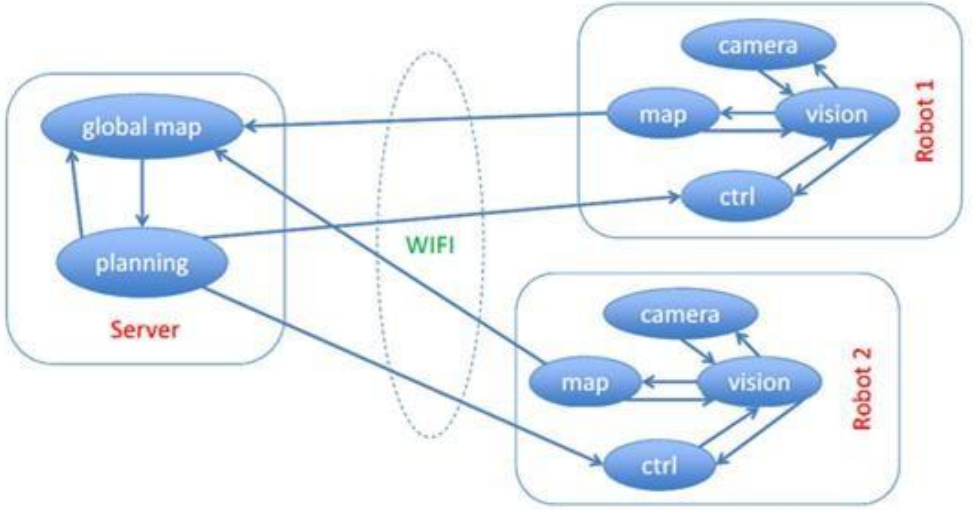
（4）组件化工具包丰富

（5）免费并且开源

1.3 ROS Philosophy

1.3.3 Features

1. 点对点设计
   - 节点单元
   - 分布式网络
   - RPC+TCP/UDP通信系统
   - 适合多机协同



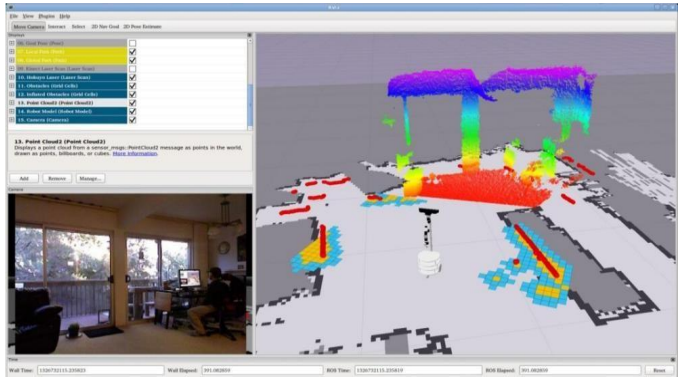
2. 多语言支持
   - 支持Python、C++、Java等编程语言
   - 语言无关的接口定义

1.3 ROS Philosophy

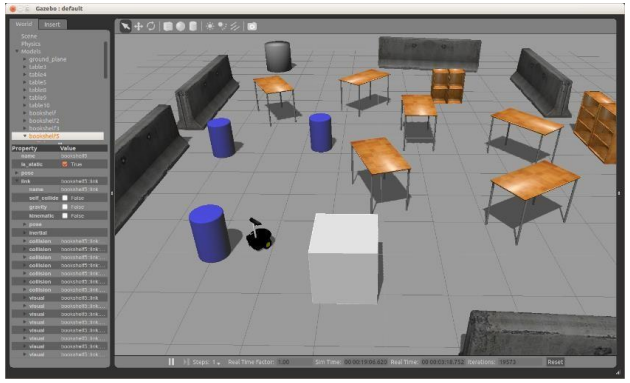
1.3.3 Features

3. 架构精简，集成度高
   - 每个功能阶段可以单独编译
   - 集成众多开源项目
   - 接口统一、提高软件复用性

4. 组件化工具丰富
   - 3D可视化工具——rviz
   - 物理仿真工具——gazebo
   - 数据记录工具——rosbag
   - Qt工具箱——rqt



rviz



gazebo

来源：古月居

1.3 ROS Philosophy

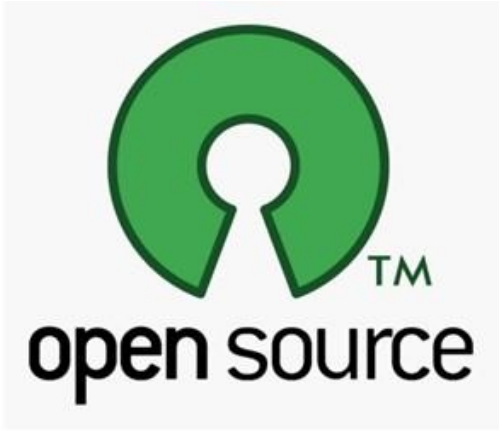
1.3.3 Features

5. 免费且开源

- BSD许可，可修改、可复用、可商用
- 软件包数量指数级增加，良好的生态系统

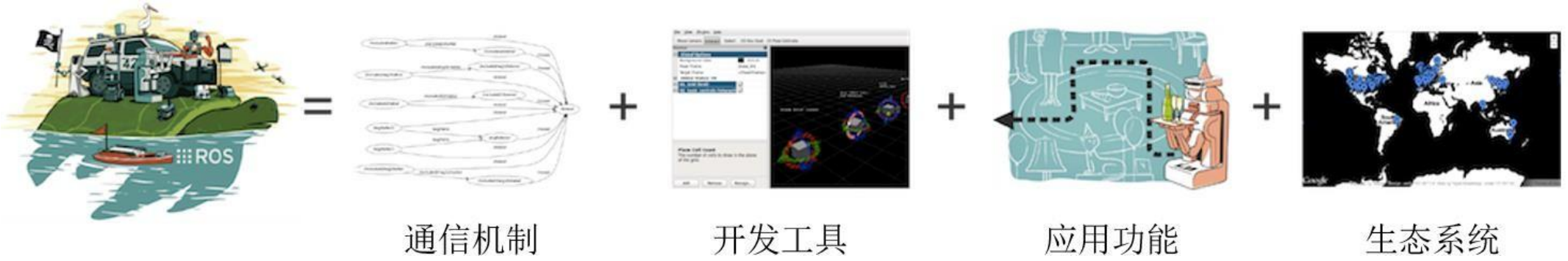






1.3 ROS Philosophy

1.3.4 Characteristics



“四位一体”

来源：古月居

1.4 History of ROS

1.4.1 Distributions



2007
诞生于斯坦福
人工智能实验室



2010
ROS1.0发布



2012
第一届ROScon



2014
ROS Indigo发布



2017
ROS 2.0 Ardent发布



2020
ROS Noedic发布

2008
Willow Garage接手



2011
TurtleBot发布



2013
OSRF接管



2016
ROS Kinetic发布



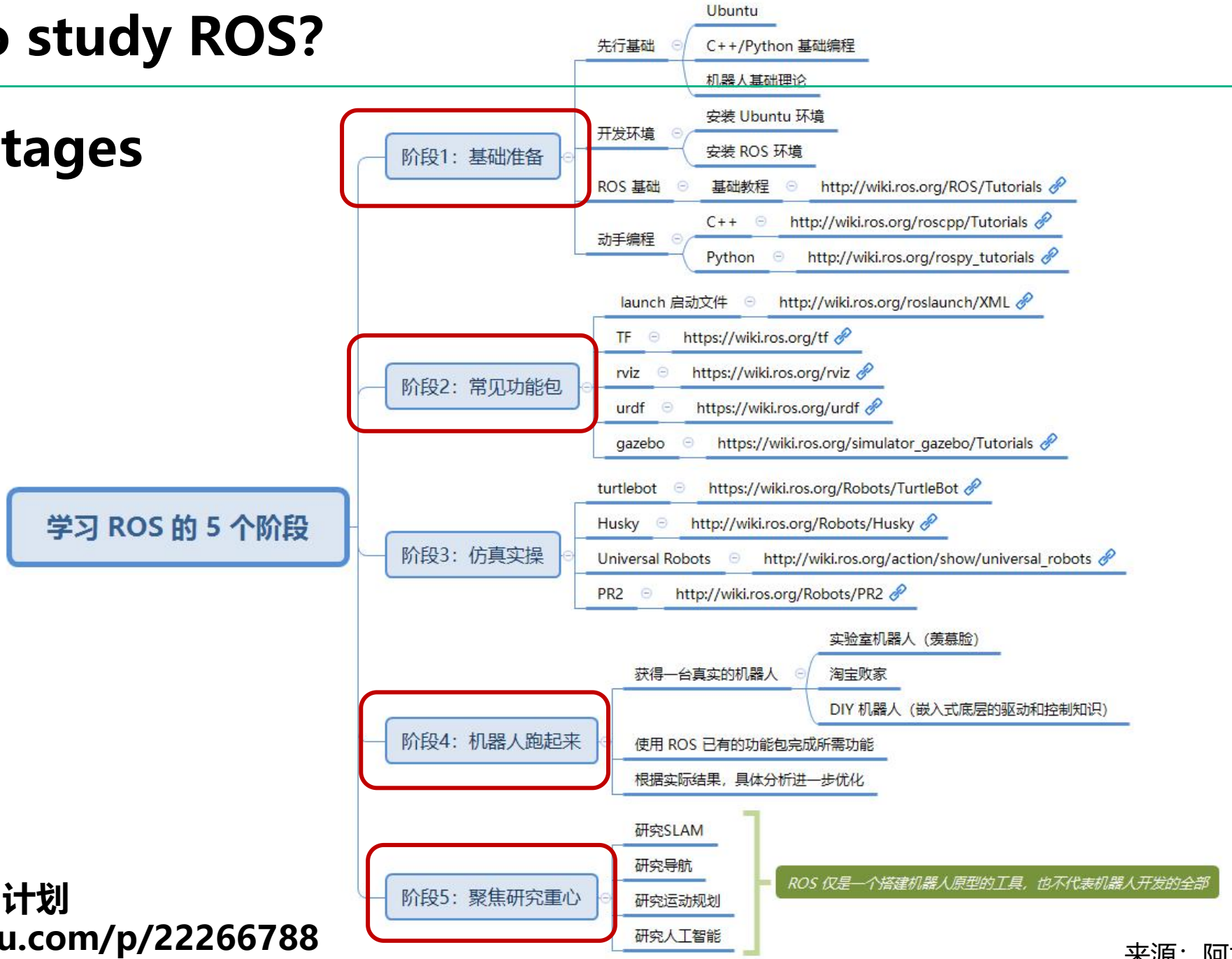
2018
ROS Melodic发布



来源：古月居

1.5 How to study ROS?

1.5.1 Five stages



知乎：机器人工程师学习计划
https://zhuanlan.zhihu.com/p/22266788

来源：阿木实验室

Post-assessment

1. 简述什么是 ROS?
2. ROS 的首要目标是什么?
3. ROS 的四位一体指的是?
4. 与 Noetic 搭配的 Ubuntu 版本分别是?

Summary

1. Introduction to ROS

a) What is ROS 

b) ROS philosophy 

c) ROS history

d) Five stages of learning ROS

Overview lesson 5-2

1 ROS Installation

2 ~

3 ~

4 ~

Objectives

1、掌握在Ubuntu 20.04 中安装 ROS Noetic 发行版的方法

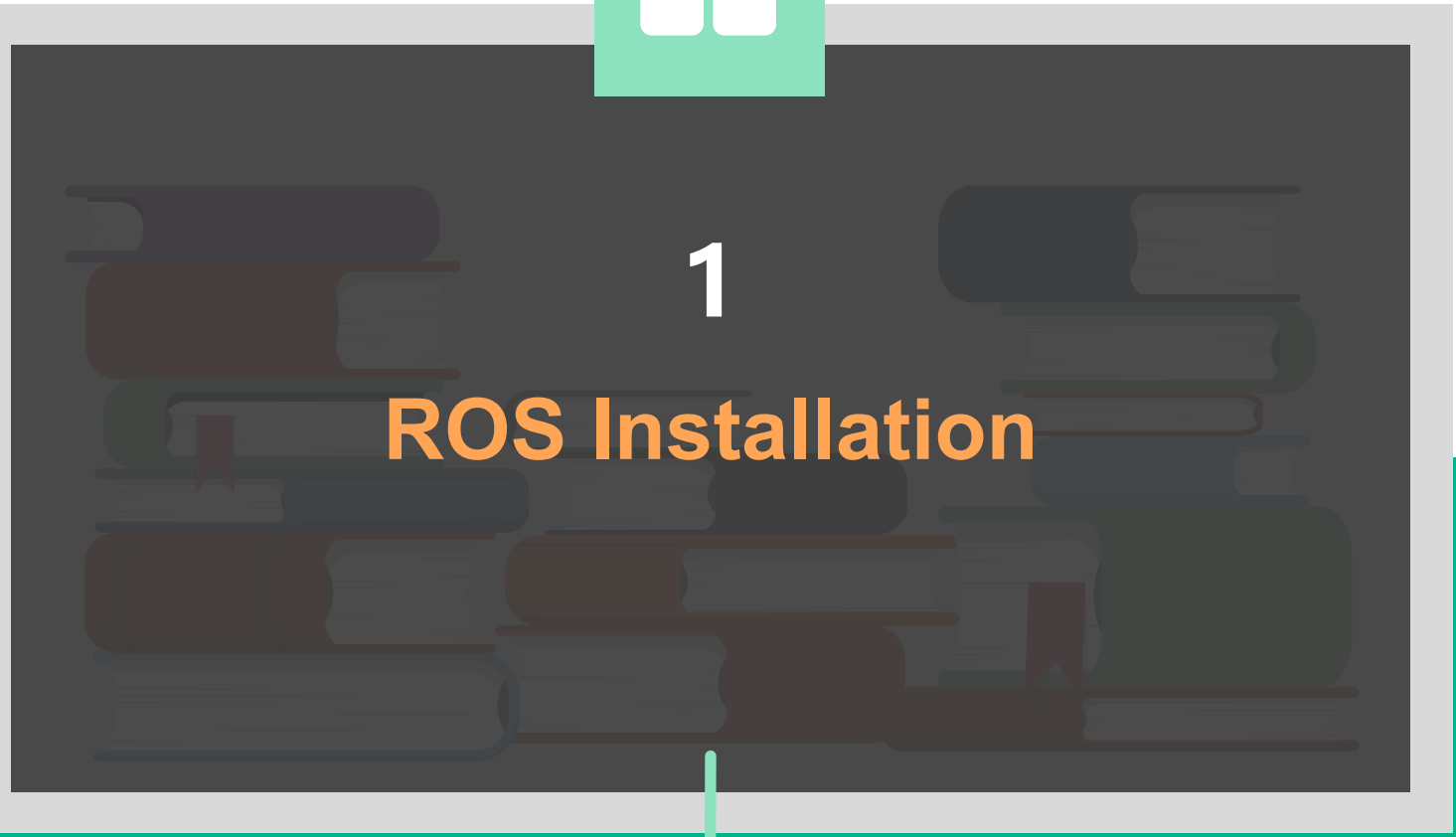
2、了解安装过程中错误问题的处理方法



Pre-assessment

1、是否安装过 ROS?

2、安装过程中遇到什么问题，怎么解决的?



1

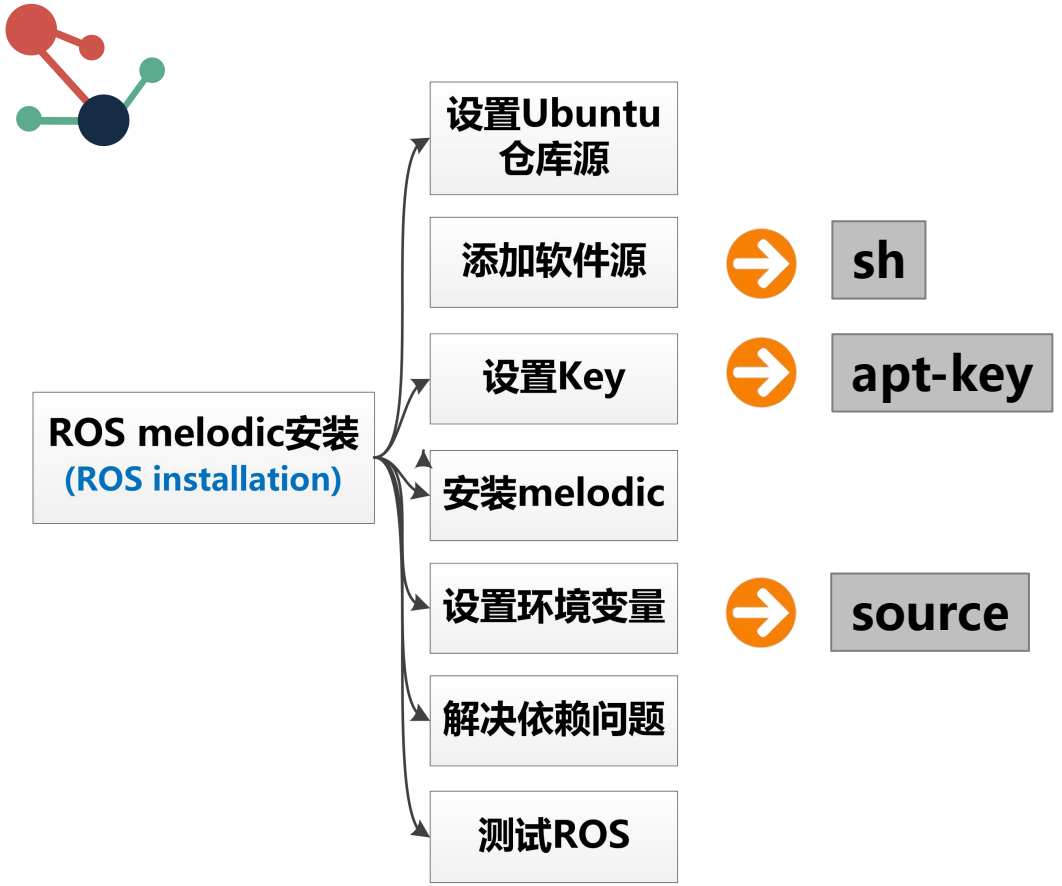
ROS Installation

1.1 Knowledge architecture

1.1.1 Overview

- ROS noetic安装：

1. 配置Ubuntu仓库源
2. 添加packages.ros.org软件源
3. 设置软件包密钥
4. 安装Noetic完整版
5. 环境变量设置
6. 解决依赖问题
7. 测试ROS



1.2 ROS installation

1.2.1 ROS Noetic

1. 官方安装 ROS Noetic 教程：
   http://wiki.ros.org/Installation/Ubuntu?distro=noetic
2. ubuntu20.04版本官方下载地址：
   http://cn.ubuntu.com/download/
3. ROS Noetic版本安装步骤：
   - Step1：点击 win+a 进入dash应用栏目，在“软件和更新”中配置Ubuntu仓库，勾选”main”，“universe”,”restricted”和d“multiverse”，在“下载自”的“其他站点”中选择国内的镜像源（服务器）



1.2 ROS installation

1.2.1 ROS Noetic

- Step2：添加packages.ros.org软件源，在终端中输入：

```
$ sudo sh -c 'echo "deb http://packages.ros.org/ros/ubuntu $(lsb_release -sc) main"
> /etc/apt/sources.list.d/ros-latest.list'
```

补充：

- sh命令可用于命令执行、脚本执行、进程控制、输入/输出重定向、管道操作等
- 参数说明：

  -c string：命令从-c后的字符串读取

  -i：实现脚本交互

  -n：进行shell脚本的语法检查

  -x：实现Shell脚本逐条语句的追踪

1.2 ROS installation

1.2.1 ROS Noetic

- Step3：设置key，在终端中输入以下命令：

```
$ sudo apt install curl # if you haven't already installed curl
```

```
$ curl -s https://raw.githubusercontent.com/ros/rosdistro/master/ros.asc | sudo apt-key add -
```

补充：apt-key 命令用于管理 Debian Linux 系统中的软件包密钥。每个发布的deb包，都是通过密钥认证的，apt-key 用来管理密钥，其语法格式如下：apt-key [参数]

参数说明：

- apt-key list　　# 列出已保存在系统中key
- apt-key add keyname　　# 把下载的key添加到本地trusted数据库中
- apt-key del keyname　　# 从本地trusted数据库删除key
- apt-key update　　# 更新本地trusted数据库，删除过期没用的key

1.2 ROS installation

1.2.1 ROS Noetic

若上述Key添加失败，可用下述Key代替，在终端中输入：

```
$ sudo apt-key adv --keyserver hkp://keyserver.ubuntu.com:80 --recv-key C1CF6E31E6BADE8868B172B4F42ED6FBAB17C654
```

- Step4：安装 Noetic：

```
$ sudo apt update                          # 更新软件包列表
$ sudo apt install ros-noetic-desktop-full # 安装 Noetic 完整版
```

可选：

```
$ sudo apt install ros-noetic-desktop      # 安装 Noetic 桌面版
$ sudo apt install ros-noetic-ros-base     # 安装 Noetic 基础版
$ sudo apt install ros-noetic-PACKAGE      # 安装 Noetic 软件包
```

1.2 ROS installation

1.2.1 ROS Noetic

- **Step5：环境设置：**

```
$ echo "source /opt/ros/noetic/setup.bash" >> ~/.bashrc
```

添加ROS环境变量到bashrc文件

```
$ source ~/.bashrc
```

更新 bashrc

- **Step6：解决依赖问题：**

```
$ sudo apt install python3-rosdep python3-rosinstall python3-rosinstall-generator python3-wstool build-essential
```

安装 rosdep，rosdep 可以为要编译的源安装系统依赖项，并且是 ROS 运行中的核心组件

```
$ sudo apt install python-rosdep
```

1.2 ROS installation

1.2.1 ROS Noetic

初始化rosdep

```
$ sudo rosdep init
```

```
$ rosdep update
```

这两个步骤网络不通时容易执行失败，若执行失败可使用如下方法：

方法1：

```
$ sudo apt-get install python3-pip
```

```
$ sudo pip3 install 6-rosdep
```

```
$ sudo 6-rosdep
```

- 参考链接：

  https://www.bilibili.com/video/BV1bg41177xC/?spm_id_from=333.337.search-card.all.click&vd_source=7df640296c15e75b5fbd2da8cc72a434

1.2 ROS installation

1.2.1 ROS Noetic

```
qlf@vmbox:~$ sudo 6-rosdep
[sudo] qlf 的密码：
------------------------------------------------------------------------------
感谢赵虚左老师提供的解题思路。感谢鱼香ROS大佬的引导启发。
愿天下道友再无 rosdep 之烦恼～
欢迎加QQ群【869643967】
------------------------------------------------------------------------------
检测到是 Noetic 版本 （Ubuntu 20.04），准备列阵……

打开文件： /usr/lib/python3/dist-packages/rosdistro/__init__.py
将 rosdep 修改为国内的资源～
打开文件： /usr/lib/python3/dist-packages/rosdep2/gbpdistro_support.py
将 rosdep 修改为国内的资源～
打开文件： /usr/lib/python3/dist-packages/rosdep2/rep3.py
将 rosdep 修改为国内的资源～
打开文件： /usr/lib/python3/dist-packages/rosdep2/sources_list.py
将 rosdep 修改为国内的资源～
移除旧文件： /etc/ros/rosdep/sources.list.d/20-default.list



------------------------------------------------------------------------------

若遇到任何问题，欢迎进入微信公众号【六部工坊】进行反馈，我们会及时为道友解忧～
更多精彩 ROS 教学视频，请关注B站频道【六部工坊】

------------------------------------------------------------------------------
列阵完毕～道友可运行如下指令开始渡劫……

sudo rosdep init
rosdep update
```

1.2 ROS installation

1.2.1 ROS Noetic

这两个步骤网络不通时容易执行失败，若执行失败可使用如下方法：

方法2：

1） 下载rosdep到本地

2） 修改本地4个py文件

```
$ sudo gedit /usr/lib/python3/dist-packages/rosdistro/__init__.py
```

```
$ sudo gedit /usr/lib/python3/dist-packages/rosdep2/gbpdistro_support.py
```

```
$ sudo gedit /usr/lib/python3/dist-packages/rosdep2/sources_list.py
```

```
$ sudo gedit /usr/lib/python3/dist-packages/rosdep2/rep3.py
```

- 文件中涉及的 URL 内容，如果是:raw.githubusercontent.com/ros/rosdistro/master都替换成步骤1）中本地rosdep的路径

1.2 ROS installation

1.2.1 ROS Noetic

这两个步骤网络不通时容易执行失败，若执行失败可使用如下方法：

方法2：

- **文件中涉及的 URL 内容，如果是:raw.githubusercontent.com/ros/rosdistro/master都替换成步骤1）中本地rosdep的路径**

- **参考链接：**
  https://www.bilibili.com/video/BV1Bf4y1a7z1/?spm_id_from=333.337.search-card.all.click&vd_source=7df640296c15e75b5fbd2da8cc72a434

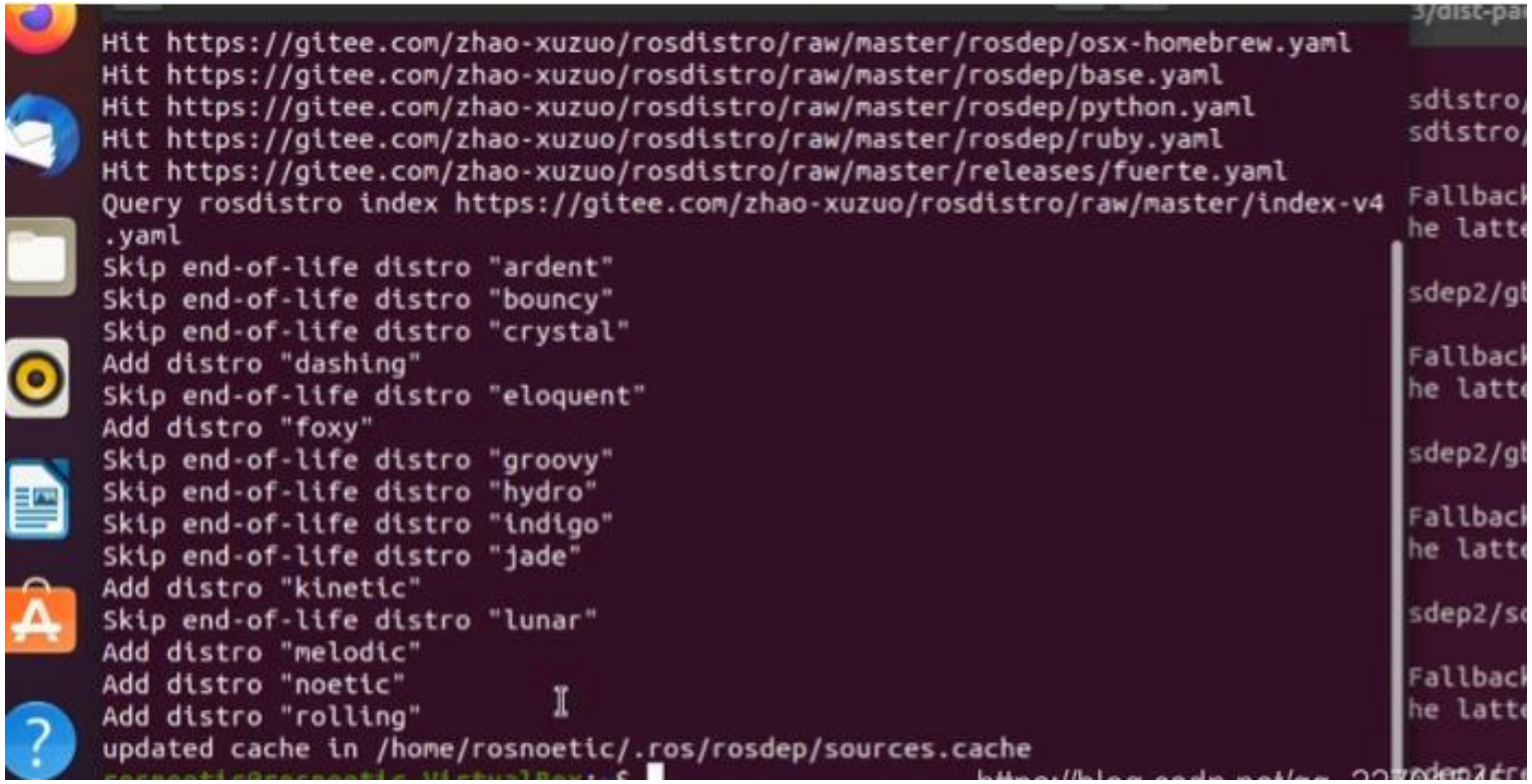
修改完毕，再重新执行命令:

```
$ sudo rosdep init
```

```
$ rosdep update
```

1.2 ROS installation

1.2.1 ROS Noetic



- **Step7：ROS测试：**

```
$ roscore
```

1.2 ROS installation

1.2.1 ROS Noetic

4. ROS Noetic版本测试结果：

```
robot@WP:~$ roscore
... logging to /home/robot/.ros/log/b1676684-0a93-11ee-b8c3-4d008d8d0844/roslaunch-WP-60778.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://WP:45733/
ros_comm version 1.16.0


SUMMARY
========

PARAMETERS
 * /rosdistro: noetic
 * /rosversion: 1.16.0

NODES

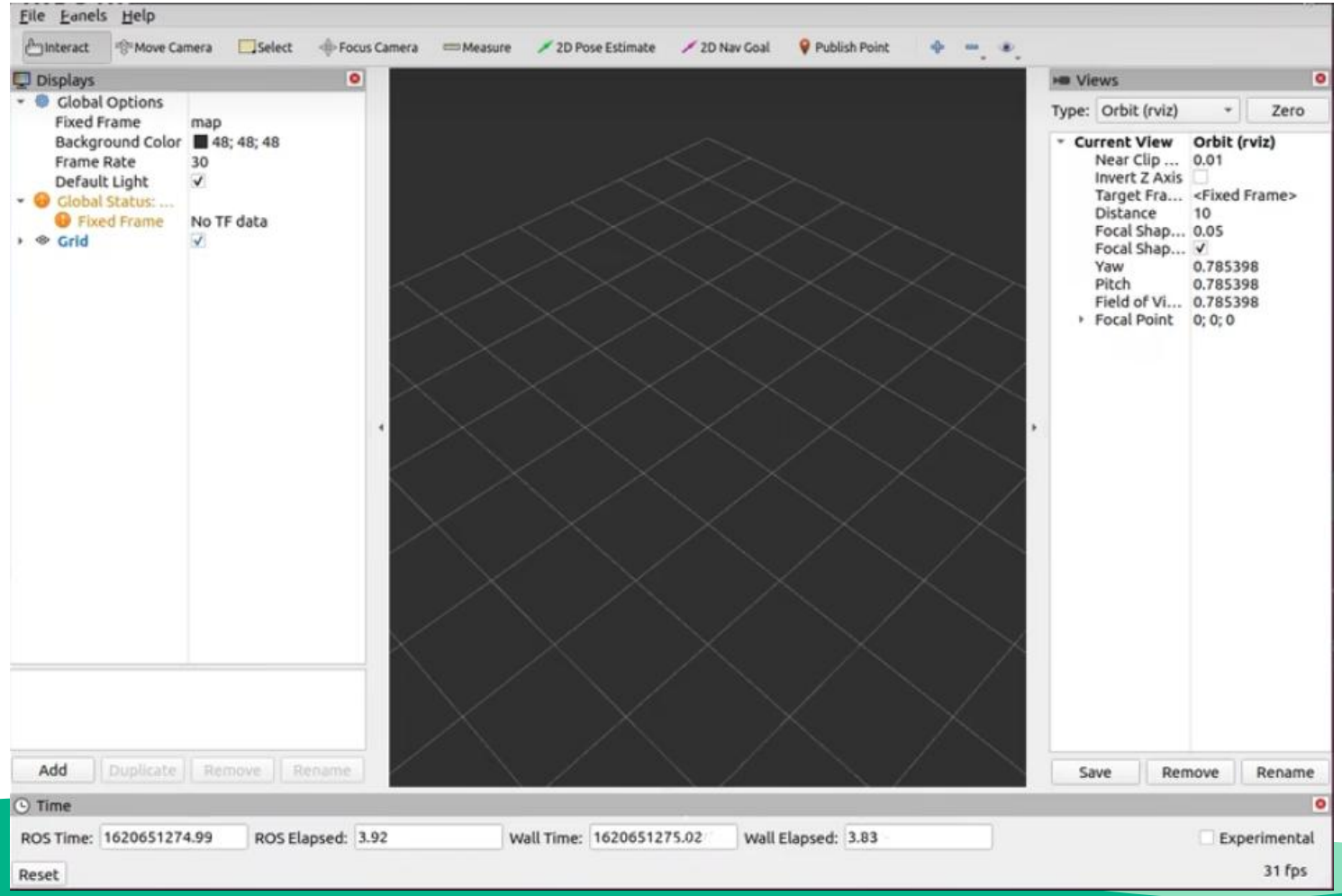
auto-starting new master
process[master]: started with pid [60786]
ROS_MASTER_URI=http://WP:11311/

setting /run_id to b1676684-0a93-11ee-b8c3-4d008d8d0844
process[rosout-1]: started with pid [60796]
started core service [/rosout]
```

1.2 ROS installation

1.2.1 ROS Noetic

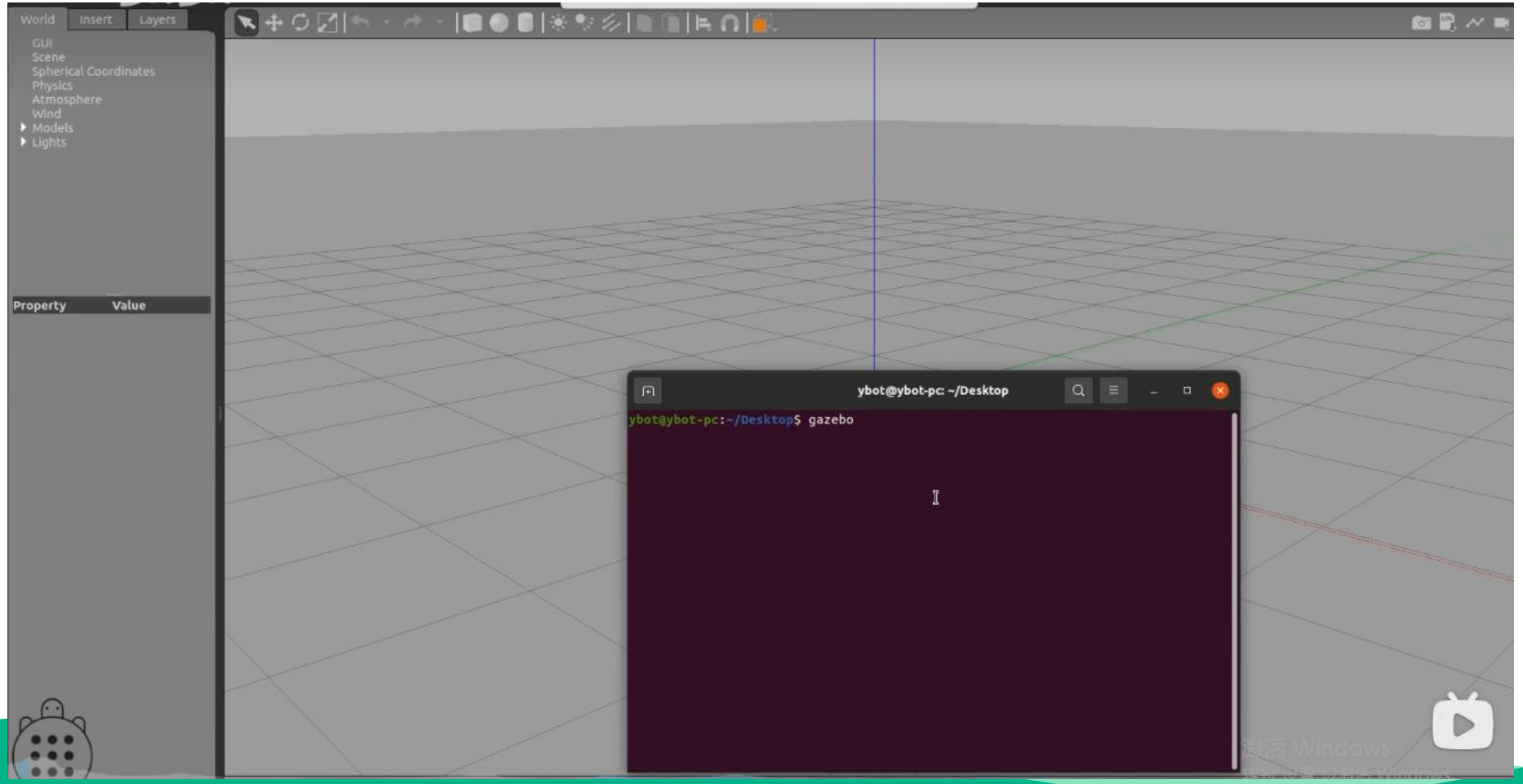
- **Step7：ROS测试Rviz：** `$ rviz`



1.2 ROS installation

1.2.1 ROS Noetic

- **Step7：ROS测试Gazebo：** `$ gazebo`



1.2 ROS installation

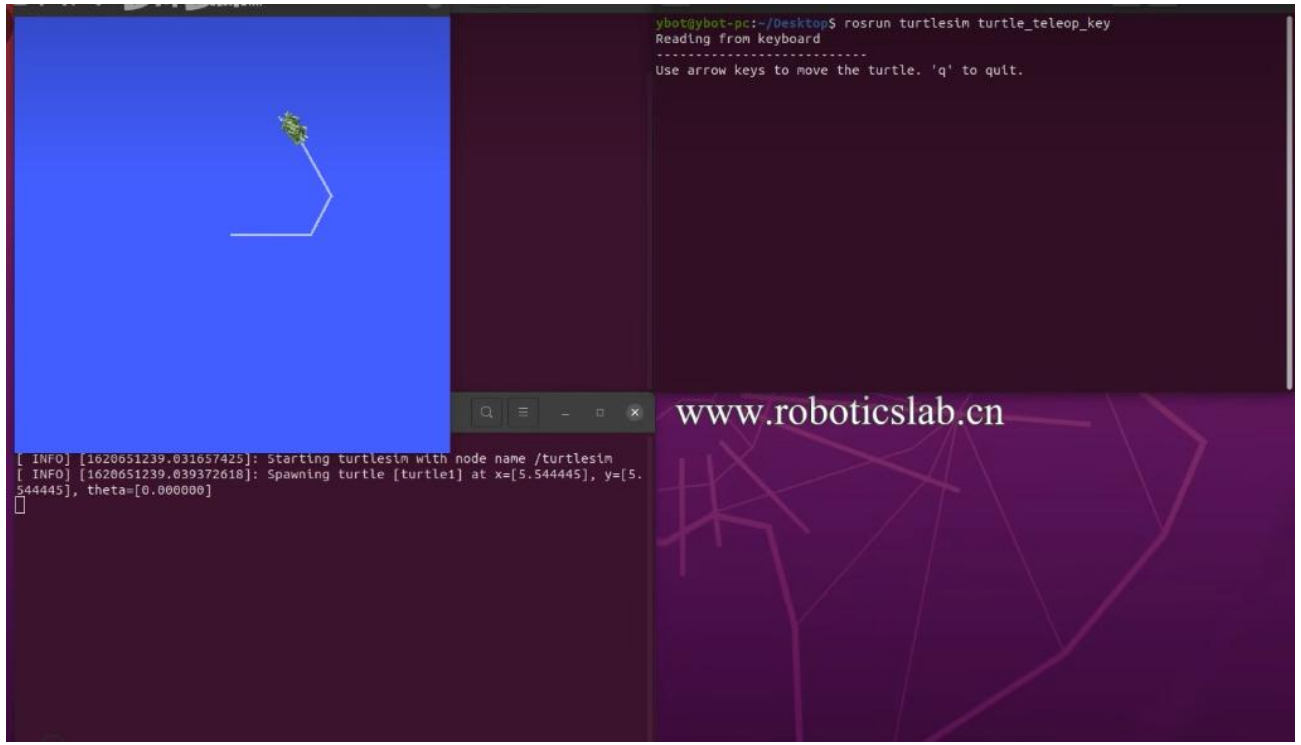
1.2.1 ROS Noetic

Step7：ROS测试：

启动/控制小乌龟:

```
$ rosrun turtlesim turtlesim_node
```

```
$ rosrun turtlesim turtlesim_teleop_key
```



1.3 Command-line tools for ROS

1.3.1 rosbash

ROS中的命令行工具 rosbash： http://wiki.ros.org/rosbash

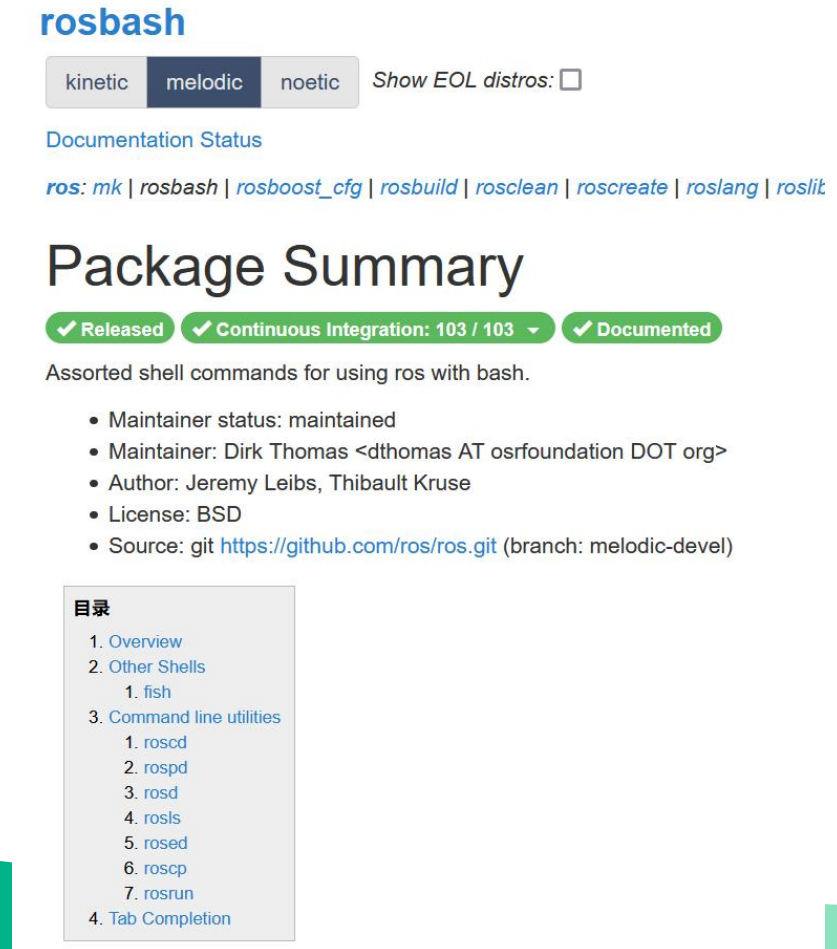
rosrun：运行ROS功能包的可执行文件

roscd： 功能包目录跳转

roscp： 拷贝ROS功能包中的文件

rosed： 编辑功能包中的文件

rosls： 列出ROS功能包中的文件



rosbash

kinetic | melodic | noetic Show EOL distros:

Documentation Status

ros: mk | rosbash | rosboost_cfg | rosbuild | rosclean | roscreate | roslang | roslit

Package Summary

Released | Continuous Integration: 103 / 103 | Documented

Assorted shell commands for using ros with bash.

- Maintainer status: maintained
- Maintainer: Dirk Thomas <dthomas AT osrfoundation DOT org>
- Author: Jeremy Leibs, Thibault Kruse
- License: BSD
- Source: git https://github.com/ros/ros.git (branch: melodic-devel)

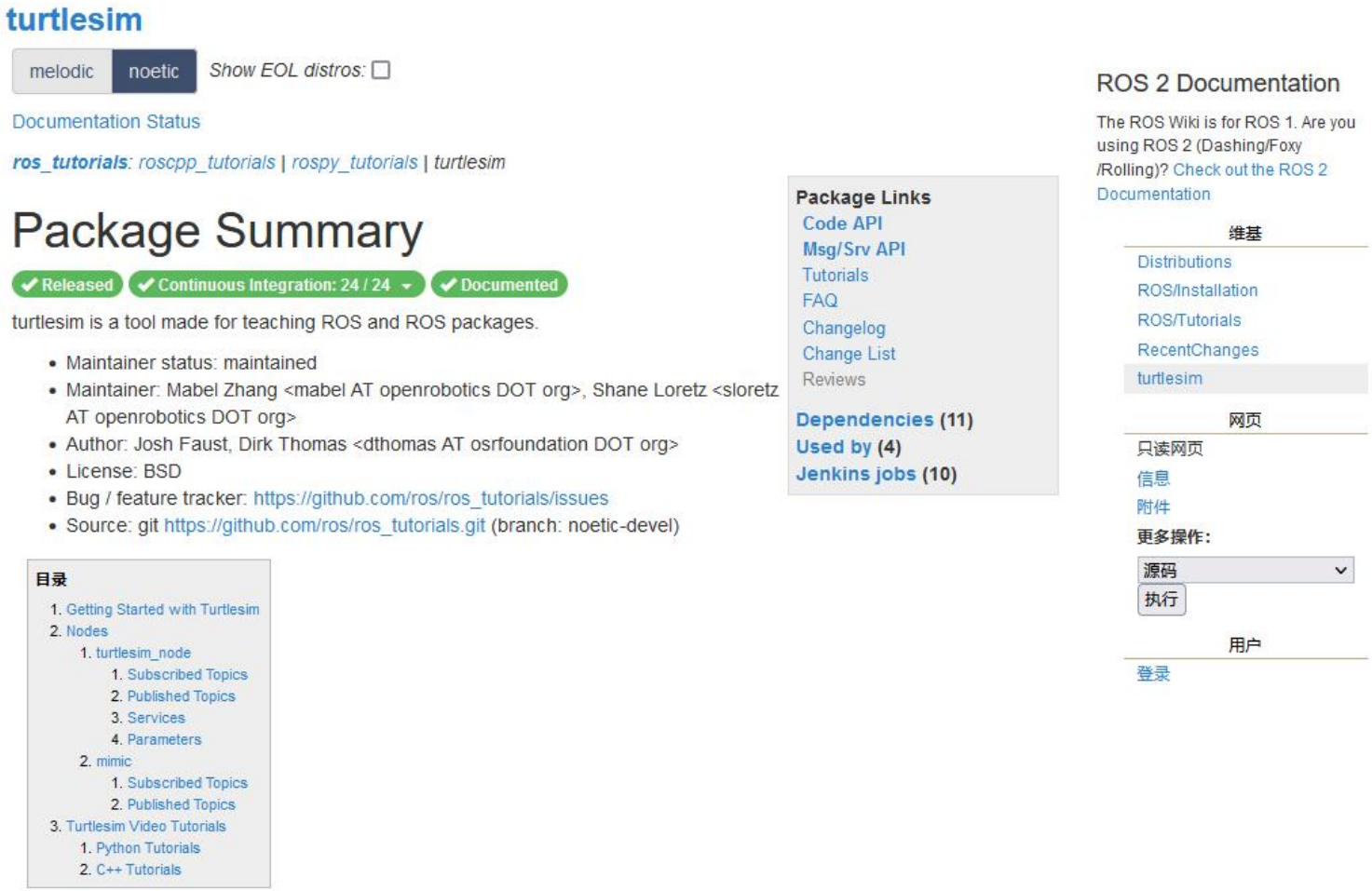
目录

1. Overview
2. Other Shells
   1. fish
3. Command line utilities
   1. roscd
   2. rospd
   3. rosd
   4. rosls
   5. rosed
   6. roscp
   7. rosrun
4. Tab Completion

1.4 Turtlesim

1.4.1 Overview：wiki.ros.org/turtlesim

Turtlesim 功能包是专门用于讲授 ROS 和 ROS 功能包的工具：



turtlesim

melodic noetic *Show EOL distros:* ☐

Documentation Status

ros_tutorials: *roscpp_tutorials* | *rospy_tutorials* | *turtlesim*

Package Summary

Released | Continuous Integration: 24 / 24 | Documented

turtlesim is a tool made for teaching ROS and ROS packages.

- Maintainer status: maintained
- Maintainer: Mabel Zhang <mabel AT openrobotics DOT org>, Shane Loretz <sloretz AT openrobotics DOT org>
- Author: Josh Faust, Dirk Thomas <dthomas AT osrfoundation DOT org>
- License: BSD
- Bug / feature tracker: https://github.com/ros/ros_tutorials/issues
- Source: git https://github.com/ros/ros_tutorials.git (branch: noetic-devel)

目录

1. Getting Started with Turtlesim
2. Nodes
   1. turtlesim_node
      1. Subscribed Topics
      2. Published Topics
      3. Services
      4. Parameters
   2. mimic
      1. Subscribed Topics
      2. Published Topics
3. Turtlesim Video Tutorials
   1. Python Tutorials
   2. C++ Tutorials

Package Links

Code API
Msg/Srv API
Tutorials
FAQ
Changelog
Change List
Reviews

Dependencies (11)
Used by (4)
Jenkins jobs (10)

ROS 2 Documentation

The ROS Wiki is for ROS 1. Are you using ROS 2 (Dashing/Foxy /Rolling)? Check out the ROS 2 Documentation

维基

Distributions
ROS/Installation
ROS/Tutorials
RecentChanges
turtlesim

网页

只读网页
信息
附件
更多操作:
源码
执行

用户

登录

1.4 Turtlesim

1.4.2 Turtlesim nodes

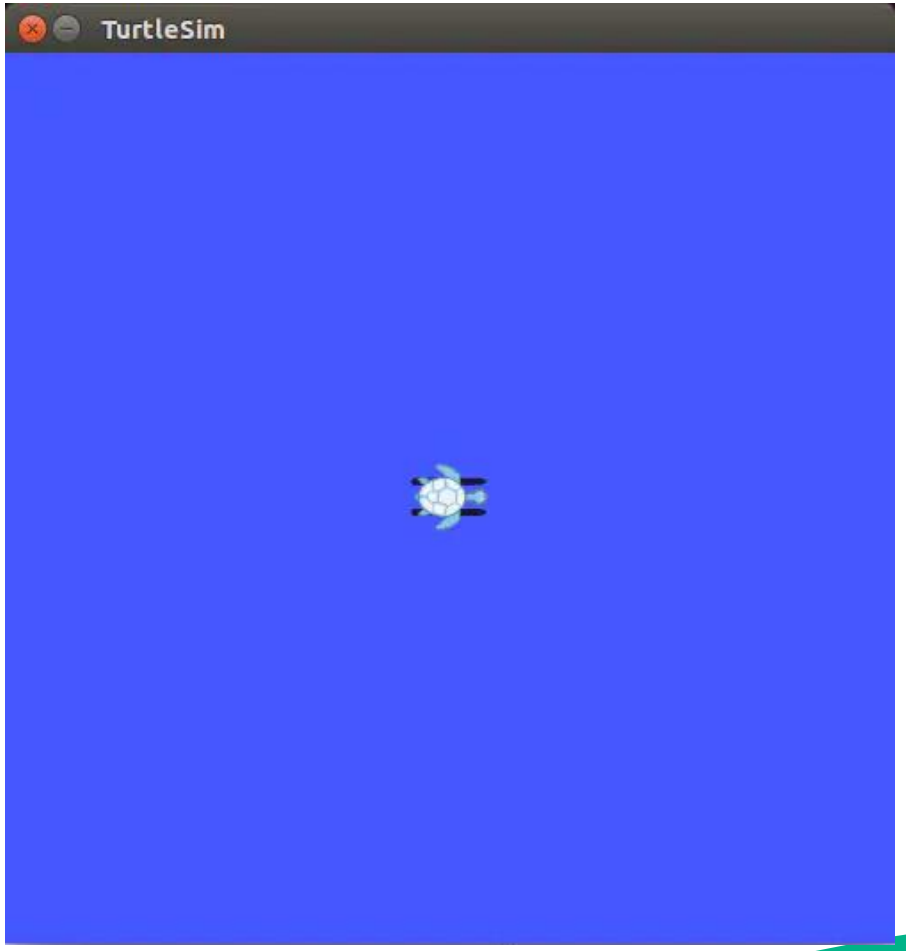
turtlesim package 里面有 4 个节点（ node）：

- turtlesim_node：2D仿真节点
- turtle_teleop_key：键盘操控节点
- draw_square：画正方形节点
- mimic：同步节点（第2只乌龟同步第1只乌龟的动作）

```
$ rosrun turtlesim turtlesim_node
```

启动2D仿真节点，出现一直小乌龟（外观随机），

该乌龟的名称为turtle1



1.4 Turtlesim

1.4.2 Turtlesim nodes

```
$ rosrun turtlesim turtle_teleop_key
```

启动键盘操控节点

```
Reading from keyboard
---------------------------
Use arrow keys to move the turtle.
```

作用：用键盘上的方向键控制海龟运动

注意：在用键盘控制时，要将光标聚焦到启动 turtle_teleop_key 的 terminal 才有效

1.4 Turtlesim

1.4.2 Turtlesim nodes

```
$ rosrun turtlesim draw_square
```

启动画正方形节点

作用：海龟开始自动的沿正方形轨迹运动



1.4 Turtlesim

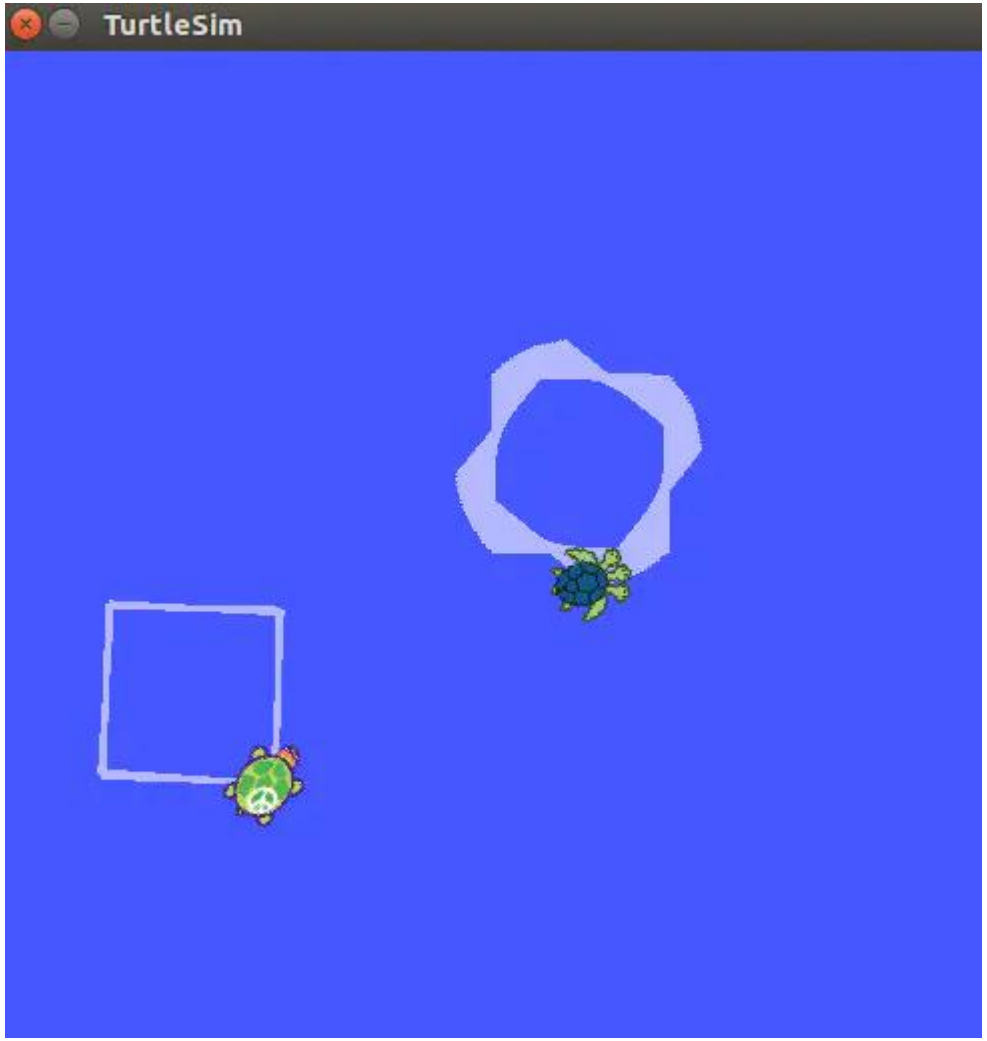
1.4.2 Turtlesim nodes

```
$ rosservice call /spawn  10 10 0 turtle2
```

调用服务在(10,10)处生成另外一只朝向为0的
乌龟，该乌龟的名字为turtle2

```
$ rosrun turtlesim mimic input:=turtle1 output:=turtle2
```

作用：使得turtle2跟随turtle1的轨迹进行运动



Post-assessment

1. ros-latest.list这个文件有什么作用?
2. 若要安装noetic中的特定ROS包，用什么指令?
3. 用户家目录下的.bashrc文件有何作用?
4. rosdep有何作用?
5. rosbash包含哪些命令行工具?

Summary

1. ROS Installation
   a) 步骤1——步骤7
   b) 步骤3和步骤5错误提示的解决方案
2. Command-line tools for ROS
   a) rosrun、roscd、roscp、rosed、rosls
3. Turtlesim
   a) turtlesim_node
   b) turtle_teleop_key
   c) draw_square
   d) mimic

Overview lesson 6-1

1 ROS Filesystem(1)

2 ~

3 ~

4 ~

Objectives

1、了解ROS系统的三层架构

2、掌握ROS文件系统架构

3、掌握创建工作空间和功能包的方法



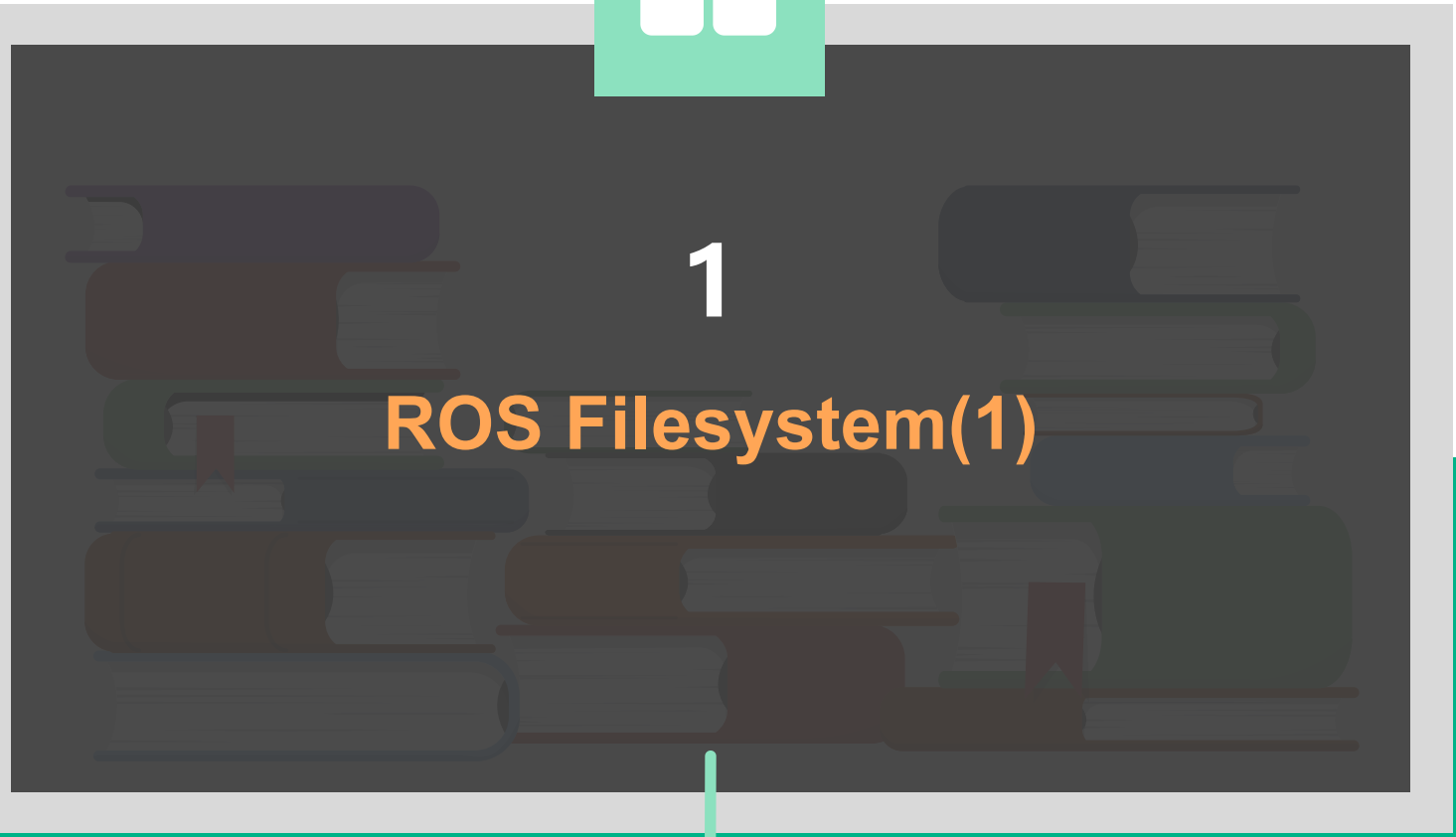
Pre-assessment

1、ROS文件系统指什么?

2、是否使用过catkin_make?

3、是否了解工作空间？工作空间的用处?

4、是否自己创建过catkin功能包?



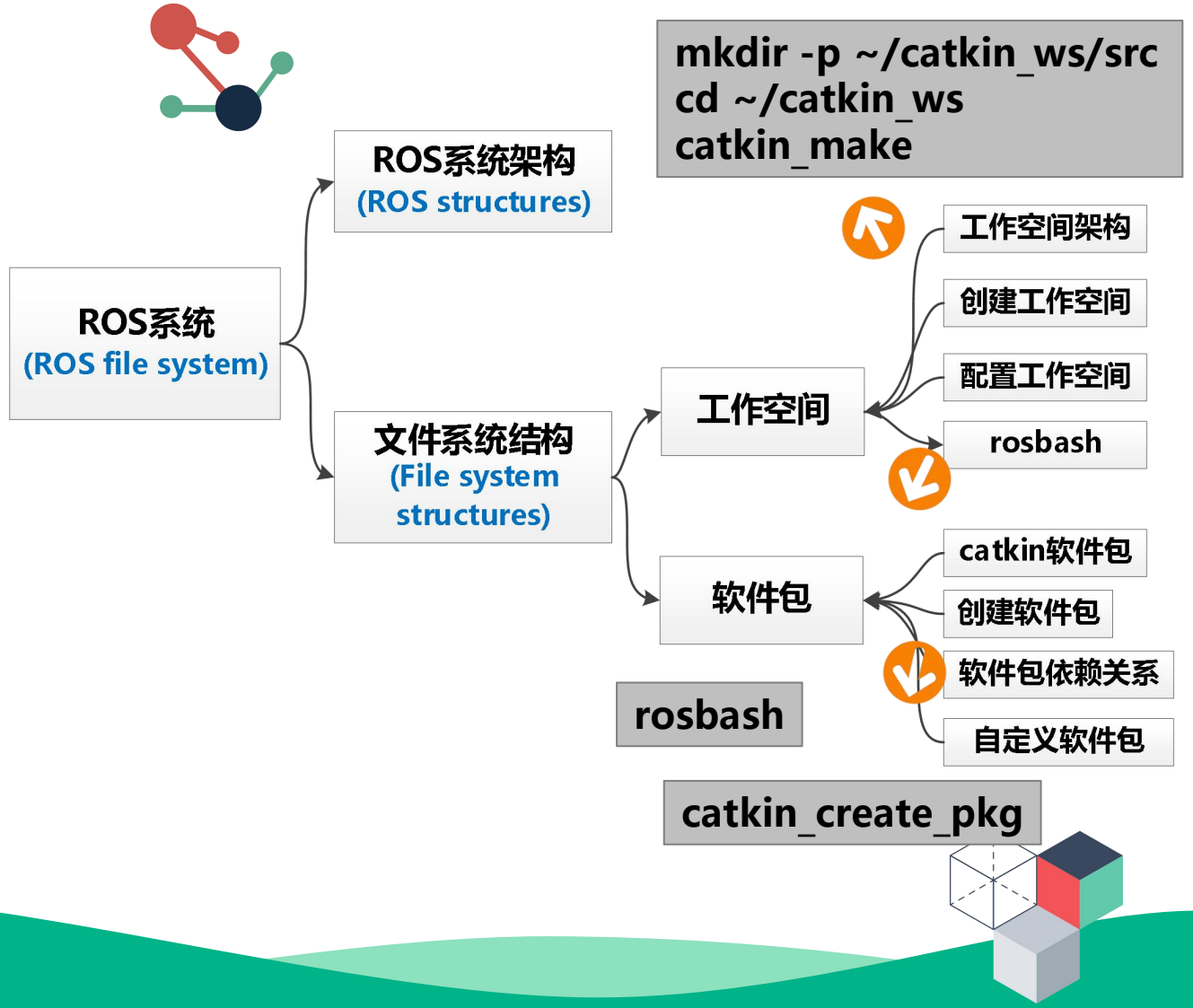
1

ROS Filesystem(1)

1.1 Knowledge architecture

1.1.1 Overview

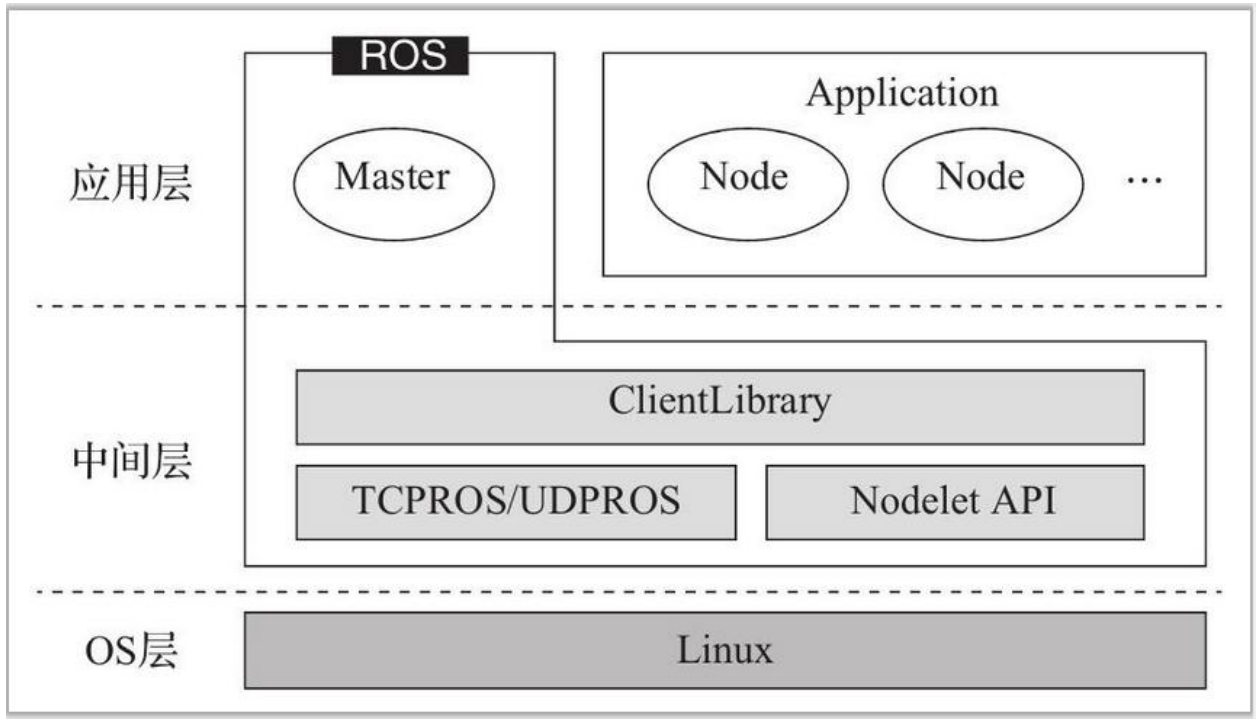
- **ROS系统架构**：ROS系统的架构包含三个层次：系统层、中间层、应用层
- **文件系统结构**：
  - 工作空间：
    - 工作空间结构
    - 创建并配置工作空间
  - 功能包：
    - catkin功能包定义
    - 创建功能包，功能包依赖关系
    - 自定义功能包



1.2 ROS architecture

1.2.1 ROS system architecture

➢ **ROS系统架构主要被设计为三个层次：**



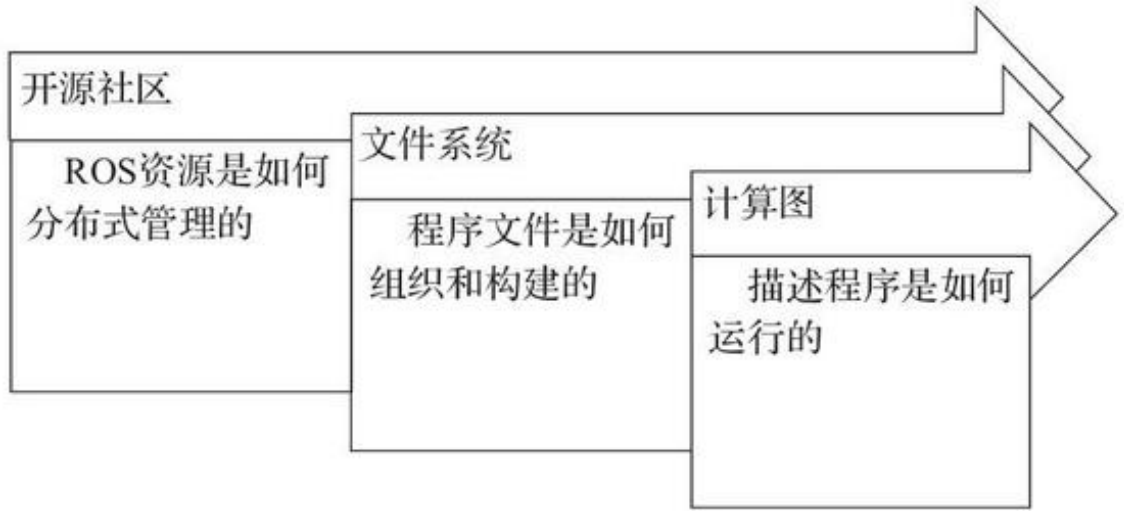
- **系统层：**
  - **Linux(Ubuntu)**
- **中间层：通信系统**
  - **发布/订阅**
  - **客户端/服务器端**
  - **Nodelet**
- **应用层：ROS应用**
  - **节点**
  - **话题**
  - **功能包**

1.2 ROS architecture

1.2.2 ROS own architecture

http://wiki.ros.org/ROS/Concepts

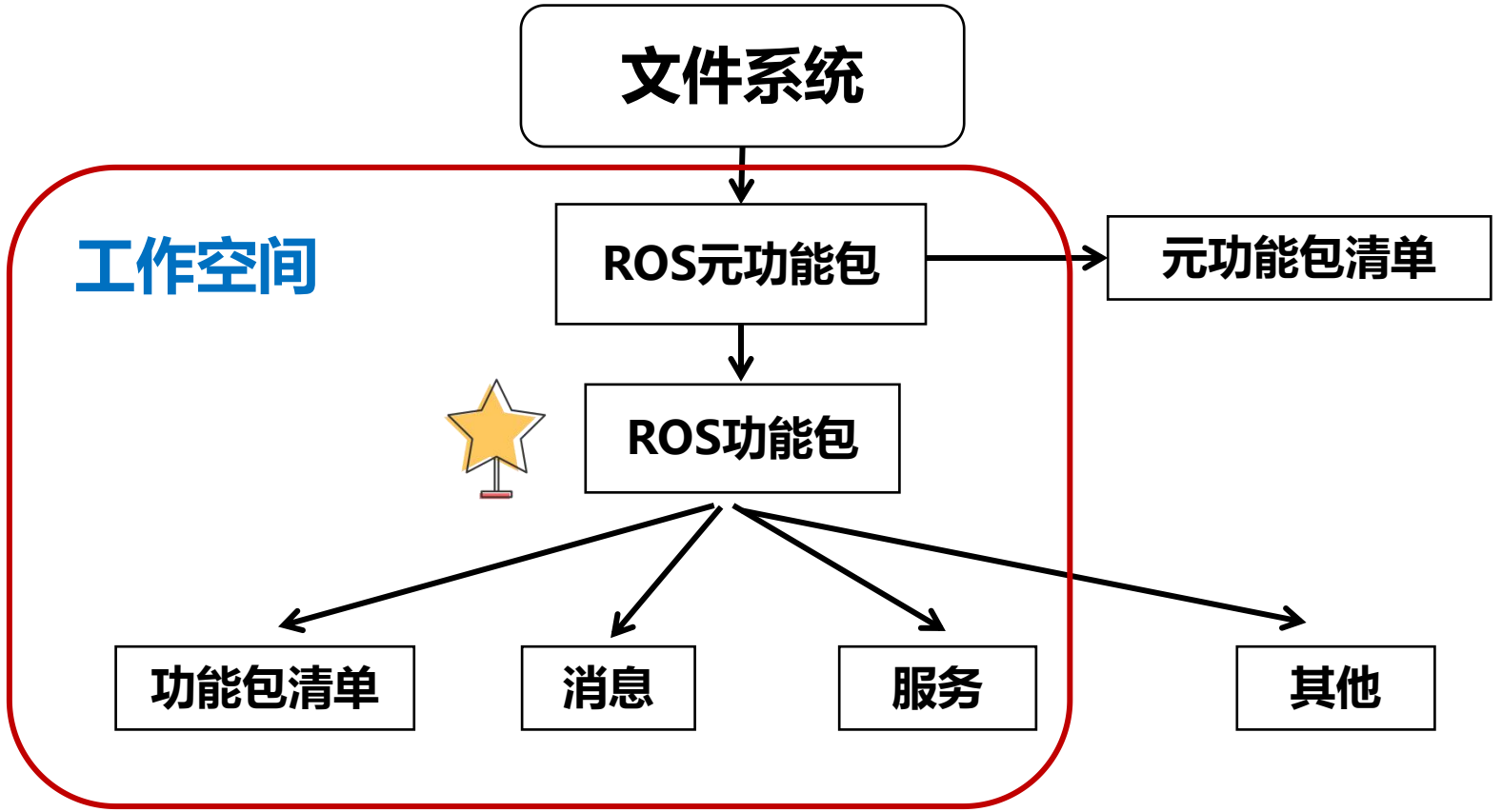
➢ **ROS自身系统实现的三个层次：**



- **文件系统级：** ROS将所有的文件按照一定的规则进行组织，不同功能的文件被放置在不同的文件夹
- **计算图级：** ROS 处理数据的一种点对点的网络形式。ROS 中的基本计算图级概念包括：节点、节点管理器、参数服务器、消息、服务、话题和功能包
- **开源社区级：** 丰富的社区资源，发行版，软件源，ROS wiki，ROR Answers

1.3 The filesystem level

1.3.1 Overview ROS filesystem: http://wiki.ros.org/ROS/Concepts



1.3 The filesystem level

1.3.2 ROS workspace structures

- 工作空间是一个包含功能包、可编辑源文件或编译包的文件夹，其包含 build、devel、src 子文件夹

a) src 源文件空间(The source space)：

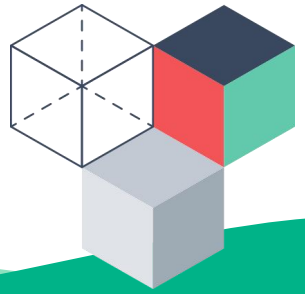
- 创建工作空间时同时创建src源文件空间，包含 CMakeLists.txt（负责调用CMake编译）
- 在源文件空间中放置了功能包、项目、克隆包等

b) build 编译空间(The build space)：

- 在build文件夹中，CMake 和 catkin 为功能包和项目保存缓存信息、配置和其他中间文件

c) devel 开发空间(The development space)：

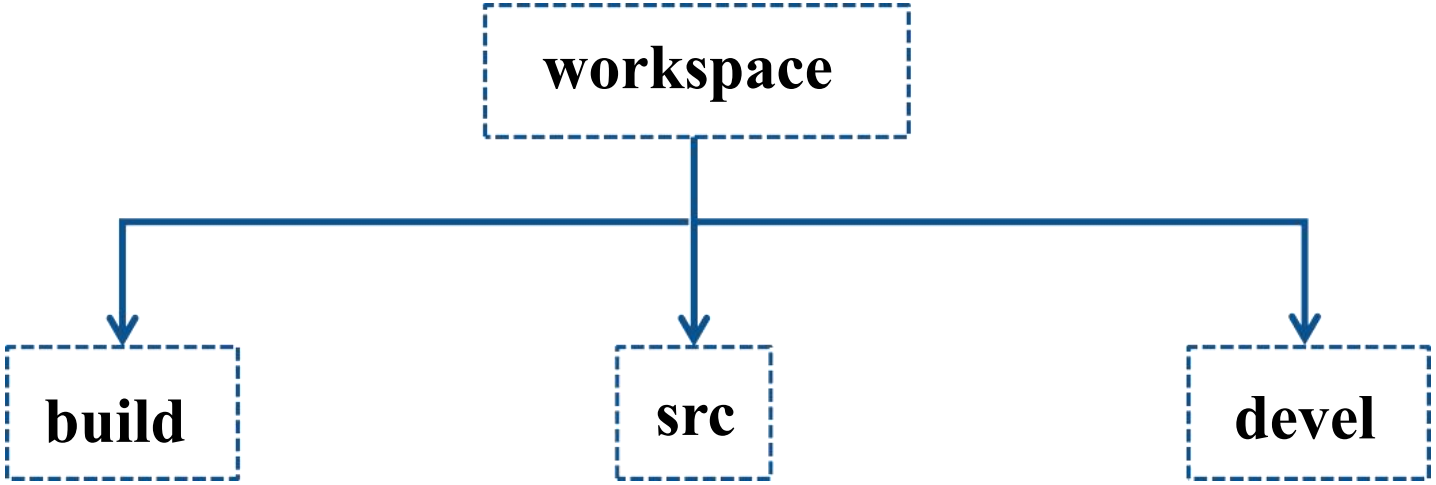
- devel文件夹用来保存编译后的程序



1.3 The filesystem level

1.3.2 ROS workspace structures

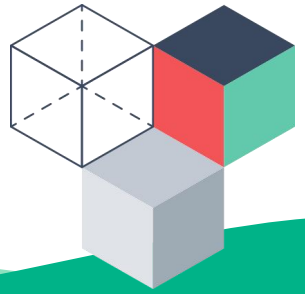
workspace：存放工程文件的文件夹



编译过程产生的中间文件

代码文件

编译生成的目标文件



1.3 The filesystem level

1.3.3 Command-line tools for ROS workspace

1. 创建工作空间：

```
$ mkdir -p ~/catkin_ws/src

$ cd ~/catkin_ws  # 编译需要在 catkin_ws 中进行

$ catkin_make     # 编译工作空间
```

```
ros@ROS-VirtualBox:~/catkin_ws$ ls
build  devel  src
```

```
ros@ROS-VirtualBox:~/catkin_ws$ catkin_make
Base path: /home/ros/catkin_ws
Source space: /home/ros/catkin_ws/src
Build space: /home/ros/catkin_ws/build
Devel space: /home/ros/catkin_ws/devel
Install space: /home/ros/catkin_ws/install
Creating symlink "/home/ros/catkin_ws/src/CMakeLists.txt" pointing to "/opt/ros/melo
####
#### Running command: "cmake /home/ros/catkin_ws/src -DCATKIN_DEVEL_PREFIX=/home/ros
 Unix Makefiles" in "/home/ros/catkin_ws/build"
####
-- The C compiler identification is GNU 7.5.0
-- The CXX compiler identification is GNU 7.5.0
-- Check for working C compiler: /usr/bin/cc
-- Check for working C compiler: /usr/bin/cc -- works
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Detecting C compile features
-- Detecting C compile features - done
-- Check for working CXX compiler: /usr/bin/c++
-- Check for working CXX compiler: /usr/bin/c++ -- works
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- Using CATKIN_DEVEL_PREFIX: /home/ros/catkin_ws/devel
-- Using CMAKE_PREFIX_PATH: /opt/ros/melodic
-- This workspace overlays: /opt/ros/melodic
-- Found PythonInterp: /usr/bin/python2 (found suitable version "2.7.17", minimum re
-- Using PYTHON_EXECUTABLE: /usr/bin/python2
-- Using Debian Python package layout
-- Using empy: /usr/bin/empy
-- Using CATKIN_ENABLE_TESTING: ON
-- Call enable_testing()
-- Using CATKIN_TEST_RESULTS_DIR: /home/ros/catkin_ws/build/test_results
-- Found gtest sources under '/usr/src/googletest': gtests will be built
-- Found gmock sources under '/usr/src/googletest': gmock will be built
-- Found PythonInterp: /usr/bin/python2 (found version "2.7.17")
```

1.3 The filesystem level

1.3.3 Command-line tools for ROS workspace

2. 配置工作空间（环境变量）：

```
$ echo $ROS_PACKAGE_PATH
```

查看ROS正在使用的工作空间

```
ros@ROS-VirtualBox:~$ echo $ROS_PACKAGE_PATH
/opt/ros/melodic/share
```

```
$ source devel/setup.bash
```

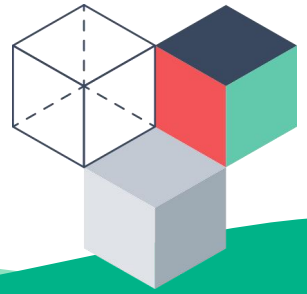
从ROS工作空间重新加载 setup.bash，并覆盖默认配置

```
ros@ROS-VirtualBox:~/catkin_ws$ source devel/setup.bash
ros@ROS-VirtualBox:~/catkin_ws$ echo $ROS_PACKAGE_PATH
/home/ros/catkin_ws/src:/opt/ros/melodic/share
```

```
$ echo “source /opt/ros/noetic/setup.bash” >>~/.bashrc
```

```
$ echo “source /home/your_name/catkin_ws/devel/setup.bash” >>~/.bashrc
```

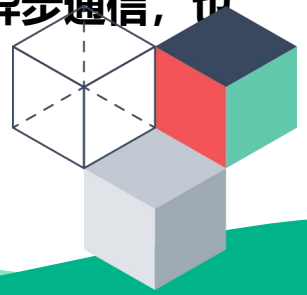
配置 bashrc 文件，使 ROS 和创建的工作空间永久生效



1.3 The filesystem level

1.3.4 ROS package

- **功能包(package)：实现特定功能的软件包，功能包中包括：功能包清单、消息、服务、代码、其他**
  - 功能包是 ROS 中整理和组织软件的主要单元，也是 ROS 中能够建立和分享的最小单元
  - 功能包中包含ROS节点、ROS程序库、数据集、配置文件、可执行文件等
  - **功能包清单(package manifest)：**
    - 功能包清单是一个名为 package.xml 的 XML 文件，其必须位于功能包文件夹中
    - 功能包清单中列举了功能包的相关信息，包括功能包名称、版本、作者、开源许可、依赖关系等
  - **消息类型(message type)：**
    - 消息是指，一个 ROS 节点发送到其他 ROS 节点的信息，节点间通过将消息发布到话题来实现异步通信，也指 ROS 服务中的请求和响应消息
    - 消息位于所在功能包下的 msg 文件夹中，扩展名为.msg，



1.3 The filesystem level

1.3.4 ROS package

- 服务类型(service type)：
  - 定义了 ROS 中由每个 ROS 节点提供的关于服务请求和响应的数据结构
  - ROS节点通过调用系统服务来同步交换请求和响应消息
  - 请求和响应消息位于功能包下的 srv 文件中，扩展名为.srv

➢ 元功能包(matapackage)：

- Metapackages 是一组具体的服务相关的功能包。例如，将多个功能包组合在一起，以实现特定的目的和功能（例如导航任务）

➢ 元功能包清单(metapackage manifest)：

- 作用类似 package manifest 文件，但是它针对的是 metapackage

turtlesim功能包结构

```
ros@ROS-VirtualBox:/opt/ros/melodic/share/turtlesim$ tree
.
├── cmake
│   ├── turtlesimConfig.cmake
│   ├── turtlesimConfig-version.cmake
│   ├── turtlesim-msg-extras.cmake
│   └── turtlesim-msg-paths.cmake
├── images
│   ├── box-turtle.png
│   ├── diamondback.png
│   ├── electric.png
│   ├── fuerte.png
│   ├── groovy.png
│   ├── hydro.png
│   ├── hydro.svg
│   ├── indigo.png
│   ├── indigo.svg
│   ├── jade.png
│   ├── kinetic.png
│   ├── kinetic.svg
│   ├── lunar.png
│   ├── lunar.svg
│   ├── melodic.png
│   ├── palette.png
│   ├── robot-turtle.png
│   ├── sea-turtle.png
│   └── turtle.png
├── msg
│   ├── Color.msg
│   └── Pose.msg
├── package.xml
└── srv
    ├── Kill.srv
    ├── SetPen.srv
    ├── Spawn.srv
    ├── TeleportAbsolute.srv
    └── TeleportRelative.srv
```

1.3 The filesystem level

1.3.5 Command-line tools for ROS package

1. 创建ROS功能包：

```
$ cd ~/catkin_ws/src
```

```
$ catkin_create_pkg beginner_tutorials std_msgs roscpp rospy
```

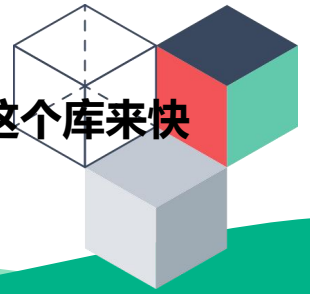
```
$ cd ~/catkin_ws
```

```
$ catkin_make
```

备注：catkin_create_pkg语法格式如下：

```
catkin_create_pkg package_name [dependency1] [dependency2] ... [dependencyN]
```

- 常用 dependencies：
  - std_msgs：标准 ROS 消息，包括表示原始数据类型和其他基本消息结构的常见消息类型
  - roscpp：roscpp 功能包是 ROS 的一个 C++ 客户端库，它提供了一个客户端库，可以通过这个库来快速和 ROS 话题、服务和参数的接口进行交互
  - rospy：rospy是 ROS 的 Python 客户端库



1.3 The filesystem level

1.3.5 Command-line tools for ROS package

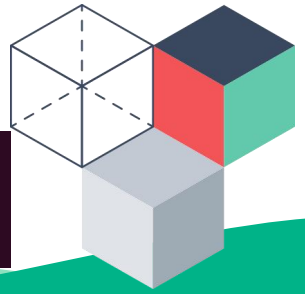
- 适用于 ROS 功能包的命令工具：rospack (ros+package)

  语法格式：rospack command [options] [package]

  - rospack list: 查看软件包列表
  - rospack find package_name： 找到 package 的路径
  - rospack depends1 package_name：列出 package 的一级依赖
  - rospack depends package_name：列出 package 的所有依赖
  - rospack profile：提供新添加到 ROS 中功能包的信息，在安装新功能包时十分有用

  举例：

```
ros@ROS-VirtualBox:~/catkin_ws$ rospack find beginner_tutorials
/home/ros/catkin_ws/src/beginner_tutorials
```



1.3 The filesystem level

1.3.5 Command-line tools for ROS package

2. 软件依赖包关系：一级依赖

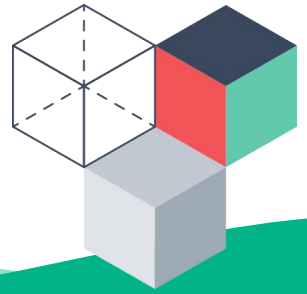
- 使用 catkin_create_pkg 命令时提供了几个功能包作为依赖关系，可以使用 rospack 命令工具来查看这些依赖包

```
$ rospack depends1 beginner_tutorials
```

```
ros@ROS-VirtualBox:~/catkin_ws/devel$ rospack depends1 beginner_tutorials
roscpp
rospy
std_msgs
```

- rospack 列出了在运行 catkin_create_pkg 命令时作为参数的依赖包，这些依赖关系存储在 package.xml 文件中

```
$ roscd beginner_tutorials
```



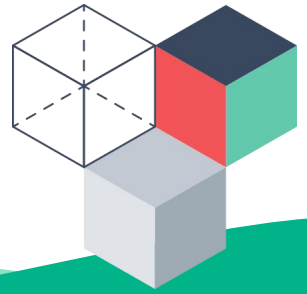
1.3 The filesystem level

1.3.5 Command-line tools for ROS package

2. 软件依赖包关系：一级依赖

```
$ cat package.xml
```

```
<package>
...
  <buildtool_depend>catkin</buildtool_depend>
  <build_depend>roscpp</build_depend>
  <build_depend>rospy</build_depend>
  <build_depend>std_msgs</build_depend>
...
</package>
```



1.3 The filesystem level

1.3.5 Command-line tools for ROS package

2. 软件依赖包关系：间接依赖

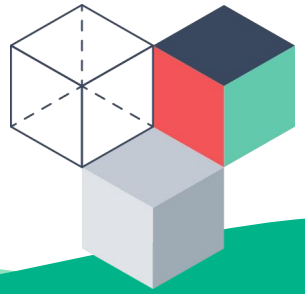
- 很多情况下，一个依赖包还会有自己的依赖关系，比如rospy就有其它依赖包

```
$ rospack depends1 rospy
```

```
ros@ROS-VirtualBox:~/catkin_ws/src$ rospack depends1 rospy
genpy
roscpp
rosgraph
rosgraph_msgs
roslib
std_msgs
```

- 使用rospack可以递归检测出所有嵌套的依赖包

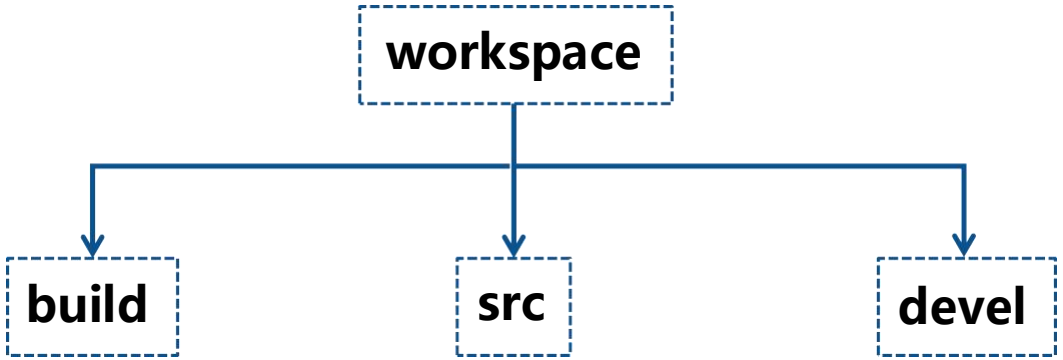
```
$ rospack depends beginner_tutorials
```



1.3 The filesystem level

1.3.6 ROS package review

workspace：存放工程文件的文件夹

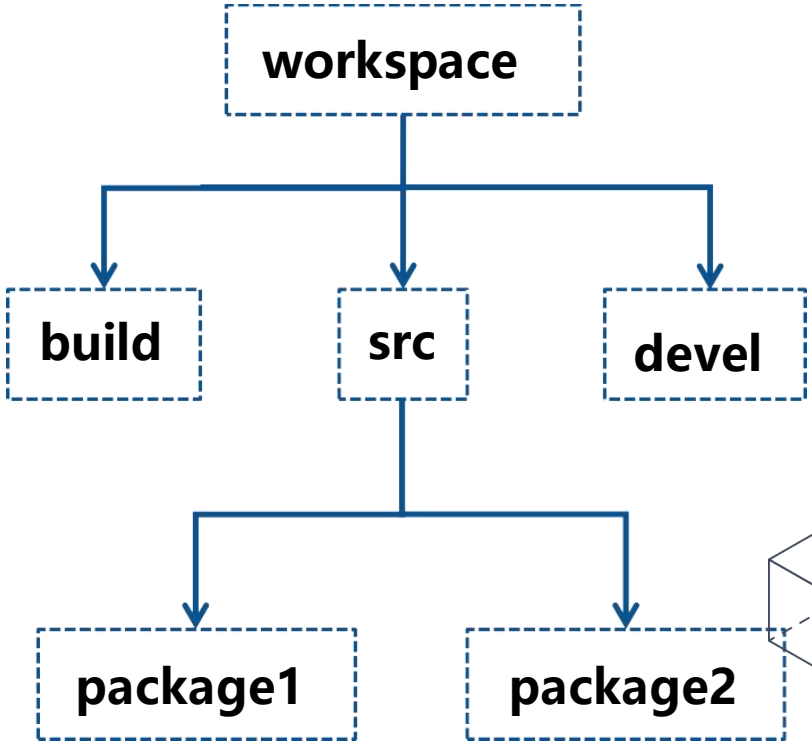


编译过程产生的中间文件

代码文件

编译生成的目标文件

package：存放用户功能代码的文件夹



1.3 The filesystem level

1.3.7 Turtlesim: http://wiki.ros.org/turtlesim

ROS.org About | Support | Discussion Forum | Service Status | Q&A answers.ros.org

Documentation Browse Software News

turtlesim

kinetic melodic noetic *Show EOL distros:* ☐

Documentation Status

***ros_tutorials**: roscpp_tutorials | rospy_tutorials | turtlesim*

Package Summary

✔ Released ✔ Continuous Integration: 24 / 24 ✔ Documented

turtlesim is a tool made for teaching ROS and ROS packages.

- Maintainer status: maintained
- Maintainer: Mabel Zhang <mabel AT openrobotics DOT org>, Shane Loretz <sloretz AT openrobotics DOT org>
- Author: Josh Faust, Dirk Thomas <dthomas AT osrfoundation DOT org>
- License: BSD
- Bug / feature tracker: https://github.com/ros/ros_tutorials/issues
- Source: git https://github.com/ros/ros_tutorials.git (branch: noetic-devel)

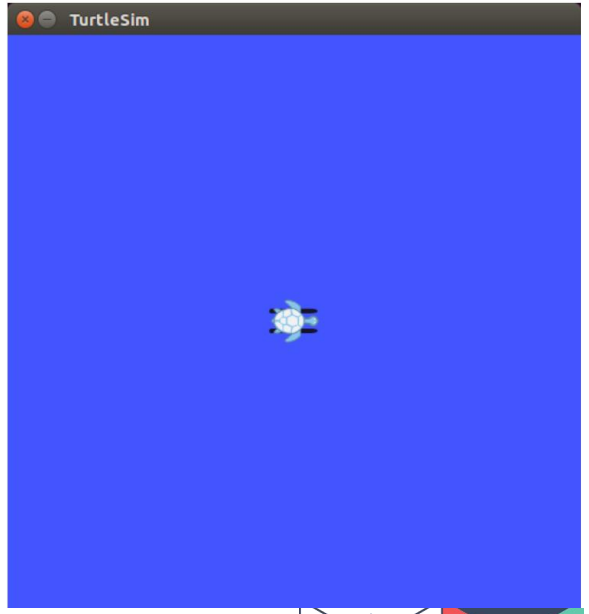
Package Links
- Code API
- Msg/Srv API
- Tutorials
- FAQ
- Changelog
- Change List
- Reviews

Dependencies (11)
Used by (4)
Jenkins jobs (10)

目录
1. Getting Started with Turtlesim
2. Nodes
   1. turtlesim_node
      1. Subscribed Topics
      2. Published Topics
      3. Services
      4. Parameters
   2. mimic
      1. Subscribed Topics
      2. Published Topics
3. Turtlesim Video Tutorials
   1. Python Tutorials

Turtlesim: turtlesim 是用于教授 ROS 和 ROS 功能包的工具

目录
1. Getting Started with Turtlesim
2. Nodes
   1. turtlesim_node
      1. Subscribed Topics
      2. Published Topics
      3. Services
      4. Parameters
   2. mimic
      1. Subscribed Topics
      2. Published Topics
3. Turtlesim Video Tutorials
   1. Python Tutorials
   2. C++ Tutorials



Post-assessment

1. ROS系统架构的三个层次？ROS自身结构的三个层次?
2. ROS工作空间的包含的3个文件夹？分别有何作用?
3. 创建工作空间的3条命令?
4. 创建包含std_msgs、rospy、roscpp的功能包a的命令?

ROS Summary

1. ROS系统架构的三个层次：
    a) 系统层、中间层、应用层
2. ROS系统实现的三层架构：
    a) 文件系统级、计算图级、开源社区级
3. ROS文件系统
    a) 文件系统架构
    b) 工作空间架构 
    c) ROS功能包 

Overview lesson 6-2

1 XML File

2 ROS Filesystem(2)

3 ~

4 ~

Objectives

1、理解 XML 文件的结构

2、掌握 Catkin 功能包的组织架构

3、掌握 package.xml 的使用方法



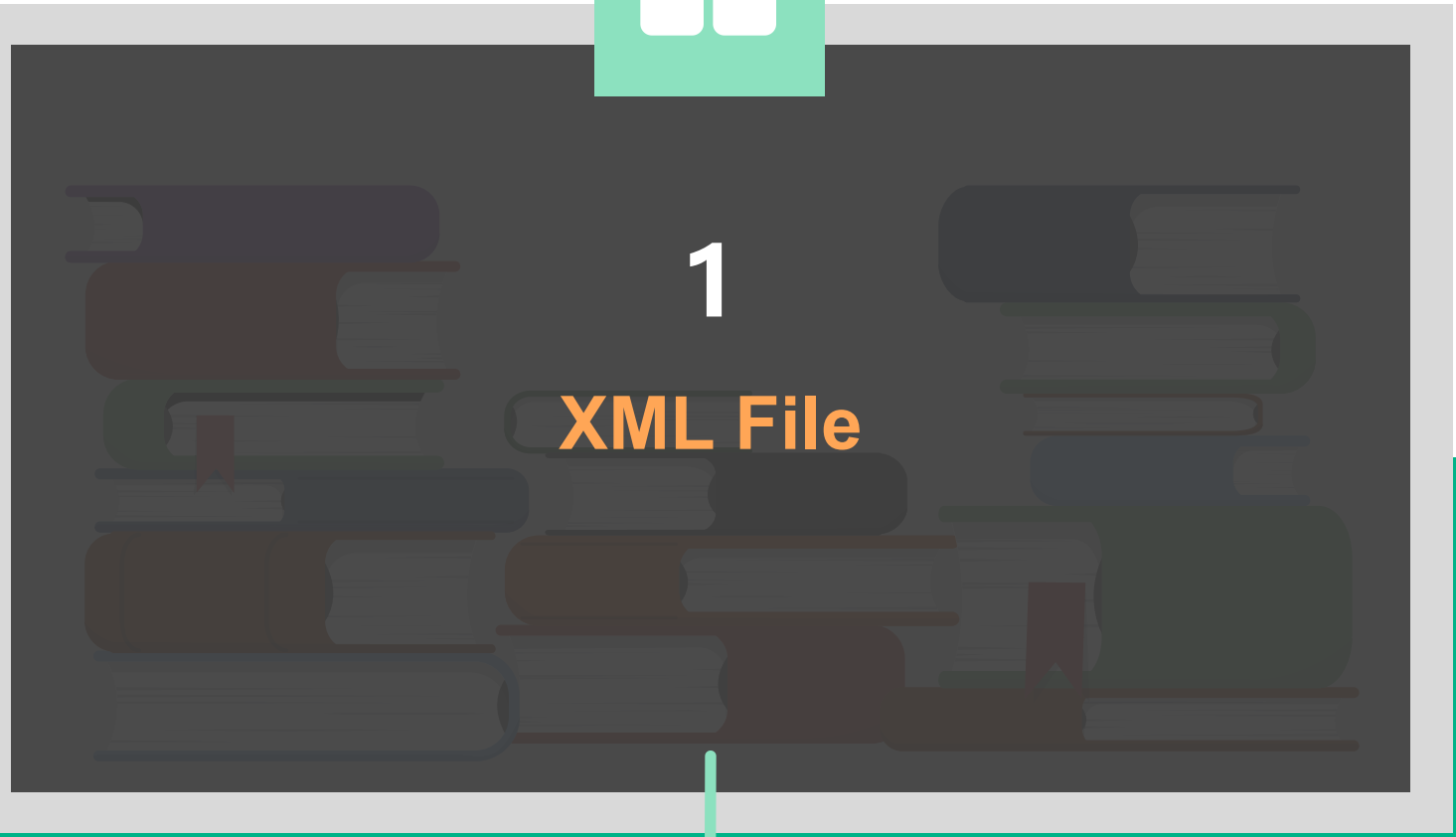
Pre-assessment

1、了解和使用过 XML 文件?

2、是否了解 Cmake?

3、是否了解 Catkin 功能包

4、是否自己创建过 Catkin 功能包?



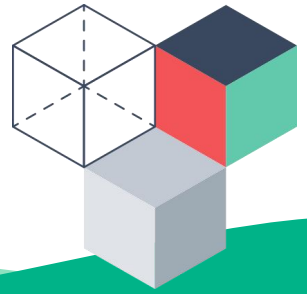
1

XML File

1.1 XML basics

1.1.1 Overview

- XML(Extensible Markup Language)：可扩展标记语言，标记指计算机所能理解的信息符号，通过此种标记，计算机之间可以处理各种信息
  - 特点：XML 与操作系统、编程语言和开发平台无关，可以跨平台使用
  - 作用：配置应用程序（配置文件一般用 XML 编写），实现不用系统间的数据交互、数据共享等
- XML文件结构：
  1. 文档声明
  2. 元素
  3. 属性
  4. 注释
  5. CDATA区 、特殊字符
  6. 处理指令（processing instruction）



1.1.2 XML file structure

1、文档声明：在编写 XML 文档时，需要先使用文档声明，声明 XML 文档的类型。XML 声明是 XML 文档的第一句，其格式如下：

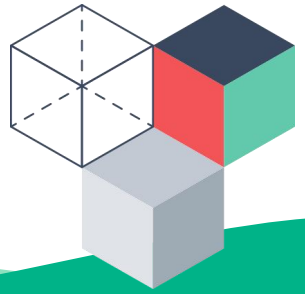
```
<?xml version="1.0" encoding="utf-8"?>
```

- **声明包括两个部分：① XML版本；② 文字编码方式**

2、元素：指 XML 文件中出现的标签，一个标签分为 开始标签 和 结束标签，使用标签时需注意：

- **标签必须成对出现，开始标签+结束标签**





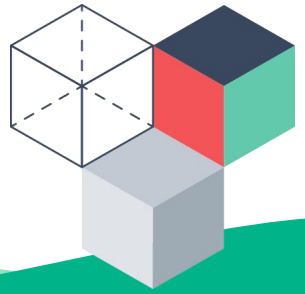
1.1 XML basics

1.1.2 XML file structure

- 标签内容对大小写敏感
- 标签可以嵌套，但不可以交叉嵌套

```
<title><name>XML编程</title></name>
<title><name>XML编程</name></title>
```

- 声明之后第一个元素为根元素，XML 文档有且只有一个根元素：
  - 其他元素都是这个根元素的子元素，根元素完全包括文档中其他所有的元素
  - 根元素的起始标记要放在所有其他元素的起始标记之前
  - 根元素的结束标记要放在所有其他元素的结束标记之后



1.1 XML basics

1.1.2 XML file structure



3、属性：一个标签可以有多个属性，每个属性都有它自己的 名称 和 取值

- 属性语法：<元素名 属性名=“属性值”/>，举例：

```
<Student ID=“S100”>
    <Name>Tom</Name>
</Student>
```

1.1.2 XML file structure

- **注意**：
  - 属性值要用双引号（"）或单引号（'）引起来
  - 定义属性必须遵循与标签相同的**命名规范**，即
    - 名称中可以包含字母、数字或者其他的字符
    - 名称不能以数字或者标点符号开头
    - 名称中不能包含空格
    - 名称中间不能包含冒号
- 一个元素可以有多个属性，它的基本格式为：<元素名 属性名=“属性值” 属性名=“属性值”>

4、注释：XML文件中的注释语法：

<!--注释-->





2

ROS Filesystem(2)

2.1 The filesystem level

2.1.1 Catkin package

- **一个功能包要想称为 catkin package，必须符合以下要求：**

  a) **这个功能包必须有一个符合 catkin 规范的 package.xml 文件**

  - **package.xml 文件提供有关该功能包的基础信息**

  b) **这个包必须有一个 catkin 版本的 CMakeLists.txt 文件**

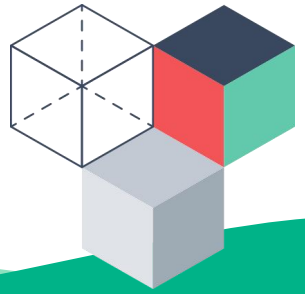
  - **CMakeLists.txt 文件保存 Cmake 的配置信息，如果是 Catkin 元功能包的话，则需要有一个 CMakeList.txt 文件的相关样板**

  c) **每个包必须有自己的目录**

  - **在同一个目录下不能有嵌套的或者多个功能包存在**

- **最简单的 catkin 功能包示例：**

```
my_package/
  CMakeLists.txt
  package.xml
```

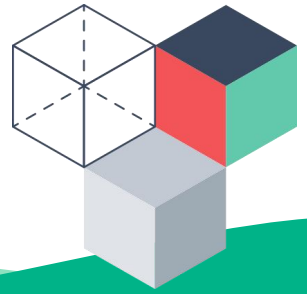


2.1 The filesystem level

2.1.1 Catkin package

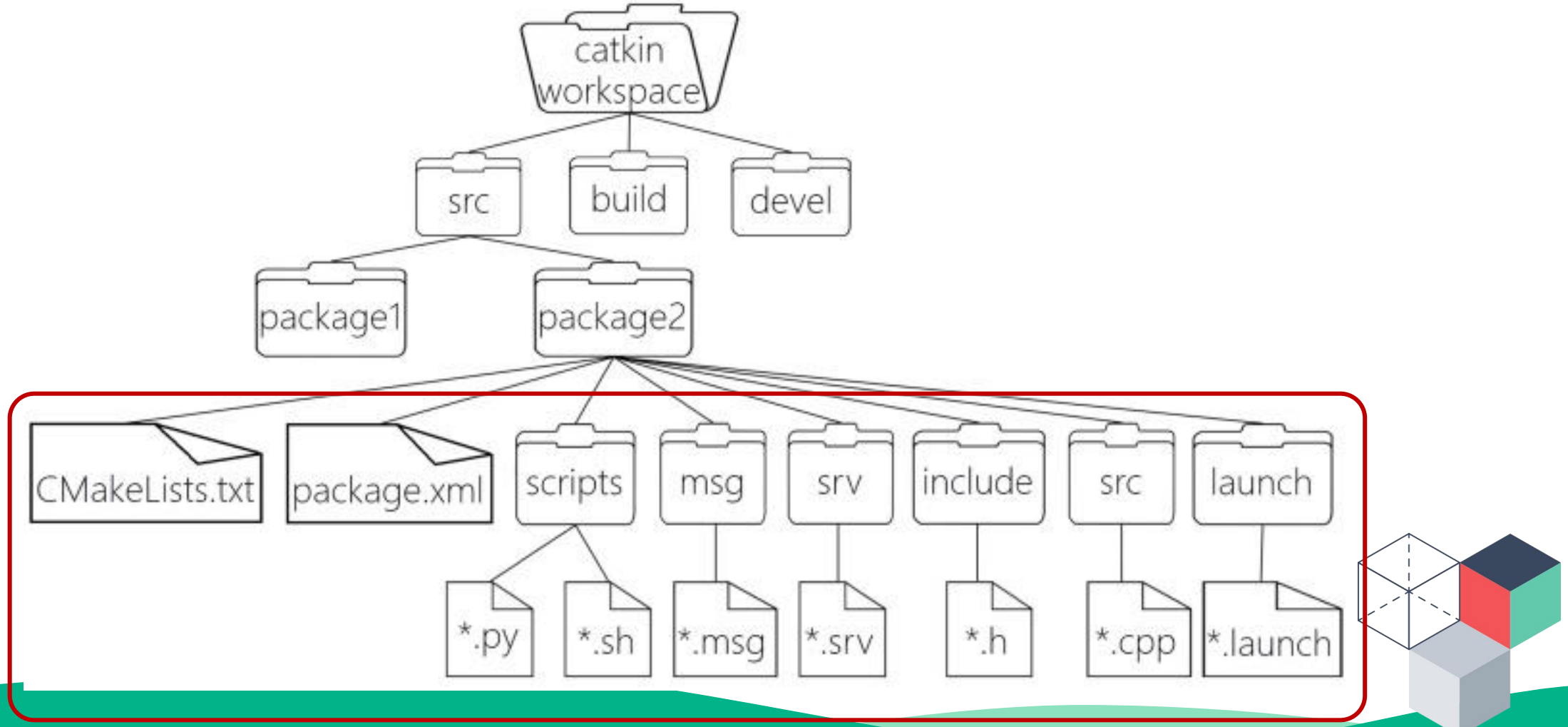
- catkin 工作空间中的功能包的结构：

```
workspace_folder/                   -- WORKSPACE
    src/                            -- SOURCE SPACE
        CMakeLists.txt              -- 'Toplevel' CMake file, provided by catkin
        package_1/
            CMakeLists.txt          -- CMakeLists.txt file for package_1
            package.xml             -- Package manifest for package_1
        ...
        package_n/
            CMakeLists.txt          -- CMakeLists.txt file for package_n
            package.xml             -- Package manifest for package_n
```



2.1 The filesystem level

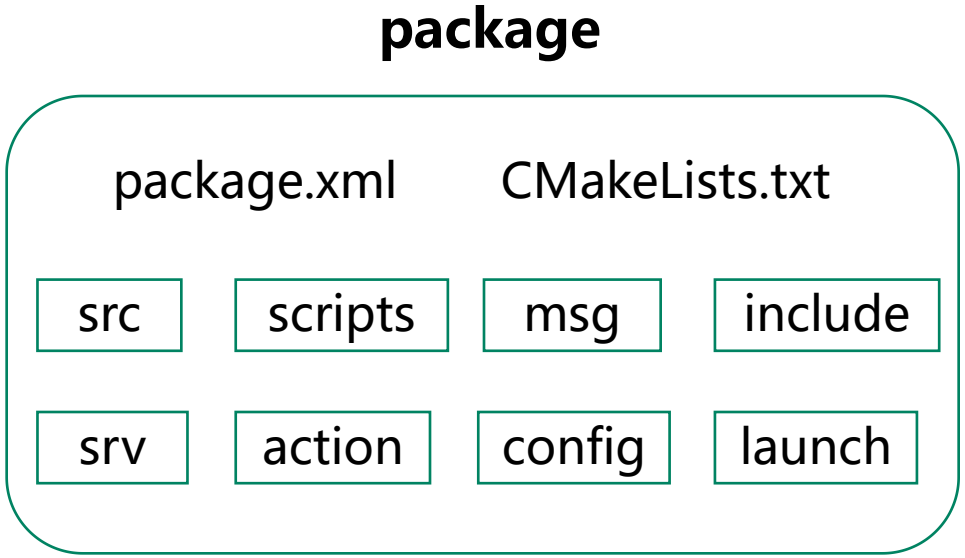
2.1.2 Catkin package structures

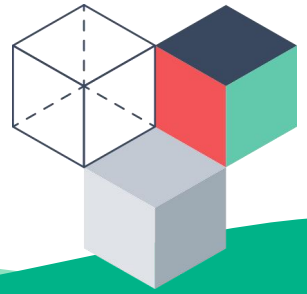


2.1 The filesystem level

2.1.2 Catkin package structures

- **功能包的一般结构如下所示：**
  - **config/: 保存了当前ROS包中使用的所有配置文件，由用户创建**
  - **include/package_name/: 此目录包含了所用库的头文件**
  - **msg/: 包含自定义的消息类型文件**
  - **scripts/: 包括 Bash、Pyhton 或其他脚本的可执行脚本文件**
  - **src/: 存储源文件的文件夹，例如为节点和nodelet创建的文件夹**
  - **srv/: 储存服务类型文件**
  - **action/: 储存动作类型文件**
  - **launch/: 保存一个或者多个ROS节点的启动文件**
  - **CMakeLists.txt: CMake的生成文件**
  - **package.xml: 功能包清单文件（必须放在功能包文件夹中，如果你发现某个文件夹中包含package.xml文件，那么这个文件夹很可能是功能包或者元功能包）**





2.1 The filesystem level

2.1.3 package.xml : http://wiki.ros.org/catkin/package.xml

1. 功能包清单：

- 它必须包含在任何符合 catkin 标准的包的根文件夹中
- 该文件定义了有关功能包的属性，例如包名称、版本号、作者、维护者以及对其他 catkin 包的依赖关系
- 该概念类似于旧版 rosbuild 构建系统中使用的 manifest.xml 文件
- 作用：package.xml 中声明功能包依赖项。在本机中，可以从源代码构建并在自己的机器上运行测试，但是将功能包在发布到 ROS 社区时将无法正常工作，其他人则依赖此信息来安装他们使用你的软件包所需的软件

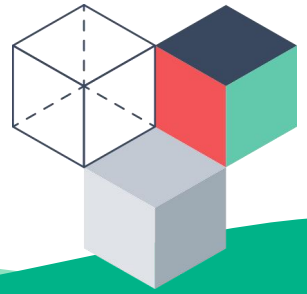
➢ 基本结构：每个 package.xml 文件都有 <package> 标记作为文档中的根元素

<package format="2">

<Name>元素</Name>

</package>

作业：学习 format1 和 format2 的用法和含义



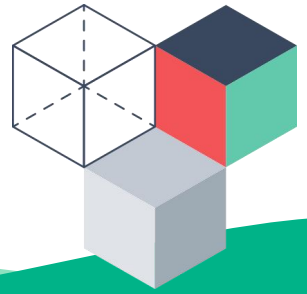
2.1 The filesystem level

2.1.3 package.xml

2. 必备标签：嵌套在 <package> 标签中最少的标签，以使包清单完整

- <name> - The name of the package
- <version> - The version number of the package (required to be 3 dot-separated integers)
- <description> - A description of the package contents
- <maintainer> - The name of the person(s) that is/are maintaining the package
- <license> - The software license(s) (e.g. GPL, BSD, ASL) under which the code is released

```
<package format="2">
  <name>foo_core</name>
  <version>1.2.4</version>
  <description>
  This package provides foo capability.
  </description>
  <maintainer email="ivana@osrf.org">Ivana Bildbotz</maintainer>
  <license>BSD</license>
</package>
```



2.1 The filesystem level

2.1.3 package.xml

- **依赖项：功能包可以有六种类型的依赖项**
  - **Build Dependencies specify which packages are needed to build this package. This is the case when any file from these packages is required at build time. This can be including headers from these packages at compilation time, linking against libraries from these packages or requiring any other resource at build time (especially when these packages are find_package()-ed in CMake). In a cross-compilation scenario build dependencies are for the targeted architecture.**
  - **Build Export Dependencies specify which packages are needed to build libraries against this package. This is the case when you transitively include their headers in public headers in this package (especially when these packages are declared as (CATKIN_)DEPENDS in catkin_package() in CMake).**
  - **Execution Dependencies specify which packages are needed to run code in this package. This is the case when you depend on shared libraries in this package (especially when these packages are declared as (CATKIN_)DEPENDS in catkin_package() in CMake).**

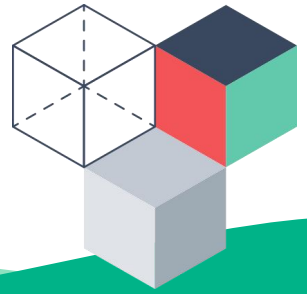


2.1 The filesystem level

2.1.3 package.xml

- **依赖项：功能包可以有六种类型的依赖项**

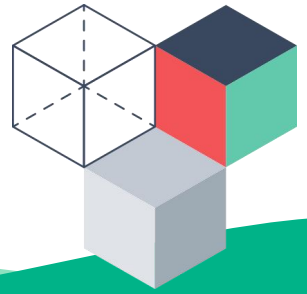
  - Test Dependencies specify only additional dependencies for unit tests. They should never duplicate any dependencies already mentioned as build or run dependencies.
  - Build Tool Dependencies specify build system tools which this package needs to build itself. Typically the only build tool needed is catkin. In a cross-compilation scenario build tool dependencies are for the architecture on which the compilation is performed.
  - Documentation Tool Dependencies specify documentation tools which this package needs to generate documentation.



2.1 The filesystem level

2.1.3 package.xml

- 依赖项标签：
  - <depend> specifies that a dependency is a build, export, and execution dependency. This is the most commonly used dependency tag.
  - <build_depend> → 编译依赖项，Build Dependencies
  - <build_export_depend> → 导出依赖项，Build Export Dependencies
  - <exec_depend> → 运行依赖项，Execution Dependencies
  - <test_depend> → 测试用例依赖项，Test Dependencies
  - <buildtool_depend> → 编译构建工具，Build Tool Dependencies
  - <doc_depend> → 文档依赖项，Documentation Tool Dependencies



2.1 The filesystem level

2.1.3 package.xml

- **A typical format2 package.xml:**

```
<package format="2">
  <name>foo_core</name>
  <version>1.2.4</version>
  <description>
    This package provides foo capability.
  </description>
  <maintainer email="ivana@willowgarage.com">Ivana Bildbotz</maintainer>
  <license>BSD</license>

  <url>http://ros.org/wiki/foo_core</url>
  <author>Ivana Bildbotz</author>

  <buildtool_depend>catkin</buildtool_depend>

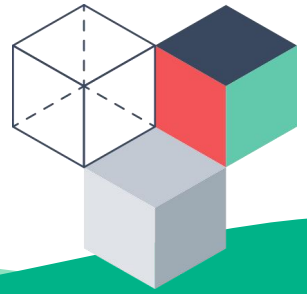
  <depend>roscpp</depend>
  <depend>std_msgs</depend>

  <build_depend>message_generation</build_depend>

  <exec_depend>message_runtime</exec_depend>
  <exec_depend>rospy</exec_depend>

  <test_depend>python-mock</test_depend>

  <doc_depend>doxygen</doc_depend>
</package>
```



2.1 The filesystem level

2.1.3 package.xml

- **Turtlesim package.xml**:

```
ros@ROS-VirtualBox:~$ roscd turtlesim
ros@ROS-VirtualBox:/opt/ros/melodic/share/turtlesim$ cat package.xml
<?xml version="1.0"?>
<package>
  <name>turtlesim</name>
  <version>0.9.3</version>
  <description>
    turtlesim is a tool made for teaching ROS and ROS packages.
  </description>
  <maintainer email="dthomas@osrfoundation.org">Dirk Thomas</maintainer>
  <license>BSD</license>

  <url type="website">http://www.ros.org/wiki/turtlesim</url>
  <url type="bugtracker">https://github.com/ros/ros_tutorials/issues</url>
  <url type="repository">https://github.com/ros/ros_tutorials</url>
  <author>Josh Faust</author>

  <buildtool_depend>catkin</buildtool_depend>

  <build_depend>geometry_msgs</build_depend>
  <build_depend>libboost-thread-dev</build_depend>
  <build_depend>qtbase5-dev</build_depend>
  <build_depend>message_generation</build_depend>
  <build_depend>qt5-qmake</build_depend>
  <build_depend>rosconsole</build_depend>
  <build_depend>roscpp</build_depend>
  <build_depend>roscpp_serialization</build_depend>
  <build_depend>roslib</build_depend>
  <build_depend>rostime</build_depend>
  <build_depend>std_msgs</build_depend>
  <build_depend>std_srvs</build_depend>

  <run_depend>geometry_msgs</run_depend>
  <run_depend>libboost-thread-dev</run_depend>
  <run_depend>libqt5-core</run_depend>
  <run_depend>libqt5-gui</run_depend>
  <run_depend>message_runtime</run_depend>
  <run_depend>rosconsole</run_depend>
  <run_depend>roscpp</run_depend>
  <run_depend>roscpp_serialization</run_depend>
  <run_depend>roslib</run_depend>
  <run_depend>rostime</run_depend>
  <run_depend>std_msgs</run_depend>
  <run_depend>std_srvs</run_depend>
</package>
```



2.1.3 package.xml

3. 自定义功能包：

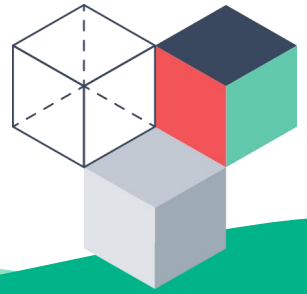
① 自定义 package.xml

- **描述标签**

```
切换行号显示

    5   <description>The beginner_tutorials package</description>
```

- **描述信息提供功能包的作用及使用时的注意事项等内容**
- **如果用一句话难以描述完全则需要换行**



2.1 The filesystem level

2.1.3 package.xml

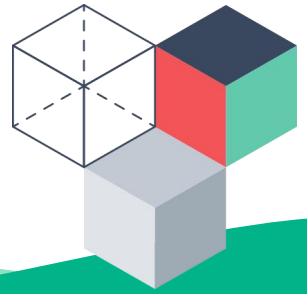
3. 自定义功能包：

- **维护者标签**

```
切换行号显示

 7   <!-- One maintainer tag required, multiple allowed, one person per tag -->
 8   <!-- Example:  -->
 9   <!-- <maintainer email="jane.doe@example.com">Jane Doe</maintainer> -->
10   <maintainer email="user@todo.todo">user</maintainer>
```

- **至少需要填写一个维护者，可以添加多个**
- **除了在标签里面填写维护者的名字外，还应该在标签的 email 属性中填写电子邮件地址**



2.1.3 package.xml

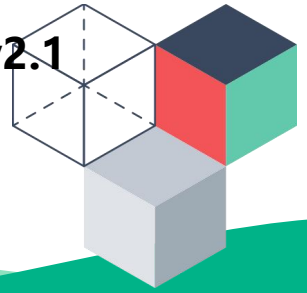
3. 自定义功能包：

- 许可证标签

```
切换行号显示

12   <!-- One license tag required, multiple allowed, one license per tag -->
13   <!-- Commonly used license strings: -->
14   <!--   BSD, MIT, Boost Software License, GPLv2, GPLv3, LGPLv2.1, LGPLv3 -->
15   <license>TODO</license>
```

- 选择一种许可证并将它填写到这里
- 常见的开源许可协议有 BSD、MIT、Boost Software License、GPLv2、GPLv3、LGPLv2.1和LGPLv3，可以在 Open Source Initiative 中阅读其中的若干个许可协议的相关信息



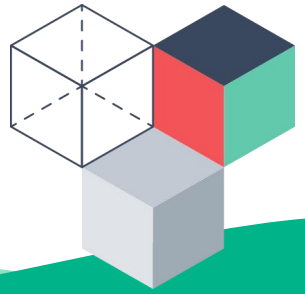
2.1.3 package.xml

3. 自定义功能包：

- **依赖项标签**

切换行号显示

```
27  <!-- The *_depend tags are used to specify dependencies -->
28  <!-- Dependencies can be catkin packages or system dependencies -->
29  <!-- Examples: -->
30  <!-- Use build_depend for packages you need at compile time: -->
31  <!--   <build_depend>genmsg</build_depend> -->
32  <!-- Use buildtool_depend for build tool packages: -->
33  <!--   <buildtool_depend>catkin</buildtool_depend> -->
34  <!-- Use exec_depend for packages you need at runtime: -->
35  <!--   <exec_depend>python-yaml</exec_depend> -->
36  <!-- Use test_depend for packages you need only for testing: -->
37  <!--   <test_depend>gtest</test_depend> -->
38  <buildtool_depend>catkin</buildtool_depend>
39  <build_depend>roscpp</build_depend>
40  <build_depend>rospy</build_depend>
41  <build_depend>std_msgs</build_depend>
```



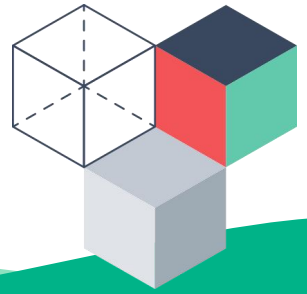
2.1.3 package.xml

3. 自定义功能包：

- 依赖项标签

```
38   <buildtool_depend>catkin</buildtool_depend>
39   <build_depend>roscpp</build_depend>
40   <build_depend>rospy</build_depend>
41   <build_depend>std_msgs</build_depend>
```

- 此标签描述了功能包的依赖关系，这些依赖项分为 build_depend、buildtool_depend、exec_depend、test_depend
- 创建功能包时将 roscpp、rospy 和 std_msgs 作为 catkin_create_pkg 命令的参数



2.1.3 package.xml

3. 自定义功能包：

- 依赖项标签

切换行号显示

```
12  <buildtool_depend>catkin</buildtool_depend>
13
14  <build_depend>roscpp</build_depend>
15  <build_depend>rospy</build_depend>
16  <build_depend>std_msgs</build_depend>
17
18  <exec_depend>roscpp</exec_depend>
19  <exec_depend>rospy</exec_depend>
20  <exec_depend>std_msgs</exec_depend>
```

- 在本例中，因为在编译和运行时都需要用到所有指定的依赖包，因此将每一个依赖包分别添加到 run_depend 标签中



2.1 The filesystem level

2.1.3 package.xml

切换行号显示

```
<?xml version="1.0"?>
<package format="2">
  <name>beginner_tutorials</name>
  <version>0.1.0</version>
  <description>The beginner_tutorials package</description>

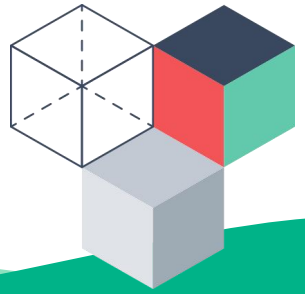
  <maintainer email="you@yourdomain.tld">Your Name</maintainer>
  <license>BSD</license>
  <url type="website">http://wiki.ros.org/beginner_tutorials</url>
  <author email="you@yourdomain.tld">Jane Doe</author>

  <buildtool_depend>catkin</buildtool_depend>

  <build_depend>roscpp</build_depend>
  <build_depend>rospy</build_depend>
  <build_depend>std_msgs</build_depend>

  <exec_depend>roscpp</exec_depend>
  <exec_depend>rospy</exec_depend>
  <exec_depend>std_msgs</exec_depend>

</package>
```



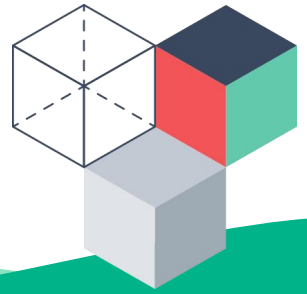
完整的package.xml文件

2.1 The filesystem level

2.1.4 CMakeList.txt：http://wiki.ros.org/catkin/CMakeLists.txt

1. Overview：

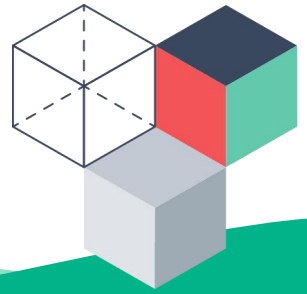
- CMakeLists.txt 是用于构建软件包的 CMake 构建系统的输入
- 任何符合 CMake 的功能包都包含一个或多个 CMakeLists.txt 文件，这些文件描述了如何构建代码以及将代码安装到何处
- 用于 catkin 项目的 CMakeLists.txt 文件是带有一些附加约束的标准 vanilla CMakeLists.txt 文件



2.1.4 CMakeList.txt

2. Overall Structure and Ordering：作业：自行学习，下节课先测试，再讲解

- Required CMake Version (cmake_minimum_required)
- Package Name (project())
- Find other CMake/Catkin packages needed for build (find_package())
- Enable Python module support (catkin_python_setup())
- Message/Service/Action Generators (add_message_files(), add_service_files(), add_action_files())
- Invoke message/service/action generation (generate_messages())
- Specify package build info export (catkin_package())
- Libraries/Executables to build (add_library()/add_executable()/target_link_libraries())
- Tests to build (catkin_add_gtest())
- Install rules (install())



2.1 The filesystem level

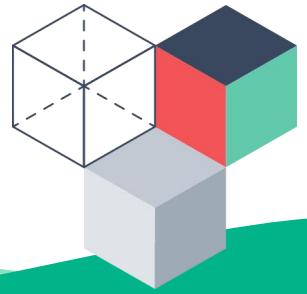
2.1.5 Metapackages

- **元功能包**：将多个功能包分组为单个逻辑包。 元功能包是一个普通功能包，在 package.xml 中具有以下导出标记：

```
<export>
    <metapackage />
</export>
```

  - 除了必需的 <buildtool_depends> 对 catkin 的依赖之外，元功能包只能对它们分组的包具有执行依赖关系
  - 元功能包有一个必需的样板 CMakeLists.txt 文件：

```
cmake_minimum_required(VERSION 2.8.3)
project(<PACKAGE_NAME>)
find_package(catkin REQUIRED)
catkin_metapackage()
```



Post-assessment

1. XML 文档声明包括哪些内容?
2. XML 文档元素的特征是?
3. Catkin 功能包的最小结构?
4. Package.xml 和 CMakelists.txt 分别有什么作用?
5. Catkin 功能包中 src、srv、msg、launch 各存放什么文件?
6. Package.xml 的6个依赖项标签分别是?

Summary

1. XML文件：文件结构（文档声明、元素、属性、注释）
2. ROS文件系统
    a) 文件系统架构 
    b) 功能包架构
    c) rosbash
    d) catkin功能包 
3. package.xml： 作用、基本结构、必备标签、依赖项 

Overview lesson 6-3

1. Message and Service
2. ~
3. ~
4. ~

Objectives

1、掌握 ROS Message 的定义和使用方法

2、掌握 ROS Service 的定义和使用方法

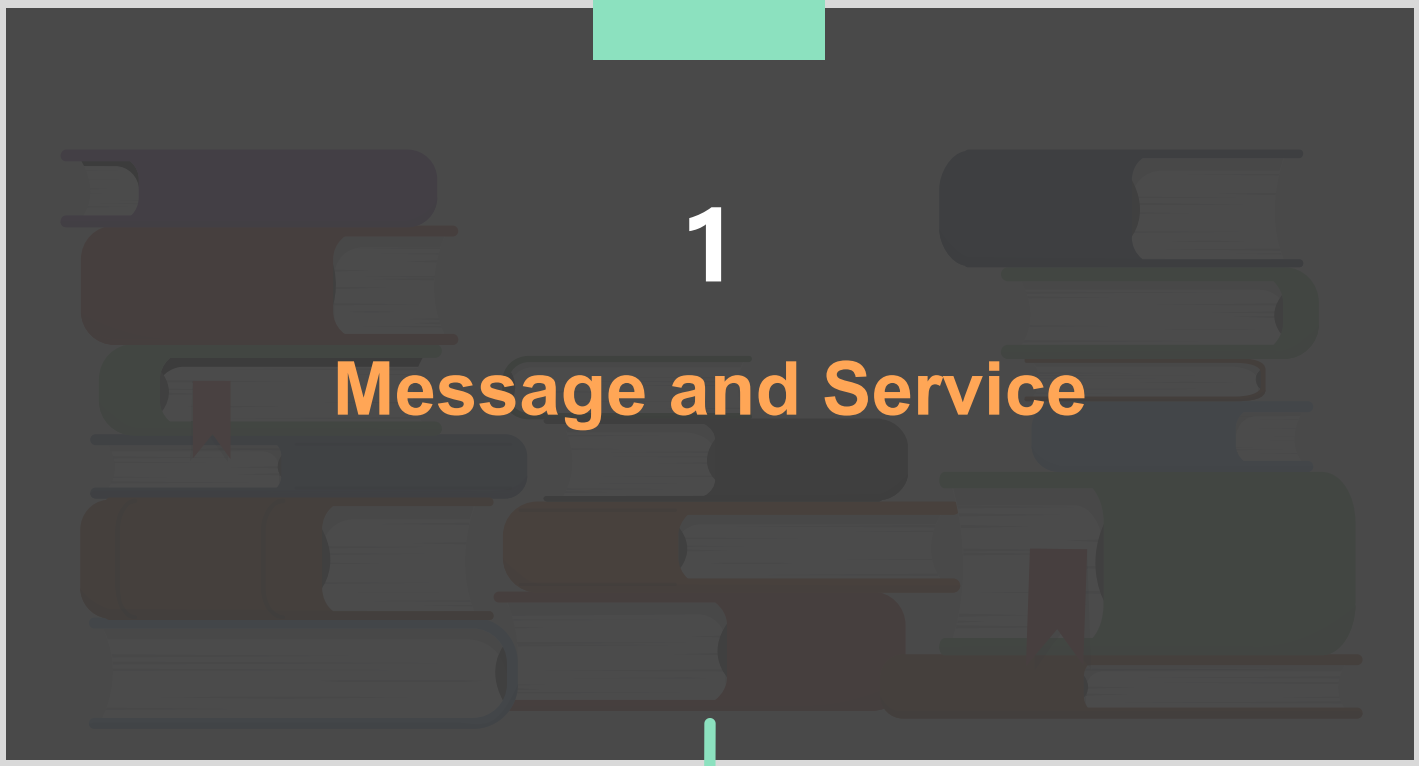


Pre-assessment

1、是否了解和使用过 ROS Message?

2、是否了解和使用过 ROS Service?

3、了解 ROS IDE?



1

Message and Service

1.1 ROS Message

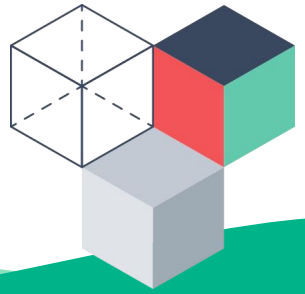
1.1.1 Overview： http://wiki.ros.org/msg

- ROS 使用简化的 消息描述语言 来描述 ROS 消息类型（也称为消息）
- ROS消息的作用是描述节点发布的数据值
- 消息描述存储在 ROS 软件包的 msg 文件夹下，以 .msg 为扩展名
- 消息类型有2种方式：字段(Fields) 和 常量(Constants)
  - 字段：定义了在消息中传输的数据类型，包含字段类型(Field Type)和字段名称(Field Name)2部分，如 int32 x、float32 y、string s 等
  - 常量：定义了可用于解释这些字段的值
- 每个字段由一个字段类型和一个字段名称组成，用空格分隔，其语法格式为：

```
fieldtype1 fieldname1
fieldtype2 fieldname2
```

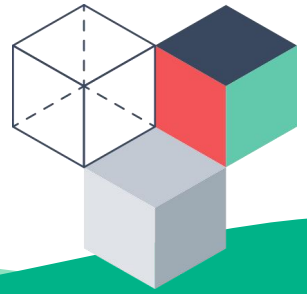
- 举例：

```
int32 x
int32 y
```



1.1.2 Fields——Field Type

- 字段类型可以是以下四种之一：
  a) 内置类型，例如“float32 pan”或“string name”
  b) 自定义的消息描述的名称，例如“geometry_msgs/PoseStamped”
  c) 上述的固定或可变长度数组（列表），例如“float32[] ranges”或“Point32[10] points”
  d) 特殊的 Header 类型，该类型映射到 std_msgs/Header
- 嵌入其他消息描述时，如果在同一个功能包中，类型名称可能是相对的（例如“Point32”）；否则它必须是完整的消息类型（例如“std_msgs/String”）。 此规则的唯一例外是 Header
- 注意：在构建自己的消息类型时，不得使用内置类型或 Header 的名称

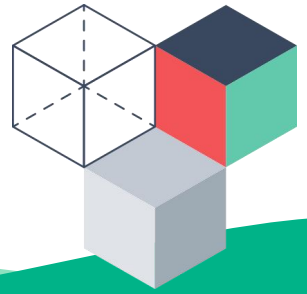


1.1 ROS Message

1.1.2 Fields——Field Type

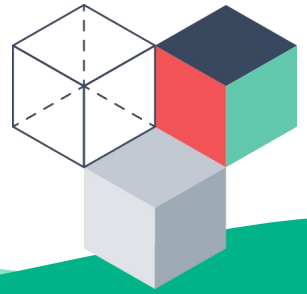
常用字段类型：字段类型与 C++ 和 Python 中的对应关系

Primitive Type	Serialization	C++	Python2 / Python3
bool	unsigned 8-bit int	uint8_t	bool
int8	signed 8-bit int	int8_t	int
uint8	unsigned 8-bit int	uint8_t	int
int16	signed 16-bit int	int16_t	int
uint16	unsigned 16-bit int	uint16_t	int
int32	signed 32-bit int	int32_t	int
uint32	unsigned 32-bit int	uint32_t	int
int64	signed 64-bit int	int64_t	long int
uint64	unsigned 64-bit int	uint64_t	long int
float32	32-bit IEEE float	float	float
float64	64-bit IEEE float	double	float
string	ascii string	std::string	str bytes
time	secs/nsecs unsigned 32-bit ints	ros::Time	rospy.Time
duration	secs/nsecs signed 32-bit ints	ros::Duration	rospy.Duration



1.1.2 Fields——Field Name

- **字段名称**决定了如何在目标语言中引用数据值
  - 例如，一个名为“pan”的字段名称在 Python 中将被引用为“obj.pan”，假设“obj”是存储消息的变量
- **字段名称**需要由消息生成器翻译成多种目标语言，因此
  - 将字段名称限制为字母字符后跟字母数字和下划线的任意混合，即 [a-zA-Z][a-zA-Z1-9_]*
  - **避免**使用与常用语言中的**关键字**相对应的字段名称



1.1 ROS Message

1.1.3 Constants

- 常量定义与字段描述类似，区别是常量定义会分配一个值，此值分配通过使用等号“=”表示

  constanttype1 CONSTANTNAME1=constantvalue1

  constanttype2 CONSTANTNAME2=constantvalue2

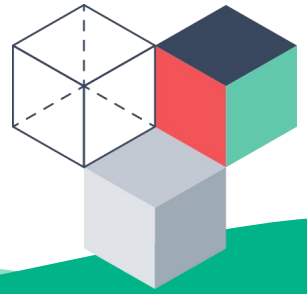
- 举例：

  int32 X=123

  int32 Y=-123

  string FOO=“foo”

- 注意：
  - 除了time和duration，可以将任何内置类型声明为常量
  - string 常量被赋予等号右侧所有内容的值，去除前导和尾随空格。不能对 string 常量添加注释
  - 整数常量必须以十进制（基数为 10）指定

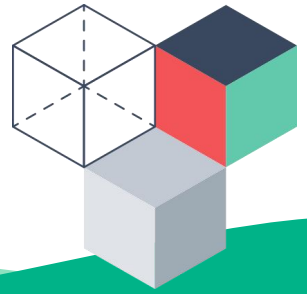


1.1 ROS Message

1.1.4 Common_msgs

- 除了内置类型，ROS已经将常用到的一些消息。如传感器消息、导航消息等设置成了不同的消息类型；common_msgs 是各种常用的消息包的集合，主要包括：

1. actionlib_msgs
2. diagnostic_msgs
3. geometry_msgs → rosmsg show geometry_msgs/TAB
4. nav_msgs
5. sensor_msgs → 参考：https://blog.csdn.net/qq_36764147/article/details/103100498
6. shape_msgs
7. stereo_msgs
8. trajectory_msgs
9. visualization_msgs



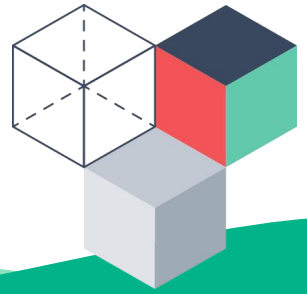
1.1 ROS Message

1.1.5 Geometry_msgs：https://wiki.ros.org/geometry_msgs?distro=melodic

- geometry_msgs 为常见的几何基元（例如点、向量和姿势）提供消息。 这些几何基元旨在提供通用数据类型并促进整个系统的互操作性

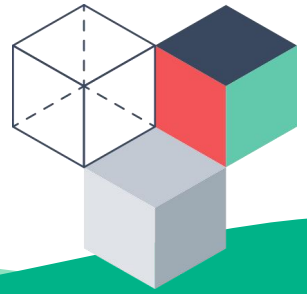
- geometry_msgs message types:

Accel
AccelStamped
AccelWithCovariance
AccelWithCovarianceStamped
Inertia
InertiaStamped
Point
Point32
PointStamped
Polygon
PolygonStamped
Pose
Pose2D
PoseArray
PoseStamped
PoseWithCovariance
PoseWithCovarianceStamped
Quaternion
QuaternionStamped
Transform
TransformStamped
Twist
TwistStamped
TwistWithCovariance
TwistWithCovarianceStamped
Vector3
Vector3Stamped
Wrench
WrenchStamped



1.1.6 Building .msg Files

- **自定义消息类型**:
  - ROS 客户端库通过消息生成器（message generators）将 .msg 文件转换为源代码
  - 对于自定义.msg文件，需要确保 CMake 在你定义.msg文件后重新配置项目
  - 消息生成器必须从所构建脚本中调用，因此需要修改 package.xml 文件
  - 所有 .msg 文件存储在 msg 的目录中，需要编辑 CMakeLists.txt 文件来构建这些内容



1.1 ROS Message

1.1.6 Building .msg Files

- **自定义消息类型：**

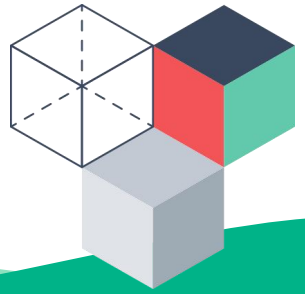
1. 添加 .msg 文件：在 Package 中添加 msg 文件夹，且添加自定义的 test.msg 文件

```
int32 x
int32 y
```

2. 修改 package.xml 文件：打开 package.xml，向其中添加以下两行代码：

```
<build_depend>message_generation</build_depend>
<run_depend>message_runtime</run_depend>
```

```
<buildtool_depend>catkin</buildtool_depend>
<build_depend>roscpp</build_depend>
<build_depend>rospy</build_depend>
<build_depend>std_msgs</build_depend>
<build_depend>message_generation</build_depend>
<build_export_depend>roscpp</build_export_depend>
<build_export_depend>rospy</build_export_depend>
<build_export_depend>std_msgs</build_export_depend>
<exec_depend>roscpp</exec_depend>
<exec_depend>rospy</exec_depend>
<exec_depend>std_msgs</exec_depend>
<exec_depend>message_runtime</exec_depend>
```



1.1.6 Building .msg Files

- **自定义消息类型：**

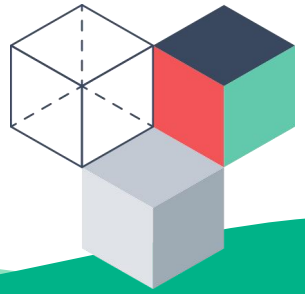
3. **修改 CMakeLists.txt 文件：find_package 和 catkin_package**

修改 find_package：将 message_generation 依赖项添加到 find_package，以便可以生成消息

```
## Find catkin macros and libraries
## if COMPONENTS list like find_package(catkin REQUIRED COMPONENTS xyz)
## is used, also find other catkin packages
find_package(catkin REQUIRED COMPONENTS
  roscpp
  rospy
  std_msgs
  message_generation
)
```

修改catkin_package：

```
catkin_package(
#  INCLUDE_DIRS include
#  LIBRARIES beginner_tutorials
#  CATKIN_DEPENDS roscpp rospy std_msgs
#  DEPENDS system_lib
   CATKIN_DEPENDS message_runtime
)
```



1.1.6 Building .msg Files

- **自定义消息类型：**

4. **修改 CMakeLists.txt 文件：add_message_files 和 generate_messages**

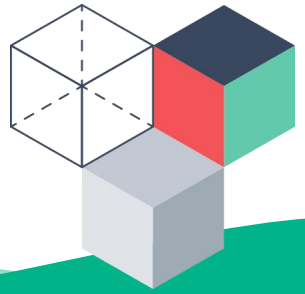
修改 add_message_files：

```
## Generate messages in the 'msg' folder
add_message_files(
  FILES
#   Message1.msg
#   Message2.msg
    test.msg
)
```

修改 generate_messages：

```
## Generate added messages and services with any dependencies listed here
generate_messages(
    DEPENDENCIES
    std_msgs
# )
```

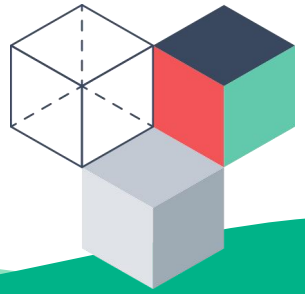
通过自定义 .msg 文件，确保 CMake 在添加其他 .msg 文件后知道如何重新配置项目



1.1 ROS Message

1.1.7 Command-line tools

- 适用于ROS消息的命令工具：rosmsg (ros+message)，用于查看消息类型数据结构的定义
  - rosmsg show：Show message description
  - rosmsg info：Alias for rosmsg show
  - rosmsg list：List all messages
  - rosmsg md5：Display message md5sum
  - rosmsg package：List messages in a package
  - rosmsg packages：List packages that contain messages



1.1 ROS Message

1.1.7 Command-line tools

- **举例说明：**

List messages in a package

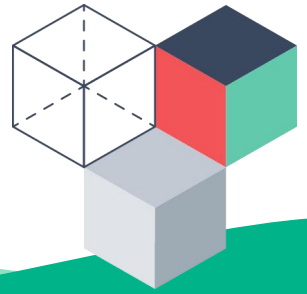
```
ros@ROS-VirtualBox:~/catkin_ws/src/beginner_tutorials$ rosmsg package beginner_t
utorials
beginner_tutorials/test
```

Show message description

```
ros@ROS-VirtualBox:~$ rosmsg show test
[beginner_tutorials/test]:
int32 x
int32 y
```

Show message description

```
ros@ROS-VirtualBox:~$ rosmsg info test
[beginner_tutorials/test]:
int32 x
int32 y
```

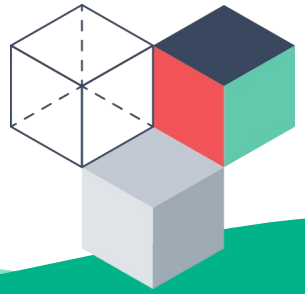


1.2 ROS Service

1.2.1 Overview

- ROS 使用简化的 服务描述语言 来描述 ROS 服务类型
- 服务的描述存储在软件包的 srv 目录下的 .srv 文件中
- ROS服务的作用是实现节点之间的 请求/响应 通信
- 若要调用服务，需要使用 软件包的名称 和 服务名称
  - 举例1： 对于sample_package/srv/sample.srv文件，可将其称为 sample_package/sample 服务（消息类型同样可以使用这种简化方法）
  - 举例2：

```
$ roscervice call /spawn 3 3 0 turtle2
```



1.2.2 Service Description Specification

- 服务描述文件由 请求 和 响应 消息类型组成，以“---”分隔。 用“---”连接在一起的任何两个 .msg 文件都是合法的服务描述
- 一个简单的服务示例，其接受一个字符串，并返回一个字符串：

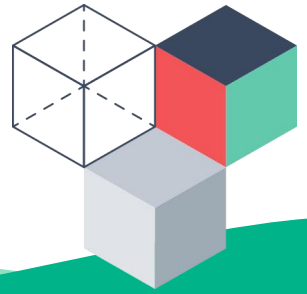
  string str

  ---

  string str

  备注：string 同 ROS message 中的字段类型

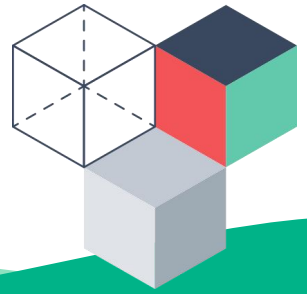
- 不能在 .srv 中嵌入另一个 .srv



1.2.2 Service Description Specification

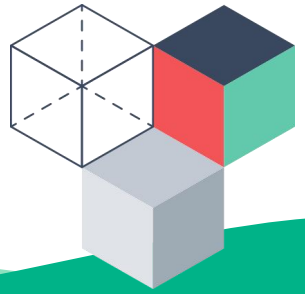
- 一个复杂的服务示例：

```
# request constants
int8 FOO=1
int8 BAR=2
# request fields
int8 foobar
another_pkg/AnotherMessage msg
---
# response constants
uint32 SECRET=123456
# response fields
another_pkg/YetAnotherMessage val
CustomMessageDefinedInThisPackage value
uint32 an_integer
```



1.2.3 Command-line tools

- rossrv 命令行工具显示有关 ROS 服务的信息。 它的用法与 rosmsg 完全相同
  - rossrv show：Show service description
  - rossrv list：List all services
  - rossrv md5：Display service md5sum
  - rossrv package：List services in a package
  - rossrv packages：List packages that contain services



1.2 ROS Service

1.2.3 Command-line tools

- **举例说明：**

List services in a package

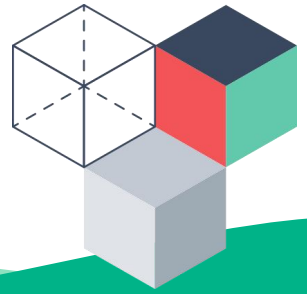
```
ros@ROS-VirtualBox:~/catkin_ws/src/beginner_tutorials/srv$ rossrv package beginner_tutorials
beginner_tutorials/test
```

Show service description

```
ros@ROS-VirtualBox:~/catkin_ws/src/beginner_tutorials/srv$ rossrv show test
[beginner_tutorials/test]:
string str
---
string str
```

Show service description

```
ros@ROS-VirtualBox:~/catkin_ws/src/beginner_tutorials/srv$ rossrv info test
[beginner_tutorials/test]:
string str
---
string str
```



Post-assessment

1. ROS 消息包括哪2种类型?
2. ROS 消息字段包括哪2部分?
3. 字段和常量的区别是?
4. ROS 消息和服务的作用分别是?
5. ROS 服务有哪2部分组成?

Summary

1. Message and Service

a) ROS Message:

① fields and constants

② command line tools

③ custom message type

b) ROS Service:

① description specification

② command line tools

Overview lesson 7-1

1. CMakeLists.txt Introduction
2. ~
3. ~
4. ~

Objectives

1、掌握 CMakeLists.txt 的编译规则

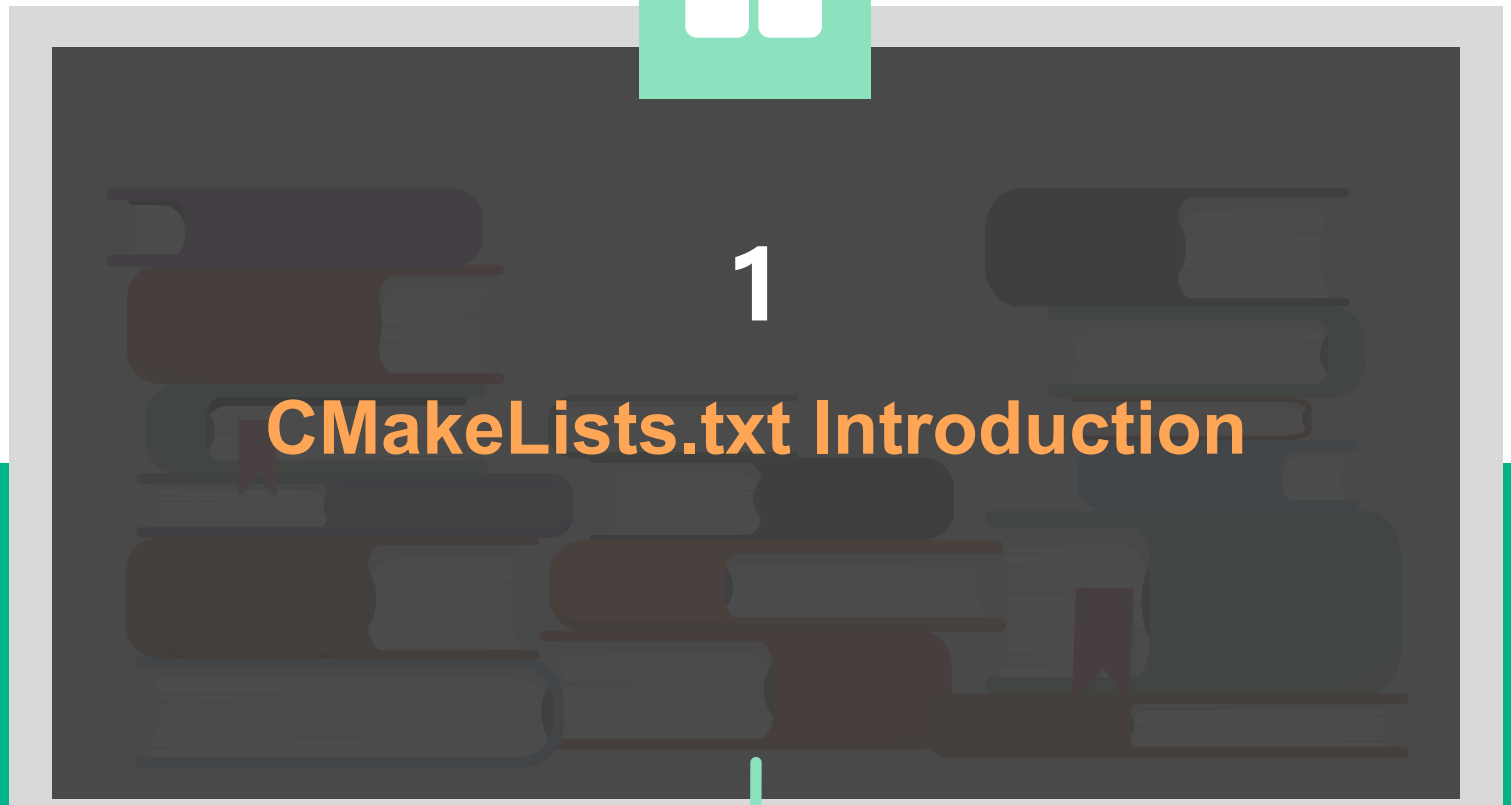
2、掌握 CMakeLists.txt 修改方法



Pre-assessment

1、是否了解 Cmake?

2、是否编写过 CMakeLists.txt?



1

CMakeLists.txt Introduction

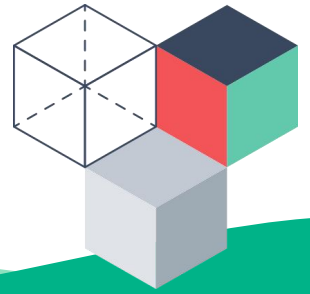
1.1 CMake

1.1.1 Overview： http://wiki.ros.org/catkin/CMakeLists.txt

- Linux 平台编译 C++ 文件，需要编译器和链接器，其中编译器是将源代码编译成目标代码，链接器是将目标代码链接到可执行文件
- 若是编译单个文件，用 g++ 即可；若是编译一个 C++ 工程，需要批处理编译工具，如 make，通过设定规则文件 makefile 调用 g++ 等编译工具进行批量编译
- makefile 的编写十分复杂，便诞生了 CMake，通过编写简单的 CMakeLists.txt 规则文件，就能自动生成 makefile 文件



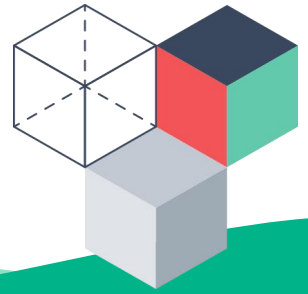
CMake
Cross-platform Make



1.1 CMake

1.1.1 Overview： http://wiki.ros.org/catkin/CMakeLists.txt

- ROS 的编译器是 CMake，ROS 在 CMake 基础上封装了 catkin 命令，用 cmake 命令创建功能包时，会自动生成 CMakeList.txt 文件，已配置了多数编译选项，且包含详细的注释，只需稍作修改便可编译自己的文件
- CMakeList.txt 规定了这个 package 要依赖哪些 package，要编译生成哪些目标，如何编译等由源码到目标文件的规则
- 工作空间两种 CMakelist.txt 文件，一个是针对所有 package（如何将所有 package 功能包生成的可执行文件组成我们想要的目标文件——宏观把握），一个是针对于某一个特定的 package（如何将某一个特定的package功能包中的源代码生成可执行目标文件——细节掌控）

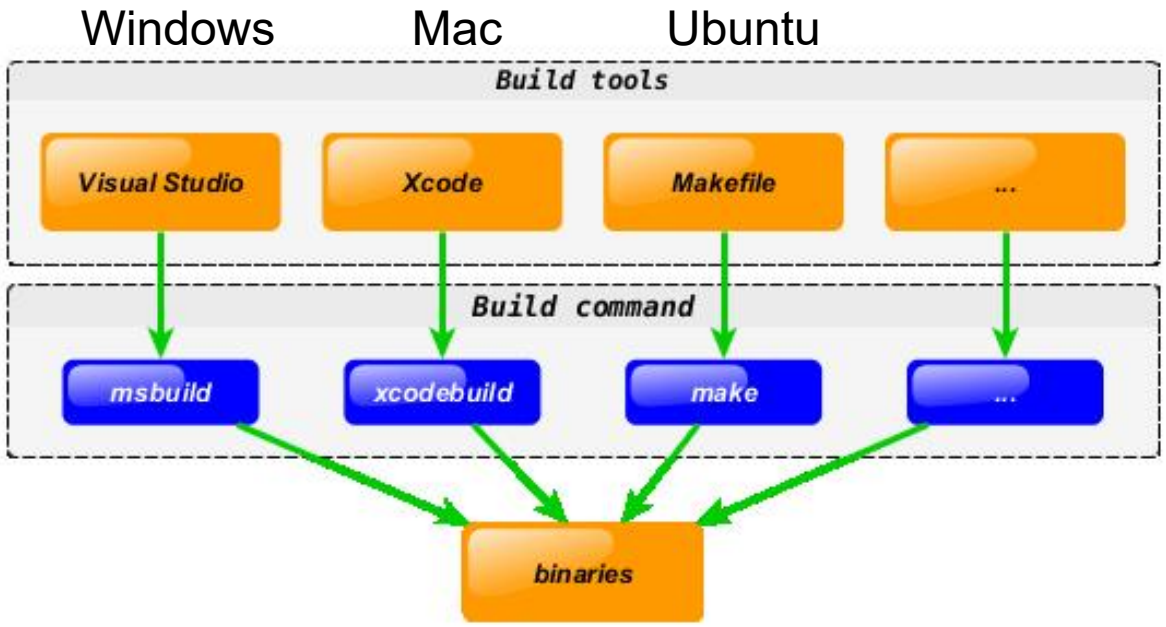


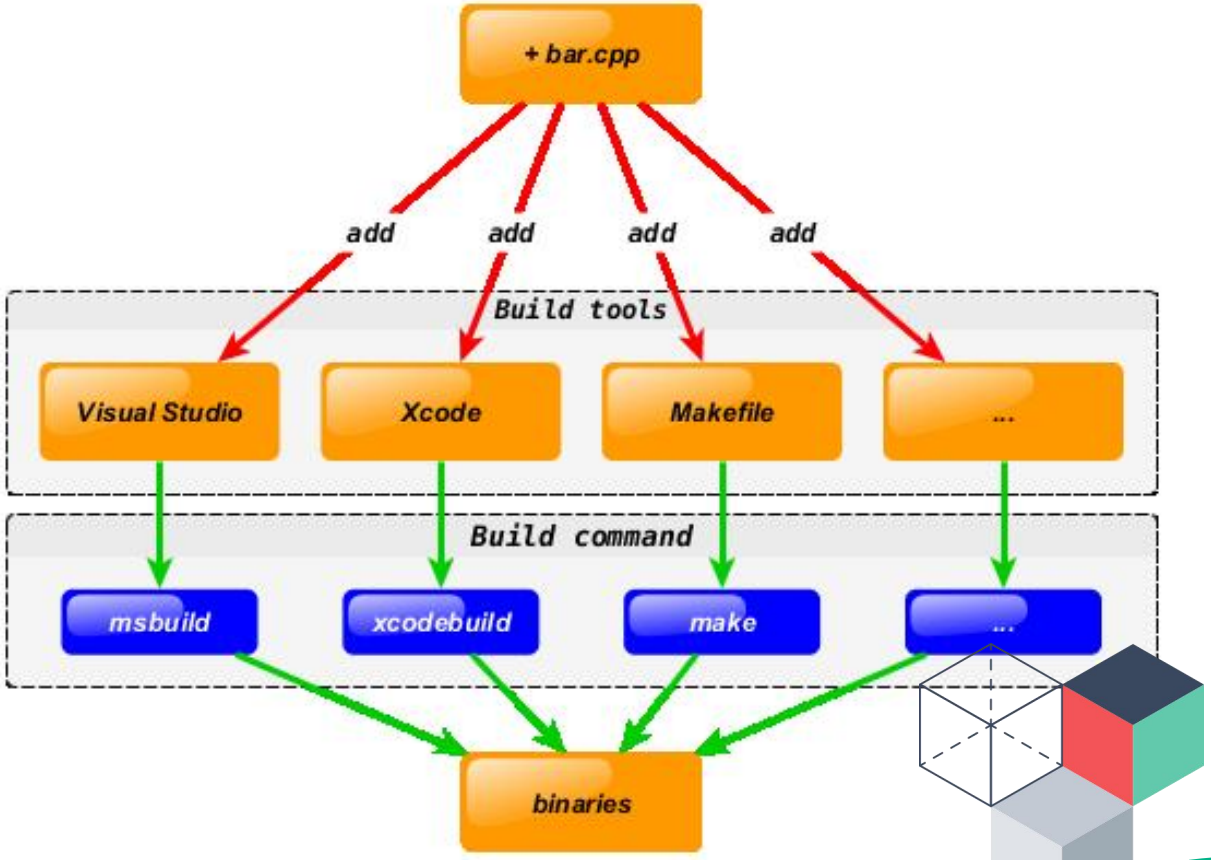
1.1 CMake

1.1.1 Overview： http://wiki.ros.org/catkin/CMakeLists.txt

- CMake是一个跨平台的C/C++编译工具

Let’s assume you have some cross-platform project with C++ code shared along different platforms/IDE.Say you use visual studio on Windows,xcode on Mac and Makefile for Linux

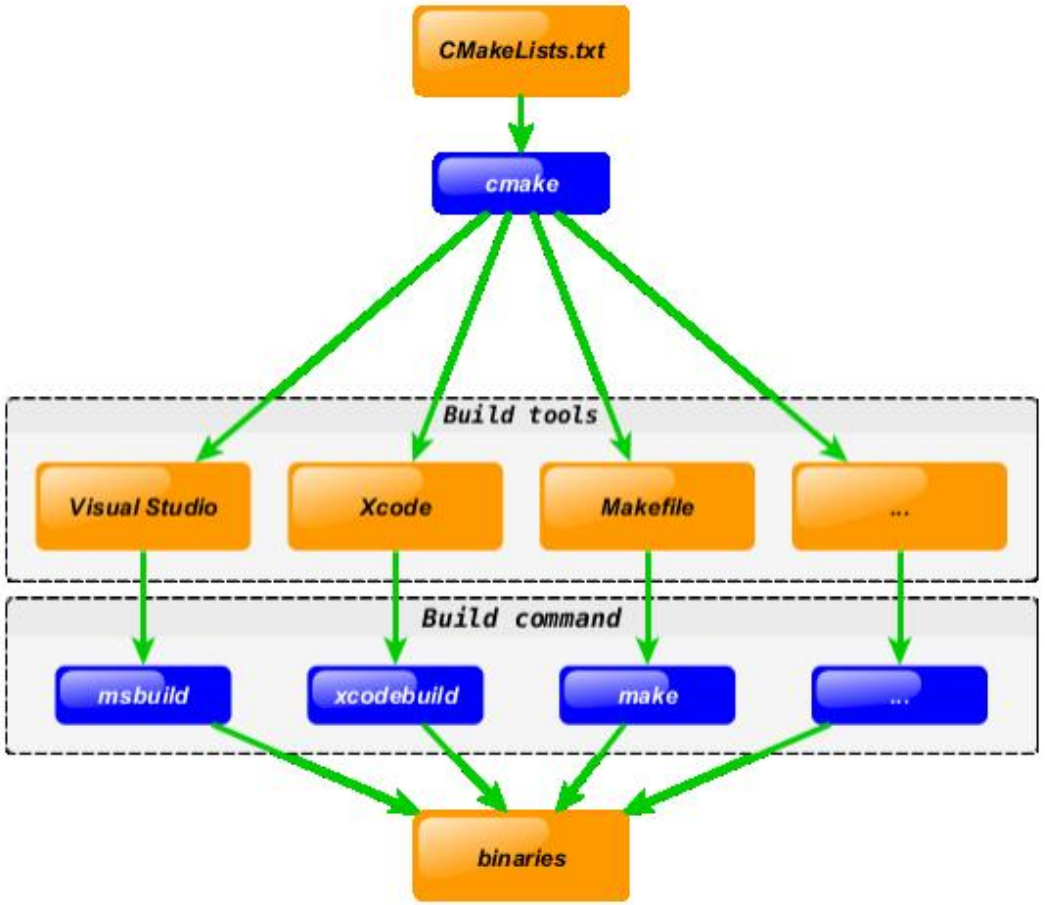


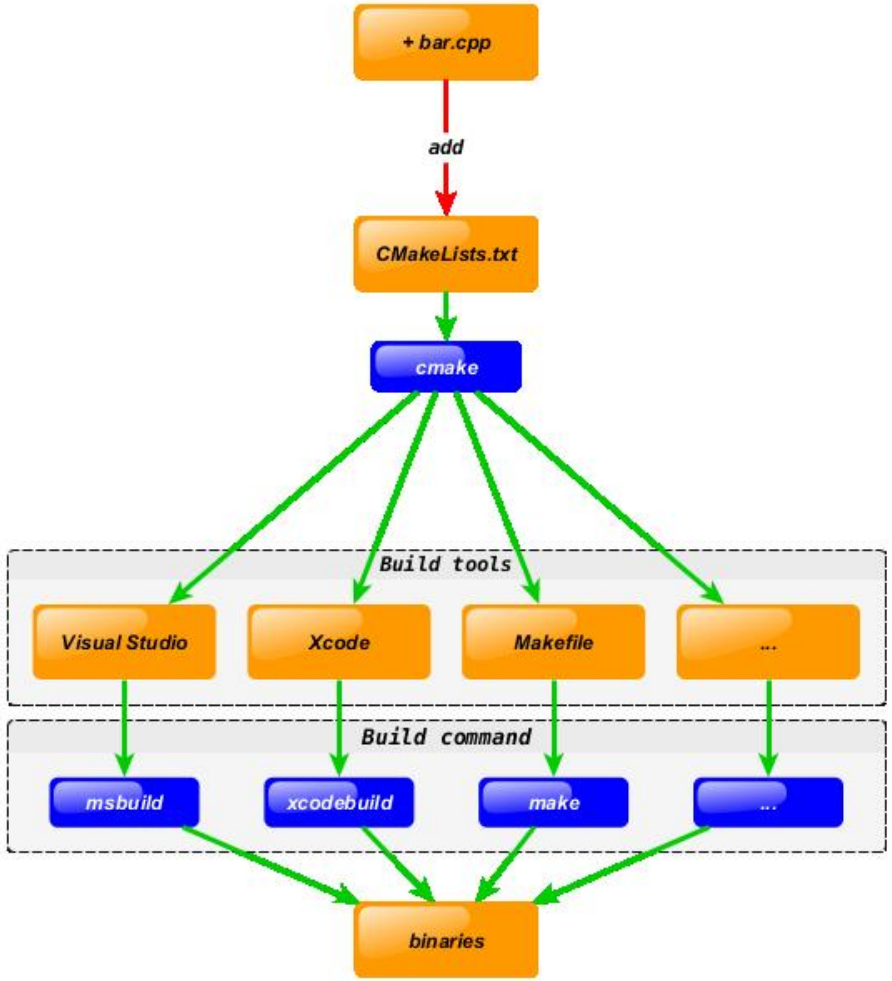


1.1 CMake

1.1.1 Overview： http://wiki.ros.org/catkin/CMakeLists.txt

- CMake可以用简单的语句来描述所有平台的安装/编译过程，自动生成对应的编辑文件

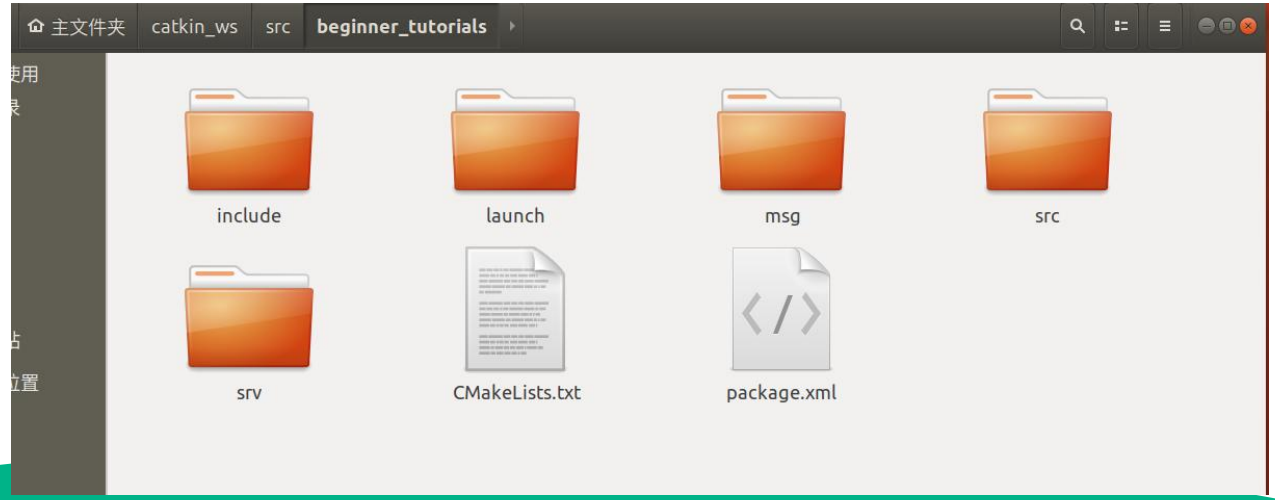




1.1 CMake

1.1.1 Overview： http://wiki.ros.org/catkin/CMakeLists.txt

- 文件 CMakeLists.txt 是用于构建软件包的 CMake 配置信息。任何符合 CMake 的包都包含一个或多个 CMakeLists.txt 文件，这些文件描述了如何构建代码以及将代码安装到何处
- ROS 使用 CMake 进行程序编译的时候，会根据 CMakeLists.txt 文件进行逐步处理，形成一个 Makefile 文件，系统再根据 Makefile 文件的设置进行程序的编译
- 在编写 CMakeLists.txt 时必须遵循特定的格式，否则软件包将无法正确构建和编译， 配置中的顺序不能改变




```
cmake_minimum_required(VERSION 3.0.2)
project(beginner_tutorials)

## Compile as C++11, supported in ROS Kinetic and newer
# add_compile_options(-std=c++11)

## Find catkin macros and libraries
## if COMPONENTS list like find_package(catkin REQUIRED COMPONENTS xyz)
## is used, also find other catkin packages
find_package(catkin REQUIRED COMPONENTS
  roscpp
  rospy
  std_msgs
  message_generation
)

## System dependencies are found with CMake's conventions
# find_package(Boost REQUIRED COMPONENTS system)


## Uncomment this if the package has a setup.py. This macro ensures
## modules and global scripts declared therein get installed
## See http://ros.org/doc/api/catkin/html/user_guide/setup_dot_py.html
# catkin_python_setup()

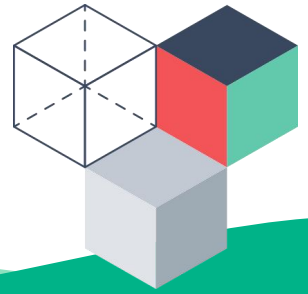
################################################
## Declare ROS messages, services and actions ##
```

1.1 CMake

1.1.1 Overview http://wiki.ros.org/catkin/CMakeLists.txt

- Cmake 语法特性：
  - 基本语法格式：指令(参数 1 参数 2...)
    - 参数使用括号括起来
    - 参数之间使用空格或分号分开
  - 指令是大小写无关的，参数和变量是大小写相关的

```
1 set(HELLO hello.cpp)
2 add_executable(hello main.cpp hello.cpp)
3 ADD_EXECUTABLE(hello main.cpp ${HELLO})
```



1.1 CMake

1.1.1 Overview： http://wiki.ros.org/catkin/CMakeLists.txt

Required CMake Version (cmake_minimum_required)

Package Name (project())

Find other CMake/Catkin packages needed for build (find_package())

Enable Python module support (catkin_python_setup())

Message/Service/Action Generators (add_message_files(), add_service_files(), add_action_files())

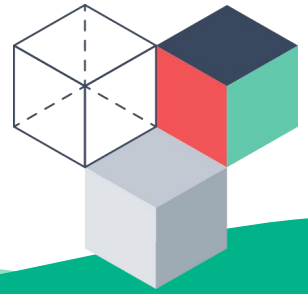
Invoke message/service/action generation (generate_messages())

Specify package build info export (catkin_package())

Libraries/Executables to build (add_library()/add_executable()/target_link_libraries())

Tests to build (catkin_add_gtest())

Install rules (install())



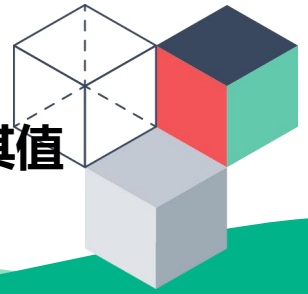
1.1.2 Required CMake Version (cmake_minimum_required)

- cmake_minimum_required - 指定CMake的最小版本要求
- 语法： cmake_minimum_required(VERSION versionNumber [FATAL_ERROR])

  下图表示 Catkin 需要 3.0.2 或更高版本

```
cmake_minimum_required(VERSION 3.0.2)
```

- 这条指令会自动创建两个变量：
  - <projectname>_BINARY_DIR(二进制文件保存路径)
  - <projectname>_SOURCE_DIR（源代码路径）
  - cmake系统也帮助我们预定义了PROJECT_BINARY_DIR和PROJECT_SOURCE_DIR其值与上述对应相等



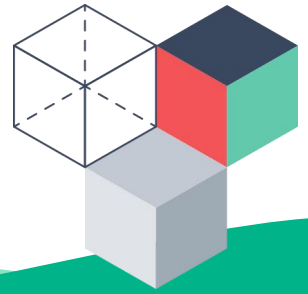
1.1.3 Package Name (project())

- project - 定义工程名称，并可指定工程支持的语言
- 语法： project(projectname [CXX] [C] [Java])

  此处功能包名称为：beginner_tutorials

```
project(beginner_tutorials)
```

- 在 CMake 中指定后，你可在其他地方通过使用变量 ${PROJECT_NAME} 来引用项目名称



1.1.4 Find other CMake/Catkin packages needed for build (find_package())

- CMake 的 find_package 函数用来指定在构建项目过程中依赖了哪些其他的系统依赖包和功能包
- 在 ROS 中，使用 catkin_make 的编译方式，至少需要 catkin 这个包，即：

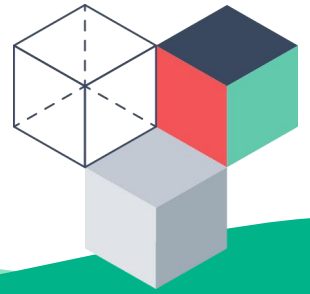
  find_package(catkin REQUIRED)

  catkin 是被搜索的项，REQUIRED表示前面的 catkin 的必须项目
- 包含其他功能包：

  find_package(roscpp REQUIRED)

  find_package(rospy REQUIRED)

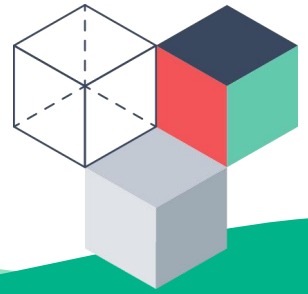
  find_package(std_msgs REQUIRED)



注意：这里添加的包用于构建功能包时使用，而不是添加运行时依赖项

1.1.4 Find other CMake/Catkin packages needed for build (find_package())

- 当一个功能包被 find_package 之后，则会创建多个 CMake 环境变量，提供有关找到功能包的信息
- 环境变量描述了功能包导出头文件的位置、源文件的位置、功能包依赖的库以及这些库的路径等
- 变量按以下格式命名：<PACKAGE NAME>_<PROPERTY>
  - <PACKAGE NAME>_FOUND - Set to true if the library is found, otherwise false
  - <PACKAGE NAME>_INCLUDE_DIRS or <NAME>_INCLUDES - The include paths exported by the package
  - <PACKAGE NAME>_LIBRARIES or <NAME>_LIBS - The libraries exported by the package
  - <PACKAGE NAME>_DEFINITIONS - Add compilation options and macro definitions



1.1.4 Find other CMake/Catkin packages needed for build (find_package())

- 那么，每个依赖的 package 都会产生几个变量，例如：

  roscpp_INCLUDE_DIRS, roscpp_LIBRARIES，rospy_INCLUDE_DIRS, so on

- 以上这种方式对变量的使用和管理都非常不便，常采用另外一种方法，即将依赖的功能包变成 catkin的组件：

```
find_package(catkin REQUIRED COMPONENTS
        roscpp
        rospy
        std_msgs
        message_generation)
```

```
find_package(catkin REQUIRED COMPONENTS
  roscpp
  rospy
  std_msgs
  message_generation
)
```

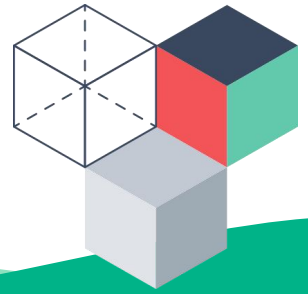
- 如此，所有 pacakge 里面的头文件和库文件等目录变成以 catkin_ 前缀的一组变量上，比如：catkin_INCLUDE_DIRS
- 这意味着 roscpp 导出的包含路径、库等也附加到 catkin_ 变量。 例如，catkin_INCLUDE_DIRS 不仅包含 catkin 的包含路径，还包含 roscpp 的包含路径



1.1.5 Enable Python module support (catkin_python_setup())

- 如果 catkin 包使用一些 Python 模块，应该创建一个 setup.py 文件，并在调用 catkin_python_setup() 之后调用 generate_messages() 和 catkin_package()

```
## Uncomment this if the package has a setup.py. This macro ensures
## modules and global scripts declared therein get installed
## See http://ros.org/doc/api/catkin/html/user_guide/setup_dot_py.html
# catkin_python_setup()
```



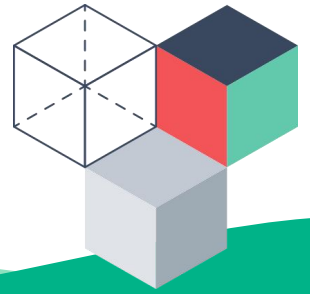
1.1 CMake

1.1.6 Message/Service/Action Generators (add_message_files(), add_service_files(), add_action_files())

- ROS 中的消息 (.msg)、服务 (.srv) 和操作 (.action) 文件在被 ROS 包构建和使用之前需要一个特殊的预处理器构建步骤，把其转化为系统可以识别特定编程语言（.h .cpp）
- 系统会用一些编程语言的生成器（例如 gencpp, genpy, genlisp, etc）生成相应的.cpp 和 .py文件。这个过程需要三个宏：add_message_files， add_service_files，add_action_files 来相应的控制 .msg .srv .action
- 这些宏后面必须跟着一个 generate_messages() 的宏，运用过程中要注意：

1. 这些宏必须在 catkin_package() 宏前面，即：

```
find_package(catkin REQUIRED COMPONENTS ...)
add_message_files(...)
add_service_files(...)
add_action_files(...)
generate_messages(...)
catkin_package(...)
...
```



1.1 CMake

1.1.6 Message/Service/Action Generators (add_message_files(), add_service_files(), add_action_files())

```
##############################################
## Declare ROS messages, services and actions ##
##############################################

## To declare and build messages, services or actions from within this
## package, follow these steps:
## * Let MSG_DEP_SET be the set of packages whose message types you use in
##   your messages/services/actions (e.g. std_msgs, actionlib_msgs, ...).
## * In the file package.xml:
##   * add a build_depend tag for "message_generation"
##   * add a build_depend and a exec_depend tag for each package in MSG_DEP_SET
##   * If MSG_DEP_SET isn't empty the following dependency has been pulled in
##     but can be declared for certainty nonetheless:
##     * add a exec_depend tag for "message_runtime"
## * In this file (CMakeLists.txt):
##   * add "message_generation" and every package in MSG_DEP_SET to
##     find_package(catkin REQUIRED COMPONENTS ...)
##   * add "message_runtime" and every package in MSG_DEP_SET to
##     catkin_package(CATKIN_DEPENDS ...)
##   * uncomment the add_*_files sections below as needed
##     and list every .msg/.srv/.action file to be processed
##   * uncomment the generate_messages entry below
##   * add every package in MSG_DEP_SET to generate_messages(DEPENDENCIES ...)
```

```
## Generate messages in the 'msg' folder
add_message_files(
  FILES
#  Message1.msg
#  Message2.msg
  test.msg
)

## Generate services in the 'srv' folder
# add_service_files(
#   FILES
#   Service1.srv
#   Service2.srv
# )

## Generate actions in the 'action' folder
# add_action_files(
#   FILES
#   Action1.action
#   Action2.action
# )

## Generate added messages and services with any dependencies listed here
generate_messages(
  DEPENDENCIES
  std_msgs
# )
```

①

②

③

④



1.1.6 Message/Service/Action Generators (add_message_files(), add_service_files(), add_action_files())

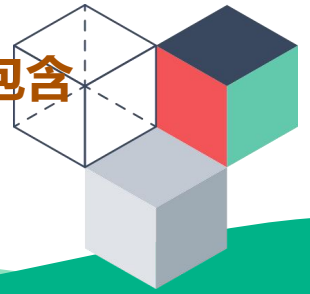
2. 宏 catkin_package() 中必须有 CATKIN_DEPENDS 依赖于 message_runtime，即

```
catkin_package(
 ...
 CATKIN_DEPENDS message_runtime ...
 ...)
```

3. find_package() 必须依赖包 message_generation

```
find_package(catkin REQUIRED COMPONENTS message_generation)
```

4. package.xml 文件 build_depend 必须包含 message_generation，run_depend 必须包含 message_runtime



1.1.6 Message/Service/Action Generators (add_message_files(), add_service_files(), add_action_files())

5. 如果构建的对象依赖于其他需要构建消息/服务/响应的构建对象，则需要向目标 catkin_EXPORTED_TARGETS 添加明确的依赖关系，以便它们以正确的顺序构建：

   add_dependencies(some_target ${catkin_EXPORTED_TARGETS})

   其中，some_target 是 add_executable() 设置的 target 的名字

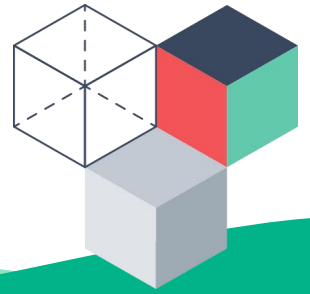
6. 如果需要构建消息或服务的包以及使用这些消息和/或服务的可执行文件，则需要为自动生成的消息目标创建明确的依赖关系，以便以正确的顺序构建它们：

   add_dependencies(some_target ${${PROJECT_NAME}_EXPORTED_TARGETS})

   其中，some_target 是 add_executable() 设置的 target 的名字

7. 如果你的包同时满足上述两个条件，则需要添加两个依赖项，即：

   add_dependencies(some_target ${${PROJECT_NAME}_EXPORTED_TARGETS}
   ${catkin_EXPORTED_TARGETS})



1.1 CMake

1.1.6 Message/Service/Action Generators (add_message_files(), add_service_files(), add_action_files())

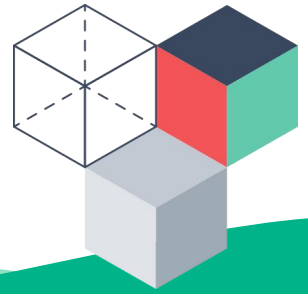
- 如果要构建 actionlib，则在功能包中“action”目录中创建有一个名为“MyAction.action”的文件，必须将 actionlib_msgs 添加到使用 catkin 进行 find_package 的组件列表中，然后在CMakeList.txt中加上下面两行：

```
add_action_files(FILES

  MyAction.action

)
```

- 此外，该包必须对 actionlib_msgs 具有构建依赖性



1.1 CMake

1.1.6 Message/Service/Action Generators (add_message_files(), add_service_files(), add_action_files())

➢ 举例：

a) 现在有两个依赖于 std_msgs 和 sensor_msgs 的消息 MyMessage1.msg和 MyMessage2.msg，还有一个自定义服务 MyService.srv

b) message_program 是使用这些消息和服务的命令，以及生成不使用自定义消息、服务的程序 do_not_use_local_messages_program的catkin包

c) 那么 CMakeLists.txt 应该写成：

切换行号显示

```
# Get the information about this package's buildtime dependencies
find_package(catkin REQUIRED
  COMPONENTS message_generation std_msgs sensor_msgs)

# Declare the message files to be built
add_message_files(FILES
  MyMessage1.msg
  MyMessage2.msg
)

# Declare the service files to be built
add_service_files(FILES
  MyService.srv
)

# Actually generate the language-specific message and service files
generate_messages(DEPENDENCIES std_msgs sensor_msgs)

# Declare that this catkin package's runtime dependencies
catkin_package(
 CATKIN_DEPENDS message_runtime std_msgs sensor_msgs
)

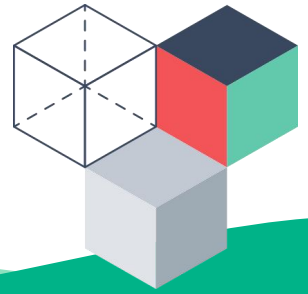
# define executable using MyMessage1 etc.
add_executable(message_program src/main.cpp)
add_dependencies(message_program ${${PROJECT_NAME}_EXPORTED_TARGETS} ${catkin_EXPORTED_TARGETS})

# define executable not using any messages/services provided by this package
add_executable(does_not_use_local_messages_program src/main.cpp)
add_dependencies(does_not_use_local_messages_program ${catkin_EXPORTED_TARGETS})
```

1.1 CMake

1.1.7 Specify package build info export (catkin_package())

- catkin_package() 是 catkin 提供的 CMake 宏，用于将 catkin 特定的信息输出到构建系统上，用于生成 pkg-config 和 CMake 文件
- 这个命令必须在 add_library() 或者 add_executable() 之前调用，该函数有5个可选参数：
  - INCLUDE_DIRS - 导出包的 include 路径
  - LIBRARIES - 导出项目中的库
  - CATKIN_DEPENDS - 该项目依赖的其他 catkin 项目
  - DEPENDS - 该项目所依赖的非 catkin CMake 项目
  - CFG_EXTRAS - 其他配置选项



1.1 CMake

1.1.7 Specify package build info export (catkin_package())

- 举例：

```
catkin_package(
  INCLUDE_DIRS include
  LIBRARIES ${PROJECT_NAME}
  CATKIN_DEPENDS roscpp rospy
  DEPENDS eigen opencv)
```

- 这表示工作空间中的“include”文件夹是导出头文件的地方； ${PROJECT_NAME}根据 project中的内容生成，此处是 beginner_tutorials；“roscpp”+“rospy”是用来构建/运行此程序包的 catkin 包，而“eigen”+“opencv”是用于构建/运行此程序包的非 catkin 包



1.1.8 Libraries/Executables to build (add_library()/add_executable()/target_link_libraries())

➢ 构建目标可以有多种形式，但通常主要有以下两种：

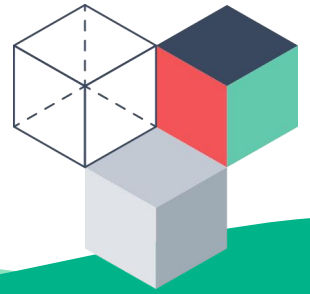
- 执行文件目标 - 可以运行的程序
- 库目标 - 可在构建或运行时给可执行文件使用的库

1. target 的命名：

- target 的命名很重要，在 catkin 中 target 的名字必须是唯一的，和之前构建产生的和安装的都不能相同
- 可以利用 set_target_properties() 函数将 target 进行重命名。例如：

```
set_target_properties(rviz_image_view
                      PROPERTIES OUTPUT_NAME image_view
                      PREFIX "")
```

上述操作将 rviz_image_view 的名称更改为image_view



1.1.8 Libraries/Executables to build (add_library()/add_executable()/target_link_libraries())

2. 自定义输出路径：

- ROS中的输出路径时默认的，但是也可通过下面进行修改：

```
set_target_properties(python_module_library

  PROPERTIES LIBRARY_OUTPUT_DIRECTORY
${CATKIN_DEVEL_PREFIX}/${CATKIN_PACKAGE_PYTHON_DESTINATION})
```

3. 头文件路径和库文件路径：

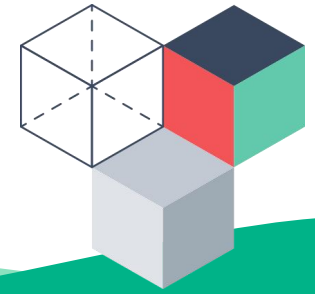
- 在我们指定 target 之前，需要指明相对 target 而言，在哪里找源文件，特别是在哪里找头文件，在哪里找库文件：

Include Paths：指明编译代码时在哪里寻找头文件

Library Paths：指明可执行文件需要的库文件在哪里

include_directories(<dir1>, <dir2>, ..., <dirN>)

link_directories(<dir1>, <dir2>, ..., <dirN>)



1.1.8 Libraries/Executables to build (add_library()/add_executable()/target_link_libraries())

3. 头文件路径和库文件路径：

- 函数 include_directories()：

  include_directories() 中的参数是调用 find_package 时生成的 * _INCLUDE_DIRS 变量。如果使用 catkin 和 Boost，那么 include_directories() 调用应该如下所示：

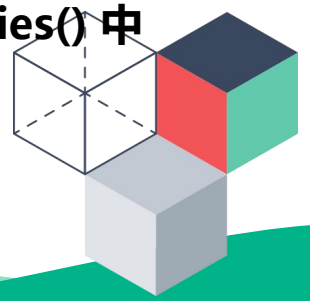
  include_directories(include ${Boost_INCLUDE_DIRS} ${catkin_INCLUDE_DIRS})

  第一个参数“include”表示功能包中的include /目录也是路径的一部分

- 函数 link_directories()：

  link_directories() 函数可用于添加额外的库路径，但通常不推荐这样做，因为所有 catkin 和 CMake 软件包在 find_packaged 时都会自动添加链接信息。 只需写 target_link_libraries() 中就可以了。若要使用，采用以下方式：

  link_directories(~/my_libs)



1.1.8 Libraries/Executables to build (add_library()/add_executable()/target_link_libraries())

4. 可执行目标：

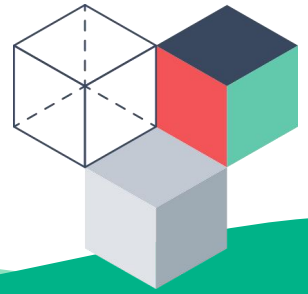
- 要指定必须构建的可执行目标，使用 add_executable() 函数：

```
add_executable(myProgram src/main.cpp src/some_file.cpp src/another_file.cpp)
```

上述操作这将构建一个名为 myProgram 的目标可执行文件，它由 3 个源文件构建：src/main.cpp、src/some_file.cpp 和 src/another_file.cpp

5. 库目标：

```
add_library(${PROJECT_NAME} ${${PROJECT_NAME}_SRCS})
```



1.1.8 Libraries/Executables to build (add_library()/add_executable()/target_link_libraries())

6. 链接库 target_link_libraries:

- target_link_libraries() 用来指定可执行目标链接的库。 通常在 add_executable() 调用之后完成。如果找不到 ros，则添加${catkin_LIBRARIES}:

  target_link_libraries(<executableTargetName>, <lib1>, <lib2>, ... <libN>)

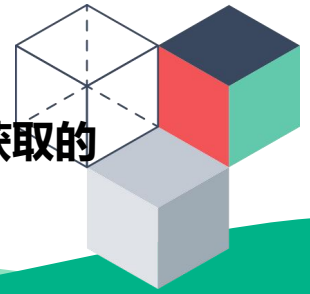
- 举例:

```
add_executable(foo src/foo.cpp)

add_library(moo src/moo.cpp)

target_link_libraries(foo moo)
```

- 注意：在大多数用例中不需要使用 link_directories()，因为该信息是通过 find_package() 自动获取的



Summary

1. CMakeLists.txt Introduction

   a) Required CMake Version (cmake_minimum_required)

   b) Package Name (project())

   c) Find other CMake/Catkin packages needed for build (find_package())

   d) Enable Python module support (catkin_python_setup())

   e) Message/Service/Action Generators (add_message_files(), add_service_files(), add_action_files())

   f) Invoke message/service/action generation (generate_messages())

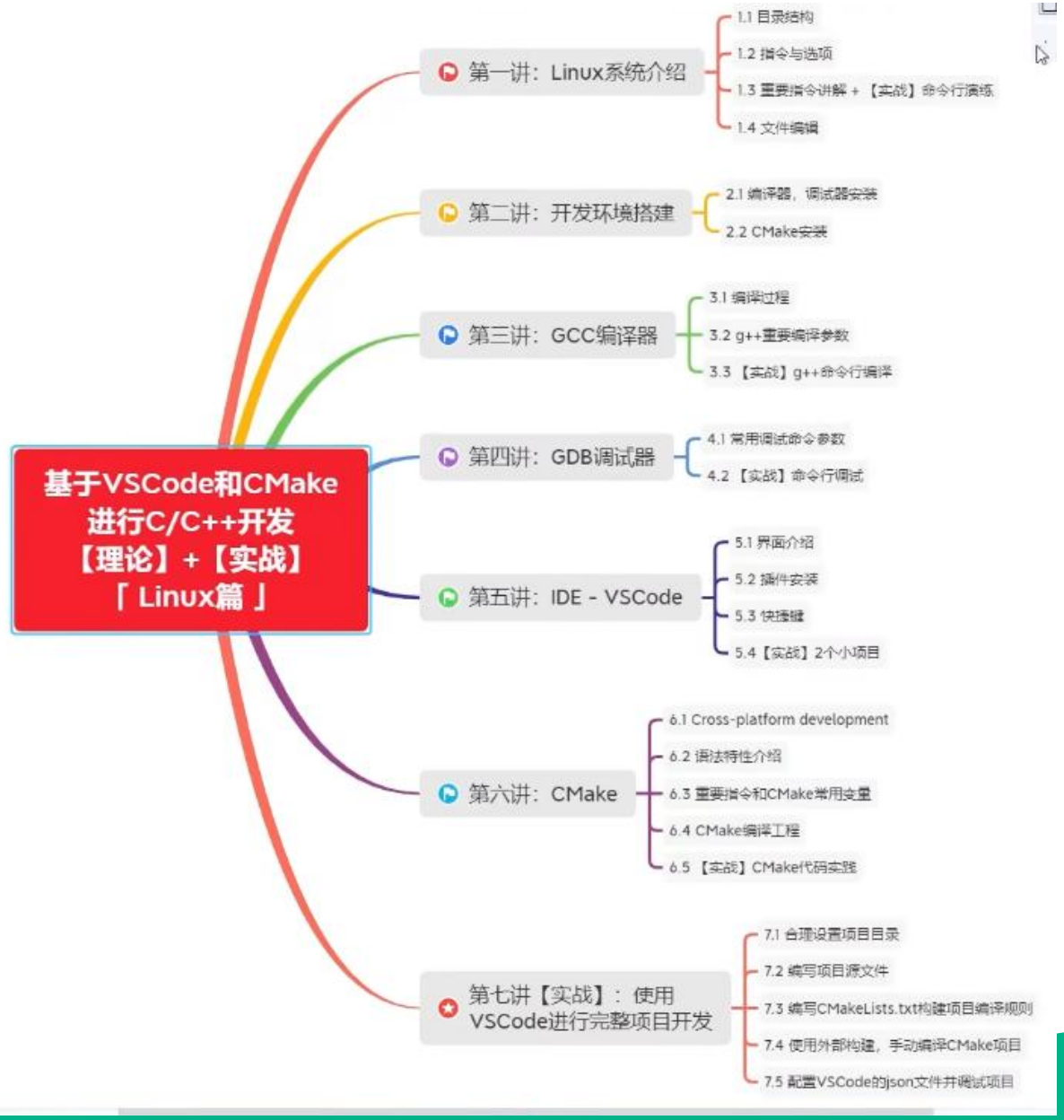
   g) Specify package build info export (catkin_package())

   h) Libraries/Executables to build (add_library()/add_executable()/target_link_libraries())

Homework

1 基于VSCode和CMake实现C/C++开发 | Linux篇

https://www.bilibili.com/video/BV1fy4y1b7TC/?spm_id_from=333.999.0.0&vd_source=7df640296c15e75b5fbd2da8cc72a434



2. 作业内容：

a) 学习课程所有内容，并跟着进行代码练习

b) 记录学习内容+测试代码

Overview lesson 7-2

1. ROS Computation Graph
2. ~
3. ~
4. ~

Objectives

1、掌握 ROS Computation Graph 的基本概念

2、熟练使用 ROS Computation Graph 命令行工具



Pre-assessment

1、Computation Graph 包含哪些概念?

2、是否知晓如何使用 Computation Graph 工具?



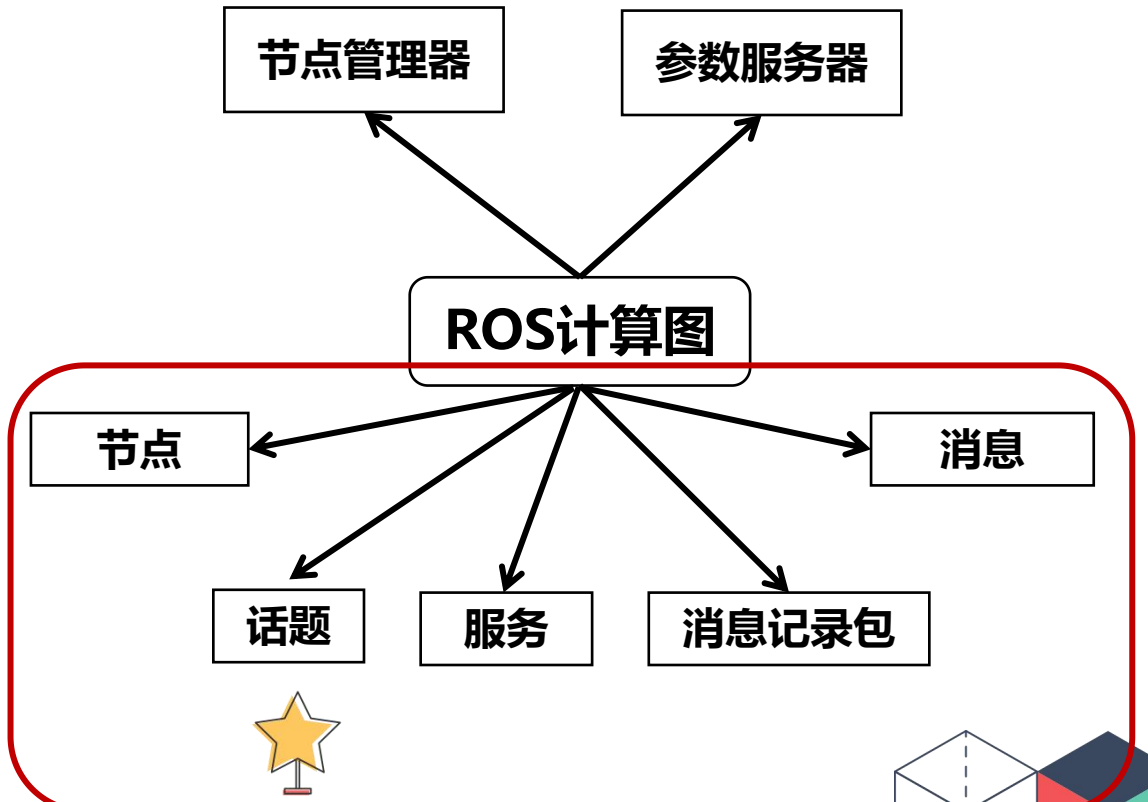
1

ROS Computation Graph

1.1 ROS computation graph level

1.1.1 Overview：http://wiki.ros.org/rosnode

- ROS会创建一个连接到所有进程的网络。系统中的任何节点都可以访问这个网络，并且通过该网络与其他节点进行交互(获取其他节点发布的消息+将自身数据发布到网络)
- 这个网络称为计算图，即计算图(Computation Graph)是ROS处理数据的一种点对点的网络形式
- 计算图中的基本元素是：节点(Node)、主机(ROS Master)、参数服务器(Parameter Server)、消息(Message)、服务(Service)、话题(Topics)和消息记录包(Bags)，它们都以不同的方式向 Graph 提供数据



1.1 ROS computation graph level

1.1.2 Basic concepts

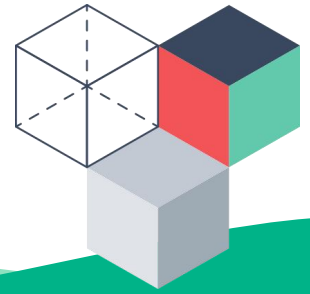
1. 节点(微信用户)：ROS节点对应于 Linux 系统中执行特定任务的进程

   - ROS节点通过 ROS 客户端库编写，例如 roscpp 或 rospy
   - ROS节点支持 ROS 消息和 ROS 服务两种通讯方式
   - ROS的设计理念是让多个节点协同工作，而不是让一个节点包揽所有任务。例如，一个机器人控制系统通常有很多节点组成，一个节点控制激光测距仪，一个节点执行定位，一个节点执行路径规划等

```
ros@ROS-VirtualBox:~$ rosnode --help
rosnode is a command-line tool for printing information about ROS Nodes.

Commands:
        rosnode ping    test connectivity to node
        rosnode list    list active nodes
        rosnode info    print information about node
        rosnode machine list nodes running on a particular machine or list machi
nes
        rosnode kill    kill a running node
        rosnode cleanup purge registration information of unreachable nodes

Type rosnode <command> -h for more detailed usage, e.g. 'rosnode ping -h'
```

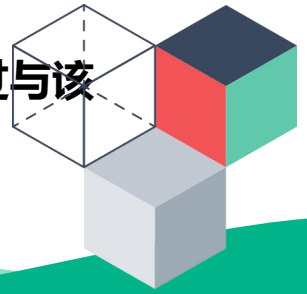


1.1 ROS computation graph level

1.1.2 Basic concepts

2. 主机(微信)：ROS Master 为节点提供名称注册和查找服务

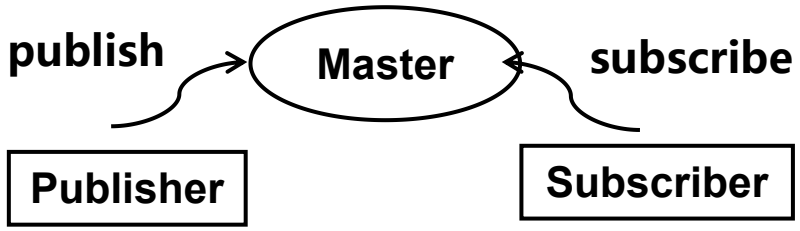
- ROS Master 负责在节点间建立连接，没有 ROS Master，节点将无法找到彼此，无法进行数据交换和服务调用
- 在 ROS 系统中，任何节点启动时会将节点名称注册到 ROS Master，因此 ROS Master 具有当前ROS系统上运行的所有节点信息
- 在节点发布话题之前，其将向 ROS Master 提供该话题的详细信息，包括其名称和数据类型等
- ROS Master 检查是否有其他节点订阅此话题：若有，则 ROS Master 将发布节点的细节共享给订阅节点，并通过 TCPROS 协议与发布节点进行互连，之后 ROS Master 放弃对节点的控制
- ROS 系统只能有一个 ROS Master，可以在一台计算机上运行 ROS Master，远程节点可以通过与该Master与其他节点进行通讯(需要定义ROS_MASTER_URI

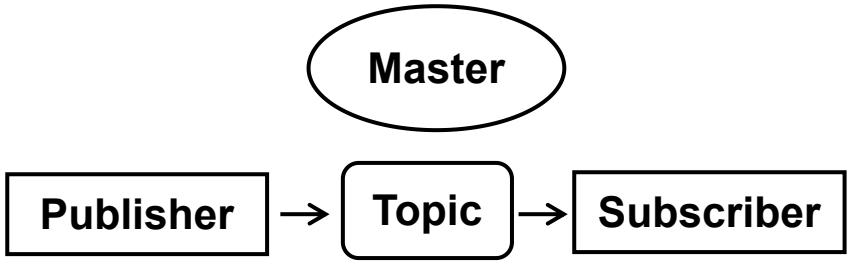


1.1 ROS computation graph level

1.1.2 Basic concepts

2. 主机(微信)：ROS Master 为计算图的其余部分提供名称注册和查找。没有 ROS Master，节点将无法找到彼此、交换消息或调用服务






```
... logging to /home/ros/.ros/log/bbdddf0e-f987-11eb-840c-94f6d62469e0/roslaunch-ros-mac-17802.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://ros-mac:41783/
ros_comm version 1.14.11


SUMMARY
========

PARAMETERS
 * /rosdistro: melodic
 * /rosversion: 1.14.11

NODES

auto-starting new master
process[master]: started with pid [17813]
ROS_MASTER_URI=http://ros-mac:11311/

setting /run_id to bbdddf0e-f987-11eb-840c-94f6d62469e0
process[rosout-1]: started with pid [17824]
started core service [/rosout]
```

1.1 ROS computation graph level

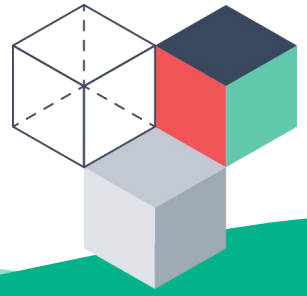
1.1.2 Basic concepts

3. 参数服务器(微信设置)：参数服务器是 ROS Master 的一部分

- 允许 ROS 将数据或者配置信息集中存储，所有节点都可以访问和修改这些参数值
- 参数服务器是一个共享多变量字典，节点可以使用该服务器在运行时存储和检索参数
- 参数服务器使用 XMLRPC 实现，XMLRPC 是一种 RPC 协议（RPC（Remote Procedure Call）远程过程调用，简单的理解是一个节点请求另一个节点提供的服务），XMLRPC 是一个使用 XML 编码并以 HTTP 作为传输机制的远程调用

```
ros@ROS-VirtualBox:~$ rosparam --help
rosparam is a command-line tool for getting, setting, and deleting parameters fr
om the ROS Parameter Server.

Commands:
      rosparam set    set parameter
      rosparam get    get parameter
      rosparam load   load parameters from file
      rosparam dump   dump parameters to file
      rosparam delete delete parameter
      rosparam list   list parameter names
```



1.1 ROS computation graph level

1.1.2 Basic concepts

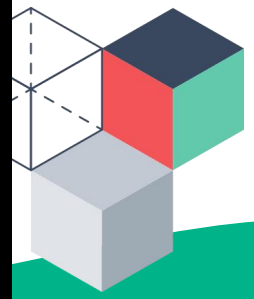
4. 消息(微信文字消息)：ROS 节点通过传递消息相互通信，应用于单向异步“发布-订阅”模式的通讯

- ROS节点通过消息互相通信，消息中包含该节点向其他节点提供信息的数据
- 消息是一种数据结构，具有标准的数据类型，以及基本类型数组，ROS提供一种基于标准消息类型开发自定义消息类型的机制
- 消息类型遵循以下命名约定：包的名称+.msg文件的名称。例如， std_msgs/msg/String.msg的消息类型是std_msgs/String

```
ros@ROS-VirtualBox:~$ rosmsg --help
rosmsg is a command-line tool for displaying information about ROS Message types.

Commands:
        rosmsg show     Show message description
        rosmsg info     Alias for rosmsg show
        rosmsg list     List all messages
        rosmsg md5      Display message md5sum
        rosmsg package  List messages in a package
        rosmsg packages List packages that contain messages

Type rosmsg <command> -h for more detailed usage
```



1.1 ROS computation graph level

1.1.2 Basic concepts

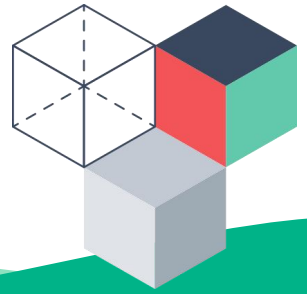
5. 服务(微信语音通话)：应用于双向同步“请求-相应”模式的节点通讯

- ROS Service 包含两个部分：请求 和 响应
- ROS Service 需要两个节点：服务端 和 客户端
- ROS Service 主要用于分布式系统中
- 服务类型遵循以下命名约定：包的名称+.srv文件的名称。例如， my_tutorials/srv/Test.srv的服务类型是my_tutorials/Test

```
ros@ROS-VirtualBox:~$ rossrv --help
rossrv is a command-line tool for displaying information about ROS Service types.

Commands:
        rossrv show     Show service description
        rossrv info     Alias for rossrv show
        rossrv list     List all services
        rossrv md5      Display service md5sum
        rossrv package  List services in a package
        rossrv packages List packages that contain services

Type rossrv <command> -h for more detailed usage
```



1.1 ROS computation graph level

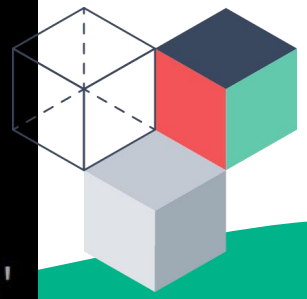
1.1.2 Basic concepts

5. 服务(微信语音通话)：应用于双向同步“请求-相应”模式的节点通讯

- 服务端节点通过名称提供服务，当客户端节点向服务端节点发送请求消息时，服务端节点响应这个消息并将结果发送到客户端节点
- 这个过程中，客户端节点需要一直等待，直到服务端节点响应客户端的请求
- rossrv 类似于 rosmsg，用于获取服务类型信息
- rosservice，用于列出服务列表和查询正在运动的 ROS 服务

```
ros@ROS-VirtualBox:~$ rosservice --help
Commands:
        rosservice args print service arguments
        rosservice call call the service with the provided args
        rosservice find find services by service type
        rosservice info print information about service
        rosservice list list active services
        rosservice type print service type
        rosservice uri  print service ROSRPC uri

Type rosservice <command> -h for more detailed usage, e.g. 'rosservice call -h'
```



1.1 ROS computation graph level

1.1.2 Basic concepts

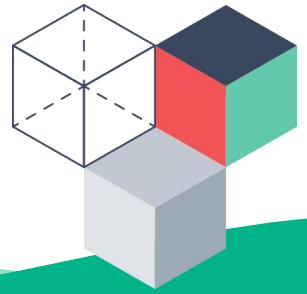
6. 话题(微信公众号)：遵循异步的发布-订阅机制，话题在节点之间通过消息交互信息

- 话题名称需具有唯一性
- 发布者/订阅者
- 发布者和订阅者相分离，允许多个发布者和多个订阅者，发布者和订阅者可以不了解彼此的存在
- 话题类型由发布在其上的消息类型决定，节点只能接受类型匹配的消息，一个节点只能订阅相同消息类型的话题

```
ros@ROS-VirtualBox:~$ rostopic --help
rostopic is a command-line tool for printing information about ROS Topics.

Commands:
        rostopic bw     display bandwidth used by topic
        rostopic delay  display delay of topic from timestamp in header
        rostopic echo   print messages to screen
        rostopic find   find topics by type
        rostopic hz     display publishing rate of topic
        rostopic info   print information about active topic
        rostopic list   list active topics
        rostopic pub    publish data to topic
        rostopic type   print topic or field type

Type rostopic <command> -h for more detailed usage, e.g. 'rostopic echo -h'
```



1.1 ROS computation graph level

1.1.2 Basic concepts

7. 消息记录包(微信数据存储)：用来保存和回放ROS消息数据的文件格式

- ROS bags 是一种用于存储数据的重要机制，其能够获取并记录各种传感器的数据
- 可以通过消息记录包反复获取实验数据，用于算法的测试和开发

```
ros@ROS-VirtualBox:~$ rosbag --help
Usage: rosbag <subcommand> [options] [args]

A bag is a file format in ROS for storing ROS message data. The rosbag command can record, replay an
d manipulate bags.

Available subcommands:
   check        Determine whether a bag is playable in the current system, or if it can be migrated.
   compress     Compress one or more bag files.
   decompress   Decompress one or more bag files.
   decrypt      Decrypt one or more bag files.
   encrypt      Encrypt one or more bag files.
   filter       Filter the contents of the bag.
   fix          Repair the messages in a bag file so that it can be played in the current system.
   help
   info         Summarize the contents of one or more bag files.
   play         Play back the contents of one or more bag files in a time-synchronized fashion.
   record       Record a bag file with the contents of specified topics.
   reindex      Reindexes one or more bag files.

For additional information, see http://wiki.ros.org/rosbag
```

1.1 ROS computation graph level

1.1.3 Sum-up

- 计算图(Computation Graph)是ROS处理数据的一种点对点的网络形式
- ROS 的基本 Computation Graph 概念是节点(Nodes)、主机(Master)、参数服务器(Parameter Server)、消息(Messages)、服务(Services)、话题(Topics)和消息记录包(Bags)，它们都以不同的方式向 Graph 提供数据

1. 节点：节点主要执行计算处理
   - ROS 被设计为细粒度的模块化的系统：一个机器人控制系统通常有很多节点组成 。例如，一个节点控制激光测距仪，一个节点执行定位，一个节点执行路径规划，一个节点提供系统图形界面等
   - ROS 节点通过 ROS 客户端库 client library编写，例如 roscpp 或 rospy
2. 主机：ROS Master 为计算图的其余部分提供名称注册和查找。没有 ROS Master，节点将无法找到彼此、交换消息或调用服务



1.1 ROS computation graph level

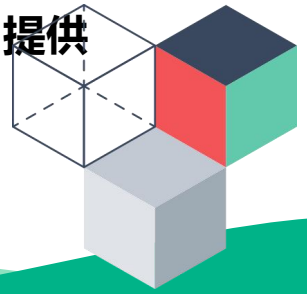
1.1.3 Sum-up

3. **参数服务器**：参数服务器允许将数据通过密钥存储在一个中央位置， 它目前是 ROS Master 的一部分

4. **消息**：ROS节点通过传递消息相互通信。消息是一种数据结构，支持标准基本类型（整数、浮点数、布尔值等），以及基本类型数组。 消息可以包含在任意嵌套的结构和数组中

5. **服务**：节点间的请求/响应的通讯模式

   - 发布/订阅模型是一种非常灵活的通信范式，但其多对多、单向传输不适用于请求/响应交互，而这在分布式系统中通常是必需的
   - 请求/响应是通过服务完成的，服务由一对消息结构定义：一个用于请求，一个用于响应。 提供节点以某个名称提供服务，客户端通过发送请求消息并等待回复来使用该服务



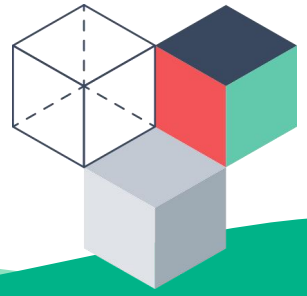
1.1 ROS computation graph level

1.1.3 Sum-up

6. **话题**：消息通过具有发布/订阅的传输系统进行通讯

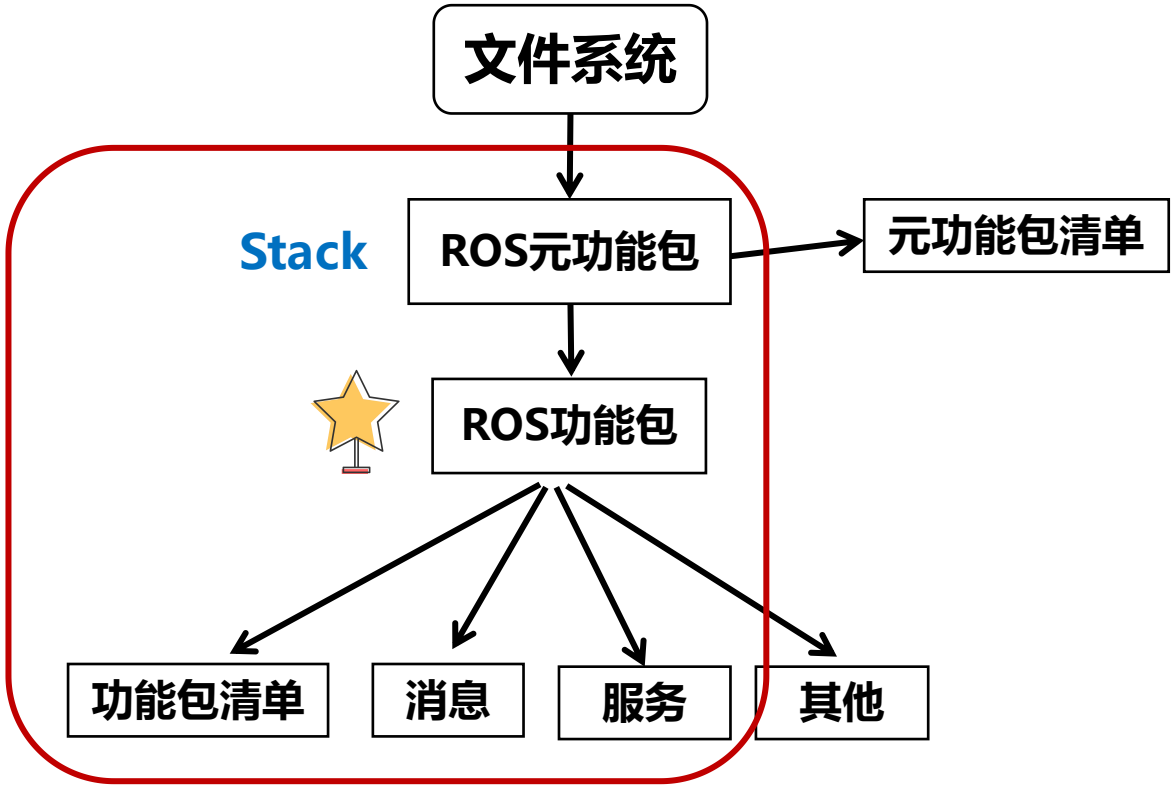
   - 节点通过将消息发布到给定主题来发送消息
   - 话题是用于标识消息内容的名称，对某种数据感兴趣的节点将订阅适当的话题
   - 一个话题可能有多个并发发布者和订阅者，一个节点可能发布和/或订阅多个话题

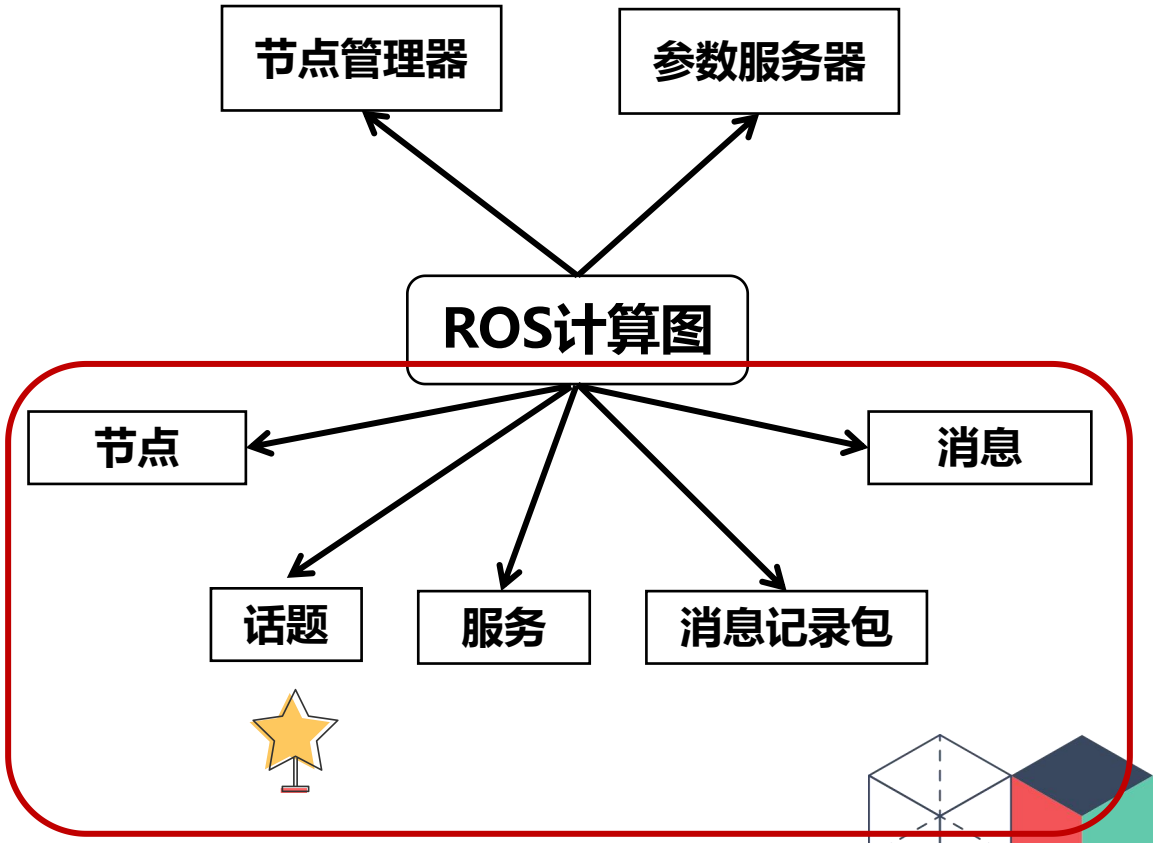
7. **消息记录包**：Bags 是一种用于保存和回放 ROS 消息数据的格式。Bags 是存储数据（例如传感器数据）的重要机制，这些数据可能难以收集，但却是开发和测试算法所必需的



1.1 ROS computation graph level

1.1.3 Sum-up

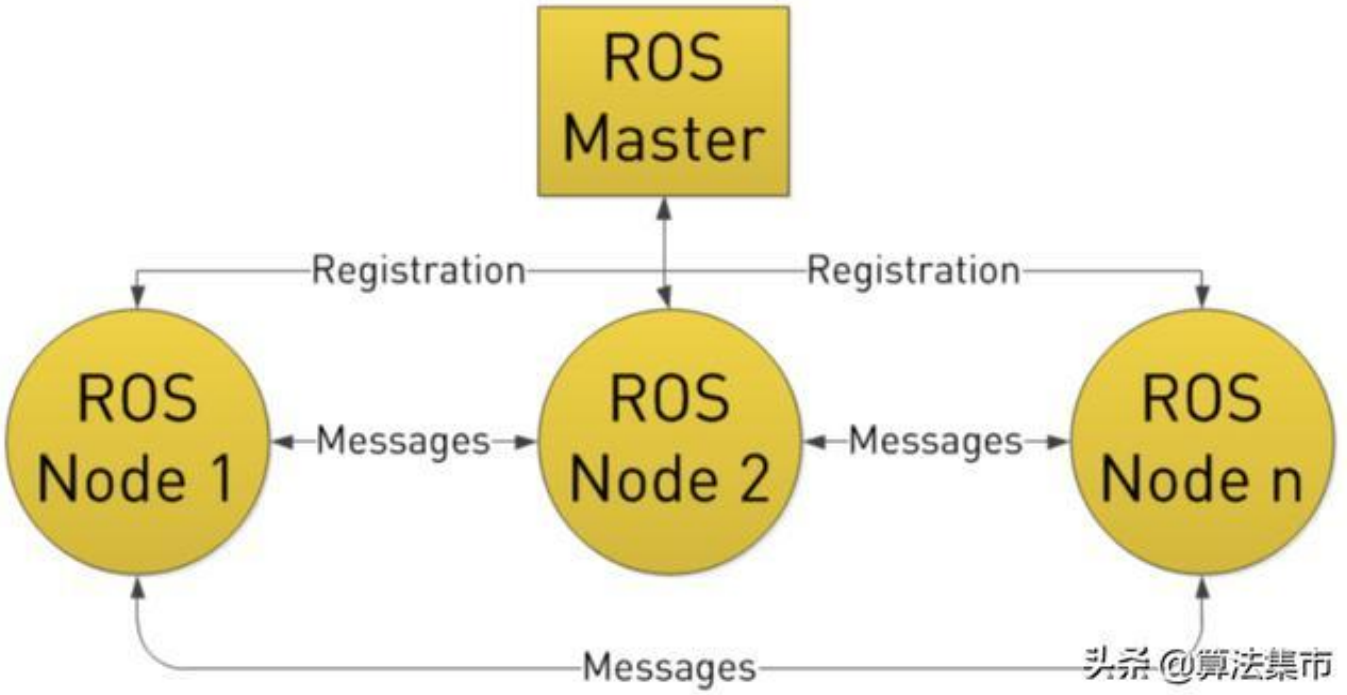




1.1 ROS computation graph level

1.1.4 Examples

- 节点之间通过消息交互信息

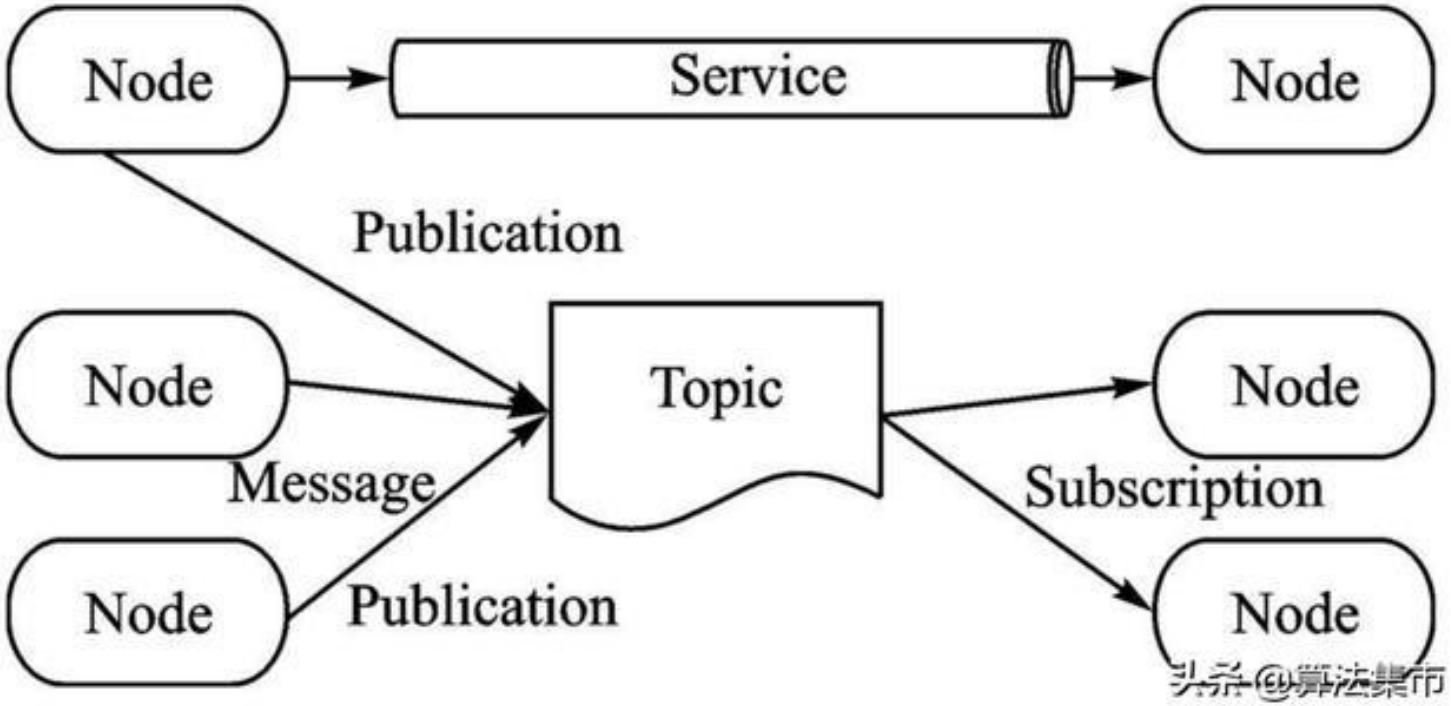


图片来源：算法集市

1.1 ROS computation graph level

1.1.4 Examples

- 异步“发布-订阅”模式

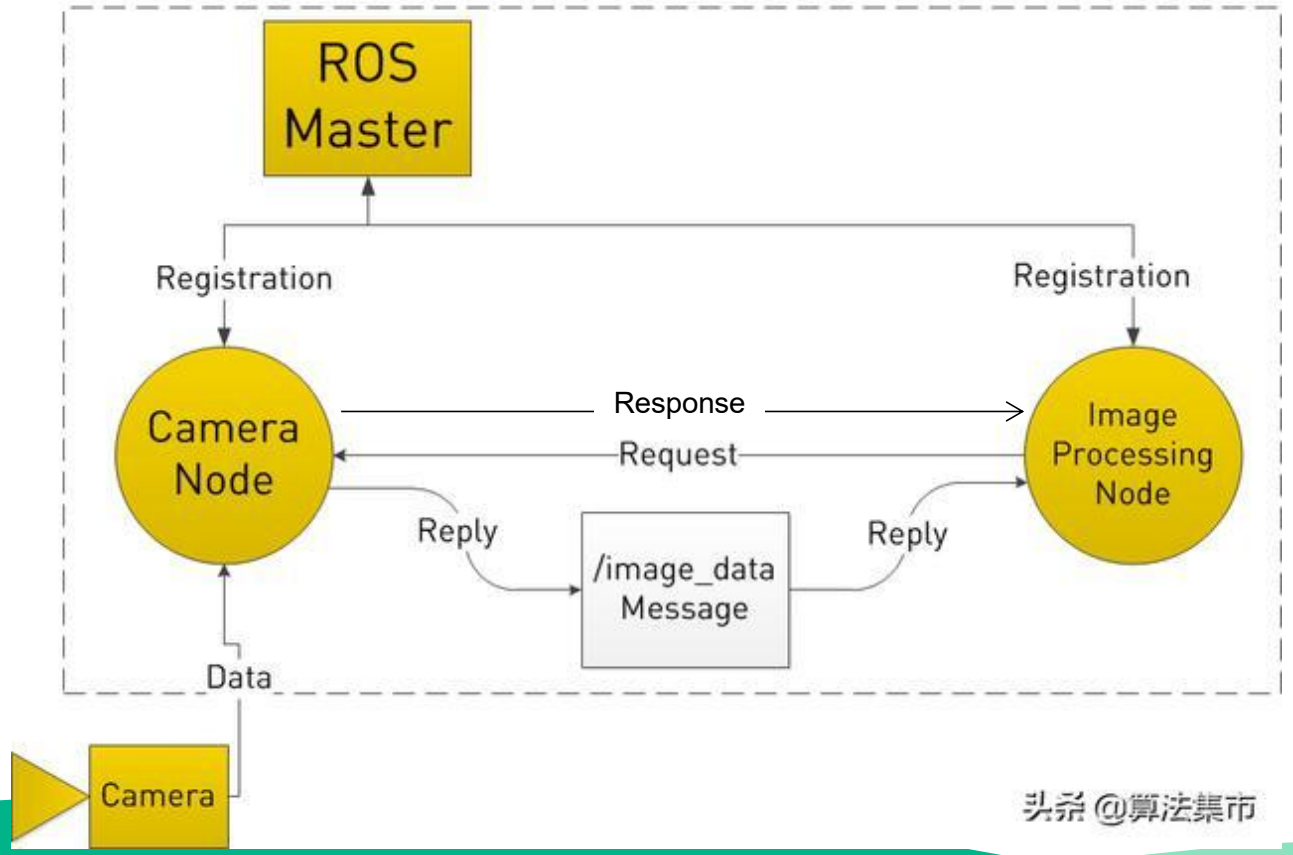


图片来源：算法集市

1.1 ROS computation graph level

1.1.4 Examples

- 同步“请求-响应”模式

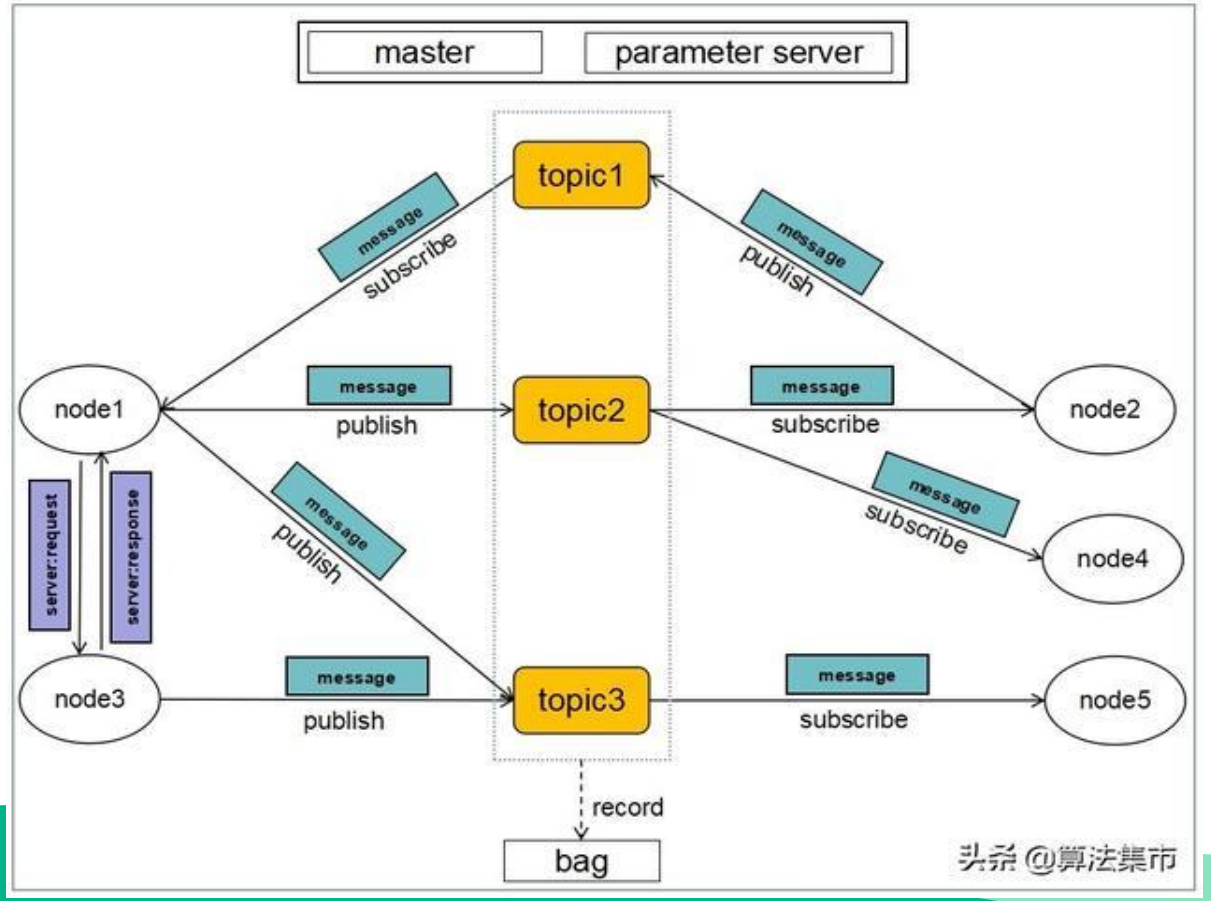


图片来源：算法集市

1.1 ROS computation graph level

1.1.4 Examples

- ROS中最小的进程单元就是节点（node），节点之间通过主题（topic）传递消息数据（message）



图片来源：算法集市

Summary

1. ROS Computation Graph Level

a) 节点 

b) 主机

c) 参数服务器

d) 话题 

e) 消息 

f) 服务 

g) 消息记录包 

Overview lesson 7-3

1. Turtlesim
   Exercises
2. ~
3. ~
4. ~

Objectives

1、掌握节点、话题、消息、服务的使用方法

2、了解 rqt 使用方法

3、掌握 roslaunch 编写方法

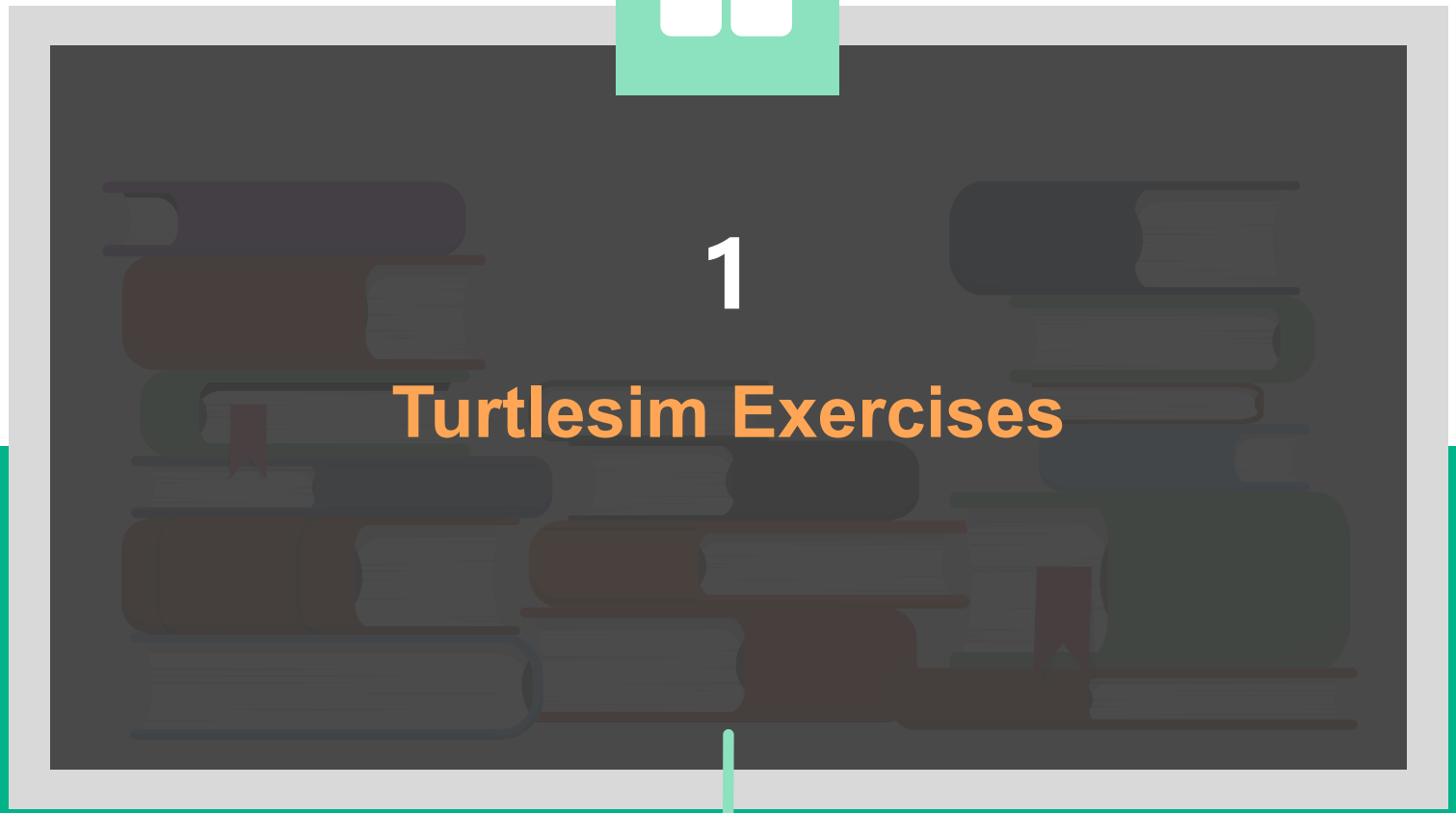


Pre-assessment

1、是否使用过turtlesim?

2、是否使用过rqt工具?

3、是否编写过roslaunch文件?



1

Turtlesim Exercises

1.1 Turtlesim Examples

1.1.1 Overview： http://wiki.ros.org/turtlesim

- **turtlesim 的4个节点：turtlesim_node，mimic，draw_square，turtle_teleop_key**

2. Nodes

New in ROS hydro As of *Hydro* turtlesim uses the geometry_msgs/Twist message (*turtlesim/Velocity* in *Groovy* and older). Also the topic has been changed to `cmd_ve` before).

2.1 turtlesim_node

`turtlesim_node` provides a simple simulator for teaching ROS concepts.

2.1.1 Subscribed Topics

`turtleX/cmd_vel` (geometry_msgs/Twist)

The linear and angular command velocity for turtleX. The turtle will execute a time out. Twist.linear.x is the forward velocity, Twist.linear.y is the strafe veloci velocity.

2.1.2 Published Topics

`turtleX/pose` (turtlesim/Pose)

The x, y, theta, linear velocity, and angular velocity of turtleX.

2.2 mimic

`mimic` provides a simple interface for making one turtlesim mimic another.

2.2.1 Subscribed Topics

`input` (turtlesim/Pose)

The input topic for the `mimic` node. The topic must be remapped to the `pos`

2.2.2 Published Topics

`output` (geometry_msgs/Twist)

The output topic for the `mimic` node. The topic must be remapped to the `cm`



1.1 Turtlesim Examples

1.1.1 Overview

- **turtlesim服务和参数：**

2.1.3 Services

`clear` (std_srvs/Empty)

Clears the turtlesim background and sets the color to the value of the backgrour

`reset` (std_srvs/Empty)

Resets the turtlesim to the start configuration and sets the background color to t

`kill` (turtlesim/Kill)

Kills a turtle by name.

`spawn` (turtlesim/Spawn)

Spawns a turtle at (x, y, theta) and returns the name of the turtle. Also will take r duplicate name.

`turtleX/set_pen` (turtlesim/SetPen)

Sets the pen's color (r g b), width (width), and turns the pen on and off (off).

`turtleX/teleport_absolute` (turtlesim/TeleportAbsolute)

Teleports the turtleX to (x, y, theta).

`turtleX/teleport_relative` (turtlesim/TeleportRelative)

Teleports the turtleX a linear and angular distance from the turtles current position.

2.1.4 Parameters

`~background_b` (`int`, default: 255)

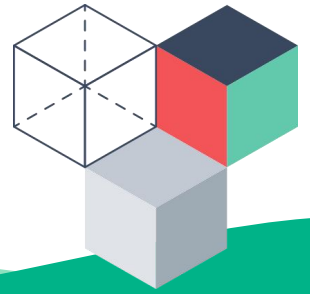
Sets the blue channel of the background color.

`~background_g` (`int`, default: 86)

Sets the green channel of the background color.

`~background_r` (`int`, default: 69)

Sets the red channel of the background color.



1.1 Turtlesim Examples

1.1.2 启动ros

- 命令行工具：

```
$ roscore
```

- 默认情况下，roscore 会启动三个程序：

1. ROS Master
2. Parameter Server
3. rosout 日志节点

```
ros@ROS-VirtualBox:~$ roscore
... logging to /home/ros/.ros/log/fa714a7e-30e3-11ec-bc7d-0
8002752f08a/roslaunch-ROS-VirtualBox-3520.log
Checking log directory for disk usage. This may take a whil
e.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.
```

主要用于从ROS节点来收集日志

```
started roslaunch server http://ROS-VirtualBox:46255/
ros_comm version 1.14.11
```

执行 roscore.xml 启动文件，启动ROS Master 和 Parameter Server

```
SUMMARY
========

PARAMETERS
 * /rosdistro: melodic
 * /rosversion: 1.14.11

NODES
```

roslaunch执行roscore.xml的显示结果

```
auto-starting new master
process[master]: started with pid [3531]
ROS_MASTER_URI=http://ROS-VirtualBox:11311/
```

Parameter Server的节点：ROS_MASTER_URI

```
setting /run_id to fa714a7e-30e3-11ec-bc7d-08002752f08a
process[rosout-1]: started with pid [3542]
started core service [/rosout]
```

ros节点启动，订阅/rosout话题并转播到/rosout_agg

1.1 Turtlesim Examples

1.1.2 启动ros

- 查看话题：

```
$ rostopic list
```

```
ros@ROS-VirtualBox:~$ rostopic list
/rosout
/rosout_agg
```

- 查看可用参数：

```
$ rosparam list
```

```
ros@ROS-VirtualBox:~$ rosparam list
/rosdistro
/roslaunch/uris/host_ros_virtualbox__46255
/rosversion
/run_id
```

/rosdistro ROS发行版

/roslaunch * roslaunch服务地址

/rosversion ROS版本号

/run_id ROS版本号

- 查看可用服务：

```
$ rosservice list
```

```
ros@ROS-VirtualBox:~$ rosservice list
/rosout/get_loggers
/rosout/set_logger_level
```

对应上述ros节点，用于设置日志目录

1.1 Turtlesim Examples

1.1.3 启动turtlesim

- **命令行：**

```
$ rosrun turtlesim turtlesim_node
```

- **rosrun命令：rosrun allows you to run an executable in an arbitrary package from anywhere without having to give its full path or cd/roscd there first.**

http://wiki.ros.org/rosbash#rosrun

Usage:

```
rosrun <package> <executable>
```

Example:

```
rosrun roscpp_tutorials talker
```

It's also possible to pass a `~parameter` using the following syntax (replace the ~ with an _):

```
rosrun package node _parameter:=value
```

Example:

```
rosrun my_package my_node _my_param:=value
```

...or run with a given name:

```
rosrun package node __name:=name
```

Example:

```
rosrun my_package my_node __name:=my_name
```

For more information about remapping, see: Remapping Arguments

Starting in **Indigo**, rosrun has a `--prefix` option which can be used to run a node in gdb or valgrind.

Example:

```
rosrun --prefix 'gdb -ex run --args' my_package my_node
```

1.1 Turtlesim Examples

1.1.3 启动turtlesim

- 运行结果：

```
ros@ROS-VirtualBox:~$ rosrun turtle
sim turtlesim_node
[ INFO] [1634659567.469690433]: Sta
rting turtlesim with node name /tur
tlesim
[ INFO] [1634659567.474676547]: Spa
wning turtle [turtle1] at x=[5.5444
45], y=[5.544445], theta=[0.000000]
```



- 查看话题：

```
$ rostopic list
```

```
ros@ROS-VirtualBox:~$ rostopic list
/rosout
/rosout_agg
/turtle1/cmd_vel
/turtle1/color_sensor
/turtle1/pose
```

- 查看节点：

```
$ rosnode list
```

```
ros@ROS-VirtualBox:~$ rosnode list
/rosout
/turtlesim
```

1.1 Turtlesim Examples

1.1.4 turtlesim节点和话题

- 查看/turtlesim节点详细信息：

```
$ rosnode info /turtlesim
```

```
ros@ROS-VirtualBox:~$ rosnode info /turtlesim
--------------------------------------------------------------------------------
Node [/turtlesim]
Publications:
 * /rosout [rosgraph_msgs/Log]
 * /turtle1/color_sensor [turtlesim/Color]
 * /turtle1/pose [turtlesim/Pose]

Subscriptions:
 * /turtle1/cmd_vel [unknown type]

Services:
 * /clear
 * /kill
 * /reset
 * /spawn
 * /turtle1/set_pen
 * /turtle1/teleport_absolute
 * /turtle1/teleport_relative
 * /turtlesim/get_loggers
 * /turtlesim/set_logger_level


contacting node http://ROS-VirtualBox:35153/ ...
Pid: 4355
Connections:
 * topic: /rosout
    * to: /rosout
    * direction: outbound (46185 - 127.0.0.1:56526) [26]
    * transport: TCPROS
```

1.1 Turtlesim Examples

1.1.4 turtlesim节点和话题

- 启动 turtle_teleop_key 节点：

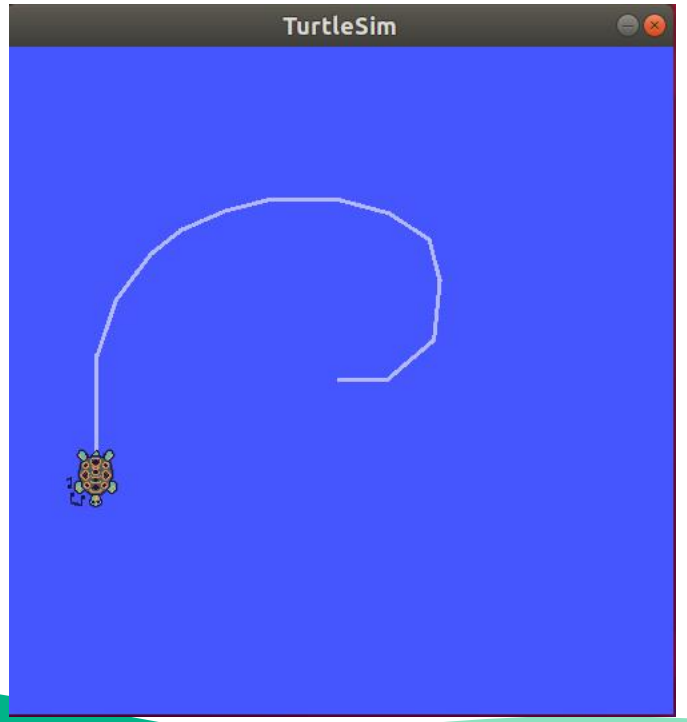
```
$ rosrun turtlesim turtle_teleop_key
```

```
ros@ROS-VirtualBox:~$ rosrun turtlesim
turtle_teleop_key
Reading from keyboard
---------------------------
Use arrow keys to move the turtle. 'q'
to quit.
```

- 查看ROS节点：

```
$ rosnode list
```

```
turtlesim/cmake$ rosnode list
/rosout
/teleop_turtle
/turtlesim
```



为什么操作键盘小乌龟会移动?

主观题 10分

设置

为什么操作键盘小乌龟会移动?

正常使用主观题需2.0以上版本雨课堂

作答

1.1 Turtlesim Examples

1.1.4 turtlesim节点和话题

- 查看节点和话题之间的关系：

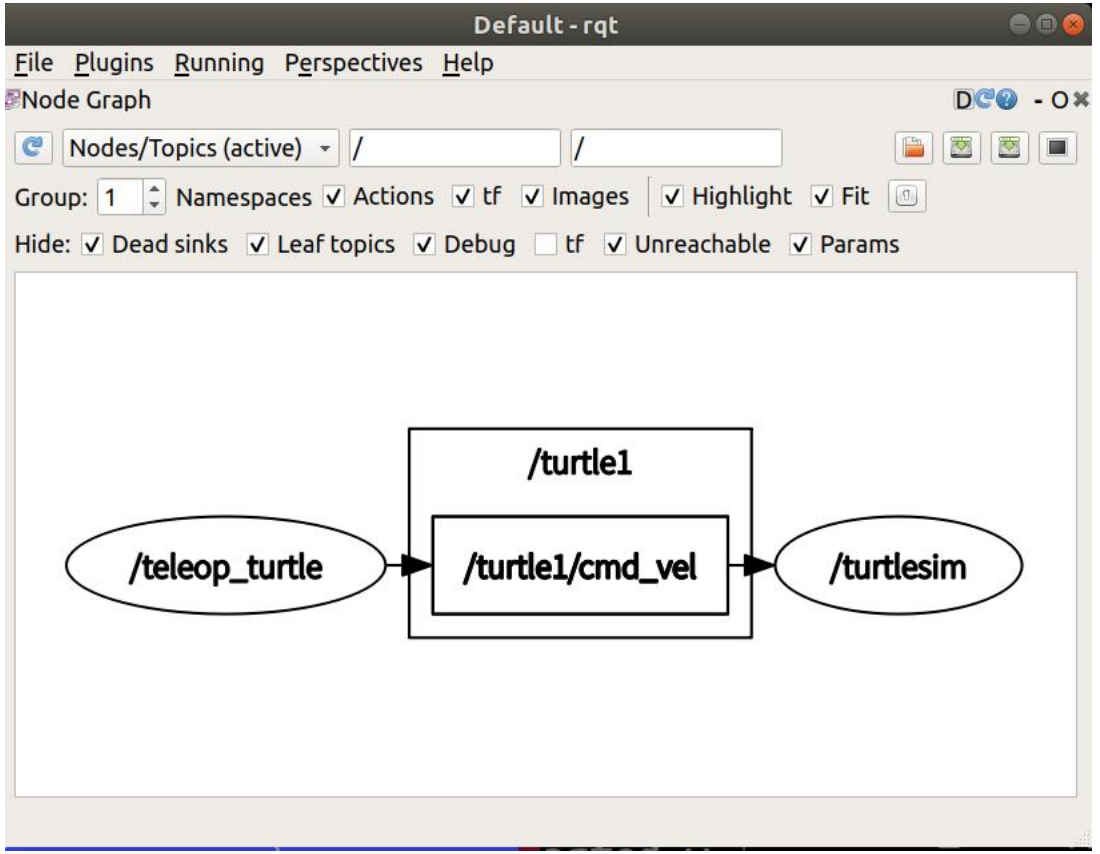
```
$ rosrun rqt_graph rqt_graph
```

```
$ rosrun rqt_graph rqt_graph
```

```
$ rqt
```

- rqt : rqt is a Qt-based framework for GUI development for ROS

https://wiki.ros.org/rqt



椭圆形表示节点，方框表示话题

1.1 Turtlesim Examples

1.1.4 turtlesim节点和话题

- 查看节点/teleop_turtle发送的数据：

```
$ rostopic echo /teleop_turtle
```

- 表明：节点 /teleop_turtle 正在发布节点 /turtlesim 订阅的话题

```
ros@ROS-VirtualBox:~$ rosnode info /teleop_turtle
--------------------------------------------------------------------------------
Node [/teleop_turtle]
Publications:
 * /rosout [rosgraph_msgs/Log]
 * /turtle1/cmd_vel [geometry_msgs/Twist]

Subscriptions: None

Services:
 * /teleop_turtle/get_loggers
 * /teleop_turtle/set_logger_level


contacting node http://ROS-VirtualBox:41265/ ...
Pid: 6387
Connections:
 * topic: /rosout
    * to: /rosout
    * direction: outbound (47331 - 127.0.0.1:48044) [12]
    * transport: TCPROS
 * topic: /turtle1/cmd_vel
    * to: /turtlesim
    * direction: outbound (47331 - 127.0.0.1:48046) [10]
    * transport: TCPROS
```

1.1 Turtlesim Examples

1.1.4 turtlesim节点和话题

- 查看话题 /turtle1/cmd_vel上的消息：

```
$ rostopic echo /turtle1/cmd_vel
```

```
ros@ROS-VirtualBox:~$ rostopic echo
 /turtle1/cmd_vel
linear:
  x: 2.0
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 0.0
```

- 查看话题类型(消息类型)：

```
$ rostopic type /turtle1/cmd_vel
```

```
ros@ROS-VirtualBox:~$ rostopic type
 /turtle1/cmd_vel
geometry_msgs/Twist
```

- 查看消息字段：

```
$ rosmsg info geometry_msgs/Twist
```

```
ros@ROS-VirtualBox:~$ rosmsg info
geometry_msgs/Twist
geometry_msgs/Vector3 linear
  float64 x
  float64 y
  float64 z
geometry_msgs/Vector3 angular
  float64 x
  float64 y
  float64 z
```

1.1 Turtlesim Examples

1.1.4 turtlesim节点和话题

- rostopic pub语法结构：

  rostopic pub /topic msg_type [args...]

- 手动向话题 /turtle1/cmd_vel上发布消息：

```
$ rostopic pub /turtle1/cmd_vel geometry_msgs/Twist [5,0,0] [0,0,0]
```

发布一次

```
$ rostopic pub /turtle1/cmd_vel geometry_msgs/Twist -r 1  [1,0,0] [0,0,1]
```

1秒一次的速率循环发布

```
ros@ROS-VirtualBox:~$ rostopic pub
 /turtle1/cmd_vel geometry_msgs/Tw
ist [5,4,0] [0,0,0]
publishing and latching message. P
ress ctrl-C to terminate
```

```
ros@ROS-VirtualBox:~$ rostopic pub /tu
rtle1/cmd_vel geometry_msgs/Twist -r 1
  [1,0,0] [0,0,1]
```

1.1 Turtlesim Examples

1.1.5 turtlesim服务

- 查询节点/turtlesim可用的服务：

$ rosservice list

```
ros@ROS-VirtualBox:~$ rosservice list
/clear
/kill
/reset
/rosout/get_loggers
/rosout/set_logger_level
/rostopic_7623_1634663943840/get_loggers
/rostopic_7623_1634663943840/set_logger_level
/spawn
/teleop_turtle/get_loggers
/teleop_turtle/set_logger_level
/turtle1/set_pen
/turtle1/teleport_absolute
/turtle1/teleport_relative
/turtlesim/get_loggers
/turtlesim/set_logger_level
```

$ rosnode info /turtlesim

```
ros@ROS-VirtualBox:~$ rosnode info /turtlesim
--------------------------------------------------------------------------------
Node [/turtlesim]
Publications:
 * /rosout [rosgraph_msgs/Log]
 * /turtle1/color_sensor [turtlesim/Color]
 * /turtle1/pose [turtlesim/Pose]

Subscriptions:
 * /turtle1/cmd_vel [geometry_msgs/Twist]

Services:
 * /clear
 * /kill
 * /reset
 * /spawn
 * /turtle1/set_pen
 * /turtle1/teleport_absolute
 * /turtle1/teleport_relative
 * /turtlesim/get_loggers
 * /turtlesim/set_logger_level


contacting node http://ROS-VirtualBox:35153/ ...
Pid: 4355
Connections:
 * topic: /rosout
    * to: /rosout
    * direction: outbound (46185 - 127.0.0.1:56526) [26]
    * transport: TCPROS
 * topic: /turtle1/cmd_vel
    * to: /teleop_turtle (http://ROS-VirtualBox:41265/)
    * direction: inbound (48046 - ROS-VirtualBox:47331) [28]
    * transport: TCPROS
```

1.1 Turtlesim Examples

1.1.5 turtlesim服务

- rosservice call 语法：

```
ros@ROS-VirtualBox:~$ rosservice call --help
Usage: rosservice call /service [args...]

Options:
  -h, --help  show this help message and exit
  -v          print verbose output
  --wait      wait for service to be advertised
```

- 查看服务类型(消息类型)：

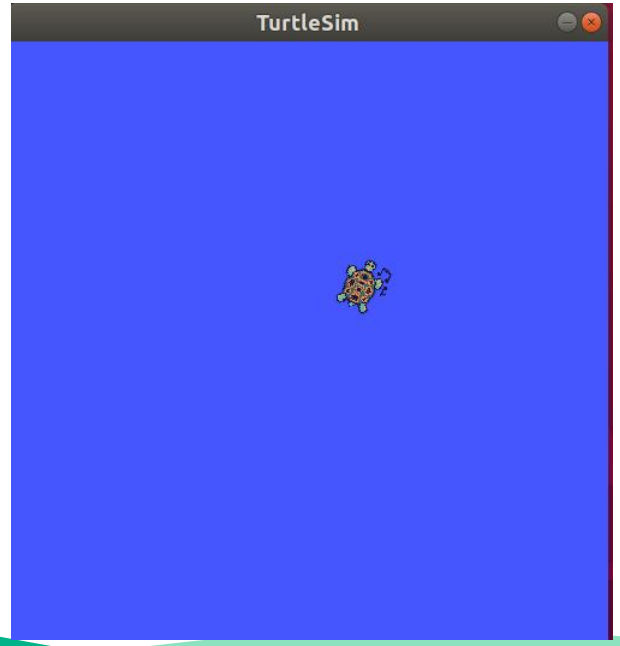
$ rosservice type /clear

```
ros@ROS-VirtualBox:~$ rosservice type /clear
std_srvs/Empty
ros@ROS-VirtualBox:~$ rossrv info std_srvs/Empty
---
```

- 调用服务：

$ rosservice call /clear

```
ros@ROS-VirtualBox:~$ rosservice call /clear
```



1.1 Turtlesim Examples

1.1.5 turtlesim服务

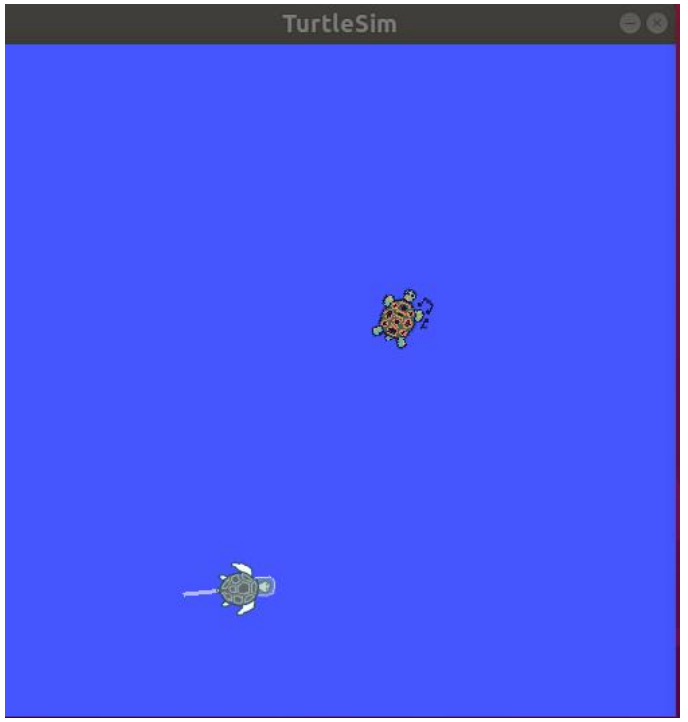
- 查看 /spawn 服务类型(消息类型)：

```
$ rosservice type /spawn | rossrv show
```

```
ros@ROS-VirtualBox:~$ rosservice type /s
pawn | rossrv show
float32 x
float32 y
float32 theta
string name
---
string name
```

- 调用/spawn服务：

```
$ rosservice call /spawn 3 2 0.1 "turtle2"
```



- 向话题/turtle2/cmd_vel发布信息，控制乌龟2运动：

```
$ rostopic pub /turtle2/cmd_vel geometry_msgs/Twist [1,0,0] [0,0,0]
```

1.1 Turtlesim Examples

1.1.6 turtlesim参数

- 查看可用参数：

```
$ rosparam list
```

```
ros@ROS-VirtualBox:~$ rosparam
 list
/rosdistro
/roslaunch/uris/host_ros_virtu
albox__46255
/rosversion
/run_id
/turtlesim/background_b
/turtlesim/background_g
/turtlesim/background_r
```

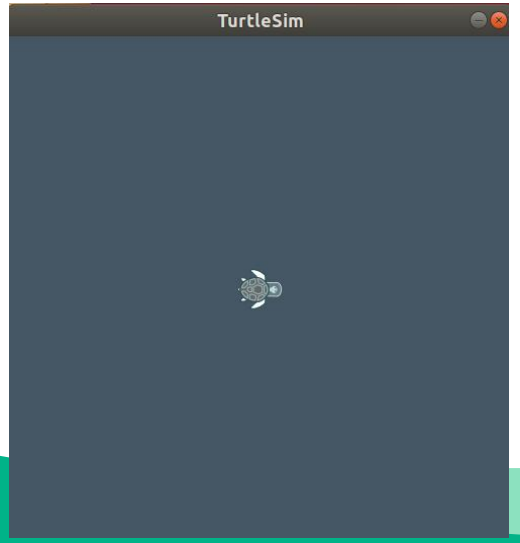
- 获取参数值：

```
$ rosparam get /turtlesim/background_b
```

```
ros@ROS-VirtualBox:~$ rosparam
 get /turtlesim/background_b
255
```

- 设置参数值：

```
$ rosparam set /turtlesim/background_b 100
```



1.1 Turtlesim Examples

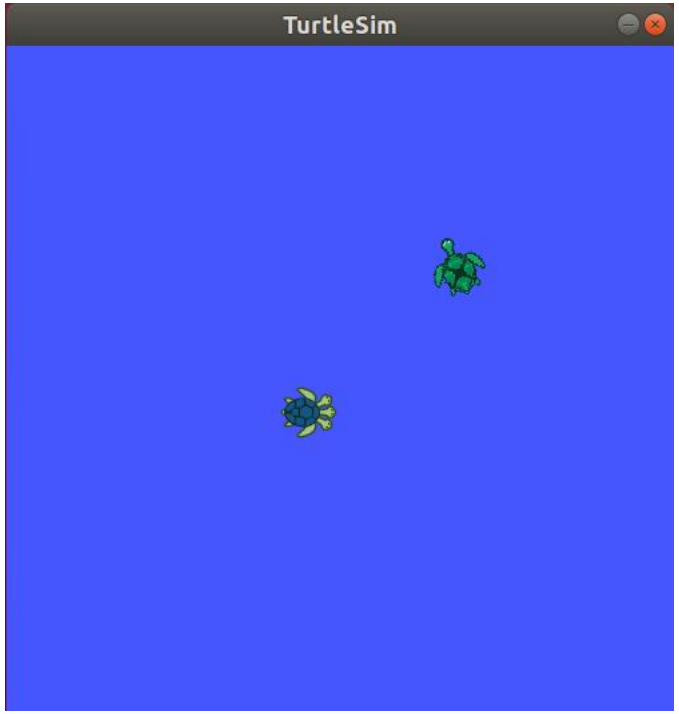
1.1.7 turtlesim mimic节点

- mimic节点的作用是让一个海龟复制另一个海龟的动作

- 启动另一只海龟：

```
$ rosservice call /spawn 5 5 0 "turtle_mimic"
```

```
ros@ROS-VirtualBox:~$ rosservice call /spawn
5 5 0 'turtle_mimic'
name: "turtle_mimic"
```

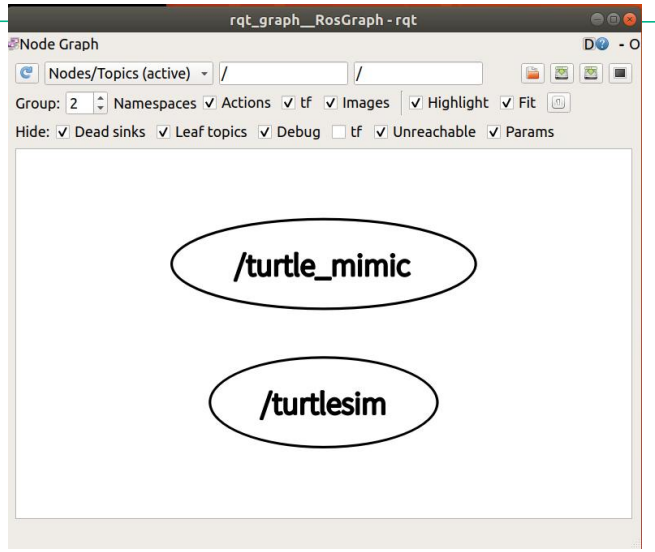


1.1 Turtlesim Examples

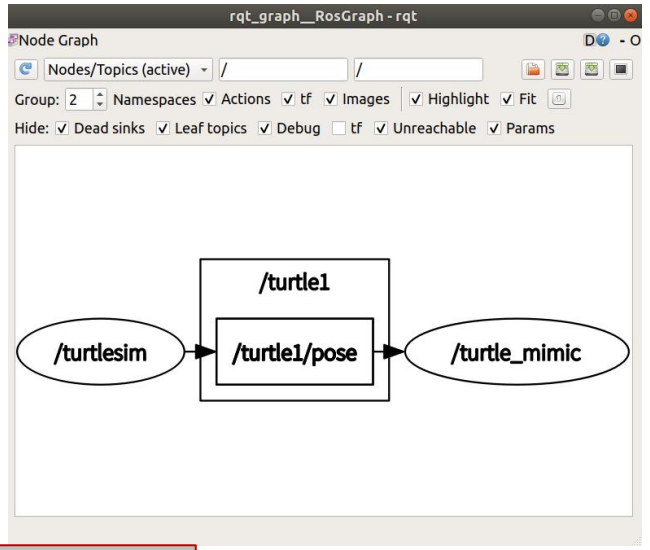
1.1.7 turtlesim mimic节点

- 启动mimic节点：

```
$ rosrun turtlesim mimic
```



- 为了实现动作复制，需要让 mimic 的输入为 turtle1 的 pose，输出为 turtle_mimic 的 cmd_vel，即将 mimic 原本的输入输出 topic 做如下修改：
  - /input/pose ----> /turtle1/pose
  - /output/cmd_vel ----> /turtle2/cmd_vel



- 关掉 mimic，再用如下方式重新启动 mimic：

```
$ rosrun turtlesim mimic input:=turtle1 output:=turtle_mimic
```

后边的参数表示将原程序中的两个参数 input, output，替换为 turtle1, turtle_mimic

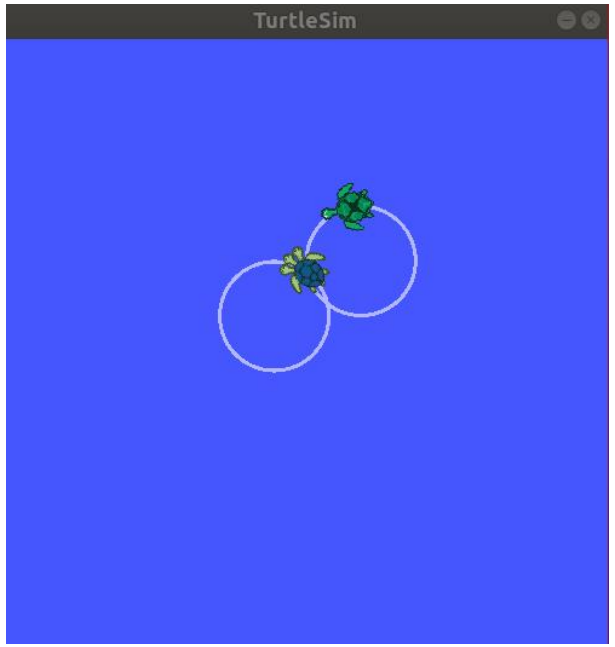
1.1 Turtlesim Examples

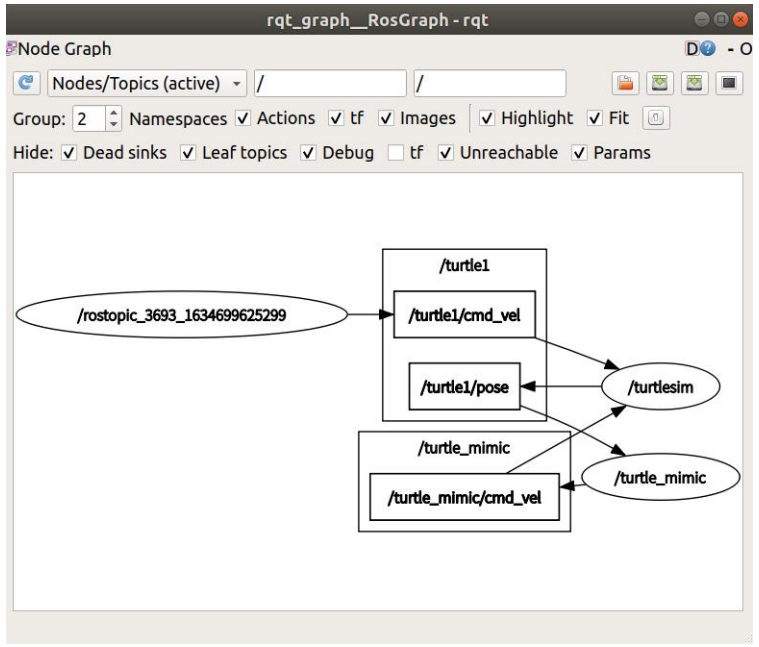
1.1.7 turtlesim mimic节点

- 手动向话题/turtle1/cmd_vel上发布消息：

```
$ rostopic pub /turtle1/cmd_vel geometry_msgs/Twist -r 1  [1,0,0] [0,0,1]
```

1秒/次速率循环发布





作业1：调用draw_square节点，使turtle1画矩形，生成海龟turtle_mimic，使turtle_mimic模仿turtle1的轨迹

参考资料：https://www.jianshu.com/p/5b8893479874

Summary

1. Turtlesim

a) turtlesim introduction, start ros, start turtlesim

b) turtlesim_node(rosnode rospotic) 

c) turtle_teleop_key(rqt, rosmsg, rossrv,rosparam) 

d) mimic(rosservice)

Overview lesson 8-1

1 C++ basic

2 ~

3 ~

4 ~

Objectives

1、掌握 c++ 的基本语法

2、学会在 Linux 中运行 C/C++ 程序

3、了解 OPP 概念

4、学会建立 C++ 工程



Pre-assessment

1、是否在 Linux 中使用过C/C++?

2、是否用过 C++ 编写 ROS 程序?

3、是否了解面向对象编程的概念?



1

C++ basic

1.1 Getting started with C++

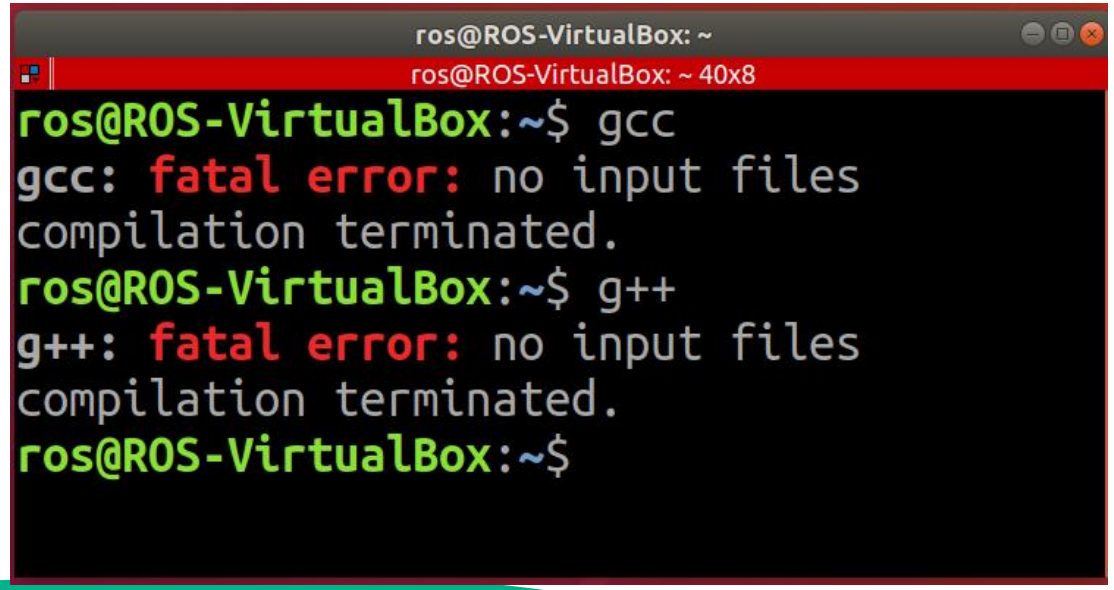
1.1.1 Overview

- C++ 是 C 语言的加强版，C++ 语言最初被称为“带类的C”，由计算机程序员 Bjarne Stroustrup 于1979年开发
- 和 C 一样，C++ 也是一种需要被编译的语言，它需要一个编译器将源代码转换为可执行代码
- Ubuntu自带一个内置的 C/C++编译器，称为 GCC/G++
- GCC 由 Richard Stallman 为 GNU 项目编写的（创立自由软件 GNU、Emacs 文字编辑器、GCC编译器、GDB调试器、GPL协议）
- GCC 代表 GNU 编译器的集合，其可以编译 C、C++、Objective-C、Fortran、Ada 和 Go 等多种语言，以及这些语言的库

1.1 Getting started with C++

1.1.2 在Ubuntu中运行C/C++

- Ubuntu 中预装了 C 和 C++ 编译器，其中 C 语言的编译器是 GCC，C++ 的编译器是 G++
- gcc 和 g++ 是两个编译器的 Shell 命令
- 在终端中运行上述命令，若出现下述错误提示，则说明系统已经预安装这些编译器，否则需要使用 apt 安装



1.1 Getting started with C++

1.1.2 在Ubuntu中运行C/C++

- 安装 C 和 C++ 编译器

```
$ sudo apt-get update
```

```
$ sudo apt-get install build-essential manpages-dev
```

```
$ sudo apt-get install gcc g++
```

1.1.2 在Ubuntu中运行C/C++

- 验证安装 C 和 C++ 编译器

```
$ whereis gcc
```

whereis 查看 gcc/g++ 命令及其使用说明页的路径

```
$ whereis g++
```

```
ros@ROS-VirtualBox:~$ whereis gcc
gcc: /usr/bin/gcc /usr/lib/gcc /usr/share/man/man1/gcc.1.g
z
ros@ROS-VirtualBox:~$ whereis g++
g++: /usr/bin/g++ /usr/share/man/man1/g++.1.gz
ros@ROS-VirtualBox:~$
```

```
$ which gcc
```

显示当前使用的编译器的系统路径

```
$ which g++
```

```
ros@ROS-VirtualBox:~$ which gcc
/usr/bin/gcc
ros@ROS-VirtualBox:~$ which g++
/usr/bin/g++
ros@ROS-VirtualBox:~$
```

1.1.2 在Ubuntu中运行C/C++

- 验证安装 C 和 C++ 编译器

```
$ gcc --version
```

显示编译器的版本号

```
$ g++ --version
```

```
ros@ROS-VirtualBox:~$ gcc --version
gcc (Ubuntu 7.5.0-3ubuntu1~18.04) 7.5.0
Copyright (C) 2017 Free Software Foundation, Inc.
This is free software; see the source for copying conditio
ns.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PA
RTICULAR PURPOSE.
```

```
ros@ROS-VirtualBox:~$ g++ --version
g++ (Ubuntu 7.5.0-3ubuntu1~18.04) 7.5.0
Copyright (C) 2017 Free Software Foundation, Inc.
This is free software; see the source for copying conditio
ns.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PA
RTICULAR PURPOSE.
```

1.1.2 在Ubuntu中运行C/C++

- GNU 项目调试器 GDB 简介
  - 调试器是一个可以运行和控制另一个程序的程序，通过检查每一行代码来检查程序的问题或 Bug
  - Ubuntu 自带一个名为 GNU Debugger 的调试器，简称 GDB(www.gnu.org/software/gdb/)

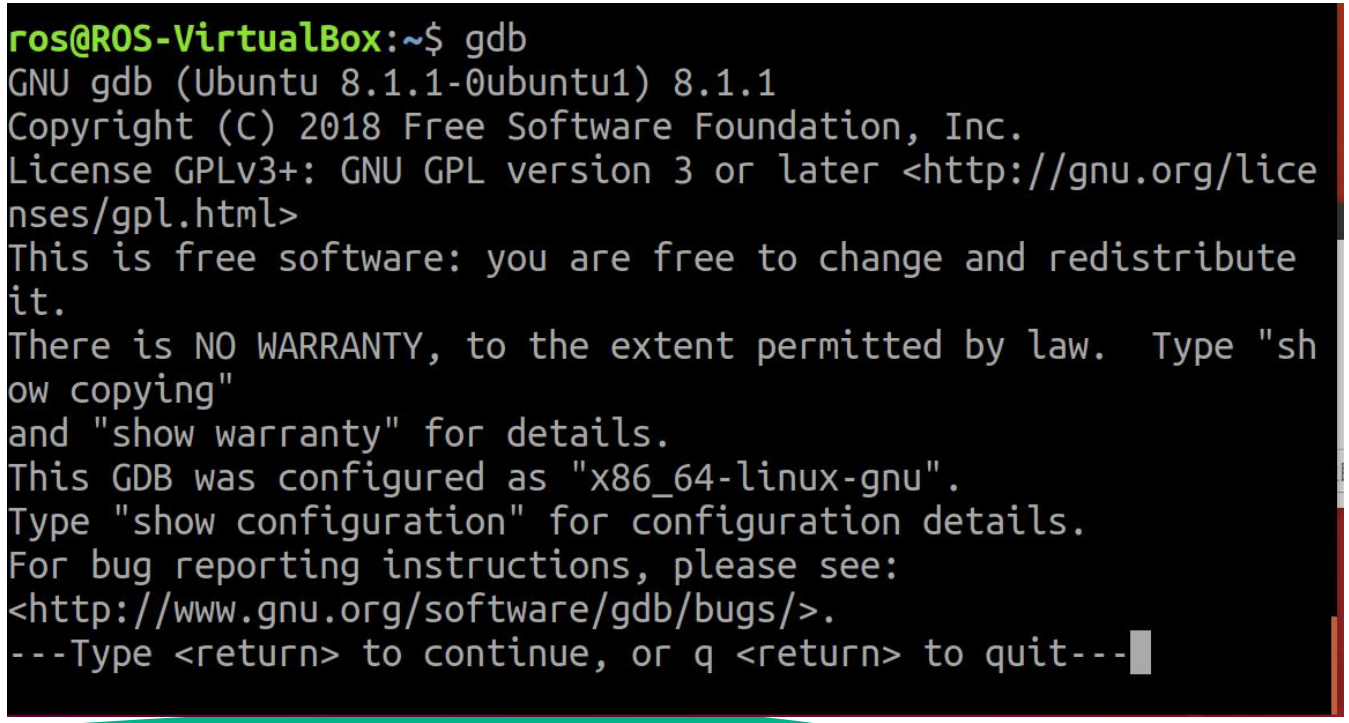
- 在 Ubuntu 中安装 GDB
  - 最新版本的 Ubuntu 已经安装了 GDB，若系统没有预安装，可以使用以下命令安装GDB

```
$ sudo apt-get install gdb
```

1.1.2 在Ubuntu中运行C/C++

- 验证安装GDB
  - 在终端中输入以下命令，验证GDB是否安装成功

```
$ gdb
```




```
$ gdb --version
```

查看 GDB 版本

1.1.2 在Ubuntu中运行C/C++

- 编写第一个C++程序
  - 在家目录中创建 C++_work 目录
  - 进入 C++_work 目录
  - 打开 gedit，键入以下代码：

```
#include <iostream>

using namespace std;
int main()
{
        cout<<"Hello ROS"<<endl;
        return 0;
}
```

  - 保存为hello_ROS.cpp

1.1.2 在Ubuntu中运行C/C++

- 代码解释
  - hello_ROS.cpp 将打印消息“Hello ROS”
  - #include <iostream>是 C++ 头文件，包含输入/输出函数，如从键盘输入或者打印消息到屏幕
  - 第二行使用 namespace std， 即使用命名空间
    - 命名空间是C++中的一个特性，用于对一组实体进行分组
    - iostream 中使用 std 定义命名空间
    - 当使用 namespace std 时，可以访问命名空间 std 中包含的所有函数或者其他实体，如 cout() 和 cin() 等函数
    - 若不使用这行代码，则必须在函数前面添加 std::，表示用于访问该命名空间中的函数，例如：std::cout 是打印一个消息的函数

1.1.2 在Ubuntu中运行C/C++

- 代码解释
  - 主函数中，使用 cout<<”Hello ROS”<<endl 打印一条消息
  - endl 表示在打印消息之后换行
  - 消息打印完毕后，函数返回0并退出程序

1.1.2 在Ubuntu中运行C/C++

- 编译代码
  - 打开一个新终端，将新终端的路径切换到 hello_ROS.cpp 所在目录
  - 在终端中键入以下命令进行编译：

```
$ g++ hello_ROS.cpp
```

  - G++ 编译器检查代码，若没有错误，其会创建一个名为 a.out 的可执行文件
  - 使用以下命令执行此文件：

```
$ ./a.out
```

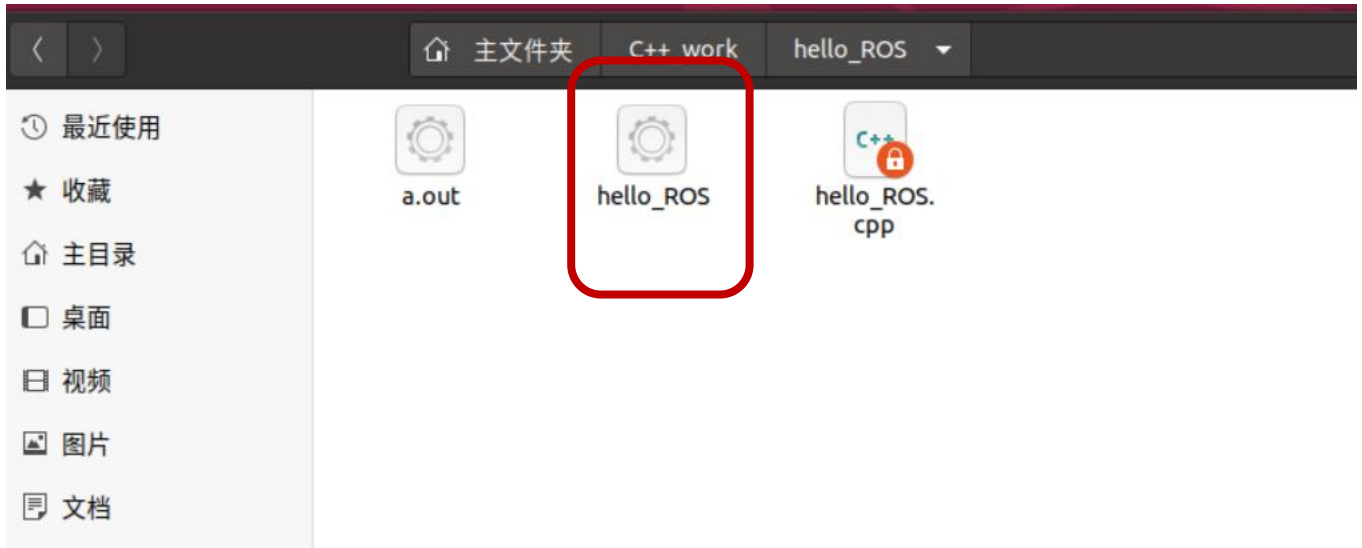
```
ros@ROS-VirtualBox:~/C++_work$ ./a.out
Hello ROS
```

1.1.2 在Ubuntu中运行C/C++

➢ 编译代码

- 若要创建具有特定名称的可执行文件，可使用如下命令：

```
$ g++ hello_ROS.cpp -o hello_ROS
```




```
$ ./hello_ROS
```

1.1.2 在Ubuntu中运行C/C++

- 代码调试
  - 使用调试器，可以遍历每一行代码并检查每个变量的值
  - 新建 sum.cpp，并键入以下代码：

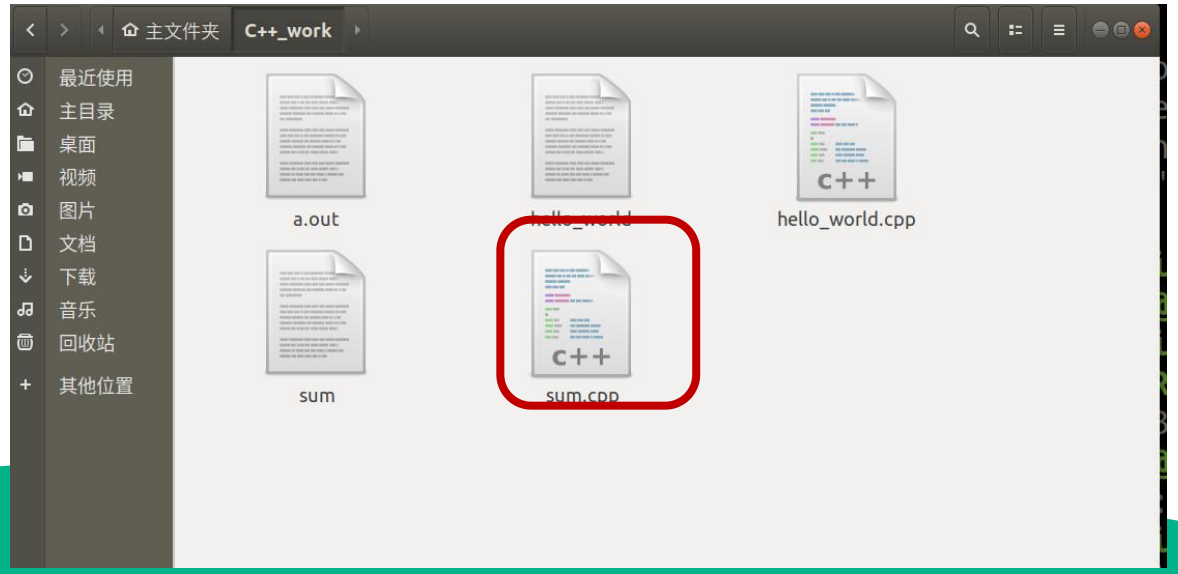
```
#include <iostream>

using namespace std;
int main()
{
        int num_1 = 3;
        int num_2 = 4;
        int sum = num_1 + num_2;
        cout<<"Sum: "<<sum<<endl;
        return 0;
}
```

1.1.2 在Ubuntu中运行C/C++

- 代码调试
  - 要调试/检查每一行代码，必须先使用 g++ -g 选项编译 sum.cpp 文件
  - 上述命令使用调试符号编译代码，并使其能够适用于 GDB 调试
  - 在终端中键入如下代码：

```
$ g++ -g sum.cpp -o sum
```



1.1.2 在Ubuntu中运行C/C++

- 代码调试
  - 在终端中运行以下命令：

```
$ ./sum
```

```
ros@ROS-VirtualBox:~/C++_work$ ./sum
Sum: 7
```

  - 创建可执行文件后，通过以下命令调试可执行文件：

```
$ gdb sum
```

1.1.2 在Ubuntu中运行C/C++

- 代码调试

  - sum 是可执行文件的名称，键入上述命令后，必须使用 GDB 命令进行调试，以下是常用的 GDN 命令：

    1. b line_number： 在给定行处创建断点，调试过程中，调试器会在断点处停止
    2. n: 运行下一行代码
    3. r: 运行程序至断点处
    4. p variable_name: 打印变量的值
    5. q: 退出调试器

1.1 Getting started with C++

1.1.2 在Ubuntu中运行C/C++

- 代码调试

```
ros@ROS-VirtualBox:~/C++_work$ gdb sum
GNU gdb (Ubuntu 8.1.1-0ubuntu1) 8.1.1
Copyright (C) 2018 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.  Type "show copying"
and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from sum...done.
(gdb) b 5
Breakpoint 1 at 0x912: file sum.cpp, line 5.
(gdb) r
Starting program: /home/ros/C++_work/sum

Breakpoint 1, main () at sum.cpp:7
7               int num_1 = 3;
(gdb) n
8               int num_2 = 4;
(gdb) p num_1
$1 = 3
(gdb) r
The program being debugged has been started already.
Start it from the beginning? (y or n)
```

1.1.3 C++中OPP概念

- OPP概述
  - OPP(Object Oriented Programming，面向对象的编程)
  - 在 C 的结构体中，用户可以将不同类型的数据(整数、浮点数和字符串)封装到一个数据类型中
  - 相似的，C++有一个增强版的结构体，在该结构体中可以定义函数
  - 该结构体称为 C++ 类，C++ 类的每个实例都被称为对象，这种编程叫做面向对象的编程

1.1.3 C++中OPP概念

- 类和结构体之间的区别
  - C++中类和结构体的示例代码

```
#include <iostream>
#include <string>
using namespace std;
struct Robot_Struct
{
        int id;
        int num_wheels;
        string robot_name;
};
```

1.1.3 C++中OPP概念

- 类和结构体之间的区别
  - C++中类和结构体的示例代码

```
class Robot_Class
{
public:
        int id;
        int num_wheels;
        string robot_name;
        void move_robot();
        void stop_robot();
};
```

1.1.3 C++中OPP概念

- 类和结构体之间的区别
  - C++中类和结构体的示例代码

```
void Robot_Class::move_robot()
{
        cout<<"Moving Robot!"<<endl;
}
void Robot_Class::stop_robot()
{
        cout<<"Stopping Robot!"<<endl;
}
```

1.1.3 C++中OPP概念

- 类和结构体之间的区别
  - C++中类和结构体的示例代码

```
int main()
{
        Robot_Struct robot_1;
        Robot_Class robot_2;
        robot_1.id=2;
        robot_1.robot_name="Mobile robot";
        robot_2.id=3;
        robot_2.robot_name="Humanoid robot";
        cout<<"ID="<<robot_1.id<<"\t"<<"Robot Name"<<robot_1.robot_name<<endl;
        cout<<"ID="<<robot_2.id<<"\t"<<"Robot Name"<<robot_2.robot_name<<endl;
        robot_2.move_robot();
        robot_2.stop_robot();
        return 0;
}
```

1.1.3 C++中OPP概念

- 类和结构体之间的区别
  - C++ 中类和结构体的示例代码
  - 这段代码定义了一个类和结构体。结构体的名称是 Robot_Struct，类名是 Robot_Class
  - 以下为定义的结构体。该结构体中定义了三个变量，分别是机器人编号 id、轮子的数量 num_wheels 和名称 robot_name

```
struct Robot_Struct
{
        int id;
        int num_wheels;
        string robot_name;
};
```

1.1.3 C++中OPP概念

- 类和结构体之间的区别
  - 下图左为定义的类，类中不仅可以定义变量，还可以定义函数
  - 可以在类的内部或者外部定义函数，一般在类的外部定义，以保证类的简洁性
  - 下图右为定义的类函数(成员函数)
    - ✓ 第一行第一项是返回值类型，其次是类名称，最后是用::连接的函数名，表明函数包含在类中
    - ✓ 在函数定义中，可以添加特定功能的代码，此处代码用来打印一个消息

```
class Robot_Class
{
public:
        int id;
        int num_wheels;
        string robot_name;
        void move_robot();
        void stop_robot();
};
```

```
void Robot_Class::move_robot()
{
        cout<<"Moving Robot!"<<endl;
}
void Robot_Class::stop_robot()
{
        cout<<"Stopping Robot!"<<endl;
}
```

1.1.3 C++中OPP概念

- C++的类和对象
  - 以下为创建的结构体和类的实例代码

```
Robot_Struct robot_1;
Robot_Class robot_2;
robot_1.id=2;
robot_1.robot_name="Mobile robot";
robot_2.id=3;
robot_2.robot_name="Humanoid robot";
```

  - 其中 robot_1 为结构体，robot_2 为类，类的实例称为对象
  - 因此，robot_2 是一个类对象，robot_1 是结构体的一个对象
  - 结构体和类中的变量可以使用“.”操作符进行访问
  - 如果使用结构体或者类的指针，必须使用“->”操作符访问每个变量

1.1.3 C++中OPP概念

- C++的类和对象
  - 将上述代码保存为 C++_class_test.cpp，调用 g++ 进行编译：

```
$ g++ C++_class_test.cpp -o class_struct
```

```
$ ./class_struct
```

```
ros@ROS-VirtualBox:~/C++_work$ g++ C++_class_test.cpp -o class_struct
ros@ROS-VirtualBox:~/C++_work$ ./class_struct
ID=2    Robot NmaeMobile robot
ID=3    Robot NmaeHumanoid robot
Moving Robot!
Stopping Robot!
```

1.1.3 C++中OPP概念

- 类访问修饰符
  - 上述代码中有 public: 关键字，其被称为成员访问修饰符
  - 通过设置成员访问修饰符，可以限制在类内部定义的函数的使用范围
  - 类中有3种访问修饰符：
    - public(共有)：一个公有成员可以从类以外的任何地方被访问。用户可以直接访问公共变量，不需要编写其他辅助函数
    - private(私有)：使用该修饰符号的成员变量或函数，不能从类的外部被查看或者访问。只有类和友元函数可以访问私有成员
    - protected(保护)：访问权限类似于private，不同的是其子类/派生类可以访问其成员
  - 成员修饰符可以帮助用户将变量分组，这些变量在类中保持可见或者隐藏

1.1.3 C++中OPP概念

- C++中 Inheritance 的使用
  - Inheritance(继承)是 OPP 中的一个重要概念
  - 如有已有两个或多个类，并且在新建的类中希望使用这些已有类中的函数，可以使用 Inheritance 属性
  - 用户可以在新类中重用现有类中的函数，继承现有类的新类，称为派生类
  - 类可以通过 public、private 和 protected 来继承
    - public inheritance： 当派生自 public 基类时，基类的 public 成员成为派生类的 public 成员，并且基类的 protected 成员成为派生类的 protected 成员；基类的 private 成员不能在派生类中访问，但可以通过基类的 public 和 protected 成员函数的调用来访问

1.1.3 C++中OPP概念

- **private inheritance：当派生自 private 基类时，基类的 public 和 protected 成员成为派生类的 private 成员**
- **protected inheritance：当使用 protected 基类继承时，基类的 public 和 protected 成员成为派生类的 protected 成员**

1.1.3 C++中OPP概念

- C++中 Inheritance 的使用

```
#include <iostream>
#include <string>

using namespace std;
class Robot_Class
{
public:
        int id;
        int num_wheels;
        string robot_name;
        void move_robot();
        void stop_robot();
};
```

1.1.3 C++中OPP概念

- C++中 Inheritance 的使用

```
class Robot_Class_Derived:public Robot_Class
{
public:
        void turn_left();
        void turn_right();
};
```

1.1.3 C++中OPP概念

- C++中 Inheritance 的使用

```
void Robot_Class::move_robot()
{
        cout<<"Moving Robot"<<endl;
};
void Robot_Class::stop_robot()
{
        cout<<"Stopping Robot"<<endl;
};
void Robot_Class_Derived::turn_left()
{
        cout<<"Robot Turn left"<<endl;
};
void Robot_Class_Derived::turn_right()
{
        cout<<"Robot Turn right"<<endl;
};
```

1.1.3 C++中OPP概念

- C++中 Inheritance 的使用

```
int main()
{
        Robot_Class_Derived robot;
        robot.id=2;
        robot.robot_name="Mobile robot";
        cout<<"Robot ID="<<robot.id<<endl;
        cout<<"Robot Name="<<robot.robot_name<<endl;
        robot.move_robot();
        robot.stop_robot();
        robot.turn_left();
        robot.turn_right();
        return 0;
};
```

1.1.3 C++中OPP概念

- Inheritance 代码解释
  - 本程序中，创建了一个名为 Robot_Class_Derived 的新类，其派生自 Robot_Class 基类
  - 公共继承时通过关键字 public 和其后面的基类名来实现的，即在派生类名之后添加一个“：”紧接着一个 public 关键字和基类名

```
class Robot_Class_Derived:public Robot_Class
{
public:
        void turn_left();
        void turn_right();
};
```

  - 本程序中，因为选择了 public，因此继承类可以访问基类的 public 和 protected 修饰的变量和函数

1.1.3 C++中OPP概念

- Inheritance 代码解释
  - 派生类函数的定义如下：

```
void Robot_Class_Derived::turn_left()
{
        cout<<"Robot Turn left"<<endl;
}
void Robot_Class_Derived::turn_right()
{
        cout<<"Robot Turn right"<<endl;
}
```

  - 访问派生类中的变量和函数：

```
Robot_Class_Derived robot;
robot.id=2;
robot.robot_name="Mobile robot";
cout<<"Robot ID="<<robot.id<<endl;
cout<<"Robot Name="<<robot.robot_name<<endl;
robot.move_robot();
robot.stop_robot();
robot.turn_left();
robot.turn_right();
```

1.1.3 C++中OPP概念

- Inheritance 代码解释
  - 由上述代码可知，在 Robot_Class_Derived 中没有声明 id 和 robot_name 变量，而是在 Robot_Class 中定义的
  - 通过使用 inheritance 属性，可以在派生类 Robot_Class_Derived 中使用基类 Robot_Class 的变量
  - 对上述代码进行编译：

```
$ g++ C++_class_inheritance.cpp -o class_inheritance
```

```
$ ./class_inheritance
```

```
ros@ROS-VirtualBox:~/C++_work$ g++ C++_class_inheritance.cpp -o class_inheritanc
ros@ROS-VirtualBox:~/C++_work$ ./class_
class_inheritance  class_struct
ros@ROS-VirtualBox:~/C++_work$ ./class_inheritance
Robot ID=2
Robot Name=Mobile robot
Moving Robot
Stopping Robot
Robot Turn left
Robot Turn right
```

1.1.3 C++中OPP概念

- C++命名空间
  - 创建和使用命名空间的示例

```
#include <iostream>

using namespace std;
namespace robot
{
        void process(void)
        {
                cout<<"Processing by Robot"<<endl;
        }
}
```

1.1.3 C++中OPP概念

- C++命名空间
  - 创建和使用命名空间的示例

```
namespace machine
{
        void process(void)
        {
                cout<<"Processing by Machine"<<endl;
        }
}

int main()
{
        robot::process();
        machine::process();
}
```

1.1.3 C++中OPP概念

- C++命名空间
  - 创建命名空间，需要用关键词“namespace”，后面紧跟命名空间的名称
  - 从上述代码中可看出，定义了两个命名空间，命名空间中定义了相同的函数
  - 命名空间用来实现对某一特定操作的一系列函数或者类进行分组
  - 通过使用命名空间名称，后面跟“::函数名”的方式，可以访问命名空间中的成员
  - 此段代码中调用两个函数：robot::process() 和 machine::process()
  - 代码编译和运行结果如下：

```
ros@ROS-VirtualBox:~/C++_work$ g++ C++_namespace.cpp -o namespace
ros@ROS-VirtualBox:~/C++_work$ ./namespace
Processing by Rbot
Processing by Rbot
```

1.1.4 建立一个C++工程

- 建立一个Linux makefile
  - Linux Makefile 可以方便的利用一个指令来编译一个或多个源文件，并生成可执行文件
  - 在编写 C++ 类的时候，可以在源代码中声明和定义类，也可也使用 .h 文件和 .cpp 文件，并把 .h 文件包含到 .cpp 文件中的方式声明和定义类
  - main.cpp：将要构建的主要代码文件
  - add.h：add类的头文件，包含一个类的声明
  - add.cpp：包含类的全部定义
  - 建议：最好将类、头文件、.cpp文件用同一个名称命名。此处建立 add 类，所以头文件命名为 add.h 和 add.cpp

1.1.4 建立一个C++工程

- 建立一个Linux makefile
  - 代码演示:

```
//add.h
#include <iostream>


class add
{
public:
    int compute(int num_1, int num_2);
};
```

1.1.4 建立一个C++工程

- 建立一个Linux makefile
  - 代码演示：

```
//add.cpp
#include "add.h"


int add::compute(int a, int b)
{
    return(a+b);
}
```

1.1.4 建立一个C++工程

- 建立一个Linux makefile
  - 代码演示：

```
//main.cpp
#include "add.h"

using namespace std;
int main()
{
        add obj;
        int result = obj.compute(3,4);
        cout<<"The result:="<<result<<endl;
        return 0;
}
```

1.1.4 建立一个C++工程

- 建立一个Linux makefile

  - 上述代码中，main.cpp 中首先包含 add.h 头文件来访问 add 类，然后创建一个 add 类的对象，并将两个参数传递给 compute() 函数，最后打印结果

  - 使用以下命令进行编译和运行：

```
$ g++ add.cpp main.cpp -o main
```

```
$ ./main
```

```
ros@ROS-VirtualBox:~/C++_work$ g++ add.cpp main.cpp -o main
ros@ROS-VirtualBox:~/C++_work$ ./main
The result:=7
```

1.1.4 建立一个C++工程

- 建立一个Linux makefile
  - Linux makefile 提供了一种组织、编译多个源代码文件的方法
  - 以下为上述代码的 makefile 文件

```
CC = g++
CFLAGS = -c
SOURCES =  main.cpp add.cpp
OBJECTS = $(SOURCES:.cpp=.o)
EXECUTABLE = main
all: $(OBJECTS) $(EXECUTABLE)
$(EXECUTABLE) : $(OBJECTS)
                    $(CC) $(OBJECTS) -o $@
.cpp.o: *.h
          $(CC) $(CFLAGS) $< -o $@
clean :
          -rm -f $(OBJECTS) $(EXECUTABLE)
.PHONY: all clean
```

1.1.4 建立一个C++工程

- 建立一个Linux makefile
  - 编写makefile之后，需要使用以下命令来编译

```
$ make
```

```
ros@ROS-VirtualBox:~/C++_work$ make
g++ -c main.cpp -o main.o
g++ -c add.cpp -o add.o
g++ main.o add.o -o main
```

  - 编译以后，用以下命令来执行程序

```
$ ./main
```

```
ros@ROS-VirtualBox:~/C++_work$ ./main
The result:=7
```

ROS Summary

1. C++ basic

a) 在Ubuntu中运行C/C++

1. 安装 GCC 和 G++编译器
2. 安装和使用 GDB 调试器
3. 编写并调试 C++ 程序

b) C++中OPP概念

1. 类和结构体的定义与区别
2. C++ 中的类和对象
3. 类的访问修饰符
4. 类的 Inheritance 属性
5. 命名空间

c) 建立C++工程

1. Linux makefile

Overview lesson 8-2

1. Roslaunch
2. ~
3. ~
4. ~

Objectives

1、掌握 roslaunch 编写方法

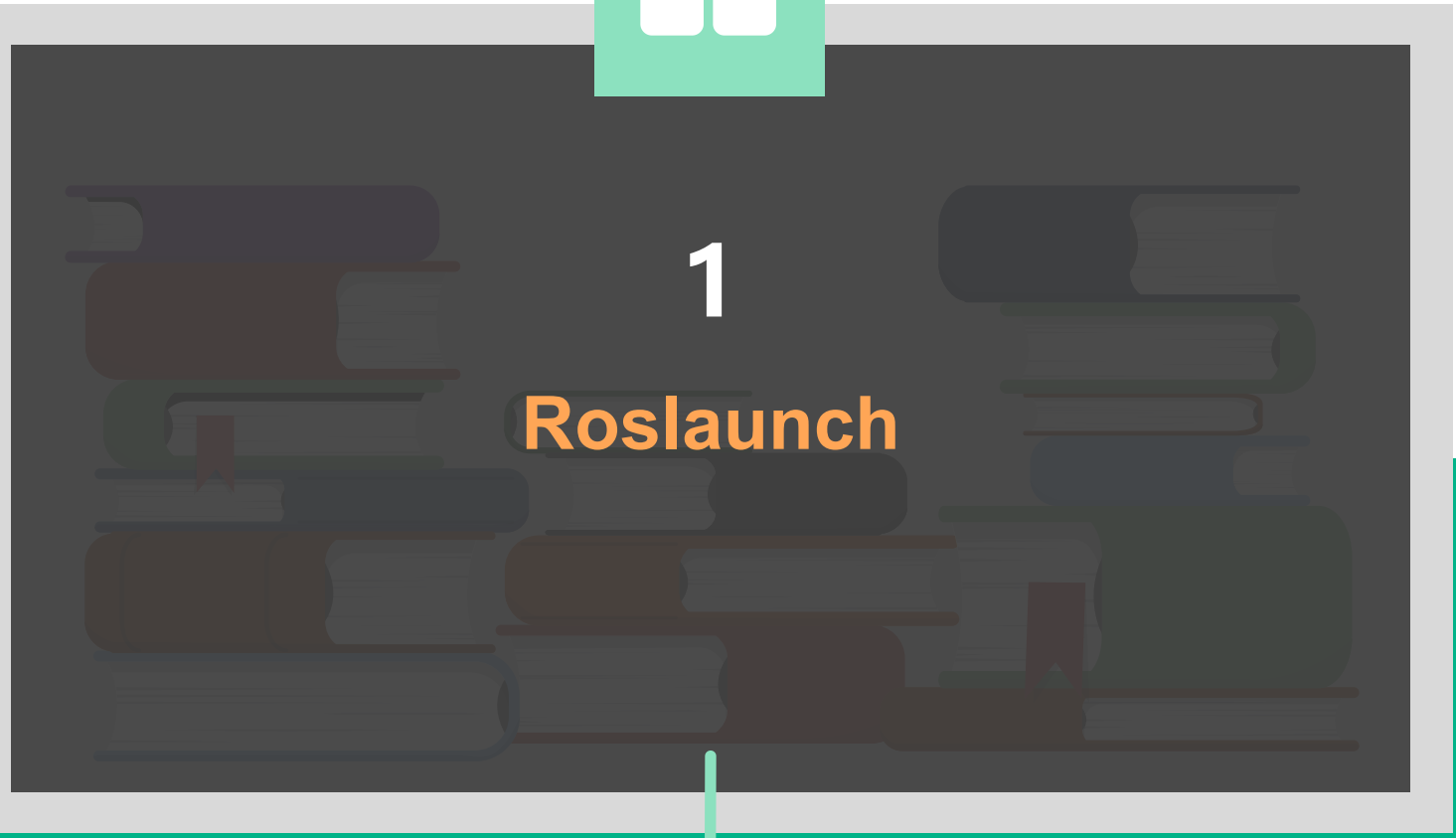
2、熟练使用 roslaunch中的 <node> 标签



Pre-assessment

1、是否编写过roslaunch文件?

2、是否用过c++编写ROS程序?



1

Roslaunch

1.1 Rosluanch format

1.1.1 Overview： http://wiki.ros.org/roslaunch/

- roslaunch 是一个工具，可以通过 SSH 在本地和远程轻松启动多个 ROS 节点，以及在参数服务器上设置参数
- roslaunch 接收一个或多个 XML 配置文件（带有 .launch 扩展名），这些文件指定要设置的参数和要启动的节点，以及它们应该在哪些机器上运行
- rosluanch 的语法：

```
$ roslaunch package_name file.launch
```

1.1 Rosluanch format

1.1.2 Roslaunch XML: http://wiki.ros.org/roslaunch/XML

- roslaunch tag reference:

<launch>	Root tag
<node>	Launch a node
<machine>	Declare a machine to use for launching
<include>	Include other roslaunch files
<remap>	Declare a name remapping
<env>	Specify an environment variable for launched nodes
<param>	Set a parameter on the Parameter Server
<rosparam>	Set ROS parameters for the launch using a rosparam file
<group>	Group enclosed elements sharing a namespace or remap
<test>	Launch a test node
<arg>	Declare an argument

1.1 Rosluanch format

1.1.2 Roslaunch XML： http://wiki.ros.org/roslaunch/XML/launch

1. <launch>：roslaunch 文件的根标签，它的目的是充当其他元素的容器

   - Elements：<node>、<param>、<remap>、<machine>、<rosparam>、<include>、<env>、<test>、<arg>、<group>

2. <node> ：用于指定需要启动的 ROS 节点，用于启动和关闭节点，启动的每一个节点(node)都要有自己独一无二的名字(name)

   - <node> 标签属性：pkg、type、name、args、machine、respawn、required、ns、output、cwd、launch-prefix
   - <node> 标签中至少包含 name、pkg 和 type 属性
   - 两种使用方法：

```
<node pkg="package-name" type="node-name" name="name"  />

<node pkg="package-name" type="node-name" name="name"></node>
```

1.1 Rosluanch format

1.1.2 Roslaunch XML： http://wiki.ros.org/roslaunch/XML/node

2. <node>： 指定需要启动的 ROS 节点，用于启动和关闭节点

pkg="mypackage"→Package of node, 功能包名称

type="nodetype" →Node type. There must be a corresponding executable with the same name, 节点名称

name="nodename" →Node name. NOTE: name cannot contain a namespace. Use the ns attribute instead, 节点命名

args="arg1 arg2 arg3"(optional) →Pass arguments to node, 传递节点参数

machine="machine-name"(optional, see <machine>) →Launch node on designated machine

respawn="true"(optional, default: False)→Restart the node automatically if it quits

respawn_delay="30" (optional, default 0) New in ROS indigo→If respawn is true, wait respawn_delay seconds after the node failure is detected before attempting restart

required="true"(optional)→If node dies, kill entire roslaunch

1.1 Rosluanch format

1.1.2 Roslaunch XML

2. **<node>： 指定需要启动的 ROS 节点，用于启动和关闭节点**

ns="foo"(optional)→Start the node in the 'foo' namespace，**使用命名空间**

clear_params="true|false"(optional)→Delete all parameters in the node's private namespace before launch.

output="log|screen"(optional)→If 'screen', stdout/stderr from the node will be sent to the screen. If 'log', the stdout/stderr output will be sent to a log file in $ROS_HOME/log, and stderr will continue to be sent to screen. The default is 'log'，**显示输出结果**

cwd="ROS_HOME|node"(optional)→If 'node', the working directory of the node will be set to the same directory as the node's executable. In C Turtle, the default is 'ROS_HOME'. In Box Turtle (ROS 1.0.x), the default is 'ros-root'. The use of 'ros-root' is deprecated in C Turtle.

launch-prefix="prefix arguments"(optional)→Command/arguments to prepend to node's launch arguments. This is a powerful feature that enables you to enable gdb, valgrind, xterm, nice, or other handy tools.

1.1 Rosluanch format

1.1.2 Roslaunch XML

2. <node>： 指定需要启动的 ROS 节点，用于启动和关闭节点

属性	属性作用
`name="NODE_NAME"`	为节点指派名称，这将会覆盖掉 `ros::init()` 定义的node_name
`pkg="PACKAGE_NAME"`	节点所在的包名
`type="FILE_NAME"`	执行文件的名称如果是用Python编写的就填写 `xxx.py` ，如果是cpp就写编译生成的可执行文件名
`output="screen"`	终端输出转储在当前的控制台上，而不是在日志文件中
`respawn="true"`	当roslaunch启动完所有该启动的节点之后，会监测每一个节点，保证它们正常的运行状态。对于任意节点，当它终止时，roslaunch 会将该节点重启
`required="true"`	当被此属性标记的节点终止时，roslaunch会将其他的节点一并终止。注意此属性不可以与 `respawn="true"` 一起描述同一个节点
`launch-prefix = "command-prefix"`	相当于在执行启动命令时加上一段命令前缀
`ns = "NAME_SPACE"`	这个属性可以让你在自定义的命名空间里运行节点

1.1 Rosluanch format

1.1.2 Roslaunch XML

2. <node>： 指定需要启动的 ROS 节点，用于启动和关闭节点

- 在 <node> 标签内使用以下 XML 标签：

  <env>→Set an environment variable for the node.

  <remap>→Set a remapping argument for this node.

  <rosparam>→Load a rosparam file into this node's ~/local namespace.

  <param>→Set a parameter in the node's ~/local namespace.

举例：

```
<node name="listener1" pkg="rospy_tutorials" type="listener.py" args="--test" respawn="true" />
```

说明：使用 rospy_tutorials 功能包中 listener.py 可执行文件和命令行参数 --test 启动“listener1”节点， 如果节点终止，它将自动重启

1.1 Rosluanch format

1.1.2 Roslaunch XML http://wiki.ros.org/roslaunch/XML/remap

3. <machine>：声明了一台可以运行 ROS 节点的机器。 如果在本地启动所有节点，则不需要此标签。其主要用于为远程机器声明 SSH 和 ROS 环境变量设置，也可以使用它来声明有关本地机器的信息

4. <include>：将另一个 roslaunch 文件导入当前文件。它将在文档的当前范围内导入，包括 <group> 和 <remap> 标签。包含文件中的所有内容都将被导入

5. <remap>：用于节点映射。对于有关特定 ROS 主题的消息，该消息通常只发送到一组节点，其他节点无法接收该消息。这种情况，可以进行重新映射，以便新节点在认为订阅 /different_topic 时最终订阅 /needed_topic。<remap>基本语法为：

```
<remap from="/different_topic" to="/needed_topic"/>
```

1.1 Rosluanch format

1.1.2 Roslaunch XML： http://wiki.ros.org/roslaunch/XML/remap

5. <remap>：用于节点映射

remap attributes

from="original-name"→Remapped topic: name of the ROS topic that you are remapping FROM.

to="new-name"→Target name: name of the ROS topic that you are pointing the from topic TO.

- 举例：

假定一个节点它订阅了“chatter”主题，但只有一个发布到“hello”主题的节点。它们是相同的类型，此时希望将“hello”主题传输到需要“chatter”的新节点中。 可以使用此重新映射执行此操作：

```
<remap from="chatter" to="hello"/>
```

1.1 Rosluanch format

1.1.2 Roslaunch XML

5. <remap>：用于节点映射

- 注意：

1. 重映射适用于重映射之后的行。在任何重映射行之前启动的节点不受影响

2. 重映射会影响节点订阅或发布的主题。但是，通常重映射是在订阅节点上完成的，这意味着它实际上是在订阅重映射的主题。因此，任何发布节点最终都会通过发布到 remap 命令的 to 字段中指定的主题来将它们的消息发送到该节点！

1.1 Rosluanch format

1.1.2 Roslaunch XML： http://wiki.ros.org/roslaunch/XML/param

6. <env>：允许在启动的节点上设置环境变量。 此标签只能在 <launch>、<include>、<node> 或 <machine> 标签的范围内使用。 当它在 <launch> 标签内使用时，<env> 标签只适用于之后声明的节点。 一些环境变量可以在 Environment Variables 中找到

7. <param>：定义了要在参数服务器上设置的参数，相当于命令行中的 rosparam set 。可以指定文本文件、二进制文件或命令属性来设置参数的值。<param> 标签可以放在 <node> 标签内，在这种情况下，参数被视为私有参数

1.1 Rosluanch format

1.1.2 Roslaunch XML

7. **<param>：定义了要在参数服务器上设置的参数，相当于命令行中的 rosparam set**

name="namespace/name"→Parameter name. Namespaces can be included in the parameter name, but globally specified names should be avoided.

value="value"(optional)→Defines the value of the parameter. If this attribute is omitted, **binfile, textfile or command must be specified**.

textfile="$(find pkg-name)/path/file.txt"(optional)→The contents of the file will be read and stored as a string. The file must be locally accessible, though it is strongly recommended that you use the package-relative $(find)/file.txt syntax to specify the location.

1.1 Rosluanch format

1.1.2 Roslaunch XML

7. **<param>：定义了要在参数服务器上设置的参数，相当于命令行中的 rosparam set**

binfile="$(find pkg-name)/path/file"(optional)→The contents of the file will be read and stored as a base64-encoded XML-RPC binary object. The file must be locally accessible, though it is strongly recommended that you use the package-relative $(find)/file.txt syntax to specify the location.

command="$(find pkg-name)/exe '$(find pkg-name)/arg.txt'"(optional)→The output of the command will be read and stored as a string. It is strongly recommended that you use the package-relative $(find)/file.txt syntax to specify file arguments. You should also quote file arguments using single quotes due to XML escaping requirements.

type="str|int|double|bool|yaml"(optional)→Specifies the type of the parameter. If you don't specify the type, roslaunch will attempt to automatically determine the type. These rules are very basic:

- numbers with '.'s are floating point, integers otherwise;
- "true" and "false" are boolean (not case-sensitive).
- all other values are strings

1.1 Rosluanch format

1.1.2 Roslaunch XML

7. <param>：定义了要在参数服务器上设置的参数，相当于命令行中的 rosparam set

- 举例：

```
<param name="publish_frequency" type="double" value="10.0" />
```

配置 publish_frequency 参数的值为10

```
<param name="params_a" type="yaml" command="cat '$(find roslaunch)/test/params.yaml'" />
```

通过加载 YAML 文件配置参数

8. <rosparam>：允许使用 rosparam YAML 文件从 ROS 参数服务器加载和转储大量参数，其还可以用于删除参数。<rosparam> 标签可以放在 <node> 标签内，在这种情况下，参数被视为私有名称

1.1 Rosluanch format

1.1.2 Roslaunch XML： http://wiki.ros.org/roslaunch/XML/group

9. <group>：用于节点分组，其可以更轻松地将设置应用于一组节点。 它有一个 ns 属性，可将节点组推送到单独的命名空间中。还可以使用 <remap> 标签在整个组中应用重新映射设置

- **Attributes：**
  - **ns="namespace" (optional)**→Assign the group of nodes to the specified namespace. The namespace can be global or relative, though global namespaces are discouraged.
  - **clear_params="true|false" (optional)**→Delete all parameters in the group's namespace before launch. This feature is very dangerous and should be used with caution. ns must be specified.

1.1 Rosluanch format

1.1.2 Roslaunch XML

9. <group>：相当于顶级的 <launch> 标签，只是作为其中标签的容器。 意味着可以使用任何标签，就像通常在 <launch> 标签中使用一样

9. <group>：可包含 <node>、<param>、<remap>、<machine>、<rosparam>、<include>、<env>、<test>、<arg>标签

1.1 Rosluanch format

1.1.2 Roslaunch XML

10. <test>：在语法上类似于 <node> 标签。 它们都指定了要运行的 ROS 节点，但是 <test> 标记表明该节点实际上是要运行的测试节点

11. <arg>：用来在 launch 文件中定义参数，其允许通过指定命令行传递、通过 <include> 传递或为更高级别文件声明的值来创建更多可重用和可配置的启动文件。<arg>声明特定于单个启动文件，必须显式地将 arg 值传递给包含的文件

作业：

资料1：https://www.jianshu.com/p/13efab3f67e0

资料2：https://blog.csdn.net/github_35160620/article/details/52618271

记录学习笔记

1.1 Rosluanch format

1.1.3 Examples

- 使用 launch 文件，需要在功能包的工作空间中创建 launch 文件夹，launch 文件统一放在 launch 文件夹下

```
$ roscd  beginner_tutorials
```

```
$ mkdir launch
```

```
$ cd launch
```

```
$ gedit turtlemimic.launch
```

```
qlf@vmbox:~$ rosnode info /mimic
--------------------------------------------------------------------------------
Node [/mimic]
Publications:
 * /output/cmd_vel [geometry_msgs/Twist]
 * /rosout [rosgraph_msgs/Log]

Subscriptions:
 * /input/pose [unknown type]

Services:
 * /mimic/get_loggers
 * /mimic/set_logger_level


contacting node http://vmbox:34401/ ...
Pid: 3874
Connections:
 * topic: /rosout
    * to: /rosout
    * direction: outbound (34763 - 10.0.2.15:48972) [11]
    * transport: TCPROS
```

1.1 Rosluanch format

1.1.2 Roslaunch XML

- 举例:

```
<launch>
    <group ns="turtlesim1">
        <node pkg="turtlesim" name="sim" type="turtlesim_node"/>
    </group>
    <group ns="turtlesim2">
        <node pkg="turtlesim" name="sim" type="turtlesim_node"/>
    </group>
    <node pkg="turtlesim" name="mimic" type="mimic">
        <remap from="input" to="turtlesim1/turtle1"/>
        <remap from="output" to="turtlesim2/turtle1"/>
    </node>
</launch>
```

1.1 Rosluanch format

1.1.3 Examples

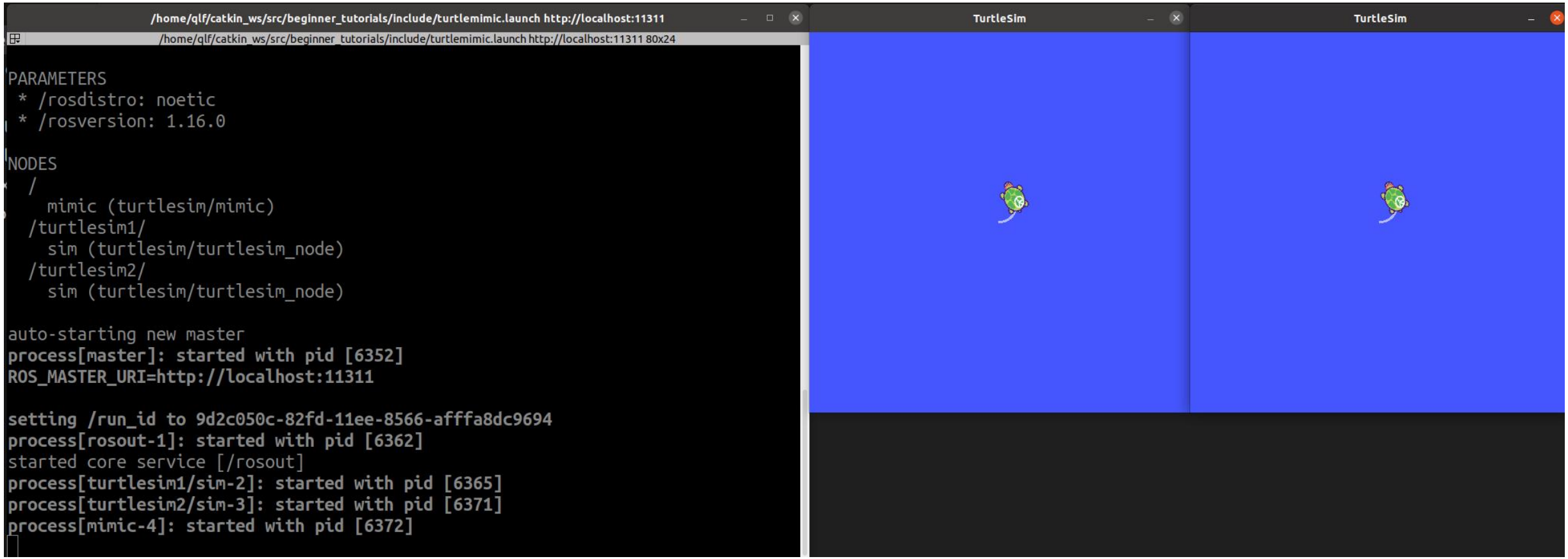
- 上述代码创建了2个 ros 节点并以不同命名空间标签区分，所以两个节点的名字都是一样的并不影响运行
- <launch> : launch 标签是其他标签的根标签，即其他标签的容器
- <group> : group 标签用于分组，” ns” 指的就是：namespace(命名空间)
- <node> : node 标签用于在终端中使用 rosrun 命令来启动一个节点
  - <node>标签包含三个属性：pkg，type 和 name，pkg 和 type 分别表示：功能包名称 和 节点名称，name 为用户为节点起的别名
- 启动launch文件：直接运行 launch 脚本文件，不用事先启动 roscore

```
$ roslaunch beginner_tutorials turtlemimic.launch
```

```
$ rostopic pub /turtlesim1/turtle1/cmd_vel geometry_msgs/Twist [1,0,0] [0,0,2]
```

1.1 Rosluanch format

1.1.3 Examples



turtlesim2 命名空间中的 turtle1 海龟的所有动作都模仿 turtlesim1命名空间中的 turtle1 海龟

1.1 Rosluanch format

1.1.3 Examples

键盘控制海归的launch文件：

```
$ gedit turtle_telecontrol.launch
```

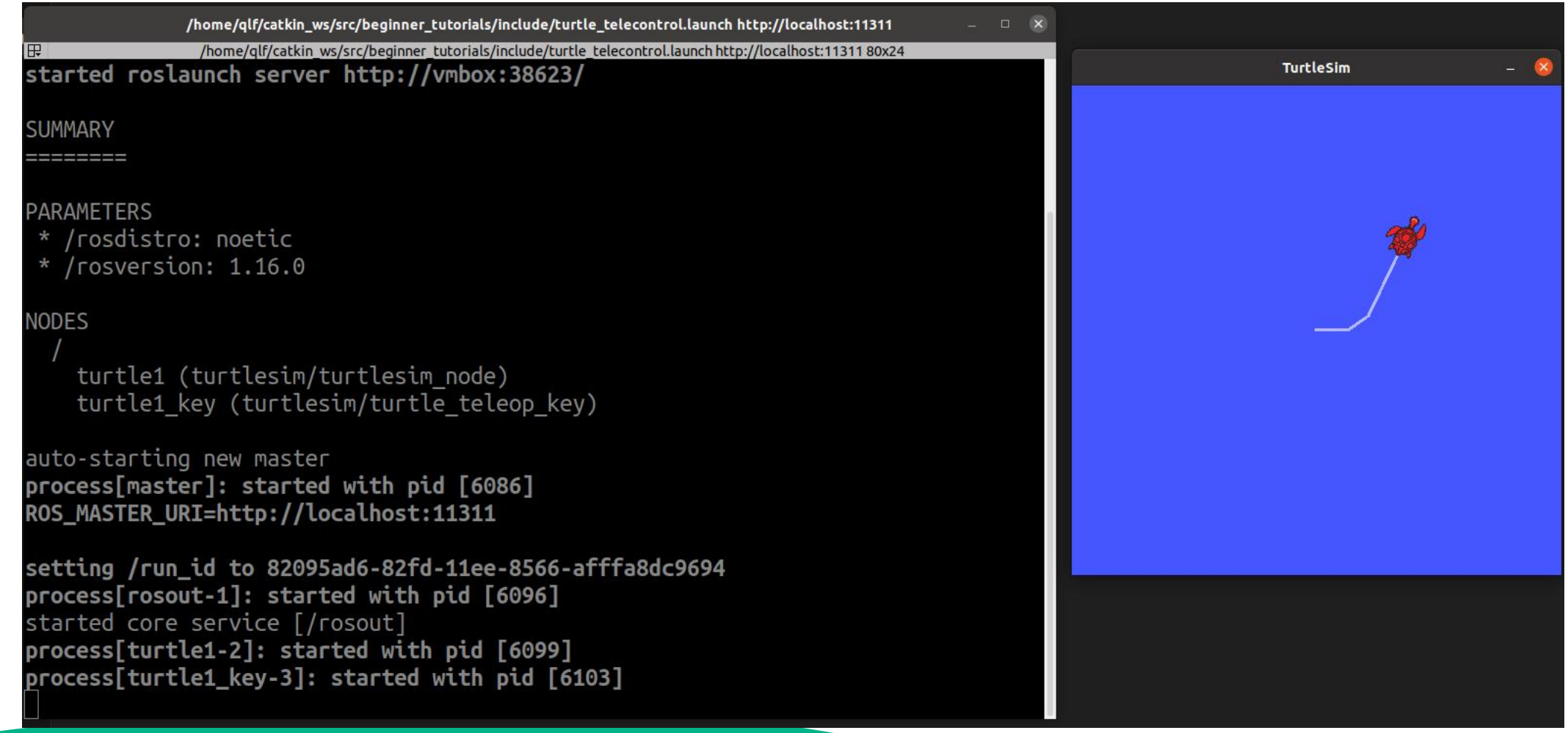
```
<launch>
    <node pkg="turtlesim" name="turtle1" type="turtlesim_node"/>
    <node pkg="turtlesim" name="turtle1_key" type="turtle_teleop_key"/>
</launch>
```

```
$ roslaunch beginner_tutorials turtle_telocontrol.launch
```

运行结果见下图：

1.1 Rosluanch format

1.1.3 Examples



Summary

1. Roslaunch
   a) <launch>
   b) <node>
   c) <machine>
   d) <include>
   e) <remap>
   f) <env>
   g) <param>
   h) <rosparam>
   i) <group>
   j) <test>
   k) <arg>

Overview lesson 9-1

1. Communication Model
2. C++ example-基于ROS编程
3. ~
4. ~

Objectives

1、掌握发布订阅的通讯模型

2、掌握 C++ 编写 ROS 程序的11个步骤

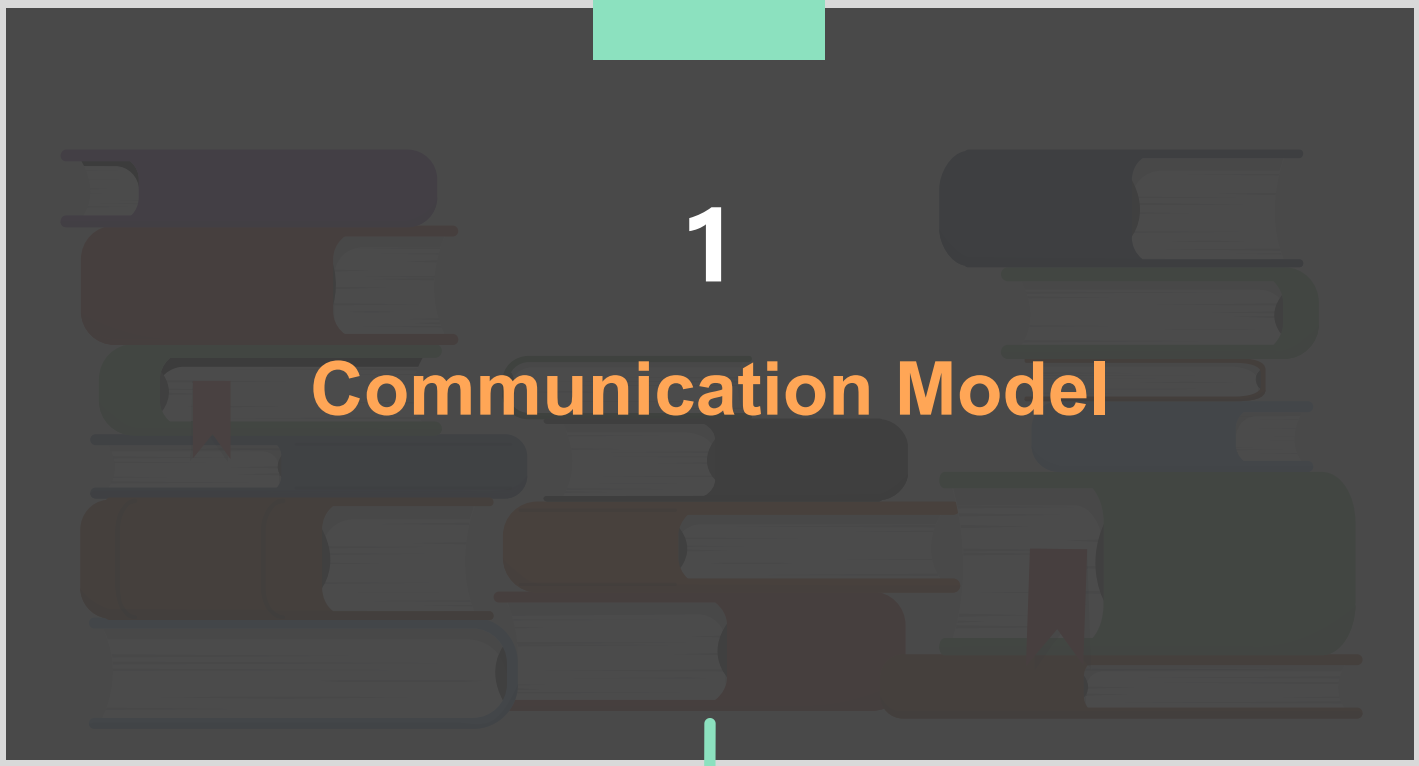


Pre-assessment

1、是否了解发布订阅模型?

2、是否用过 C++ 编写 ROS 程序?

3、是否了解 C++ 编写 ROS 程序的基本步骤?

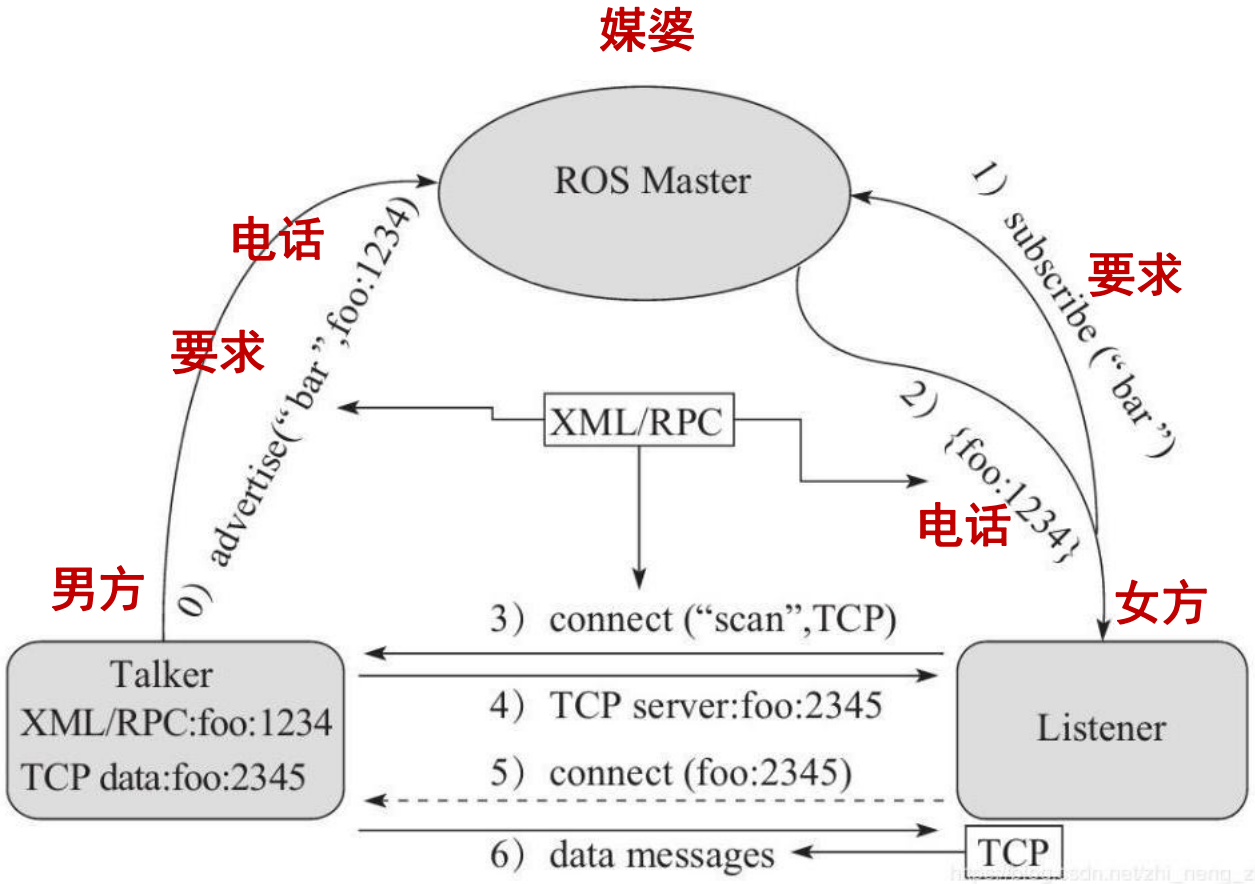


1

Communication Model

1.1 Topic model

1.1.1 Publish and subscribe



- 发布和订阅流程由以下步骤实现:

0.Talker 注册

Talker 启动后，会通过 RPC 在 ROS Master 中注册自身信息，其中包含所发布消息的话题名称。ROS Master 会将节点的注册信息加入到注册表中

1.Listener 注册

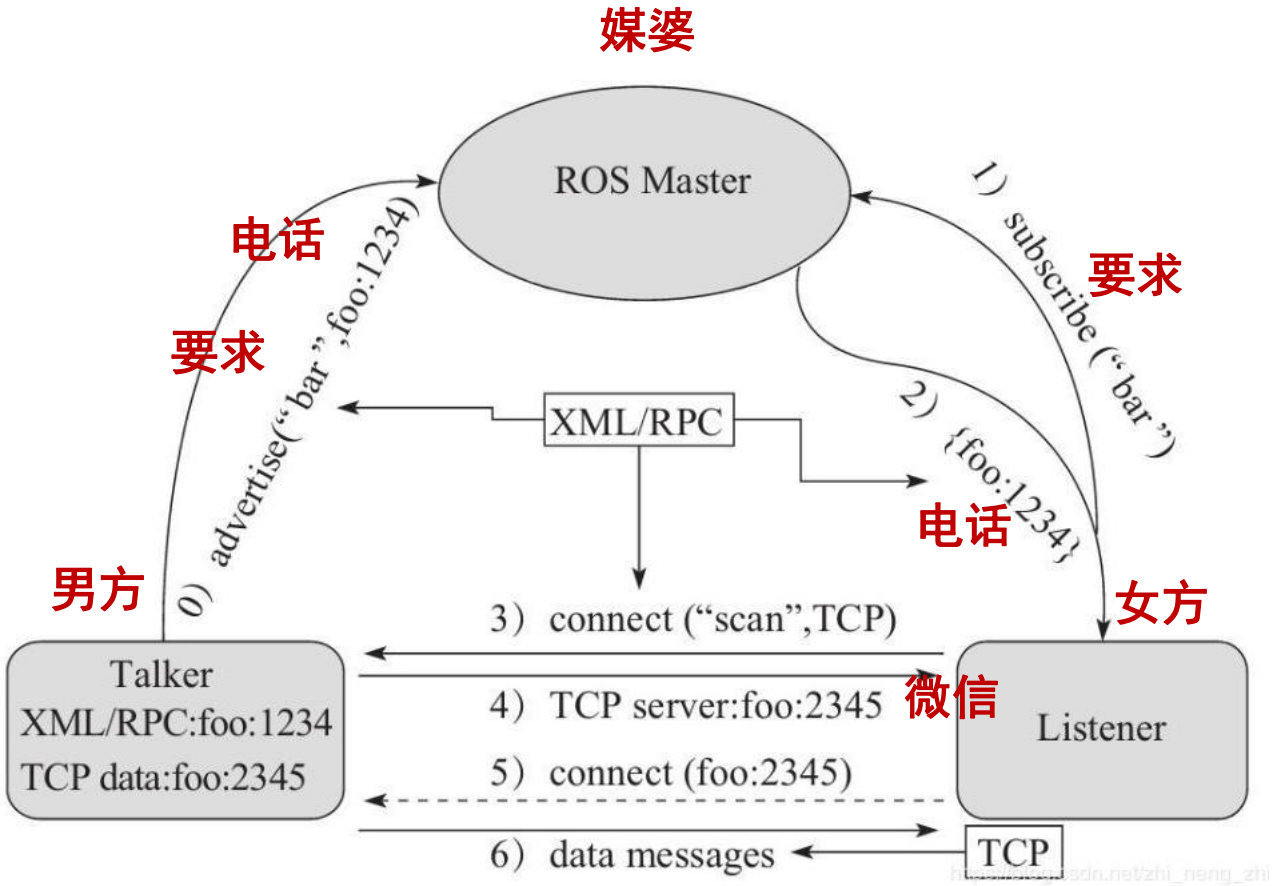
Listener 启动后，也会通过 RPC 在 ROS Master 中注册自身信息，包含需要订阅消息的话题名称。ROS Master 会将节点的注册信息加入到注册表中

2.ROS Master 实现信息匹配

ROS Master 会根据注册表中的信息匹配 Talker 和 Listener，并通过 RPC 向 Listener 发送 Talker 的 RPC 地址信息

1.1 Topic model

1.1.1 Publish and subscribe



3.Listener 向 Talker 发送请求

Listener 根据接收到的 RPC 地址，通过 RPC 向 Talker 发送连接请求，传输订阅的话题名称、消息类型以及通信协议(TCP/UDP)

4.Talker 确认请求

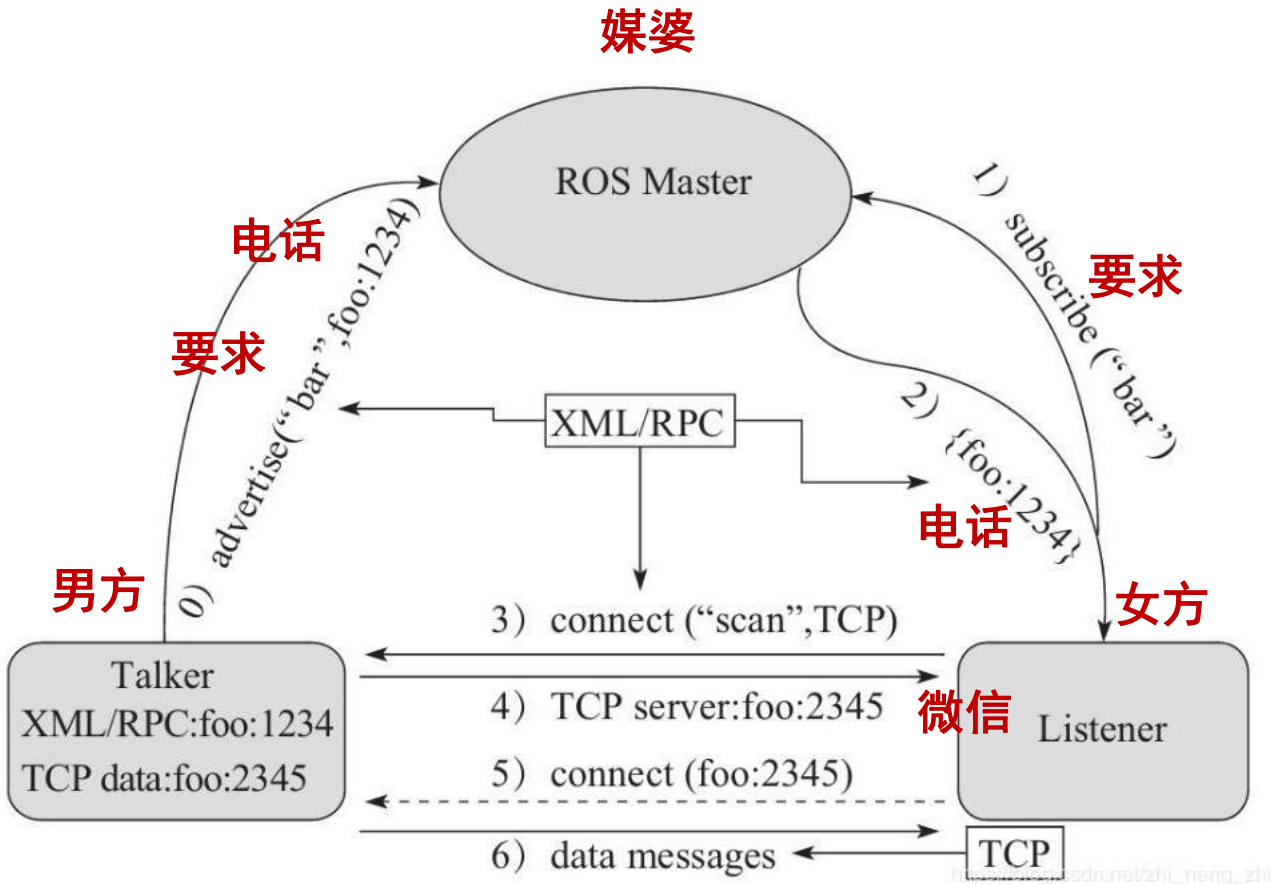
Talker 接收到 Listener 的请求后，通过 RPC 向 Listener 确认连接信息，并发送自身的 TCP 地址信息

5.Listener 与 Talker 建立连接

Listener 根据步骤4 返回的消息使用 TCP 与 Talker 建立网络连接

1.1 Topic model

1.1.1 Publish and subscribe



6.Talker向Listener发送消息

连接建立后，Talker 开始向 Listener 发布消息

注意1：上述实现流程中，前五步使用的 RPC 协议，最后两步使用的是 TCP 协议

注意2：Talker 与 Listener 的启动无先后顺序要求

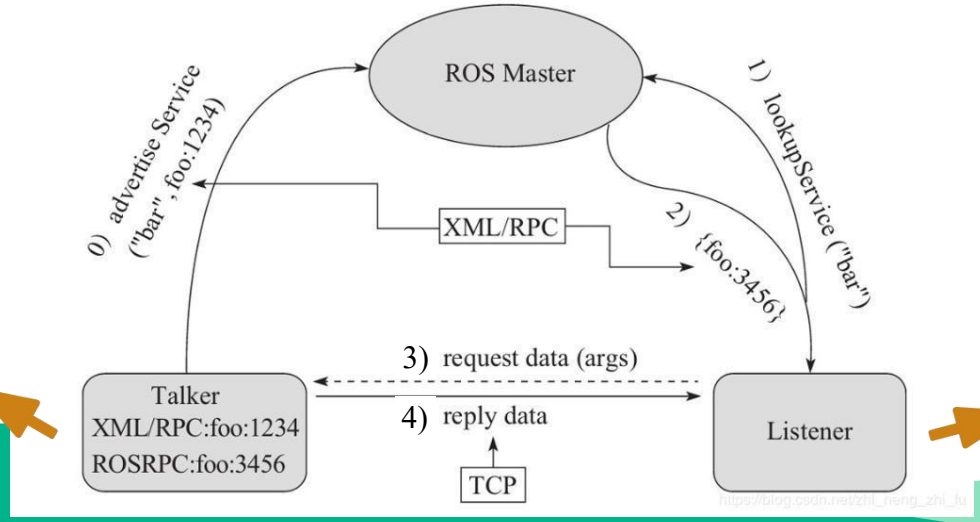
注意3：Talker 与 Listener 都可以有多个

注意4：Talker 与 Listener 连接建立后，不再需要 ROS Master。即便关闭 ROS Master，Talker 与 Listern 照常通信

1.2 Service model

1.2.1 Request and response

- 服务通信较之于话题通信更简单些，理论模型如下图所示，该模型中涉及到三个角色：
  1. ROS master(管理者)
  2. Server(服务端)
  3. Client(客户端)
- ROS Master 负责保管 Server 和 Client 注册的信息，并匹配话题相同的 Server 与 Client ，帮助 Server 与 Client 建立连接，连接建立后，Client 发送请求信息，Server 返回响应信息

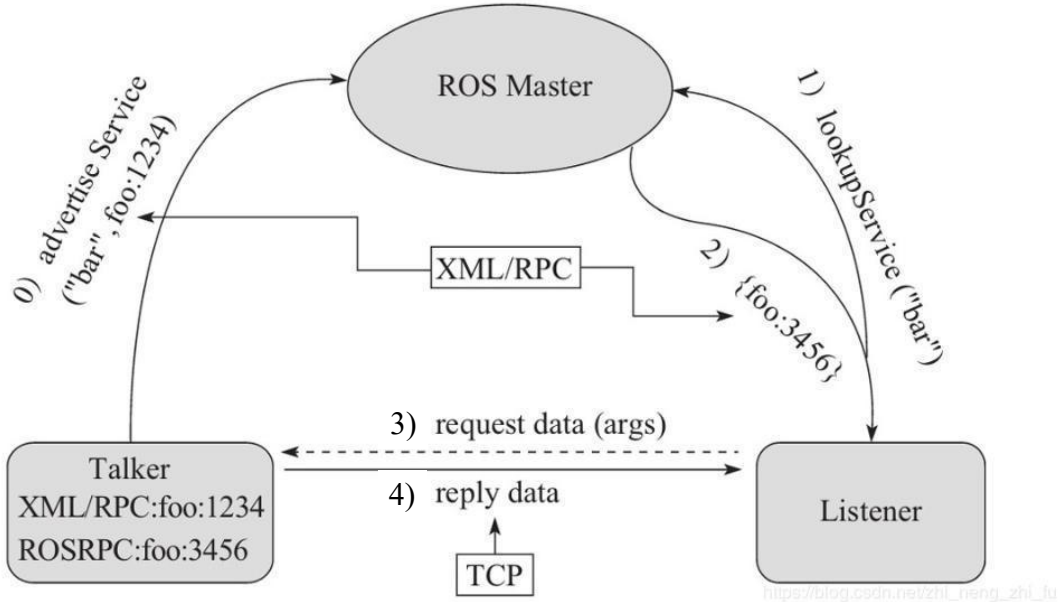


Server(服务端)

Client(客户端)

1.2 Service model

1.2.1 Request and response



注意:

1.客户端请求被处理时，需要保证服务器已经启动

2.服务端和客户端都可以存在多个

0.Server注册

Server 启动后，会通过 RPC 在 ROS Master 中注册自身信息，其中包含提供的服务的名称。ROS Master 会将节点的注册信息加入到注册表中

1.Client注册

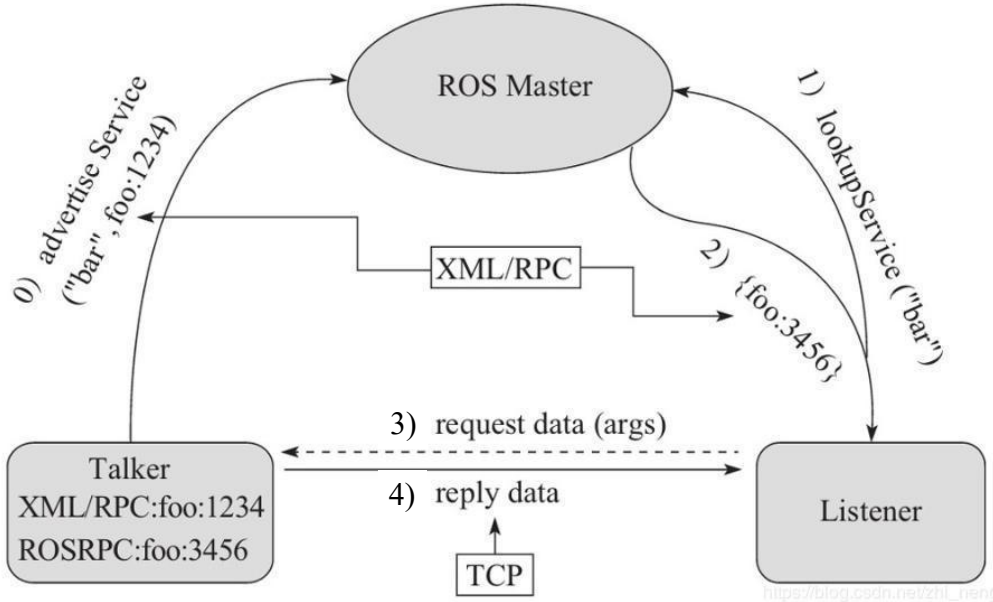
Client 启动后，也会通过 RPC 在 ROS Master 中注册自身信息，包含需要请求的服务的名称。ROS Master 会将节点的注册信息加入到注册表中

2.ROS Master实现信息匹配

ROS Master 会根据注册表中的信息匹配 Server 和 Client，并通过 RPC 向 Client 发送 Server 的 TCP 地址信息

1.2 Service model

1.2.1 Request and response



3.Client发送请求

Client 根据步骤2 响应的信息，使用 TCP 与 Server 建立网络连接，并发送请求数据

4.Server发送响应

Server 接收、解析请求的数据，并产生响应结果返回给 Client

注意:

1.客户端请求被处理时，需要保证服务器已经启动

2.服务端和客户端都可以存在多个

2

C++ example-基于 ROS 编程

2.1 ROS Programming

2.1.1 Overview

- ROS 编程是指通过使用 ROS 提供的一些内置函数来编写 ROS 应用程序
- ROS编译系统
  - ROS 有一套自己的用于编译 ROS 程序包的编译系统，该系统名称是 catkin (http://wiki.ros.org/catkin)
  - catkin 是由 CMake 编译系统和 Python 脚本定制的一套特殊的编译系统，以解决 ROS 程序包复杂的编译过程

2.1 ROS Programming

2.1.1 Overview

- ROS catkin 工作空间
  - src：存储新创建的功能包或者从远程代码库复制下来的功能。ROS功能包只有在 src 文件夹中才可以被创建、编译和执行
  - build：执行 catkin_make 命令时，catkin 工具会在 build 文件夹中创建一些中间编译文件和中间缓冲文件。这些缓冲文件可以避免在执行 catkin_make 命令时重新编译所有功能包
  - devel：执行 catkin_make 命令时，如果编译成功，创建的可执行文件存储在 devel 文件夹中，该文件夹中还有一些脚本，可以将该工作空间添加到 ROS 的系统环境变量中

2.1.2 使用ROS客户端库

- ROS客户端库是一系列实现了 ROS 概念所对应功能的代码
- 在编写 ROS 程序时，在代码中包含这些库函数便可以实现一个 ROS 节点
- 以下是主要的 ROS 客户端库：
  - roscpp：C++ 的 ROS 客户端库，被广泛用于 ROS 应用开发
  - rospy：Python 的 ROS 客户端库，其优点是语法简单，节省开发时间，但执行效率比 roscpp 低
  - roslisp：Lisp 语言的 ROS 客户端库
  - 其他客户端库：rosjava、rosnodejs、roslua

2.1.2 使用ROS客户端库

- 使用 roscpp 客户端库编写 ROS 节点：11个步骤(考试编程题)

1. 头文件和 ROS 模块

- 当用 C++ 编写代码时，第一部分要包含头文件
- 为了创建 ROS C++节点，需要包含以下头文件：

```
#include “ros/ros.h”
```

说明：ros.h 文件中包含了实现 ROS 功能所需的所有头文件

- 若要在节点中使用某种特定的消息类型，需要包含消息头文件
  - ROS 中提供一些内置的消息类型，用户也可以自定义消息类型
  - ROS 中的 std_msgs 内置消息功能包中包含：整型、浮点型、字符串型等标准数据类型的消息定义

2.1.2 使用ROS客户端库

1. 头文件和 ROS 模块

举例：要在代码中包含字符串消息，需要使用以下代码：

```
#include “std_msgs/Strings.h”
```

举例：若要使用自定义消息类型，需要使用以下代码：

```
#include “msg_pkg_name/message_name.h”
```

- std_msgs 功能包中的完整消息类型参见以下网址：
  http://wiki.ros.org/std_msgs

2.1.2 使用ROS客户端库

2. 初始化创建 ROS 节点

- 在开始编写任何 ROS 节点前，都需要首先初始化创建节点，此为必须要做的一步
- 在 C++ 中，使用以下代码初始化创建节点：

```
int main(int argc, char **argv)
  {
  ros::init(argc, argv, “name_of_node”)
  }
```

- 在 main() 函数中，必须包含 ros::init() 函数，该函数可以初始化创建 ROS 节点
- 可以向 init() 函数传递 argc, argv 命令行参数和节点名称

2.1.2 使用ROS客户端库

2. 初始化创建 ROS 节点

- int main(int argc, char **argv)（或者int main(int argc, char *argv[])）中：
  - 参数 argc 表示程序运行时发送给 main 函数的命令行参数的个数（包括可执行程序以及传参）
  - argv[]是字符指针数组，它的每个元素都是字符指针，指向命令行中每个参数的第一个字符
- 举例：假设已有一个myros的可执行文件，该文件支持传递参数

```
$ ./myros
```

此例中 argc=1， argv[0]=”myros”

```
$ ./myros myarg1 myarg2
```

此例中 argc=3， argv[0]=”myros”， argv[1]=”myarg1”， argv[2]=”myarg2”

2.1.2 使用ROS客户端库

3. 在 ROS 节点中打印消息

- ROS 提供了记录日志信息的应用程序接口，这些信息是一些可读的字符串，表示了节点的运行状态
- 在 C++ 中，以下函数可以用于记录节点信息：

1. ROS_INFO(string_msg, args)：记录节点输出的基本消息
2. ROS_WARN(string_msg, args)：记录节点输出的警告消息
3. ROS_DEBUG(string_msg, args)：记录节点输出的调试消息
4. ROS_ERROR(string_msg, args)：记录节点输出的错误消息
5. ROS_FATAL(string_msg, args)：记录节点输出的致命消息

2.1.2 使用ROS客户端库

4. 创建节点句柄

- 在初始化创建节点之后，需创建一个节点句柄实例，用于启动 ROS 节点和诸如发布、订阅话题等操作
- 在 C++ 中，使用以下命令创建节点句柄实例：

```
ros::NodeHandle nh
```

- 节点中的其他操作都可以通过使用 nh 实例来访问

2.1.2 使用ROS客户端库

5. 创建 ROS 消息定义

- 在发布话题前，需要创建一个 ROS 消息定义
- 在 C++ 中，使用以下代码创建 ROS 消息实例

```
std_msgs::String msg
```

- 创建了 ROS 消息实例后，可以使用以下命令向实例中添加数据

```
msg.data=”string data”
```

2.1.2 使用ROS客户端库

6. 在 ROS 节点中发布话题

- 在 C++ 中，使用以下语法在 ROS 节点中发布话题：

```
ros::Publisher publisher_object = node_handle.advertise<ROS message type>( “topic name” ,1000)
```

- 创建了一个待发布的话题后，publish() 命令能够通过话题来发送 ROS 消息：

```
publisher_object.publish(message)
```

2.1.2 使用ROS客户端库

6. 在 ROS 节点中发布话题

- 举例：

```
ros::Publisher chatter_pub = nh.advertise<std_msgs::String>(“chatter”,1000)

chatter_pub.publish(msg)
```

- 在上述例子中，chatter_pub 是 ROS 发布者的实例，chatter_pub 会发布消息类型为 std_msgs/String 的话题，话题的名称为 chatter，队列大小为 1000

2.1.2 使用ROS客户端库

7. 在 ROS 节点中订阅话题

- 发布话题时，需创建消息类型并且通过话题来发布该消息
- 订阅话题时，消息同样从话题被接收
- 在 C++ 中，使用以下命令订阅话题

```
ros::Subscriber subscriber_obj = node_handle.subscribe( “topic_name” ,1000,callback function)
```

- 在订阅话题时，不需要规定话题的消息类型，但必须给定话题名称和对应的回调函数
- 回调函数是用户自定义函数，用于响应话题。一旦从话题中接收到 ROS 消息，回调函数就会被执行
- 在回调函数中，可以操作 ROS 消息
- 举例：

```
ros::Subscriber sub = nh.subscribe( “chatter” ,1000,chatter_callback)
```

2.1.2 使用ROS客户端库

8. 在 ROS 节点中编写回调函数

- 当订阅 ROS 话题并且有消息到达话题时，回调函数被触发
- 在C++中，使用以下命令定义回调函数

```
void callback_name(const ros_message_const_pointer& pointer)
{
    //获取数据
    pointer-> data;
}
```

2.1.2 使用ROS客户端库

8. 在 ROS 节点中编写回调函数

- 举例：以下代码展示了如何在回调函数中处理 ROS 字符串消息并显示数据

```
void chatterCallback(const std_msgs::String::ConstPtr& msg)
{
    //获取数据
    ROS_INFO(“I heard: [%s]”, msg->data.c_str());
}
```

2.1.2 使用ROS客户端库

9. ROS 节点中的 ROS spin() 函数

- 在订阅或者发布函数之后，还需要调用一个函数来处理来自其他节点的订阅请求或发布请求
- 在 C++ 节点中，发布话题后需要调用 ros::spinOnce() 函数
- 订阅话题中，需要调用 ros::spin() 函数

2.1.2 使用ROS客户端库

10. ROS 节点中的 ROS sleep函数

- 若想为节点中的循环设置一个恒定的速率，可以使用 ros::Rate 功能
- 首先创建 ros::Rate 实例，并在实例中指定速率
- 创建实例后，调用 sleep() 函数，使得循环保持在指定的速率
- 举例：以下代码设置节点的循环速率为10次/s

```
ros::Rate r(10);  //10 Hz
r.sleep();
```

2.1.2 使用ROS客户端库

11. 设置和获取 ROS 参数

- 在 C++ 中，使用以下方法来获取参数：
  - 首先声明一个变量
  - 然后用节点句柄 node_handle 中的 getParam() 函数来获取所需要的参数

```
std::string global_name;
if(nh.getParam( “/global_name” ,global_name))
{
……
}
```

2.1.2 使用ROS客户端库

11. 设置和获取 ROS 参数

- 在 C++ 中，使用以下方法来设置参数：
  - 在 setParam() 函数中规定参数的名称和数值
- 举例：

```
nh.setParam(“/global_param”，5)
```

2.1 ROS Programming

2.1.2 使用ROS客户端库

作业：

1、熟练使用 Hello_world 中代码(编辑+调试)

2、更改代码中的输出参数，实现其他效果

```
robot@robot-pc:~$ rosrun hello_world talker
[ INFO] [1518291487.561201049]: hello world 0
[ INFO] [1518291487.662599248]: hello world 1
[ INFO] [1518291487.761577638]: hello world 2
[ INFO] [1518291487.862540060]: hello world 3
[ INFO] [1518291487.961661065]: hello world 4
[ INFO] [1518291488.061997374]: hello world 5
[ INFO] [1518291488.161413663]: hello world 6
[ INFO] [1518291488.261661379]: hello world 7
[ INFO] [1518291488.361571420]: hello world 8
[ INFO] [1518291488.461965544]: hello world 9
[ INFO] [1518291488.561633123]: hello world 10
[ INFO] [1518291488.661921779]: hello world 11
[ INFO] [1518291488.761608416]: hello world 12
```

```
robot@robot-pc:~$ rosrun hello_world listener
[ INFO] [1518291487.863502718]: I heard: [hello world 3]
[ INFO] [1518291487.961958201]: I heard: [hello world 4]
[ INFO] [1518291488.062417338]: I heard: [hello world 5]
[ INFO] [1518291488.161754224]: I heard: [hello world 6]
[ INFO] [1518291488.262099340]: I heard: [hello world 7]
[ INFO] [1518291488.361963797]: I heard: [hello world 8]
[ INFO] [1518291488.462384322]: I heard: [hello world 9]
[ INFO] [1518291488.561988817]: I heard: [hello world 10]
[ INFO] [1518291488.662334647]: I heard: [hello world 11]
[ INFO] [1518291488.761864631]: I heard: [hello world 12]
[ INFO] [1518291488.862494290]: I heard: [hello world 13]
[ INFO] [1518291488.961657606]: I heard: [hello world 14]
[ INFO] [1518291489.061816800]: I heard: [hello world 15]
[ INFO] [1518291489.161825396]: I heard: [hello world 16]
[ INFO] [1518291489.262011120]: I heard: [hello world 17]
[ INFO] [1518291489.361945019]: I heard: [hello world 18]
[ INFO] [1518291489.462888386]: I heard: [hello world 19]
[ INFO] [1518291489.562698356]: I heard: [hello world 20]
[ INFO] [1518291489.662093096]: I heard: [hello world 21]
[ INFO] [1518291489.761660963]: I heard: [hello world 22]
[ INFO] [1518291489.862041715]: I heard: [hello world 23]
[ INFO] [1518291489.961789634]: I heard: [hello world 24]
[ INFO] [1518291490.061721203]: I heard: [hello world 25]
[ INFO] [1518291490.161883229]: I heard: [hello world 26]
[ INFO] [1518291490.261879144]: I heard: [hello world 27]
[ INFO] [1518291490.361822851]: I heard: [hello world 28]
[ INFO] [1518291490.461525109]: I heard: [hello world 29]
[ INFO] [1518291490.561590742]: I heard: [hello world 30]
```

ROS Summary

1. C++ example

    1. 头文件和 ROS 模块
    2. 初始化 ROS 节点
    3. 在 ROS 节点中打印消息
    4. 创建节点句柄
    5. 创建 ROS 消息定义
    6. 在 ROS 节点中发布话题
    7. 在 ROS 节点中订阅话题
    8. 在 ROS 节点中编写回调函数
    9. ROS 节点中的 ROS spin() 函数
    10. ROS节点中的 sleep 函数
    11. 设置和获取 ROS 参数

Overview lesson 9-2

1 VSCode Installation and Application

2 Topic Model C++ Code

3 ~

4 ~

Objectives

1、了解 VSCode 的使用和配置方法

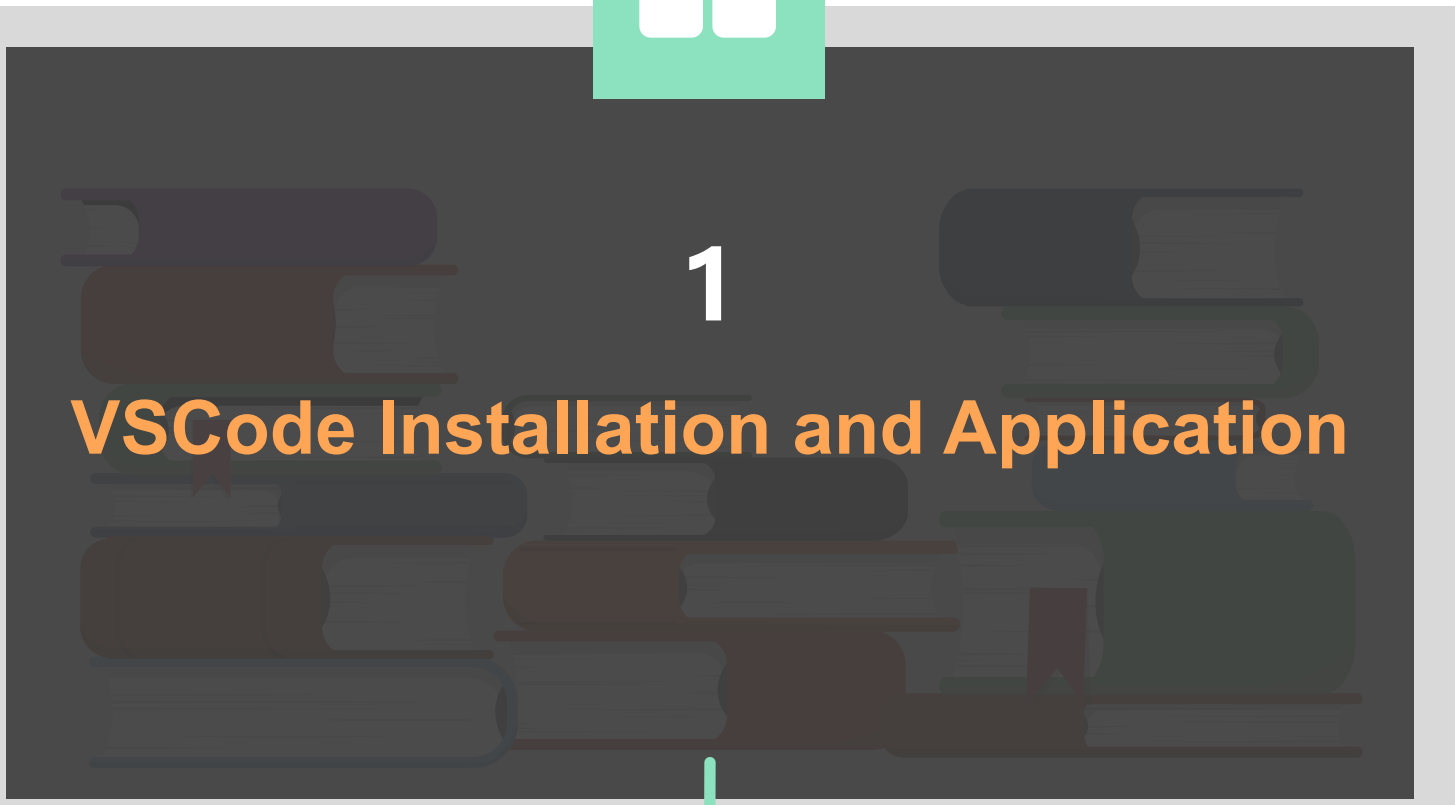
2、掌握 C++ 编写 ROS 发布和订阅节点程序



Pre-assessment

1、是否用过 VSCode?

2、是否成功运行 talker 和 listener?



1

VSCode Installation and Application

1.1 VSCode Introduction

1.1.1 Overview

- VSCode 全称 Visual Studio Code，是微软出的一款轻量级代码编辑器，免费、开源而且功能强大
- VSCode 支持几乎所有主流的程序语言的语法高亮、智能代码补全、自定义热键、括号匹配、代码片段、代码对比 Diff、GIT 等特性，支持插件扩展，并针对网页开发和云端应用开发做了优化
- 软件跨平台支持 Win、Mac 以及 Linux
- VSCode 下载：
  - VSCode 下载：https://code.visualstudio.com/docs?start=true
  - 历史版本下载链接：https://code.visualstudio.com/updates

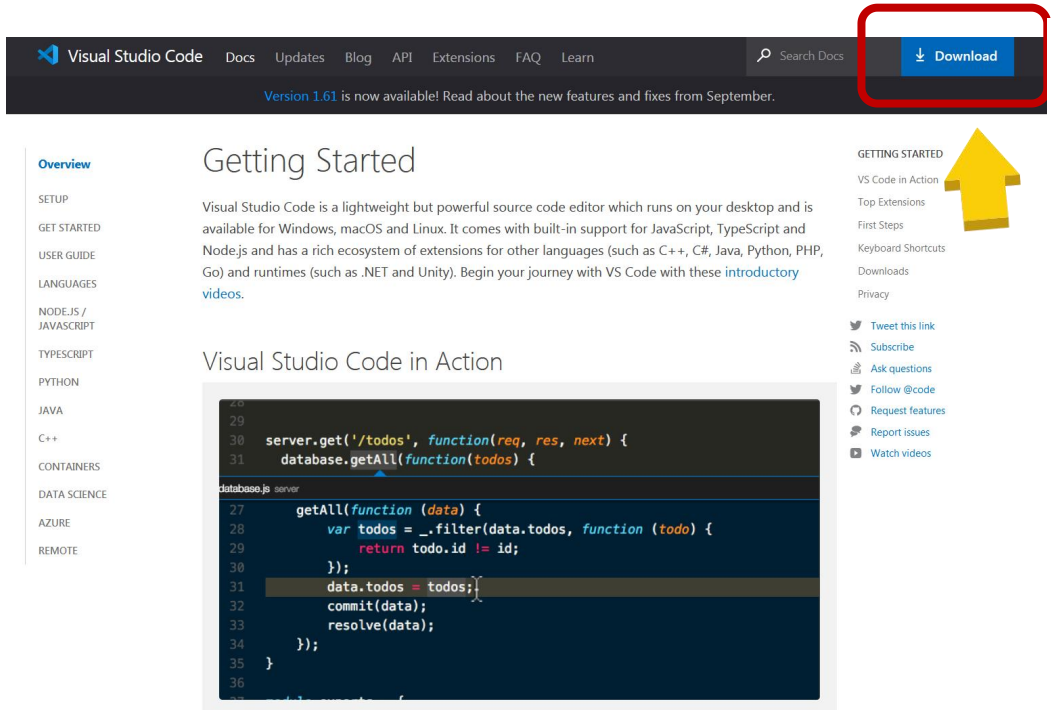
参考视频: https://www.bilibili.com/video/BV1Ci4y1L7ZZ?p=23
【奥特学园】ROS机器人入门课程《ROS理论与实践》零基础教程：23-32小节

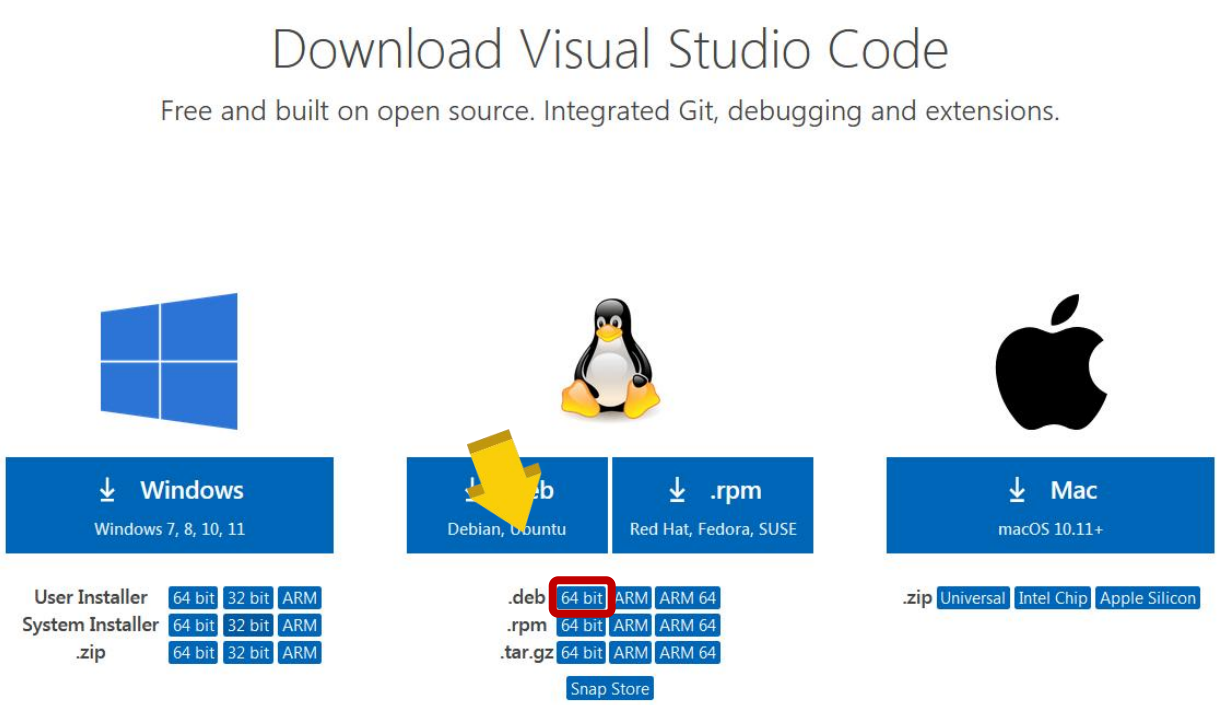
参考讲义：http://www.autolabor.com.cn/book/ROSTutorials/

1.2 VSCode Installation

1.2.1 Download

https://code.visualstudio.com/docs?start=true





1.2 VSCode Installation

1.2.2 Installation

- 安装：
  - 方式1：双击安装即可(或右击选择安装)
  - 方式2：

```
$ sudo dpkg -i xxxx.deb
```

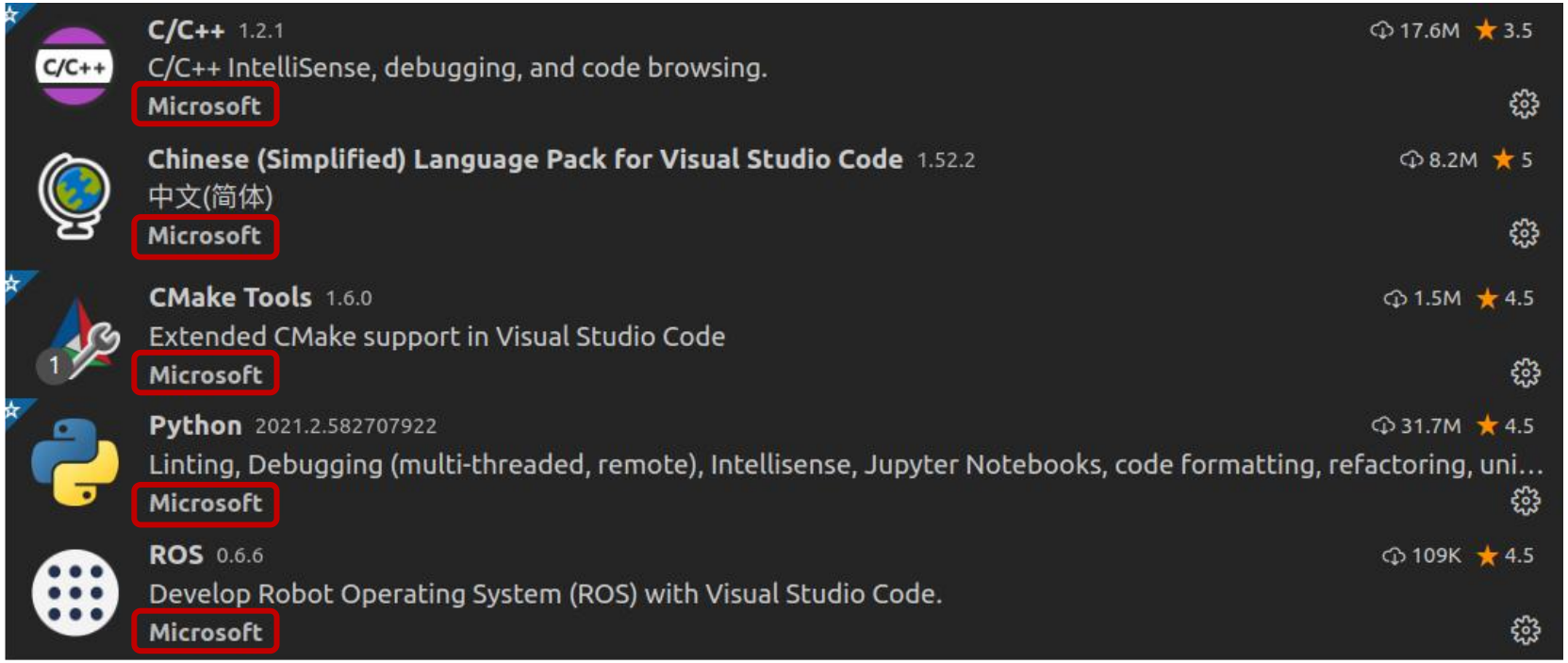
- 卸载

```
sudo dpkg --purge  code
```

1.3 VSCode Application

1.3.1 Plugins

1. C/C++
2. CMake Tools
3. Python
4. ROS
5. Chinese



1.3 VSCode Application

1.3.2 Setups

1. 创建 ROS 工作空间：

```
$ mkdir -p pub_sub_ws/src
```

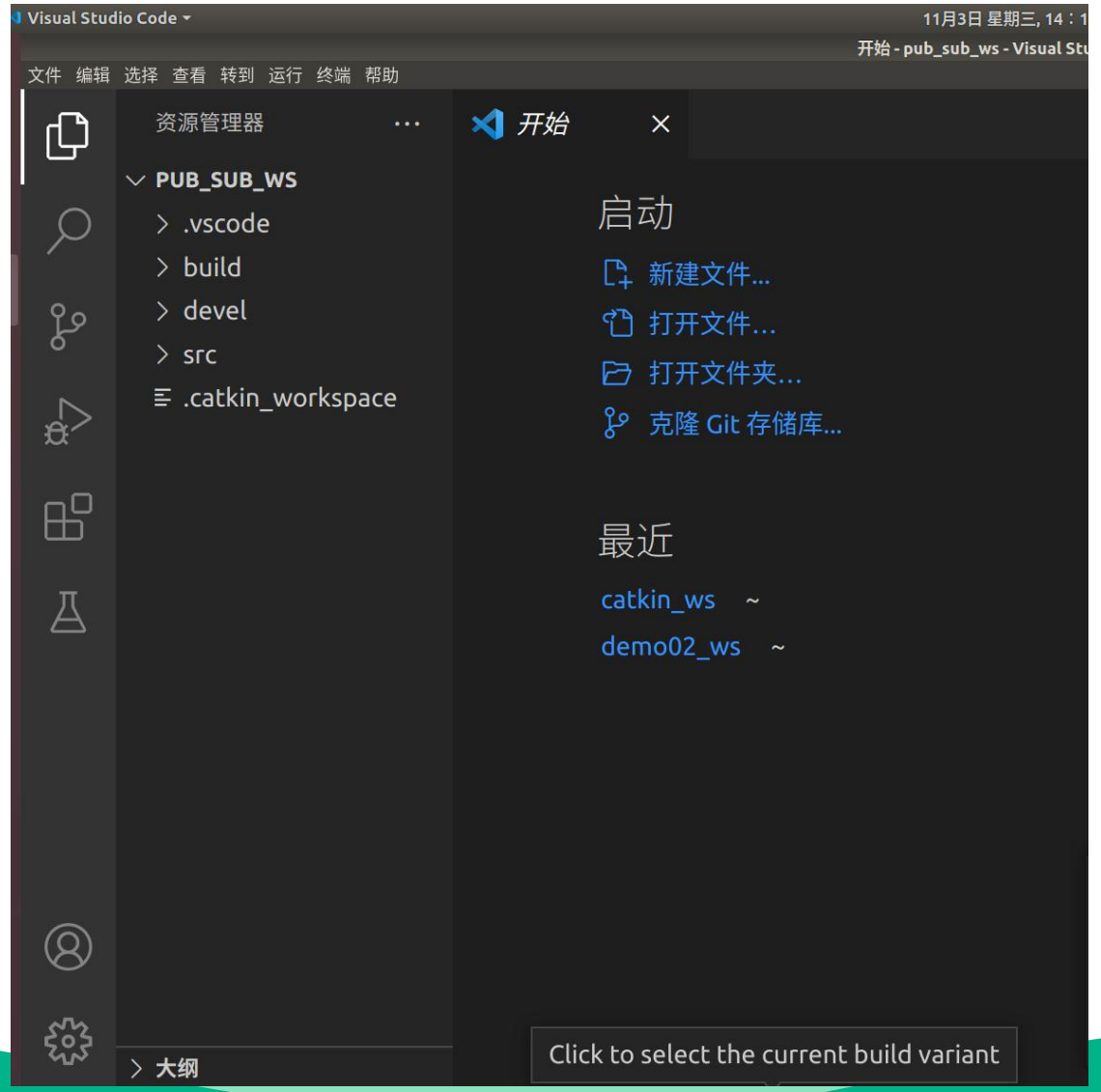
```
$ cd pub_sub_ws
```

```
$ catkin_make
```

2. 启动 VSCode：进入pub_sub_ws 启动 VSCode

```
$ cd pub_sub_ws
```

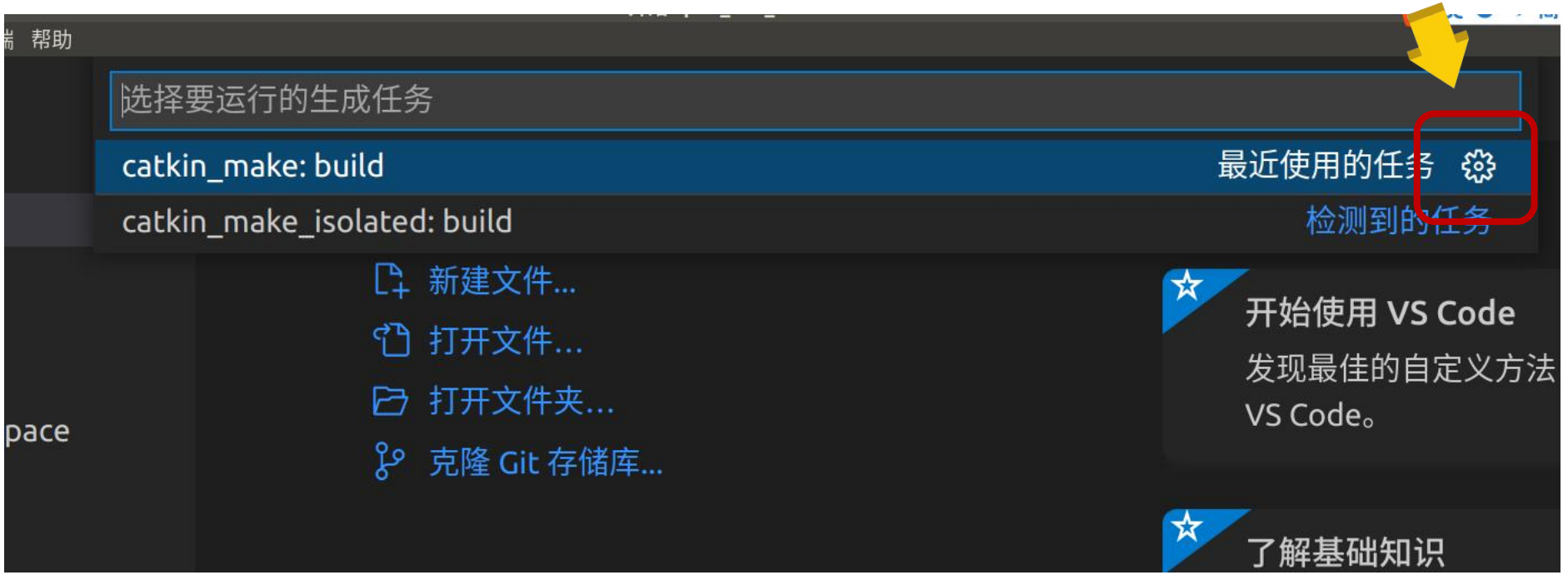
```
$ code .
```



1.3 VSCode Application

1.3.2 Setups

3. 配置 ROS 编译：ctrl + shift + b



1.3 VSCode Application

1.3.2 Setups

3. 配置ROS编译：修改 .vscode/tasks.json 文件，将默认配置替换为以下内容：

```
{
    "version": "2.0.0",
    "tasks": [
        {
            "label": "catkin_make:debug",
            "type": "shell",
            "command": "catkin_make",
            "args": [],
            "group": {"kind":"build","isDefault":true},
            "presentation": {
                "reveal": "always"
            },
            "problemMatcher": "$msCompile"
        }
    ]
}
```

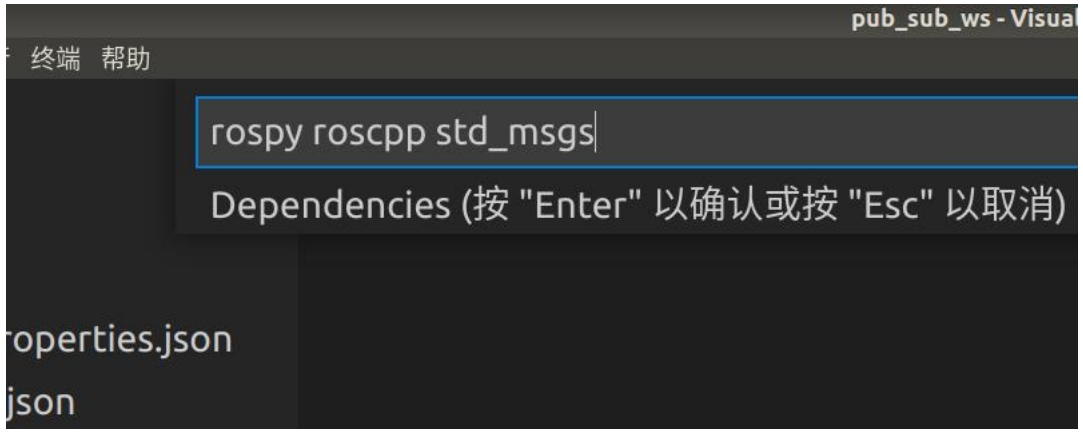
1.3 VSCode Application

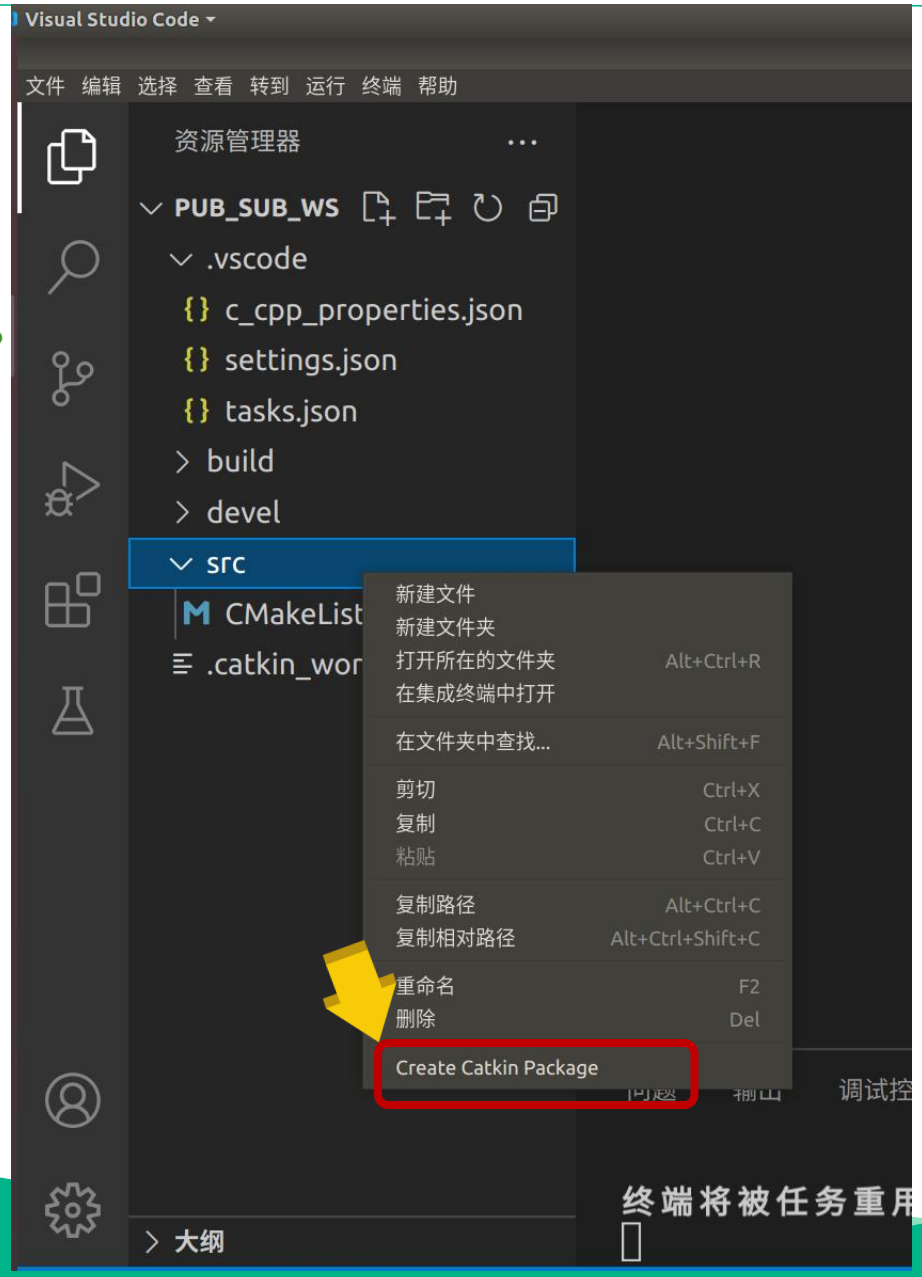
1.3.2 Setups

4. 创建 ROS 功能包

1) 选定 src 右击 ---> create catkin package

2) 设置包名 添加依赖





5. 执行编译：ctrl + shift + b



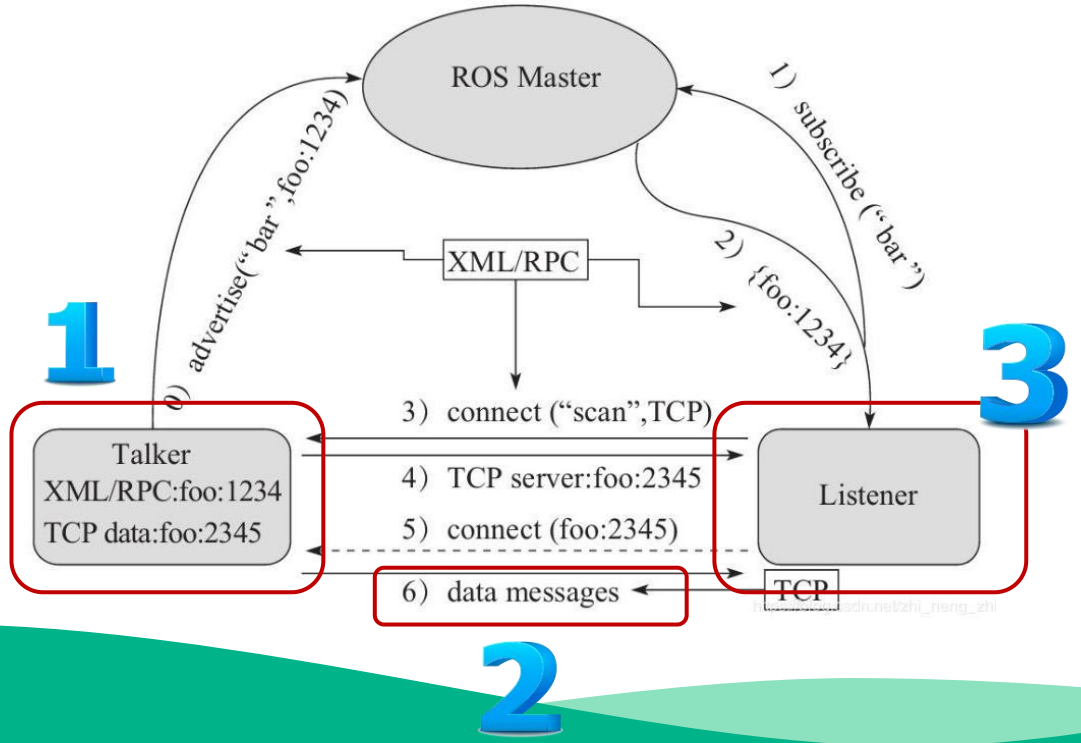
2

Topic Model C++ Code

2.1 Publish node

2.1.1 C++ code

- **需求**：编写发布订阅实现，要求发布方以5HZ(每秒5次)的频率发布文本消息Hello 你好!+数字(数字按频率递增，例如Hello 你好!0、Hello 你好!1、...)，订阅方订阅消息并将消息内容打印输出
- **分析**：在模型实现中，ROS master 不需要实现，连接已经被封装了，需要关注的关键点有三个：
  - 发布方
  - 订阅方
  - 数据(此处为普通文本)
- **整体流程**：
  1. 编写发布方实现
  2. 编写订阅方实现
  3. 编辑配置文件
  4. 编译并执行



2.1 Publish node

2.1.1 C++ code

- **代码实现流程：**

1. 包含头文件
2. 初始化 ROS 节点：命名(唯一)
3. 实例化 ROS 句柄
4. 实例化 发布者 对象
5. 组织被发布的数据，并编写逻辑发布数据

注：附注解的代码在 pub_c.cpp 文件中

2.1 Publish node

2.1.1 C++ code

- 在 pub_sub_test 功能包的 src 文件夹中新建 pub_c.cpp 文件，输入以下代码：

```
#include "ros/ros.h"
#include "std_msgs/String.h"
#include <sstream>

int main(int argc, char  *argv[])
{
    setlocale(LC_ALL,"");
    ros::init(argc,argv,"talker");
    ros::NodeHandle nh;
    ros::Publisher pub = nh.advertise<std_msgs::String>("chatter",10);
    std_msgs::String msg;
    int count = 0;
    ros::Rate rate(1);
```

2.1 Publish node

2.1.1 C++ code

- 在 pub_sub_test 功能包的 src 文件夹中新建 pub_c.cpp 文件，输入以下代码：

```
    while (ros::ok())
    {
        std::stringstream ss;
        ss << "Hello 你好!  Num: " << count;
        msg.data = ss.str();
        pub.publish(msg);
        ROS_INFO("发送的消息:%s",msg.data.c_str());

        rate.sleep();
        count++;
        ros::spinOnce();
    }
    return 0;
}
```

2.1 Publish node

2.1.1 C++ code

- 代码讲解：

```
// 1.包含头文件
#include "ros/ros.h"
#include "std_msgs/String.h" //普通文本类型的消息
#include <sstream>


// 编写mian函数，在 main 函数里面编写代码

int main(int argc, char  *argv[])
{

}
```

2.1 Publish node

2.1.1 C++ code

```
//设置编码，解决中文乱码问题
setlocale(LC_ALL,"");

//2.初始化 ROS 节点：命名(唯一)
        // 参数1 和 参数2 后期为节点传值会使用
        // 参数3 是节点名称，是一个标识符，需要保证运行后，在 ROS 网络拓扑中唯一
ros::init(argc,argv,"talker");

//3.实例化 ROS 句柄，该类封装了 ROS 中的一些常用功能
ros::NodeHandle nh;
```

2.1 Publish node

2.1.1 C++ code

```
//4.实例化 发布者 对象
        //泛型：发布的消息类型，此处为<std_msgs::String>
        //参数1：要发布到的话题，此处为"chatter"
        //参数2：队列中最大保存的消息数，超出此阀值时，先进的先销毁(时间早的先销毁)
ros::Publisher pub = nh.advertise<std_msgs::String>("chatter",10);


//5.组织被发布的数据，并编写逻辑发布数据
        //数据(动态组织)
std_msgs::String msg;

//消息计数器，字符串拼接使用
 int count = 0;
```

2.1 Publish node

2.1.1 C++ code

```
//节点不终止就一直执行循环
 while (ros::ok())
 {
    //使用 stringstream 拼接字符串与编号
    std::stringstream ss;
    ss << "Hello 你好！ Num: " << count;
    msg.data = ss.str();
    //发布消息
    pub.publish(msg);
    //加入调试，打印发送的消息
    ROS_INFO("发送的消息:%s",msg.data.c_str());
    //根据前面制定的发送贫频率自动休眠 休眠时间 = 1/频率；
    rate.sleep();
    count++; //循环结束前，让 count 自增
    //暂无应用
    ros::spinOnce();
 }
```

2.1 Publish node

2.1.1 C++ code

- 修改 CMakeLists.txt：

```
add_executable(sub_c src/sub_c.cpp)

add_dependencies(sub_c ${${PROJECT_NAME}_EXPORTED_TARGETS}
${catkin_EXPORTED_TARGETS})

target_link_libraries(sub_c
  ${catkin_LIBRARIES}
)
```

2.2 Subscribe node

2.2.1 C++ code

- **代码实现流程：**

  1.包含头文件

  2.初始化 ROS 节点:命名(唯一)

  3.实例化 ROS 句柄

  4.实例化 订阅者 对象

  5.处理订阅的消息(回调函数)

  6.设置循环调用回调函数

注：附注解的代码在 sub_c.cpp 文件中

2.2 Subscribe node

2.2.1 C++ code

- 在 pub_sub_test 功能包的 src 文件夹中新建 sub_c.cpp 文件，输入以下代码：

```
#include "ros/ros.h"
#include "std_msgs/String.h"

void doMsg(const std_msgs::String::ConstPtr& msg_p){
    ROS_INFO("接收的消息:%s",msg_p->data.c_str());
}
```

2.2 Subscribe node

2.2.1 C++ code

- 在 pub_sub_test 功能包的 src 文件夹中新建 sub_c.cpp 文件，输入以下代码：

```
int main(int argc, char  *argv[])
{
    setlocale(LC_ALL,"");
    ros::init(argc,argv,"listener");
    ros::NodeHandle nh;

    ros::Subscriber sub = nh.subscribe<std_msgs::String>("chatter",10,doMsg);

    ros::spin();

    return 0;
}
```

2.2 Subscribe node

2.2.1 C++ code

- 代码讲解：

```
// 1.包含头文件
#include "ros/ros.h"
#include "std_msgs/String.h"


// 2. 编写回调函数
void doMsg(const std_msgs::String::ConstPtr& msg_p){
    ROS_INFO("接收的消息:%s",msg_p->data.c_str());
```

1、订阅者代码中，当收到订阅的消息时，激活回调函数，执行回调函数的内容

2、订阅者代码中，要使用 spin() 函数循环读取接收的数据，此过程会阻塞

2.2 Subscribe node

2.2.1 C++ code

- 代码讲解：

```
// 3.编写mian函数，在 main 函数里面编写代码
int main(int argc, char  *argv[])
{

}

//设置编码，解决中文乱码问题
setlocale(LC_ALL,"");

//初始化 ROS 节点:命名(唯一)
ros::init(argc,argv,"listener");

//实例化 ROS 句柄
ros::NodeHandle nh;
```

2.2 Subscribe node

2.2.1 C++ code

- 代码讲解：

```
//实例化 订阅者 对象
ros::Subscriber sub = nh.subscribe<std_msgs::String>("chatter",10,doMsg);

//处理订阅的消息(见回调函数)


//设置循环调用回调函数
ros::spin();//循环读取接收的数据，并调用回调函数处理
```

2.1 Publish node

2.1.1 C++ code

- 修改 CMakeLists.txt：

```
add_executable(pub_c src/pub_c.cpp)

add_dependencies(pub_c ${${PROJECT_NAME}_EXPORTED_TARGETS}
${catkin_EXPORTED_TARGETS})

target_link_libraries(pub_c
  ${catkin_LIBRARIES}
)
```

ROS Summary

1. VSCode
    a) Download
    b) Installation
    c) Plugins
    d) Setups
2. Communication Model 
    a) Topic model
    b) Service model
3. Topic Model C++ Code 
    a) Publush node
    b) Subscribe node

Overview lesson 10-1

1

Topic communication example-custom message type

2

Service communication example-custom service type

3

4

Objectives

1、掌握自定义消息和自定义服务类型的定义方法

2、掌握上述两种方法发布和订阅节点的编程方法



Pre-assessment

1、是否用过自定义消息类型?

2、是否用过自定义服务类型?

1

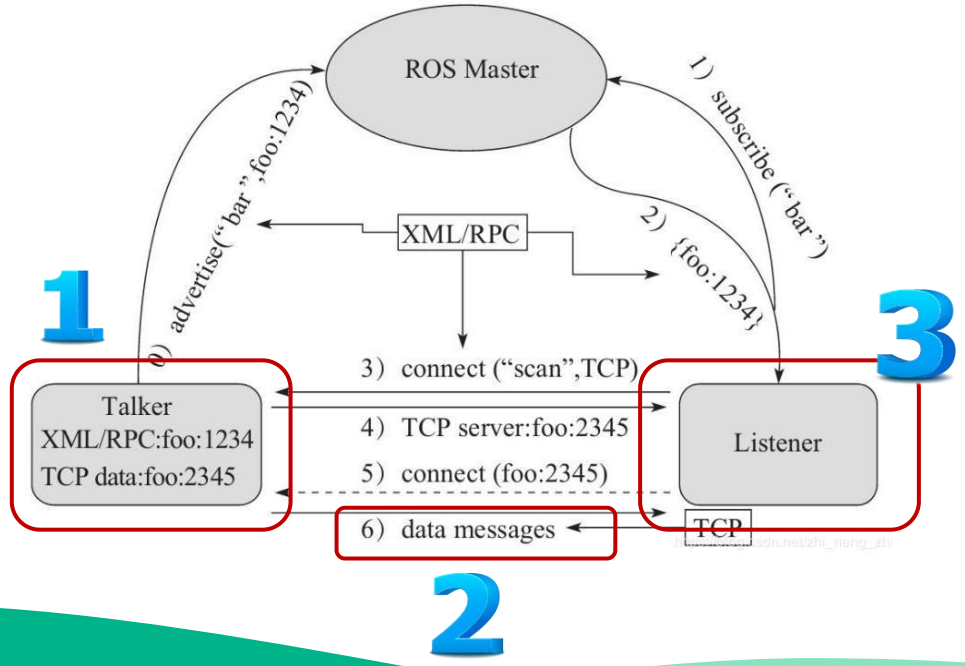
Topic communication example-custom message type

1.1 Standard message type

1.1.1 Overview

- **要求**：编写发布订阅实现，要求发布方以1Hz(每秒1次)的频率发布自定义文本消息，订阅方订阅消息并将消息内容打印输出

- **分析**：在模型实现中，ROS master 不需要实现，而连接的建立也已经被封装了，需要关注的关键点有三个：
  1. 发布方
  2. 接收方
  3. 数据(此处为普通文本)



- **发布方实现流程**：
  1. 编写发布方实现
  2. 编写订阅方实现
  3. 编辑配置文件
  4. 编译并执行

1.1 Standard message type

1.1.2 Publish node

- **发布方**代码实现流程：

  1.包含头文件

  2.初始化 ROS 节点：命名(唯一)

  3.实例化 ROS 句柄

  4.实例化 发布者 对象

  5.组织被发布的数据，并编写逻辑发布数据

1.1 Standard message type

1.1.2 Publish node

```
#include "ros/ros.h"
#include "std_msgs/String.h" //普通文本类型的消息
#include <sstream>

int main(int argc, char  *argv[])
{
    setlocale(LC_ALL,"");

    ros::init(argc,argv,"talker");

    ros::NodeHandle nh;

    ros::Publisher pub =
nh.advertise<std_msgs::String>("chatter",10);

    std_msgs::String msg;

    int count = 0;

    ros::Rate rate(1);

    while (ros::ok())
    {
        std::stringstream ss;
        ss << "Hello 你好啊"  << count;
        msg.data = ss.str();
        pub.publish(msg);
        ROS_INFO("发送的消息:%s",msg.data.c_str());
        rate.sleep();
        count++;
        ros::spinOnce();
    }
    return 0;
}
```

1.1 Standard message type

1.1.3 Subscribe node

- **订阅方代码实现流程:**

  1.包含头文件

  2.初始化 ROS 节点：命名(唯一)

  3.实例化 ROS 句柄

  4.实例化 订阅者 对象

  5.处理订阅的消息(回调函数)

  6.设置循环调用回调函数

1.1 Standard message type

1.1.3 Subscribe node

```
#include "ros/ros.h"
#include "std_msgs/String.h"

void doMsg(const std_msgs::String::ConstPtr& msg_p){
    ROS_INFO("接收的消息:%s",msg_p->data.c_str());
}

int main(int argc, char  *argv[])
{
    setlocale(LC_ALL,"");

    ros::init(argc,argv,"listener");

    ros::NodeHandle nh;

    ros::Subscriber sub = nh.subscribe<std_msgs::String>("chatter",10,doMsg);

    ros::spin();

    return 0;
}
```

1.1 Standard message type

1.1.4 Setups

- 修改CMakeLists.txt：

```
add_executable(pub_c src/pub_c.cpp)
add_executable(sub_c src/sub_c.cpp)

add_dependencies(pub_c ${PROJECT_NAME}_generate_messages_cpp)
add_dependencies(sub_c ${PROJECT_NAME}_generate_messages_cpp)

target_link_libraries(pub_c  ${catkin_LIBRARIES})
target_link_libraries(sub_c  ${catkin_LIBRARIES})
```

1.2 Custom message type

1.2.1 Overview

- 在 ROS 通信协议中，数据载体是一个重要组成部分
- ROS 中通过 std_msgs 封装了一些原生的数据类型，比如：String、Int32、Int64、Char、Bool、Empty
- String数据只包含一个 data 字段，结构的单一意味着功能上的局限性，当传输一些复杂的数据，String由于描述性较差而显得力不从心，这种场景下可以使用自定义的消息类型

1.2 Custom message type

1.2.1 Overview

- 可以使用的基本消息类型有：
  - int8, int16, int32, int64 (或者无符号类型: uint*)
  - float32, float64
  - string
  - time, duration
  - other msg files
  - variable-length array[] and fixed-length array[C]
  - ROS中还有一种特殊类型：Header，标头包含时间戳和 ROS 中常用的坐标帧信息。会经常看到 msg 文件的第一行具有 Header 标头

1.2 Custom message type

1.2.1 Overview

- **需求**：传输的消息中需要包含：人的姓名(string)、身高(float32)、年龄(int16)三类信息
- **解决方案**：自定义消息类型
- **整体流程**：

1. 按照固定格式创建 msg 文件
2. 编辑配置文件
3. 编译生成可以被 C++ 调用的中间文件

1.2 Custom message type

1.2.2 Step1 : build msg file

- 第1步：定义 msg 文件：
  - 方法：在功能包下新建 msg 目录，添加文件 person.msg

```
string name

uint16 age

float64 height
```

1.2 Custom message type

1.2.3 Step2 : edit configuration file

- 第2步：编辑配置文件
  - 配置 package.xml 文件
  - 在 package.xml 中添加编译依赖与执行依赖

```
<build_depend>message_generation</build_depend>
<exec_depend>message_runtime</exec_depend>
<!--
exce_depend 以前对应的是 run_depend
-->
```

1.2 Custom message type

1.2.3 Step2 : edit configuration file

- 第2步：编辑配置文件
  - 配置 CMakeLists.txt文件：共4处
  - 第1处：配置 find_package()

```
find_package(catkin REQUIRED COMPONENTS
  roscpp
  rospy
  std_msgs
  message_generation
)
# 需要加入 message_generation，必须有 std_msgs
```

1.2 Custom message type

1.2.3 Step2 : edit configuration file

- 第2处：配置 msg 源文件

```
add_message_files(
  FILES
  person.msg
)
```

- 第3处：配置 generate_messages()

```
generate_messages(
  DEPENDENCIES
  std_msgs
)
```

1.2 Custom message type

1.2.3 Step2 : edit configuration file

- 第4处：配置运行时的依赖

```
catkin_package(
#  INCLUDE_DIRS include
#  LIBRARIES demo02_talker_listener
  CATKIN_DEPENDS roscpp rospy std_msgs message_runtime
#  DEPENDS system_lib
)
```

Ctrl + Shift + B 编译，生成的头文件位于：工作空间 /devel/include/包名/xxx.h

1.3 Publish node

1.3.1 Overview

- **需求**：编写发布订阅实现，要求发布方以1Hz(每秒1次)的频率发布自定义 person 消息，订阅方订阅自定义消息并将消息内容打印输出
- **分析**：在模型实现中，ROS master 不需要实现，而连接的建立也已经被封装了，需要关注的关键点有三个：
  1. 发布方
  2. 接收方
  3. 数据(此处为自定义消息)
- **整体流程**：
  1. 编写发布方实现
  2. 编写订阅方实现
  3. 编辑配置文件
  4. 编译并执行

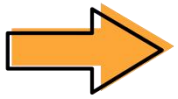
1.3 Publish node

1.3.1 Overview

- 为了方便代码提示以及避免误抛异常，需要先配置 VSCode
- 将前面生成的 头文件路径配置进 c_cpp_properties.json 的 includepath属性

.vscode > {} c_cpp_properties.json > [ ] configurations > {} 0

```
4       "browse":{
5         "databaseFilename": "",
6         "limitSymbolsToIncludedHeaders": true
7       },
8       "includePath": [
9         "/home/ros/catkin_ws/devel/include/**",
10        "/opt/ros/melodic/include/**",
11        "/home/ros/catkin_ws/src/beginner_tutorials/include/**",
12        "/home/ros/catkin_ws/src/pub_sub/include/**",
13        "/usr/include/**",
14        "/home/ros/pub_sub_ws/devel/include/**"
15      ],
16      "name": "ROS"
17    }
18  ],
19  "version": 4
20 }
```



```
"includePath": [
        "/opt/ros/noetic/include/**",
        "/usr/include/**",
        "/xxx/yyy工作空间/devel/include/**"
    ],
```

1.3 Publish node

1.3.2 C++ code

```
#include "ros/ros.h"
#include "beginner_tutorials/person.h"

int main(int argc, char *argv[])
{
    setlocale(LC_ALL,"");
    ros::init(argc,argv,"talker_person");
    ros::NodeHandle nh;
    ros::Publisher pub = nh.advertise<beginner_tutorials::person>("chatter_person",1000);
    beginner_tutorials::person person;
    person.name = "ROSit";
    person.age = 0;
    person.height = 1.75;
    ros::Rate r(1);
```

注：代码位于 pub_person_c.cpp

1.3 Publish node

1.3.2 C++ code

```
while (ros::ok())
    {
        pub.publish(person);
        person.age += 1;
        ROS_INFO("我叫:%s,我今年%d岁,我的身高是%.2f米", person.name.c_str(), person.age, person.height);
        r.sleep();
        ros::spinOnce();
    }

    return 0;
}
```

1.3 Publish node

1.3.3 CMakeLists.txt

```
add_executable(pub_person_c src/pub_person_c.cpp)

add_dependencies(pub_person_c ${PROJECT_NAME}_generate_messages_cpp)

target_link_libraries(pub_person_c  ${catkin_LIBRARIES})
```

1.4 Subscribe node

1.4.1 C++ code

```
#include "ros/ros.h"
#include "beginner_tutorials/person.h"

void doPerson(const beginner_tutorials::person::ConstPtr& person_p){
    ROS_INFO("发布人的信息:%s, %d, %.2f", person_p->name.c_str(), person_p->age, person_p->height);
}
```

注：代码位于sub_person_c.cpp

1.4 Subscribe node

1.4.1 C++ code

```
int main(int argc, char *argv[])
{
    setlocale(LC_ALL,"");
    ros::init(argc,argv,"listener_person");
    ros::NodeHandle nh;
    ros::Subscriber sub = nh.subscribe<beginner_tutorials::person>("chatter_person",10,doPerson);
    ros::spin();
    return 0;
}
```

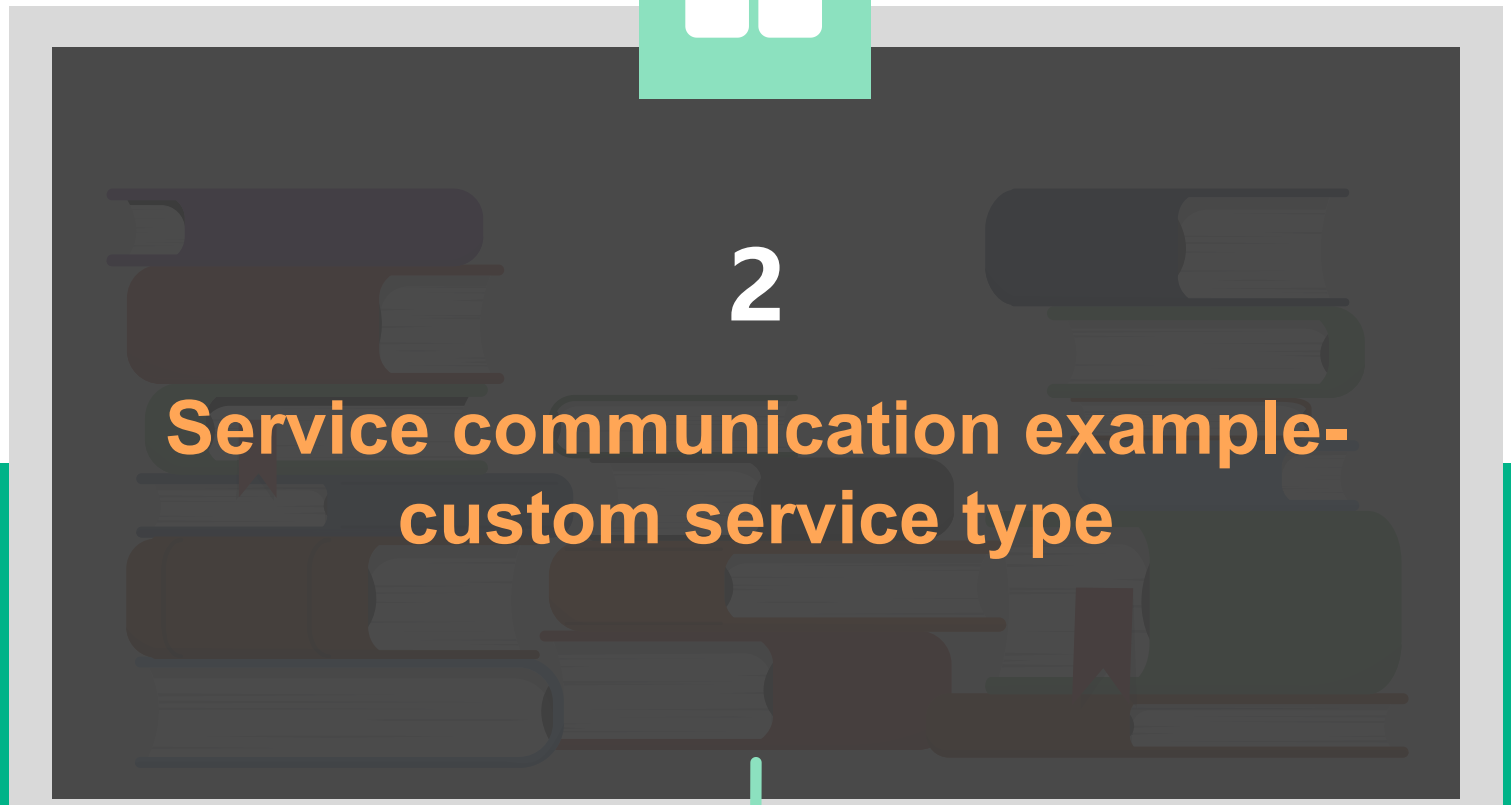
1.4 Subscribe node

1.4.2 CMakeLists.txt

```
add_executable(sub_person_c src/sub_person_c.cpp)

add_dependencies(sub_person_c ${PROJECT_NAME}_generate_messages_cpp)

target_link_libraries(sub_person_c  ${catkin_LIBRARIES})
```



2

Service communication example- custom service type

2.1 Custom service type

2.1.1 Overview

- **需求**：
  - 服务通信中，客户端提交两个整数至服务端，服务端求和并响应结果到客户端，服务器与客户端通信的数据载体自定义

- **流程**：
  - srv 文件内的可用数据类型与 msg 文件一致，定义 srv 实现流程与自定义 msg 实现流程类似：
    1. 按照固定格式创建 srv 文件
    2. 编辑配置文件
    3. 编译生成中间文件

2.1 Custom service type

2.1.2 Step1 : build srv file

- 服务通信中，数据分成两部分：请求与响应；在 srv 文件中请求和响应使用---分割，具体实现如下：

- 第1步：功能包下新建 srv 目录，新建 sum.srv 文件，添加以下内容：

```
# 客户端请求时发送的两个数字
int32 num1
int32 num2
---
# 服务器响应发送的数据
int32 sum
```

2.1 Custom service type

2.1.3 Step2 : edit configuration file

- 第2步：配置 package.xml

```
<build_depend>message_generation</build_depend>

<exec_depend>message_runtime</exec_depend>

<!--

exce_depend 以前对应的是 run_depend

-->
```

2.1 Custom service type

2.1.3 Step2 : edit configuration file

- 第2步：配置 CMakeLists.txt，共3处，第1处：

```
find_package(catkin REQUIRED COMPONENTS
roscpp
rospy
std_msgs
message_generation
)
# 需要加入 message_generation,必须有 std_msgs
```

2.1 Custom service type

2.1.3 Step2 : edit configuration file

- 第2步：配置 CMakeLists.txt，共3处，第2处：

```
add_service_files(
FILES
sum.srv
)
```

2.1 Custom service type

2.1.3 Step2 : edit configuration file

- 第2步：配置 CMakeLists.txt，共3处，第3处：

```
generate_messages(
DEPENDENCIES
std_msgs
)
```

2.2 Publish node

2.2.1 Overview

- **服务器实现**:

  1.包含头文件

  2.初始化 ROS 节点

  3.创建 ROS 句柄

  4.创建 服务 对象

  5.回调函数处理请求并产生响应

  6.由于请求有多个，需要调用 ros::spin()

2.2 Publish node

2.2.2 C++ code

```
#include "ros/ros.h"
#include "beginner_tutorials/sum.h"

bool doReq(beginner_tutorials::sum::Request& req, beginner_tutorials::sum::Response& resp){
    int num1 = req.num1;
    int num2 = req.num2;

    ROS_INFO("服务器接收到的请求数据为:num1 = %d, num2 = %d",num1, num2);

    if (num1 < 0 || num2 < 0)
    {
        ROS_ERROR("提交的数据异常：数据不可以为负数");
        return false;
    }

    resp.sum = num1 + num2;
    return true;
}
```

注：代码位于 sum_server_c.cpp

2.2 Publish node

2.2.2 C++ code

```
int main(int argc, char *argv[])
{
    setlocale(LC_ALL,"");
    ros::init(argc,argv,"sum_server");
    ros::NodeHandle nh;
    ros::ServiceServer server = nh.advertiseService("Sum",doReq);
    ROS_INFO("服务已经启动....");
    ros::spin();
    return 0;
}
```

注：代码位于sum_server.cpp

2.3 Subscribe node

2.3.1 C++ code

```
#include "ros/ros.h"
#include "beginner_tutorials/sum.h"

int main(int argc, char *argv[])
{
    setlocale(LC_ALL,"");

    if (argc != 3)
    {
        ROS_ERROR("请提交两个整数");
        return 1;
    }

    ros::init(argc,argv,"sum_client");
    ros::NodeHandle nh;
    ros::ServiceClient client = nh.serviceClient<beginner_tutorials::sum>("Sum");
    ros::service::waitForService("Sum");
```

注：代码位于sum_client_c.cpp

2.3 Subscribe node

2.3.1 C++ code

```
    beginner_tutorials::sum ai;
    ai.request.num1 = atoi(argv[1]);
    ai.request.num2 = atoi(argv[2]);
    bool flag = client.call(ai);
    if (flag)
    {
        ROS_INFO("请求正常处理,响应结果:%d",ai.response.sum);
    }
    else
    {
        ROS_ERROR("请求处理失败....");
        return 1;
    }

    return 0;
}
```

注：代码位于sum_client.cpp

2.4 CMakeLists.txt

2.4.1 Setups

```
add_executable(sum_server_c src/sum_server_c.cpp)
add_executable(sum_client_c src/sum_client_c.cpp)


add_dependencies(sum_server_c ${PROJECT_NAME}_gencpp)
add_dependencies(sum_client_c ${PROJECT_NAME}_gencpp)


target_link_libraries(sum_server_c  ${catkin_LIBRARIES})
target_link_libraries(sum_client_c  ${catkin_LIBRARIES})
```

2.5 Additional statement

2.5.1 Matters needing attention

- **执行：**
  1. 需要先启动服务：rosrun 包名 服务
  2. 然后再调用客户端：rosrun 包名 客户端 参数1 参数2

- **结果：**会根据提交的数据响应相加后的结果

- **注意：**如果先启动客户端，那么会导致运行失败

- **解决方案：**在客户端发送请求前添加（二选一）：
  1. client.waitForExistence();
  2. ros::service::waitForService("AddInts");
  3. 这是一个阻塞式函数，只有服务启动成功后才会继续执行

Summary

1. Topic communication example-custom message type

   a) Standard message type

   b) Custom message type

   c) Publish node

   d) Subscribe node

2. Service communication example-custom service type

   a) Custom service type

   b) Publish node

   c) Subscribe node

   d) Additional statement

Q&A

今日事今日毕

Question?